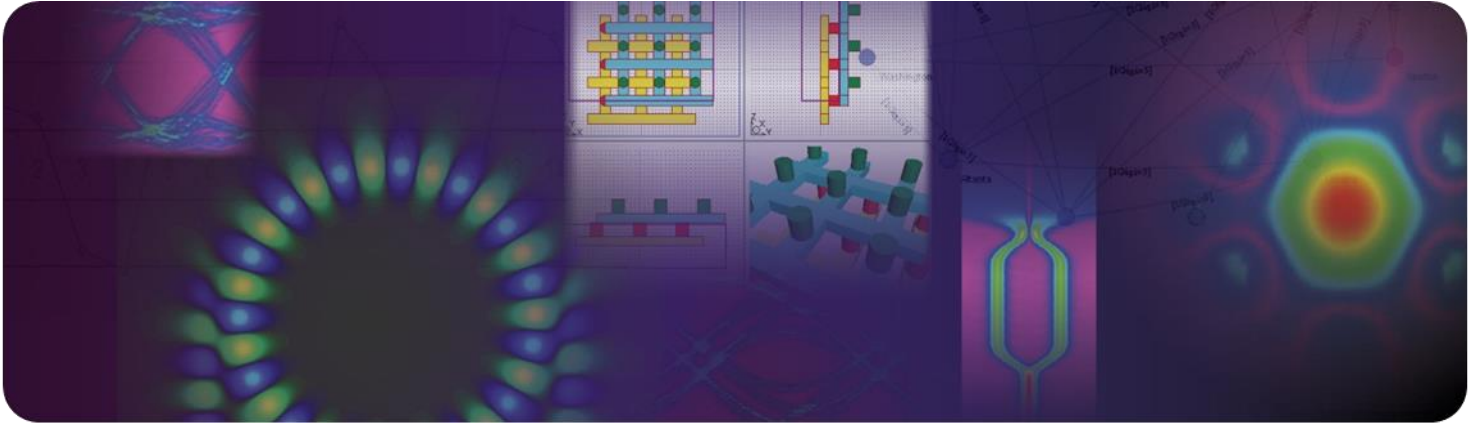




Photonic Solutions

RSoft CAD

v2024.09

User Guide

Copyright Notice and Proprietary Information

Copyright © 2025 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Right to Copy Documentation

The license agreement with Synopsys permits licensee to make copies of the documentation for its internal use only. Each copy shall include all copyrights, trademarks, service marks, and proprietary rights notices, if any. Licensee must assign sequential numbers to all copies. These copies shall contain the following legend on the cover page:

"This document is duplicated with the permission of Synopsys, Inc., for the exclusive use of _____ and its employees. This is copy number _____."

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys' company and certain product names are trademarks of Synopsys, as set forth at:

<http://www.synopsys.com/Company/Pages/Trademarks.aspx>. All other product or company names may be trademarks of their respective owners.

FOSS Notices

Free and Open Source Software (FOSS) notices are located in the \readme subfolder of the RSoft installation folder.

Contents

Preface	1
Foreword	1
How to Read This Manual	3
 1 Introduction	 7
1.A. Program Installation & System Requirements	7
1.B. Physical Conventions	7
1.C. Program Executables	8
1.D. Example Files	9
1.E. README Files	9
1.F. Product Support, Upgrades, and Resources	10
 2 A Simple Tutorial	 11
Step 1: Opening the CAD Window	11
Step 2: Creating a New Design File	12
Step 3: Defining Variables	14
Step 4: Drawing a Component	15
Step 5: Moving a Component	16
Step 6: Set Component Properties Exactly	17
Step 7: Creating a Tapered Component	18
Step 8: Verifying Structure Layout	20
Step 9: Simulation & Advanced CAD Features	20
 3 Basic CAD Usage	 22
3.A. The CAD Window	22
3.A.1. Menubar	22
3.A.2. Toolbars	25

3.A.3. Status Bar	30
3.A.4. Tree View Toolbar	30
3.A.5. RSoft Tips	31
3.B. Creating a New Design File	31
3.C. The Symbol Table & Variables	34
3.C.1. Why use the Symbol Table & Variables?	35
3.C.2. Using the Symbol Table	35
3.D. Drawing Components	37
3.D.1. Drawing Modes	37
3.D.2. Placing Components in a Design File	38
3.D.3. Drawing Preferences	39
3.D.4. Try It!	39
3.E. Viewing Components	39
3.E.1. Single-Pane Mode	40
3.E.2. Multi-Pane Mode (3D Only)	40
3.E.3. Zooming	41
3.E.4. Panning	41
3.E.5. Display Aspect Ratio:	41
3.E.6. Displaying 'Simulation' Objects	42
3.F. Editing Components	42
3.F.1. Selecting Components	42
3.F.2. Modifying Components	43
3.G. Calculating Material Profiles	45
3.G.1. Computing a Material Profile	45
3.G.2. Saving a Material Profile	49
3.G.3. Producing Index Profile Plots from the Command Line	49
3.H. Viewing the Current Value of a Parameter	50
3.I. Easily Edit Long Expressions	50
3.J. Editable Drop-Down Lists	50

4 Segment Components 52

4.A. Segment Overview	52
4.A.1. Basic Segment Properties	52
4.A.2. Segment Orientation Overview	53
4.A.3. Setting Segment Properties	54
4.A.4. Setting Segment Display Color	54
4.B. Vertices	55
4.B.1. Setting Position	55
4.B.2. Index Definition	56
4.B.3. Controlling Geometry	57
4.C. Basic 2D Segments	58
4.D. 3D Structure Types	58
4.D.1. Fiber	59

4.D.2. Channel.....	59
4.D.3. Diffused	60
4.D.4. Rib/Ridge	62
4.D.5. Multilayer	64
4.D.6. Polygon Structure Types	67
4.D.7. Mixing 3D Structure Types	69
4.E. Profiles & Tapers.....	69
4.F. Segment Types (z-Axis, Seg-Axis, Extended)	69
4.G. Drawing Common Extended Segments	71

5 Other Component Types 74

5.A. General Overview	74
5.B. Lenses	74
5.C. Circles.....	77
5.D. Polygons.....	79
5.E. Registration Marks	81
5.F. Circuit References, Simulation Regions, Vertices, and Monitors	82

6 Advanced CAD Features 83

6.A. Profiles.....	83
6.A.1. Built-in Profile Types.....	83
6.A.2. Types of User Profiles	85
6.A.3. Creating & Using User Profiles	87
6.A.4. Example User Profile	89
6.B. Tapers.....	90
6.B.1. Component Options that Support Tapering.....	91
6.B.2. Built-In Taper Types	91
6.B.3. Built-In Arc Taper Types	92
6.B.4. Creating & Using User Tapers	93
6.B.5. An Example User Taper	95
6.C. Priority (When Components Overlap).....	96
6.C.1. Combine Mode	96
6.C.2. Priority	96
6.C.3. General Notes	97
6.D. Global Index Generation Options	97
6.D.1. Simulating Curved Segments to Large Angles.....	98
6.D.2. Sidewall Angle Control	99
6.D.3. Simulating the Effects of Finite Lithographic Resolution	99
6.D.4. Simulating the Effects of Lithographic Roughness.....	99
6.D.5. Interpreting User Profiles Absolutely.....	101
6.E. Geometry Import/Export.....	102
6.E.1. Export.....	102

6.E.2. Import	102
6.E.3. Import/Export Settings and Important Notes	103
6.F. Circuit References	104
6.F.1. Creating a Circuit Reference	105
6.F.2. Setting Circuit Reference Properties	106
6.F.3. Editing a Circuit Reference	108
6.F.4. Controlling Variables in Child File From Parent File	108
6.F.5. Hierarchy Exports	109
6.F.6. Creating Python Functions	111
6.F.7. Adding Vertices to a Circuit Reference	113
6.F.8. Dynamically Importing GDS files via Circuit References	113
6.G. Dynamically-Sized Arrays	114
6.G.1. Creating a Dynamic Array	115
6.G.2. Setting Lattice Properties	115
6.H. Defining Pathways	116
6.H.1. Creating a Pathway	116
6.H.2. Using Pathways	117
6.I. Importing Symbols or Components from Another Design File	118
6.I.1. Importing Symbols	118
6.I.2. Importing Components	120
6.J. CAD Preferences	120
6.K. Additional Component Properties	122
6.K.1. General Properties	123
6.K.2. Extended Segment Properties	124
6.K.3. Crystal Axes	125
6.K.4. Component Tags	126
6.L. Using EIM to Simulate 3D Structures in 2D (2.5D)	128
6.M. Expanded Referencing of Vertices	129
6.N. Warning Options	130
6.O. Component Filters	130
6.O.1. Defining Filters	130
6.O.2. Using Filters	133
6.P. Simulation Log Files	133

7 Layout Generation Utilities 135

7.A. Motivation	135
7.B. Lattice Parameters	136
7.A.1. 1D Lattice Types	137
7.B.2. 2D Lattice (XZ and XY) Types	137
7.B.3. 3D Lattice Types	138
7.C. Unit Cell Parameters	139
7.C.1. 1D Unit Cells	139
7.C.2. 2D Unit Cells (XZ and XY)	140

7.C.3. 3D Unit Cells	141
7.C.4. Additional Options	142
7.D. Creation Options	143
8 Advanced Materials	144
8.A. Using the Material Editor	144
8.A.1. The Material Tree	145
8.A.2. The Material Tabs.....	146
8.A.3. Creating and Organizing Materials in the Material Editor	146
8.A.4. Using Materials in a Project (Local Design File)	147
8.A.5. Displaying Material Definition.....	148
8.A.6. Material Properties.....	148
8.A.7. Tips & Traps	149
8.B. Linear Epsilon Properties	151
8.B.1. Linear Epsilon Overview	151
8.B.2. Simple Linear Epsilon	152
8.B.3. Anisotropic Linear Epsilon	154
8.B.4. Dispersive Linear Epsilon	155
8.C. Non-Linear Epsilon Properties	159
8.D. Linear Mu Properties.....	162
8.E. Non-Linear Mu Properties	163
8.F. Electro-Optic Properties	164
8.G. Thermo-Optic Properties.....	165
8.H. Stress-Optic Properties.....	166
8.I. Semiconductor Properties	167
8.K. Special Materials.....	171
9 Non-Uniform Grids	173
9.A. Grid Basics.....	173
9.B. Advanced Grid Controls	174
9.C. Overriding Grid Settings for Specific Components or Materials	177
9.D. Viewing the Grid.....	178
10 Scripting & Scanning	179
10.A. Layout Scripts	179
10.A.1. Using Scripts within the RSoft CAD	179
10.A.2. Generating Design Files Completely via Scripts	191
10.B. Parameter Scanning and Optimization	195
10.C. Data Manipulation	195
10.D. Simulation Scripts	196
10.D.1. Running the Simulation Programs from the Command Line.....	196

10.D.2. Creating a Simulation Script	198
10.D.3. Using Schedulers	199
11 Transverse Mode Solving	200
11.A. BPM-Based Mode Solvers	200
11.A.1 Basic Theory of BPM-Based Mode Solving	201
11.A.2. Using the BPM Mode Solvers	202
11.B. TMM-Based Mode Solver (TmmSIM)	204
11.B.1. Using TmmSIM.....	204
11.B.2. Output Options	205
11.C. FEM-based Mode Solver (FemSIM).....	205
Appendix A Tips and Traps	206
A.A. Common RSoft CAD mistakes.....	206
A.B. Some Good RSoft CAD habits to learn	207
Appendix B File Formats	209
B.A. Standard RSoft File Format	209
B.A.1. Syntax Parameters.....	209
B.A.2. File Syntax	211
B.A.3. Example Files.....	213
B.B. TDR File Format	217
B.B.1. Using TDR Files as Inputs to the RSoft Software	217
B.B.2. Enabling TDR File Output from the RSoft Software	218
B.C. User-Profile Functions	218
B.D. User-Taper Functions.....	219
B.E. Polygon Files	219
Appendix C RSoft Expressions	220
C.A. Arithmetic Operators & Order of Operations	220
C.B. Built-in Constants	221
C.C. Using File Names in Variables/Expressions	221
C.D. Defining Default Value for Variable when used in Expression.....	221
C.E. Square Bracket Notation for Hierarchy Export Symbols.....	222
C.F. Built-In Functions.....	222
C.F.1. Basic Functions.....	222
C.F.2. Comparison Functions	223
C.F.3. Logical Functions:	224
C.F.4. Step Functions	225
C.F.5. Random Functions	225
C.F.6. Data-File Functions	226

C.F.7. Material Functions	229
C.F.8. Effective Index Functions	231
C.F.9. Polygon Functions.....	232
C.F.10. PDK Functions	232
C.F.11. String Functions	233
C.F.12. Unit Conversion Functions	233
C.F.12. Sentaurus Workbench (TCAD) Functions.....	234

Appendix D Symbol Table Variables 235

D.A. Startup Window/Global Settings	235
D.B. Special Effects.....	236
D.C. Advanced Grid Parameters	236
D.D. Advanced Features	237
D.E. Display Material Profile	238
D.F. CAD Preferences:	239

Appendix E Utilities 240

bdconv – Matrix Manipulation	242
bdutil – Overlap Integrals, Far Fields, Spot Sizes, and LightTools Ray Files	246
Syntax.....	246
Overlap Integrals: -i.....	247
Coupling Coefficients: -k	249
Field Size Calculation: -s	250
Far Field Calculations: -f	251
LightTools Interface	254
Simple File Arithmetic and Data Transformations: -o, -sc, -tr	255
bmp2ind – Convert Bitmap Image to a User Profile	256
Syntax:.....	256
Usage:	256
Indexed-Color Bitmap Background	258
Examples	259
bp2codev and codev2bp – Ray Tracing Converter for CODE V	262
Converting from RSoft to CODE V (bp2codev)	262
Converting from CODE V to RSoft (codev2bp)	263
BSDF Utilities (bsdfgen and bsdfutil)	265
Custom PDK Utility (smatgen)	266
coatuf – Create Coating for User Profile	267
disperse – Calculate Dispersion Relations.....	269
dx2ind – Convert DXF Mask to .ind File.....	270
ffutil – Far-Field Utility	271
fhakit – Fast Harmonic Analysis of Complex Exponential Time Series	272

findpks – Find Peaks Utility	277
fitdisp – Dispersion Fitting Utility	279
fwmodekit – Extract Data From FullWAVE State File.....	282
ind2gds, ind2dxf – Convert .ind File to Mask	284
gds2ind – Convert GDS Mask to .ind File	285
killall – Utility to stop processes	286
Mask Conversion Utilities	287
GDS Utilities	287
DXF Utilities	288
mat2bp – Convert Matrix Data to RSoft Format.....	289
mathmat – Do Arithmetic with Data Files	290
parutil – Generate nk data files from TCAD Sentaurus parameter files	293
Ray-Tracing Conversion Utility	294
rsanimate – Create Animations from Plot Files	295
rscombine – Combine Frequency Results	296
rspython – Python Environment.....	297
shufflemat – Rearrange Data Files	298
smatgen - Custom PDK Utility	301
tailmon – Extract Final Value Utility	302
tdrutil – TDR File Utility.....	303
wgmode – Waveguide Mode Utility.....	307

Appendix F Color Scales 308

F.A. What are Scale Files?	308
F.B. Using Color Scales	309
F.C. Scale File Format.....	310

Appendix G Polarization Conventions 313

G.A. Background	313
G.B. The RSoft Convention	314
G.C. Simple rules to remember	317
G.D. Complications	317

Appendix H Python Usage 319

H.A. Python Version Support.....	319
Note: Numeric Array Support	320
H.B. Layout API: RSoftCircuit() class.....	320
H.B.1. RSoftCircuit() class	320
H.B.2. RSoftComponent() class	328
H.B.3. RSoftVertex() class	332
H.B.4. Using Hierarchy Exports	333
H.B.5. Best Practices	333

H.B.6. Example Usage.....	334
H.C. RSoft Data API: RSoftUserFunction()	337
H.C.1. Reference	337
H.C.2. Example Usage	339
 Appendix I RSoft CAD Release Notes	 341
 Index	 348

Preface

Foreword

The RSoft CAD is the core program in the RSoft Photonic Device Tools, and acts as a control program for the passive device simulation modules BeamPROP BPM, FullWAVE FDTD, BandSOLVE, GratingMOD, DiffractMOD RCWA, FemSIM FEM, and ModePROP EME. It is used to define the most important input required by these simulation modules: the material properties and structural geometry of a photonic device. A user will typically first design a structure in the CAD interface and then use one or more simulation engines to model various aspects of the device performance.

This modular approach to the design and simulation of photonic devices is one of RSoft's Photonic Device Tool's greatest strengths. Each program in the suite is designed to "play nice" with the other programs, creating an environment in which data can be shared between the modules. Virtually all the input and output files are in a simple ASCII text format, which allows even greater user control over program operation as well as third-party programs to be integrated into the suite.

While the tools are designed to be used via the GUI (Graphical User Interface), command line operation is also possible. This, coupled with the modularity of the Suite, allows for complex scripting capability. The tools are not limited to a single scripting language, but rather uses the native scripting language of your operating system. For example, Windows users can use DOS batch files, while Unix users can use bash scripts. Additionally, users familiar with languages such as Perl, Python, C, or C++ can create custom scripts in these languages. The RSoft

Component Design Suite provides the best of both worlds: it allows for simulations to be performed via the GUI, and for complicated custom simulations to be performed via a script. New and advanced users alike are able to realize the full power of the tools.

Associated Simulation Modules

The RSoft CAD is the main control program for a series of simulation modules which are licensed separately from RSoft. These simulation modules, along with typical applications, are:

BeamPROP BPM

BeamPROP is a simulation engine for the design of integrated and fiber-optic waveguide devices and circuits. The software incorporates advanced finite-difference beam propagation (BPM) techniques for simulation. The software has been commercially available since 1994, and is in use by researchers and development engineers in both university and industrial environments.

FullWAVE FDTD

FullWAVE is ideal for the design of complex photonic devices. The software employs the finite-difference time-domain (FDTD) method for simulation, which allows analysis of devices, such as photonic bandgaps and ring resonators, that cannot be modeled with techniques such as the efficient beam propagation method (BPM).

BandSOLVE

BandSOLVE is a simulation module for generating and analyzing photonic band structures. This simulation module is based on an advanced optimized implementation of the plane-wave expansion technique for periodic structures. It is ideal for producing band structures for classic photonic bandgap structures such as 2D and 3D photonic crystal waveguides and defect sites. In addition, it can be applied to fiber structures such as photonic crystal fibers and photonic bandgap fibers, which are particularly challenging for other simulation techniques. It is especially useful for optimizing the band structure properties of photonic crystal structures, which can be subsequently simulated in FullWAVE to examine time-dependent properties such as waveguide loss and coupling.

GratingMOD

This software tool helps model and analyze many devices that incorporate gratings and various kinds of filters, and is easily integrated with RSoft's award-winning simulation tools. It provides both a forward and backward analysis, allowing for the calculation of grating spectra from known geometries and the synthesis of grating structures from grating spectra.

DiffractMOD RCWA

DiffractMOD is a general design tool for diffractive optical structures such as diffractive optical elements, subwavelength periodic structures, and photonic bandgap crystals. It is based on the Rigorous Coupled Wave Analysis (RCWA) technique, and can simulate both 2D and 3D structures with an arbitrary lattice structure and unit cell index profile. In addition to dielectric materials, dispersive and lossy material structures such as metallic systems can also be used. Typical applications include Diffractive optical elements (DOEs), photonic bandgap structures, wavelength filters, optical metrology, nano-lithography, polarization sensitive devices, artificial dielectric coatings, photovoltaic systems, 3D displays, optical interconnections, optical data storage, spectroscopy, microlens arrays, and beam splitting, combining, and shaping.

FemSIM FEM

FemSIM uses the finite element method to calculate modes, both waveguide and cavity, of any 2D cross-section. A generally non-uniform and irregular mesh is used to solve for all field components of supported modes, both guided and leaky. FemSIM can easily find modes for highly hybrid structures, highly leaky structures, and structures with small feature sizes.

ModePROP EME

ModePROP is an eigenmode propagation tool that accounts for both forward and backward propagation and radiation modes. It provides a rigorous steady-state solution to Maxwell's equations that is based on the highly stable Modal Transmission Line Theory. A full array of analysis and simulation features makes this tool flexible and easy to use.

How to Read This Manual

While this manual can (and should!) be used as a reference manual at times, it is recommended that you read the first five chapters in their entirety to get a firm foundation from which to interpret the rest of the documentation. This is not only important for learning to use the RSoft CAD, but also for using any associated simulation modules as the correct usage of the CAD is critical to attaining meaningful simulation results.

If you are a new user, please reject the notion that you can simply turn to a section of interest in this manual or any simulation manuals for BeamPROP, FullWAVE, BandSOLVE, GratingMOD, DiffractMOD, FemSIM, or ModePROP. It can be hard to resist this urge, but doing so will reward you with better usage habits, a fuller understanding of how the software works, and will help you from suffering from common usage problems.

How is This Manual Organized?

This manual can be logically split into several main parts:

Introductory Information

Chapters 1 and 2 provide an overview of the installation of the program, an introduction to the program and its components, information about technical support and product upgrades, and an introductory tutorial. All new users should read these chapters in their entirety.

Basic CAD Usage

Chapters 3-5 explain the basic usage of the RSoft CAD including an overview of the CAD window, the symbol table, creating new design files, adding/removing components from a design, setting component properties such as material and geometry information, and the various component types. All new users should read these chapters in their entirety.

Advanced CAD Usage

Chapters 6 and higher cover advanced topics, automatic layout utilities, advanced material properties, non-uniform grids, parameter scanning/scripting, the multi-physics utility, and basic transverse mode calculations. These chapters can be treated as a reference manual and do not need to be read by all new users.

Appendices

The appendices provide detailed discussion of file formats, expressions, utilities, and other relevant information. The appendices can be treated as a reference manual and do not need to be read by all new users.

This manual should be used alongside the simulation tool manuals as described in the next section.

Where Is The Documentation For...

The documentation for the RSoft Photonic Device Tools is divided into several manuals. The manuals are structured using a simple rule:

Anything defining geometry and/or material parameters is in the CAD manual. Anything else is in an appropriate simulation manual.

Using this rule, almost any topic can be found. As with any rule, there are a few exceptions. The major exceptions are:

- *Installation*

The installation procedure is covered in the Photonic Solutions Installation Guide.

- *Parameter Scanning/Scripting/Batch Operation*

These topics are very similar, and are shared by all the simulation modules. They are discussed in Chapter 10 of the CAD manual.

- *Computing the Index Profile*

Computing the index profile is discussed in Section 3.G of the CAD manual.

- *Pathways*

Pathways define the location of pathway monitors (used by BeamPROP and FemSIM) and the location and geometry of launch (initial) fields, in BeamPROP, FullWAVE, and ModePROP. They are documented in Section 6.G of the CAD manual.

- *Command Line Utilities*

The RSoft Photonic Device Tools ship with several command line utilities that perform a variety of tasks. These utilities are documented in Appendix E of the CAD manual.

- *RSoft Expressions*

Virtually any numeric field can accept an analytical expression involving pre-defined and user-defined variables. The form of these expressions, including valid arithmetic operators and functions can be found in Appendix C of the CAD manual.

- *Mode Solving*

A variety of mode solvers is included, see Chapter 11 in the CAD manual for an overview of the mode solving methods available.

Anytime this rule is violated, a note will direct the reader to the proper section in the proper manual.

Where are these manuals located?

All documentation is placed on your computer during the program installation. Online versions can be accessed through the RSoft CAD via the Help menu item, or the two help buttons on the right of the top toolbar. The actual files can be found in the subdirectory `help` in the installation directory. Additionally, PDF versions can be found in the subdirectory `docs`. These files require the Adobe Acrobat Reader, which can be obtained from Adobe (www.adobe.com) at no charge.

Typographical Conventions

A number of typeface and layout conventions are followed in this manual.

- The *names* of fields and controls in the GUI dialogs are written in **boldface**
- The *values* of pull-down menus and radio button controls are written in *italics*.
- File names and paths, symbol table variables and values, expressions typed in GUI edit fields, and code snippets are written in `monospace`.

- In referring to example CAD files, the installation directory for the CAD tool is specified as `<rsoft_dir>`, and should be replaced with the correct value for your installation.

1

Introduction

This chapter explains the installation procedure for the RSoft CAD, a discussion of physics conventions used in this manual, information on running the program, and notes about getting the product updates and technical support.

1.A. Program Installation & System Requirements

The installation process is outlined in the Photonic Solutions Installation Guide. Please check the 'System Requirements' section of the relevant product on Synopsys' Photonic Solutions website (<https://www.synopsys.com/photonic-solutions/product-system-requirements.html>) for a list of the specific OS versions we currently support.

Note that once a product has been successfully installed, it may not function correctly after changes such as OS updates or other software/hardware modifications are made to the computer system. In such cases, we will attempt to resolve any issues for customers that have a current annual maintenance contract, but we do not guarantee success.

1.B. Physical Conventions

As with any branch of science, there are a number of concepts in the study of photonic devices for which there exist several different definitions exist in the literature. These are our conventions:

Units

The units used in the CAD are as follows:

- The standard unit of length is measured in microns [μm].
- The angular unit used is degrees.
- The units of time (for FullWAVE) are also in microns [μm], and therefore in units of cT , where c is the speed of light in a vacuum which is approximately $3\text{e}14 \mu\text{m/s}$.
- The units of imaginary refractive index are defined as:

$$n_{\text{imag}} = \frac{\gamma\lambda}{4\pi}$$

where λ is the wavelength and γ is the usual exponential loss coefficient defined such that the power decays as $e^{-\gamma z}$ and is given in units of μm^{-1} .

Polarization

Since the RSoft CAD is shared by many programs, the polarization conventions used can vary by tool. Please consult [Appendix G](#) as well as the individual simulation tool manuals for information specific to a particular tool.

1.C. Program Executables

The directory `<rsoft_dir>\bin\` contains many executable files. The following is a list of the executables used by the RSoft CAD, associated simulation engines, the WinPLOT graphing tool, and MOST, RSoft's scanning and optimization tool, and other utilities.

Product	Windows Name	Linux Name
The RSoft CAD tool	bcadw32.exe	xbcad
BeamPROP BPM simulation tool	bsimw32.exe	xbeam
BSDF Utilities	See Appendix E	See Appendix E
FullWAVE FDTD simulation tool	fullwave.exe	xfullwave
BandSOLVE simulation tool	bandsolve.exe	xbandsolve
Custom PDK Utility	smatgen.exe	xsmatgen
GratingMOD simulation tool	grmod.exe	xgrmod
DiffractionMOD RCWA simulation tool	dfmod.exe	xdfmod
FemSIM FEM simulation tool	femsim.exe	xfemsim

LaserMOD CAD tool	lasermod.exe	Xlasermod
LaserMOD simulation tool	lmsim.exe	xlmsim
LED Utility	led.exe	xled
ModePROP EME simulation tool	modeprop.exe	xmodeprop
Solar Cell Utility	scutil.exe	xscutil
Tapered Laser Utility	lmbpm.exe	xlmbpm
WinPLOT graphing tool	winplot.exe	xplot
MOST tool	rsmost.exe	xrsmost

Note that each simulation tool is licensed separately, and you will only be able to run those you have a valid license for.

These executables have different names under Windows and Linux though in most documentation the Windows names are used. Also, these programs can be run from the command line; more information on this can be found in [Section 10.D.1](#).

1.D. Example Files

The subdirectory `<rsoft_dir>\examples\` contains example device files. These files are sorted into subdirectories based on the simulation modules. These files have an extension `.ind` (named for the refractive index distribution which describes the device). Each examples directory has a `\Tutorial` subdirectory that contains the index files described in the Tutorials chapter(s) of that product's manual.

To open an example file, open the RSoft CAD, select **File/Open** from the CAD menu, and choose the desired index file. To start a simulation, click the **Perform Simulation** (green light) icon and press **OK** in the simulation parameters dialog.

These example and tutorial files are specific to each simulation engine; you can only perform a simulation if you are licensed for that particular simulation engine.

1.E. README Files

The latest product information can be found in the README files located in the `/readme` subdirectory of the installation directory. These files contain important last-minute information

about RSoft software that is not contained in other documentation, including new or improved features and options.

1.F. Product Support, Upgrades, and Resources

The software includes technical support, product updates, and other resources. Software sales and technical support worldwide is handled by Synopsys, except in those countries where we have local representatives whom you can contact directly.

Visit <https://www.synopsys.com/optical-solutions/support.html> for information about how to access training resources, knowledge articles, product downloads/updates, documentation, how to open support cases, and other resources. Many of these resources are in Synopsys'

SolvNetPlus portal (<https://solvnetplus.synopsys.com/>), eligible users can find information about how to create an account at the main support link above.

2

A Simple Tutorial

This chapter contains a brief tutorial that shows the basic usage of the RSoft CAD. It breaks the design process into nine steps in order to illustrate a sample workflow. In practice, some of these steps may be skipped or repeated several times to achieve a working design.

Even though this tutorial creates a simple fiber structure, the ideas and techniques introduced will be exactly the same for producing complex structures. While not necessary, it is recommended that you follow the steps in this tutorial at a computer while reading this chapter.

Step 1: Opening the CAD Window

To open the Rsoft CAD Environment:

- *Windows:* Use the Windows Start Menu.
- *Linux:* Enter `xbcad` at a command prompt.

The CAD program appears as in Fig. 2-1. There is a menubar at the top of the window, two toolbars with icons just below it, another toolbar along the left edge of the window, a tree-view toolbar along the right edge of the window, and a status line at the bottom of the window. These CAD menus and icons allow for standard editing operations as well as other common functions. Their use is completely documented in [Section 3.A](#).

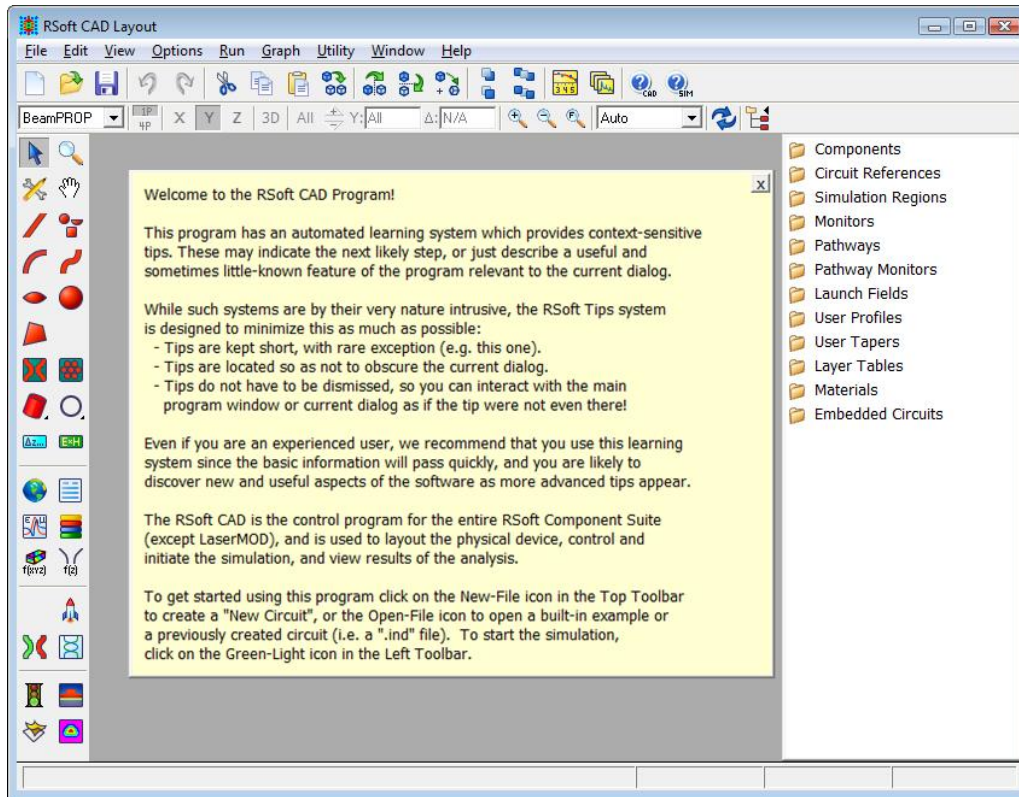


Figure 2-1: The RSoft CAD window, showing the menubars at the top and left, the tree-view toolbar at the right, and the status line at the bottom.

If this is your first time opening the RSoft CAD, a “Welcome to the RSoft CAD Program!” tip window will appear, as appears in Fig. 2-1. These tip windows are designed to convey relevant information about the software to both new and experienced users, and will occasionally pop-up as you use the software.

Step 2: Creating a New Design File

The RSoft CAD provides the key input for each of the RSoft simulation engines: the refractive index distribution of the problem to be modeled. This is determined by the geometry, arrangement, and optical properties of components that the user has placed. The RSoft CAD allows users to specify this information in a straightforward, user-friendly manner.

To create a new design file, click on the **New Circuit** icon (the leftmost icon on the top toolbar) or choose **File/New** from the menu. The startup dialog will appear as shown in Fig. 2-2 where basic information about the structure and simulation tool to be used.

After creating a new structure, changes can be made to these settings in the Global Settings dialog that can be accessed via the Globe icon in the left toolbar.

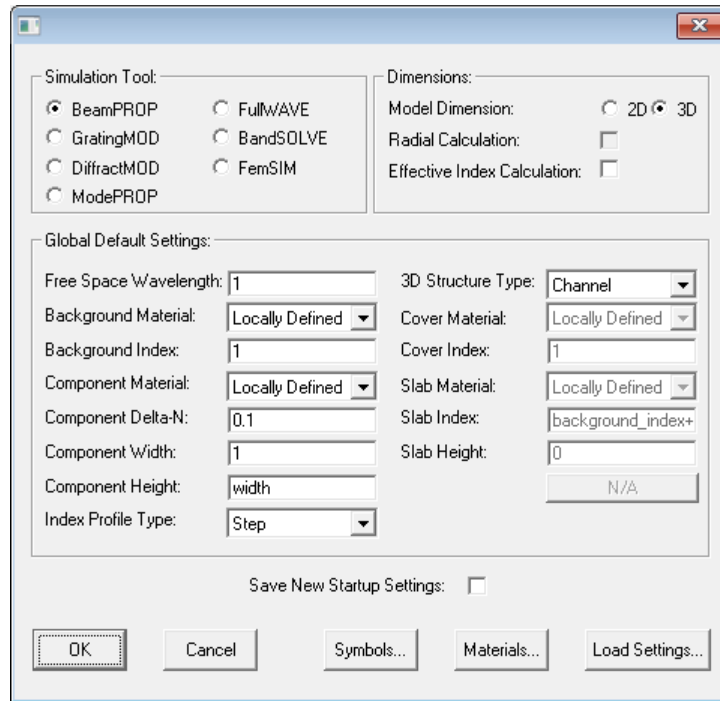


Figure 2-2: The startup window where basic information about the structure is set.

Note the following settings:

Option	Description
Model Dimension	The dimension of the structure.
Component Delta-N	The default index difference between a component and the Background Index
Background Index	The index of the background (white) region.
Simulation Tool	The simulation tool to be used. Since this tutorial will only look at the CAD, the tool chosen does not matter.
Load Settings	Loads a pre-defined list of settings, for example typical Silicon waveguide settings.

For this tutorial, all the default settings are acceptable except the **Model Dimension**. Change the **Model Dimension** to 3D and then click **OK** to continue to create the layout window within the main CAD window as shown in Fig. 2-3.

The view shown in Fig. 2-3 shows the multi-pane view. There are three panes, an XZ plane, YZ, XY plane, and 3D pane. Axes labels are shown in each pane. If this were a 2D structure, only the XZ pane would be shown. To view a single pane, click the **1P** button in the view toolbar (the second toolbar on the top). Click the **X**, **Y**, and **Z** buttons in view toolbar to switch between the views. To switch back to multi-pane view, click the **4P** button in the view toolbar.

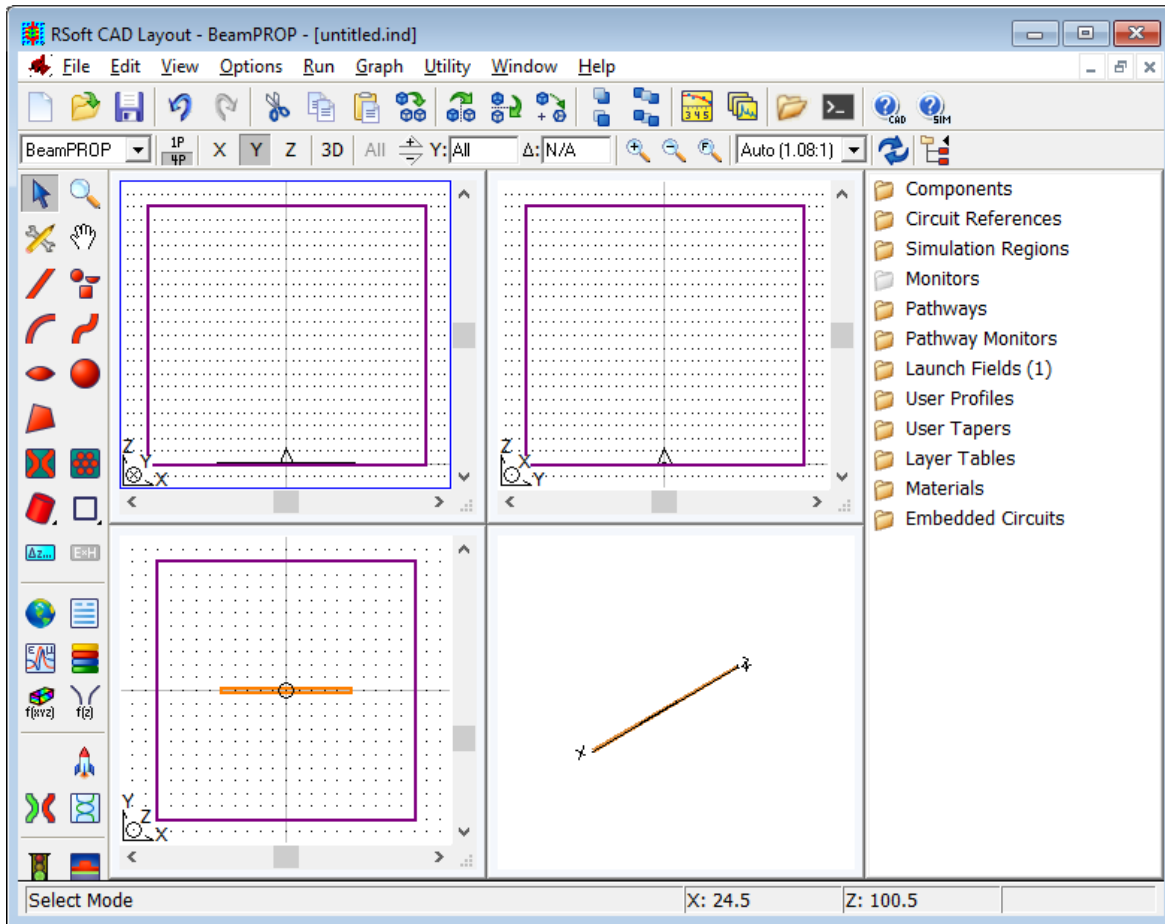


Figure 2-3: Illustration of a layout window where components are added to the circuit. The simulation domain (solid or dashed line) and launch field are also shown.

Finally, note that the mouse appears as a cross-hair, and the coordinate display in the status line indicates the mouse position in real coordinates [μm].

Step 3: Defining Variables

It is usually beneficial to create variables that will represent various aspects of the structure. As an example, we will create a variable to represent the length of a component. Click on the **Edit Symbols** icon in the left toolbar to open the Symbol Table Editor as shown in Fig. 2-4.

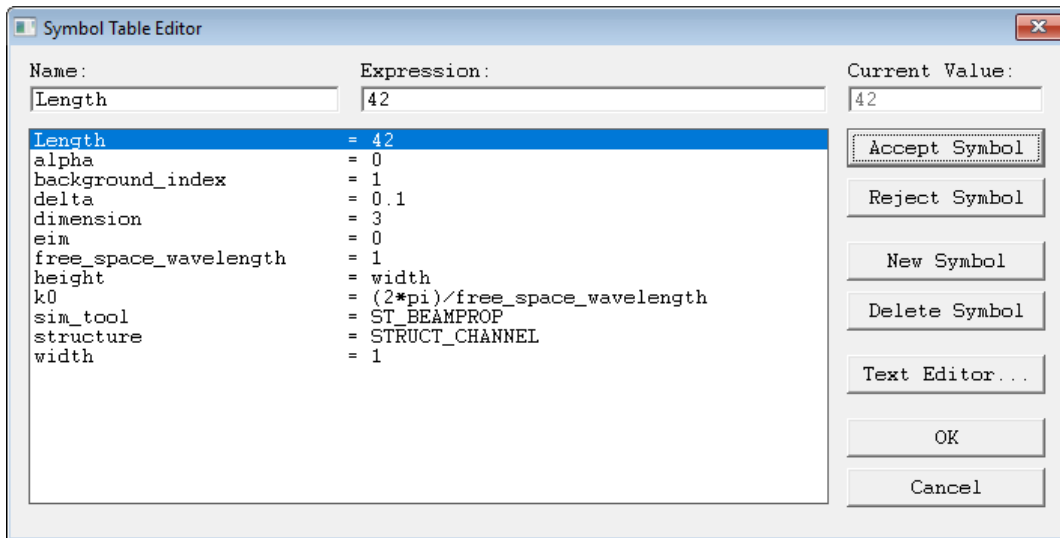


Figure 2-4: The Symbol Table Editor; Length has been defined with a value of 42.

The symbol table will have several symbols already defined that correspond to the settings made in the Startup window. We are going to define a symbol `Length`, and set it equal to 42: click **New Symbol**, enter the symbol **Name** (`Length`) and **Expression** (42), and then click **Accept Symbol**. The variable `Length` should now appear in the symbol table. This variable will be used in a later step. More information on the usage of the Symbol Table can be found in [Section 3.B](#).

Step 4: Drawing a Component

CAD Components (usually referred to as just ‘components’) are the basic building blocks from which a design is created. A wide variety of components can be used, including segments (rectangular, cylindrical, multilayer, diffused, etc.), tapered objects, lenses, user-defined polygons, spheres, cylinders, and completely user-defined objects. Any number and combination of these components, which have their own local, geometric and material parameters, can be embedded in a background material of a specific index to create the refractive index distribution of the entire structure.

This tutorial will create a simple rectangular segment along the Z axis. The type of component to be added to a design is selected by the Drawing Mode. For this example, the **Segment (In Plane)** mode will be used, and is the default selection on the left toolbar. By default, a 3D segment has a rectangular cross-section. This can be changed to other cross-sections as discussed in [Section 4.D](#).

Move the cross-hair cursor to the desired start of the fiber component ($X=0, Z=0$). After positioning the cursor, press and hold the left mouse button and move the cursor to the desired end of the fiber component as shown in Fig 2-4. As you move the cursor, the center line of the component is shown in blue and the status line coordinate display is continuously updated with

the coordinate of the end point. If a mistake is made when adding any component, use the **Undo** icon in the top toolbar.

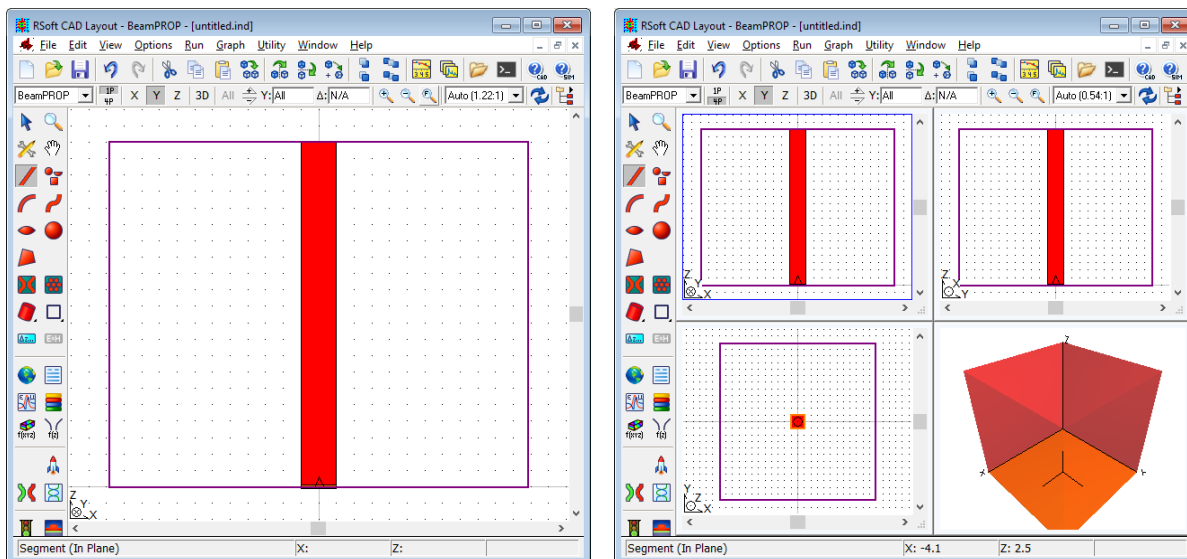


Figure 2-5: a) A fiber component as it appears in the XZ plane after drawing, and b) Multi-pane view clearly showing the fiber component. Multi-pane view is not available for 2D structures. Here, the tree view has been hidden by clicking on the **Toggle Component Tree** button.

You may notice that the CAD display does not appear as shown in Figure 2-5. This is most likely due to a different display aspect ratio. There is a pulldown menu on the view toolbar to control the aspect ratio, set it to 1:1. You can switch between the single pane view shown in Fig 2-5a and four pane view in Fig 2.5b by clicking the **1P** or **4P** button in the view toolbar. Use the pulldown menu on the view toolbar to choose an appropriate aspect ratio for each view.

In general, the next step would be to continue to add elements to the CAD layout to create the desired device. This process is described further in [Section 3.D](#). For this example, there are no more elements to add.

In addition to viewing all components in the CAD at once, it is also possible to view only the components that lie within a specific cut plane by using the “Cut Mode” option. See [Section 3.E.1](#) for details.

Step 5: Moving a Component

There are two methods to move a component within the CAD:

- *Via the Mouse*

Components can be selected with the mouse and then dragged to a new position. This allows for the quick movement of a waveguide segment.

- *Via the Component Properties window*

The exact coordinates of a component can be set in the Component Properties dialog box allowing for precise control of component position. The use of this box will be discussed in the next step.

Step 6: Set Component Properties Exactly

It is desirable to set component properties exactly in order to have precise control over the component properties such as refractive index, position, and size. This is done via the Component Properties dialog shown in Fig. 2-7 which can be opened by right-clicking on a component in the CAD window, or in the component's icon in the tree view.

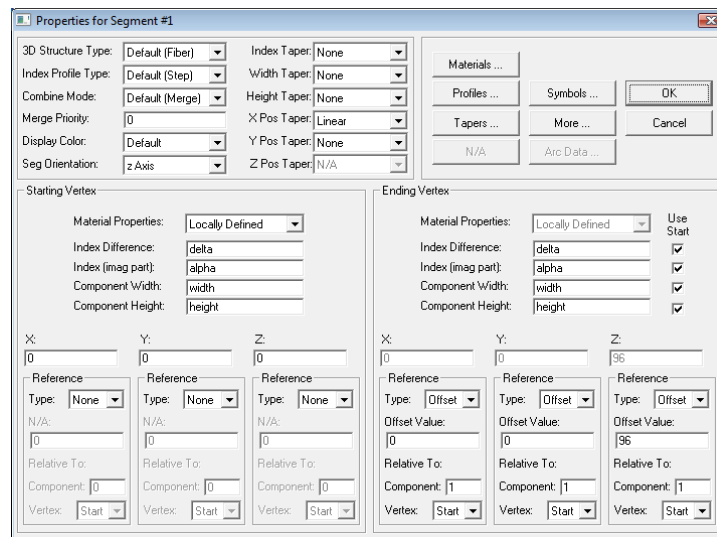


Figure 2-7: The Component Properties dialog for the fiber component.

The RSoft CAD is an object oriented design environment so each component has a different Component Properties dialog, and therefore a different set of parameters. A detailed description of this dialog can be found in [Chapter 4](#).

The current value of any field in the software, including those in the Component Properties dialog shown in Fig. 2-7, can be found by holding the **CTRL** key and double-clicking on the field.

Setting Position

The length of the fiber component was set using the mouse in Step 4. It is possible to control this length exactly in the Component Properties dialog. This will be done using the variable `Length` that has been previously defined and the concept of “references”. In order to set the length of this component, the ending vertex will be offset by the variable `Length` from the starting vertex resulting in a fiber with a length of 42 μm . Set the **Reference Type** of the Z coordinate of the Ending Vertex to Offset, the **Offset Value** to `Length`, and relative to the Starting Vertex of component 1. Thus, the Z coordinate of the ending vertex of this component, or component 1, should be computed as an offset from the starting vertex of this component, or component 1. This offset will therefore equal the length of the segment.

Setting Refractive Index & Other Material Properties

The refractive index of this component can be set in one of two ways:

- *Locally Defined:*

Set the **Index Difference** field of the starting vertex (the ending vertex is equal to the starting vertex by default). This value is defined relative to the **Background Index** for the structure set in the Startup window in Step 2. For this example, the **Background Index** is 1 and the **Index Difference** is 0.1, so the real index of the structure will be 1.1. See [Section 4.B.2](#) for additional details.

- *Via a Material*

The Material Editor can be used to define materials that can be simple dielectrics, but can also have material properties such as anisotropy, non-linearity, and dispersive effects. Note that these material properties are not enabled/appropriate for all simulation tools. More information about the specification of materials can be found in [Chapter 8](#).

Step 7: Creating a Tapered Component

Some components such as the fiber in this tutorial have two vertices and allow for properties such as index, width, height, and position to be tapered, or varied, along the length of the component.

Right click on the fiber component to open its Component Properties dialog. Change the ending vertex **Component Width**, which corresponds to the fiber diameter along X, to `width*4`. The built-in variable `width` was set in the Startup window in Step 2 and can be changed via the Global Settings window. Click **OK** and the fiber will appear tapered as in Fig. 2-8: one vertex will have a circular cross-section, the other elliptical.

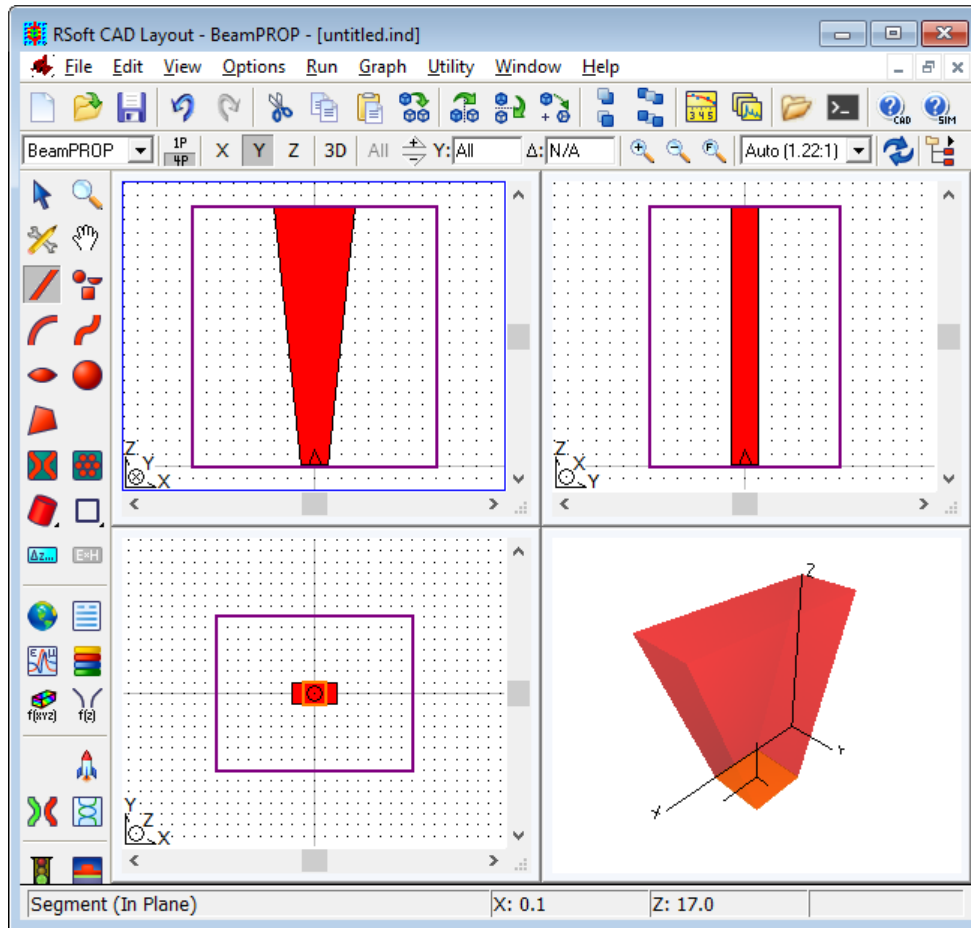


Figure 2-8: The tapered fiber as seen in the CAD window.

The width taper function, or how the width varies between the two vertices, is linear by default. It can be changed via the **Width Taper** setting in the Component Properties dialog. See [Section 4.E](#).

The view shown in Fig. 2-8 shows the entire structure; it is also possible to view ‘slices’ of each cross-section at a particular position. This is most readily illustrated with the XY view. On the view toolbar, select **Z** and click the +/- buttons to begin stepping through the structure: Both the absolute Z position and the step along Z between slices are shown on the view toolbar, and can be directly set by the user. As the slices step through the structure, the cross-section will change from circular to elliptical (and/or back). To view the entire structure again, click the **All** button on the view toolbar.

Step 8: Verifying Structure Layout

It is important to simulate the desired index distribution! Performing a simulation with the wrong index distribution will result in erroneous results. There are two main methods to verify the structure layout:

- *Via the CAD*

Various aspects of the geometry can quickly be seen using the multi-pane view and cut feature described briefly above. See [Section 3.E](#) for more details.

- *By Displaying the Index Profile*

Index profiles show the index distribution and can help verify that both the geometry and material properties have been set correctly. To compute an index profile, click the **Display Material Profile** icon in the left toolbar to open the dialog shown in Fig. 2-9.

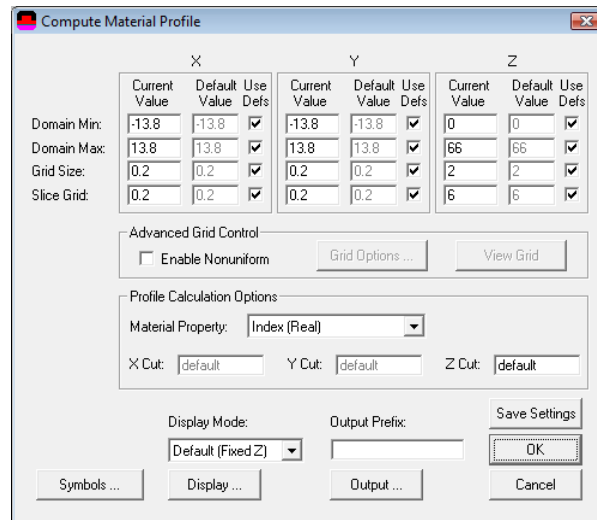


Figure 2-9: The Compute Index Profile dialog.

Pressing **OK** in this dialog will display the index profile at the Z **Domain Min** value. More information on the display of material profiles such as the refractive index can be found in [Section 3.G](#).

Step 9: Simulation & Advanced CAD Features

Once the structure has been created, the next step is to use the selected simulation tool to perform a simulation. See the simulation tool documentation for more information.

This tutorial illustrates basic CAD layout features and a sample workflow for using the RSoft CAD. The rest of this manual explains the simple concepts introduced in this tutorial in more detail and also introduces advanced concepts.

3

Basic CAD Usage

This chapter should be read by all users and covers the CAD window, the symbol table, creating new structures, creating/editing components, viewing the structure, and calculating material profiles. This chapter does not cover component types or component properties. See [Chapter 4](#) for segment components and [Chapter 5](#) for other component types.

3.A. The CAD Window

This section describes the menu, toolbars, and status bar in the CAD (Fig. 2-1).

3.A.1. Menubar

Some options can be accessed via the toolbars; keyboard shortcuts are shown in parenthesis if available.

Under **F**ile:

New... (Ctrl-N)
Create new design
file: [Section 3.B](#)

Open... (Ctrl-O)
Open existing

Close
Close current
design file

Close All
Close all

Import Circuit
Import mask file: [Section 6.E](#)

Export Circuit

design file

Save (Ctrl-S)

Save current
design file

Save As...

Save current design
file with a different name

design files

Print (Ctrl-P)

Print current
design file

Printer Setup

Set printer related
parameters

Export mask file: [Section 6.E](#)

Exit

Close RSoft CAD

Under Edit:

Undo (Ctrl-Z)

Reverse last
change

Redo (Ctrl-Y)

Redo last
change

Cut (Ctrl-X)

Move selected components
to the clipboard

Copy (Ctrl-C)

Copy selected components
to the clipboard

Paste (Ctrl-V)

Paste clipboard contents
into design file

Delete (Del)

Delete selected
components

Duplicate

Duplicate
selected components

Select All

Selects all
components

Select/Edit Components...

Select or edit a component
based on its number or name

Edit Default Tags...

Edit default component tags:
[Section 6.K.4](#)

Edit Hierarchy Exports...

Edit hierarchy exports: [Section 6.F.5](#)

Change Order

Change order in which
components are drawn.

Flip Horizontal

Flip selected
components horizontally

Flip Vertical

Flip selected
components vertically

Rotate...

Rotate selected
components

Convert to Polygons

Convert selected components
to polygons

Flatten Circuit

Flatten hierarchy
into one level

Tables

Access various setting dialogs

Under View:

These features are discussed in [Section 3.E](#).

In

Zoom in
CAD window

Out

Last

Revert viewing window
to the last setting

Set View

ReGrid

Recalculate
the grid

Clear Log

Zoom out
CAD window

Full

Set viewing domain so
the entire structure is visible

Directly set the
viewing window

Redraw

Redraw all components

Clear log file, see [Section 6.P](#)

Open Log

Open log file, see [Section 6.P](#)

Under Options:

Preferences

Open CAD preferences
dialog: [Section 3.D.4](#)

Index Generation

Open Global Index Generation
Options dialog: [Section 6.D](#)

Import Symbols

Import symbol table from
another file: [Section 6.I](#)

Global Settings

Open Global Settings
dialog: [Section 3.B](#)

Warnings

Enable/disable
warnings: [Section 6.M](#)

Insert

Insert components
by type

Under Run:

Go

Open Simulation Parameters
dialog to start simulation

Compute Modes

Compute transverse
modes: [Chapter 11.A](#)

Run Script

Run an RSoft
script.

Display Material Profile

Open Display Material
Profile dialog: [Section 3.G](#)

MOST Optimizer/Scanner

Open MOST optimization
and scanning utility

Cluster Settings

Set FullWAVE
cluster parameters

Multi-Threading Settings

Set Multi-Threading Options:
See Simulation Manuals

GPU Settings

Set GPU Settings: see
Simulation Manuals

Under Graph:

View Single Plot

Open a single plot
in WinPLOT

View S-Matrix File

Open an RSoft Matrix File

Close All Browsers

Close all
DataBROWSER windows

Launch DataBROWSER

Open DataBROWSER

Close All

Close all WinPLOT and
DataBROWSER windows

View BSDF File

Open an RSoft BSDF

Close All Plots

Close all

file in the BSDF Viewer

WinPLOT windows

Under Utility

Array Layout

Create periodic structures: [Chapter 7](#)

Multi-Physics

RSoft's Multi-Physics Utility

Q-Finder

RSoft's Q-Finder Utility: FullWAVE manual

WDM Router Layout...

Create WDM Router or AWG: AWG Utility Manual

Solar-Cell

RSoft's Solar Cell Utility: Solar Cell Utility manual

Raytracing-Converters

Convert field data to and from common ray-tracing formats

WDM Router Layout...

Simulate WDM Router or AWG: AWG Utility Manual

LED

RSoft's LED Utility: LED Utility manual

TMM-Mode-Solver

TmmSIM mode solver: [Section 11.B](#)

GratingMOD Grating Layout

Create simple grating structures for GratingMOD

Tapered-Laser

RSoft's Tapered-Laser Utility: Tapered-Laser Utility manual

Under Window:

Tile

Tile all open design files in the CAD window

Arrange Icons

Arrange minimized windows within the CAD window

Close All

Close all open file.

Cascade

Cascade all open design files in the CAD window

Close

Close the current file

Under Help

This menu option provides online help for all RSoft products that share the CAD interface.

3.A.2. Toolbars

The CAD has several toolbars:

The Top Toolbar:



New Circuit

Create new design file: [Section 3.B](#)



Flip Section Vertically

Flips selected components vertically

**Open Circuit**

Open existing design file

**Save Circuit**

Save current design file

**Undo Last Change**

Reverse last change made to the design file

**Redo Last Change**

Redo last change made to the design file

**Cut Selection**

Cut selected components

**Copy Selection**

Copy selected components

**Paste Clipboard Contents**

Paste clipboard contents

**Duplicate Selection**

Duplicate selected components

**Flip Section Horizontally**

Flip selected components horizontally

**Rotate Selection**

Rotate selected components

**Send Forward/Send Backward**

Change component order of selected components

**Send to Front/Send to Back**

Change component order of selected components

**View Graphs**

Open plot file in WinPLOT.

**Launch DataBROWSER**

Open DataBROWSER

Open Folder In Working Directory

Open a file browser in the working directory

Open Terminal in Working Directory

Open a terminal (DOS) window in working directory

**Help for CAD program**

Open online help for RSoft CAD

**Help for Active Sim Program**

Open online help for current simulation tool

The View Toolbar

**Choose Simulation Tool**

Select active simulation tool

**Single-Pane/Four-Pane**

Switch between single-pane and multi-pane view

**Cut View Controls**

Display specific slice of the structure

**Zoom In**

Zoom in CAD window

**Edit X Axis View**

Display YZ plane

**Edit Y Axis View**

Display XZ plane

**Edit Z Axis View**

Display XY plane

**Show 3D View**

Display 3D view

**Toggle Component Tree**

Hide/Reveal tree view toolbar

**Zoom Out**

Zoom out
CAD window

**Zoom Full**

Set viewing domain of the
CAD so entire structure is visible

**Aspect Ratio**

Set aspect ratio for
the selected cut view

**Redraw Circuit**

Redraw components

The Side Toolbar

**Select Mode**

Select components:
[Section 3.F.1](#)

**Drawing Tool Options**

Set drawing preferences:
[Section 3.D.3](#)

**Segment (In Plane)**

Draw straight in-plane segment
component: [Section 3.D.1](#)

**Arc**

Draw arc segment
component: [Section 3.D.1](#)

**Lens**

Draw lens component:
[Section 3.D.1](#)

**Polygon**

Draw polygon objects:
[Section 3.D.1](#)

**Zoom Mode**

Zoom in/out:
[Section 3.E.3](#)

**Pan Mode**

Pans CAD display:
[Section 3.E.4](#)

**Segment (Out of Plane)**

Draw straight out-of-plane segment
component: [Section 3.D.1](#)

**S-Bend**

Draw S-bend segment
component: [Section 3.D.1](#)

**Circle**

Draw circle/cylinder/sphere:
[Section 3.D.1](#)

	Circuit Reference Draw circuit references: Section 6.F		Dynamic Array Draw a dynamic arrays: Section 6.G
	Extended Segments Nested menu to draw extended segments, see section below for all sub-options		Structure Preference Nested menu for structure type preference, see section below for all sub-options
	Simulation Region Draw BeamPROP simulation region: BeamPROP		Monitor Draw monitor: FullWAVE, DiffractMOD, ModePROP, BeamPROP
	Edit Global Settings Open Global Settings dialog: Section 3.B		Edit Symbols Open Symbol Table: Section 3.C
	Edit Materials Open Material Editor: Chapter 8		Edit Layers Open Layer Table Editor: Section 4.C.5
	Edit User Profiles Open User Profile Editor: Section 6.A		Edit User Tapers Open User Taper Editor: Section 6.B
	Edit Pathways Pathways are used to define launch positions and for analysis: Section 6.H		Edit Launch Field Open Launch Parameters dialog
	Edit Global Settings Open Simulation Parameters dialog to start simulation		Edit Pathway Monitors Create and modify pathway monitors: BeamPROP and FemSIM
	Display Material Profile Open the Display Material Profile dialog: Section 3.G		Launch MOST Open the MOST optimization and scanning utility
			Compute Modes Compute transverse modes: Chapter 11.A

Extended Segments:

Extended Segments are discussed in more detail in [Section 4.G](#). Though illustrated with a circular cross-section, these segments can have any **3D Structure Type**.

**Straight Extended Segment**

Draw straight
extended segment

**Half-Taper Extended Segment**

Draw half-taper
extended segment

**Full-Taper Extended Segment**

Draw full-taper
extended segment

**Arc Extended Segment**

Draw arc
extended segment

**Toroidal Extended Segment**

Draw toroidal
extended segment

**Helical Extended Segment**

Draw helical
extended segment

Structure Preference:

The structure preference is discussed in [Section 3.D.3](#) and sets the **3D Structure Type** of segments to be drawn. The default choices indicate that the setting in the Global Settings dialog should be used.

**Fiber Structure Preference**

Fiber structure type:
[Section 4.D.1](#)

**Channel Structure Preference**

Channel structure type:
[Section 4.D.2](#)

**Diffused Structure Preference**

Diffused structure type:
[Section 4.D.3](#)

**Rib/Ridge Structure Preference**

Rib/ridge structure type:
[Section 4.D.4](#)

**Multilayer Structure Preference**

Multilayer structure type:
[Section 4.D.5](#)

**Polygon Structure Preference**

Polygon structure type:
[Section 4.D.6](#)

**Triangle Structure Preference**

Polygon structure type
with 3 sides: [Section 4.D.6](#)

**Pentagon Structure Preference**

Polygon structure type
with 5 sides: [Section 4.D.6](#)

**Hexagon Structure Preference**

Polygon structure type
with 6 sides: [Section 4.D.6](#)

**Octagon Structure Preference**

Polygon structure type
with 8 sides: [Section 4.D.6](#)

**N-sided Structure Preference**

Polygon structure type
with N sides: [Section 4.D.6](#)

**Polygon Datafile Structure Preference**

Polygon structure type defined
by a data file: [Section 4.D.6](#)

3.A.3. Status Bar

The status bar is located on the bottom of the CAD window, and provides information about the current mode of the CAD interface as well as the current coordinates of the mouse.

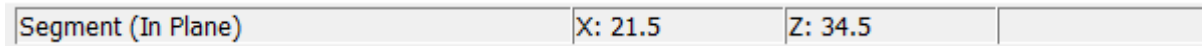


Figure 3-1: The status bar at the bottom of the CAD window.

3.A.4. Tree View Toolbar

The tree view toolbar is located on the right side of the CAD window, and provides a hierarchical view of the elements present in the CAD window. Double-clicking on any branch folder (Components, Monitors, Launch Fields, etc.) in the tree-view will list all elements of that type that are currently present in the CAD window.

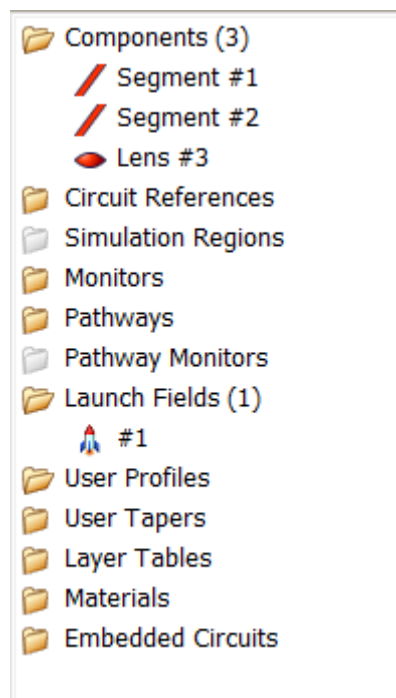


Figure 3-2: Tree view for a simple structure consisting of 2 Segments, a Lens, and a Launch Field.

Any element in the CAD can be easily modified by right-clicking on it in the tree view toolbar (for example, Segment #1 or Launch #1 in Figure 3-2). New elements of any type can be created by right-clicking on the appropriate branch folder in the tree view. A branch folder will be greyed out if it is not relevant to the simulation tool being used.

If desired, the tree-view toolbar can be hidden by clicking on the **Toggle Component Tree** icon in the view toolbar.

The Component Filters folder can be used to create custom list(s) of components based on user-defined searches/filters. For example, a filter can be made to list components that have a material of 'Si', or to list all monitors, or all components with a priority of 4. The resulting objects can also be hidden in the drawing windows, so for example, you can hide all components that have a material of 'Si', etc. See [Section 6.O](#) for more details.

Multiple elements in the CAD can be selected by right-clicking on them in the tree view toolbar and holding the `Ctrl` and/or `Shift` keys down. Holding down the `Ctrl` key adds them one by one whereas holding down the `Shift` key selects a range. In the case of multiple selections, the last element is highlighted in the tree view toolbar and the other elements in the selection are marked by an * in front of their element name. Operations can be performed on the elements in the multiple selection list through right-clicking the highlighted element. One such operation is adding these elements to a Group which will then appear as a named Group under the Component Filters. The resulting Group can now perform similar operations like the Component Filters, see [Section 6.O](#) for more details

Project materials are shown in the tree. If material colors are used (see [Section 8.A.6](#)), the material color is shown in the tree.

3.A.5. RSoft Tips

The RSoft CAD has an automatic learning system which provides context-sensitive tips. These tip windows are designed to convey relevant information about the software to both new and experienced users, and will occasionally pop-up as you use the software.

While such systems are by their very nature intrusive, the RSoft Tips system is designed to minimize this as much as possible. Tips are kept short with rare exceptions, and are located as so not to obscure the current dialog. Tips also do not have to be dismissed; you can interact with the main program window or current dialog as if the tip were not even there!

Even if you are an experienced user, we recommend that you use the tip learning system since the basic information will pass quickly, and you are likely to discover new and useful aspects of the software as more advanced tips appear.

3.B. Creating a New Design File

Click the **New Circuit** icon in the top toolbar or select `File/New` from the CAD menu to create a new design file. The startup dialog appears where basic information about the structure is specified. This information can be changed at any time via the Global Settings dialog. Also, all options may not be relevant for all simulation tools or simulation types.

The **Load Settings** button can be used to pre-load a set of simulation settings, for example typical Silicon wire settings. It is recommended to load the closest settings for your specific case and then modify as needed. New settings packages can also be saved, see below.

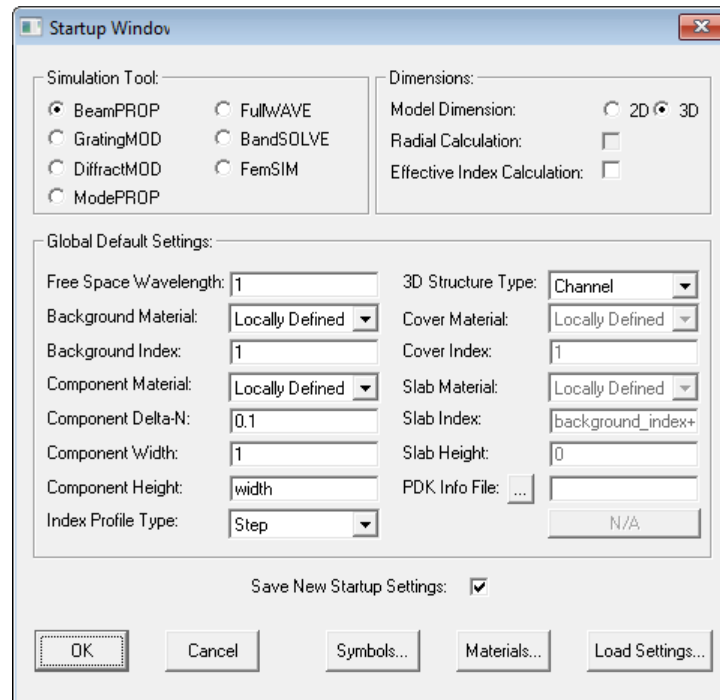


Figure 3-3: The Startup window.

Simulation Tool and Dimensions:

Simulation Tool

All of these products share the RSoft CAD interface and are licensed separately. The simulation tool can also be chosen from the view toolbar in the CAD.

Model Dimension

The dimension of the structure which can be either 2D or 3D. A 2D structure is in the XZ plane. Note that in some cases, 3D structures can be automatically simulated in 3D using the **Effective Index Calculation** option described below.

Radial Calculation

Performs a 2D radial simulation of structures that have azimuthal symmetry; this option is not enabled for all simulation tools and is described in relevant simulation tool manuals.

Effective Index Calculation (2.5D)

The Effective Index Method (EIM) can be used to greatly reduce the computation time and memory requirements of a simulation by converting a 3D structure into an approximate 2D structure. This is sometimes called ‘2.5D’. This option only affects the simulation; the definition of the original 3D structure is unaltered allowing this feature to be safely turned on and off without changing the defined structure. See [Section 6.L](#) for details including supported simulation tools and other requirements.

Global Default Settings:

These settings control the global default settings for components.

Free Space Wavelength

The wavelength [μm] of light used for simulation. It is measured in free space, i.e. $\lambda=c/f$ where c is the velocity of light in vacuum and f is the frequency of the light.

For most simulation tools, the wavelength is an input. Some tools solve for wavelength(s) as a part of the simulation (BandSOLVE for example). For such tools this setting will, in most cases, have no effect. Exceptions include using dispersive materials where the material properties are defined as a function of the wavelength or when variables are defined as a function of the built-in symbol for this option `free_space_wavelength`.

Background Index or Background Material

The real refractive index of the background material in which components are embedded. A real value can be given via the **Background Index**, or a material can be assigned via a **Background Material**. For 3D simulations, the definition of this property depends on the default **3D Structure Type** as described in [Section 4.D](#). See [Chapter 8](#) for more information about materials.

Component Material and Component Delta-N

These options set the default material properties of a component:

- The **Component Material** is the default material for components. If set, it overrides the **Component Delta-N** setting below and any components with the default material properties will use the defined **Component Material**.
- **Component Delta-N** sets the default difference between the real refractive index of a component and the background index. The default imaginary refractive index (loss) of a component can be specified through the variable `alpha` in the symbol table.

Individual components can be assigned different real and/or imaginary values as described in [Section 4.B](#).

Component Width and Component Height

The default width and height of a component along the X and Y axes respectively. Individual components can also have a different width and/or height described in [Section 4.B](#). The height is only used for 3D simulations and is set equal to the width by default.

Index Profile Type

This option determines the default functional form of the refractive index profile along the transverse direction of a component. Individual components can have different profiles. See [Section 4.E](#).

3D Structure Type

The default 3D structure type. See [Section 4.D](#).

Cover Index, Cover Material, Slab Index, Slab Material, and Slab Height

These options set the refractive index when **3D Structure Type** is set to Rib/Ridge or Multilayer. See [Section 4.D.4](#) and [Section 4.D.5](#) for more details.

PDK Info file

Selects the PDK Info file to use to set relevant settings set via the `pdkinfo()` function. See [Section C.F.10](#), this option can be automatically set for new files using the Preference described in [Section 6.J](#). PDK info files can also be selected from one or more RSoft CAD Info folders, see the Custom PDK manual for more information on using RSoft CAD Info folders.

Symbols, Materials, Loading Settings:

The buttons in the bottom of the Startup Window dialog can be used to set Symbols, select Materials, or Load pre-defined settings.

Settings Files:

Settings files are `.ind` file that contain convenient starting points for a structure/simulation. There are built-in settings files, users can create their own setting files, or can be loaded from one or more RSoft CAD Info folders. See the Custom PDK Utility manual for more information on using Settings files and RSoft CAD Info folders.

3.C. The Symbol Table & Variables

The Symbol Table is a powerful concept in the RSoft CAD and is easily the most-used feature. As such, a proper understanding of the concepts in this section is critical to using the CAD and associated simulation tools effectively.

3.C.1. Why use the Symbol Table & Variables?

The Symbol Table has two main uses:

- *Modify existing built-in variables*

Every option in the RSoft CAD and simulation tools is controlled by a built-in variable in the symbol table. While most options can be set in the graphical interface, advanced users can modify these variables in the symbol table to access these features. This concept is the basis for RSoft's convenient scripting capabilities: see [Chapter 10](#) for more details.

- *Create user-defined variables*

User-defined variables can be used to set various aspects of a design. These variables can be a simple numerical value, or an arbitrary mathematical function of other variables. This is an extremely powerful and unique feature of the RSoft CAD which allows for the creation of parametric designs, facilitates parameter scanning and optimization, and simplifies simulation scripting.

It is recommended that the names user-defined variables start with a capital letter. Variables that start with a capital letter are displayed on top of the variable list making them easier to find.

3.C.2. Using the Symbol Table

To open the Symbol Table, click the **Edit Symbols** button in the left toolbar or select Edit/Tables/Edit-Symbols... from the CAD menu.

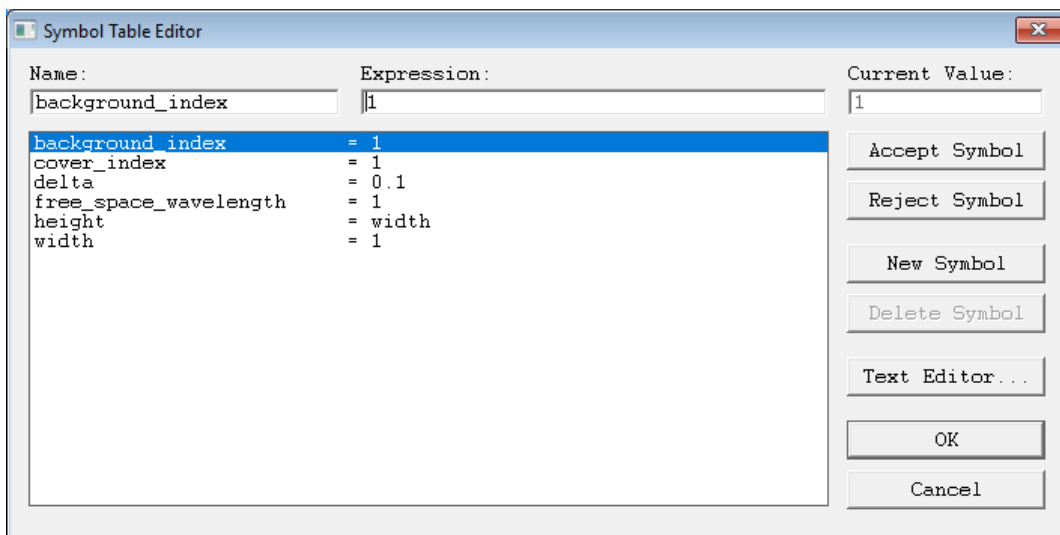


Figure 3-4: The Symbol Table Editor dialog which allows access to both built-in and user-defined symbols.

Most of the variables shown in Fig. 3-4 correspond to the settings shown in the startup dialog described in [Section 3.B](#). Altering any of the values will immediately affect the corresponding parameter in the design file and is equivalent to changing the parameter in the Global Settings dialog.

Adding a New Variable

To add a new variable to the Symbol Table, simply click **New Symbol**, type the desired variable name in the **Name** field, enter the value in the **Expression** field, and click **Accept Symbol**. To reject a variable without saving it to the variable list, click **Reject Symbol** instead of **Accept Symbol**.

Modifying an Existing Variable

To modify an existing variable, select the desired variable, change the value in the **Expression** field, and then click **Accept Symbol**. To reject a variable without saving the modification, click **Reject Symbol** instead of **Accept Symbol**.

Removing a Variable

To remove a variable from the symbol table, select the variable and click **Delete Symbol**.

An Example

As an example, open the file `<rsoft_dir>\examples\beamprop\branch.ind`. Open the symbol table and note that the variable `width` is set to 4 indicating that the default width of components in this design is 4 μm . Change the value of this variable to 10 and click **OK** to return to the layout window. The structure will have adjusted accordingly.

Additional Notes

- Variables names are case-sensitive.
- Variables can be defined as an analytic expression involving any other variable in the symbol table. See `k0` shown in Fig. 3-4. [Appendix C](#) documents the syntax and functions that can be used.
- Variables can be edited via a text editor using the **Text Editor** button. Any changes here will be automatically reloaded back into the Symbol Table when the text file is closed. A default text editor is used, set the environment variable `RSOFT_EDITOR` to the path of the text editor of your choice.
- Numerous built-in variables control all CAD and simulation options. A list of built-in variables that correspond to CAD options can be found in [Appendix D](#); built-in variables that correspond to simulation tool settings can be found in similar appendices in each simulation tool manual.

- Built-in variables that do not appear in the symbol table are internally set to their default values. Do not use the name of a built-in variable as a user-defined variable to avoid conflict. The easiest way to avoid this problem is to start the name of any user-defined symbol with a capital letter since the names of all built-in symbols are entirely lower case.
- Variables can be used in place of filenames where applicable in the graphical interface. To use this feature, create the variable and set its value to the name of the data file. Then, enter `$name` in the graphical interface where `name` is the name of the variable.

3.D. Drawing Components

Drawing components in the CAD is very simple. Most geometric shapes (e.g. tapered or curved structures, periodically varying structures, etc.) can be produced from variants these components or by overlaying more than one component.

3.D.1. Drawing Modes

The CAD has several drawing modes that control the type of components to be added to the design. They can be selected through the left CAD toolbar and can be broadly categorized by the number of vertices (end points):

Two-Vertex Components:

All two-vertex components in the CAD are generalized forms of the same component, the generalized segment. However, several drawing modes are provided to simplify the creation of these different types:

See [Chapter 4](#) for a complete description of the segment component type. Also, one type of segment can be changed into another by modifying its settings.

Segment (In Plane), Segment (Out of Plane), and Extended Segments

These drawing modes produced segments whose axis follows either a straight or curved line in the drawing plane or perpendicular to the drawing plane. Its properties can be changed so that it lies in any plane after it is drawn. Extended segments can be arbitrarily curved.

Arc and S-Bend

This drawing mode produces curved segments whose axis follows either a circular arc or an s-bend shape. The exact shape can be set using the arc properties as described in [Section 6.B.3](#). Note that custom tapers can be used as well to define custom arc shapes.

One-Vertex Components:

One-vertex components that can be added to a design file are:

See [Chapter 5](#) for a complete description of these component types.

Lens and Circle

These drawing modes produce lens, cylindrical, or spherical components.

Polygon

This drawing mode produces polygon components defined by a series of points in the XZ plane.

Circuit Reference

This drawing mode produces circuit reference components.

Simulation Region

This drawing mode produces simulation region components and is only available for BeamPROP.

Monitor

This drawing mode produces monitor components and is only available for FullWAVE, DiffractMOD, ModePROP, and BeamPROP.

3.D.2. Placing Components in a Design File

Components are drawn by first selecting the desired drawing mode and then using the mouse in the layout window. One-vertex components are added by simply clicking at the desired location of the vertex; two-vertex components are added by rubberbanding. Rubberbanding involves left-click-dragging the mouse to draw the extent of the component. Please note:

- The center line of the component is shown while rubberbanding.
- The drawing process can be canceled by right-clicking or pressing `ESC`.
- Alternatively you can first click without moving the mouse to define the starting point, move the mouse, and click again to define the ending point.
- The exact coordinates (as well as other component properties) can be directly set in the Component Properties dialog discussed in [Chapter 4](#).

Snap Modes

Snap modes provide simple ways to align or connect components:

- *Snap To Drawing Grids*

The layout window maintains both a fine and a coarse grid. Drawing coordinates can snap to either of these grids reducing required hand-eye coordination: the fine grid is used by default, and the coarse grid can be used by holding `Shift`. Holding `Ctrl` disables snapping.

- *Snap To Components*

When drawing components a vertex will automatically snap to the vertex of another component if the two vertices are near enough. This is extremely helpful when drawing components that need to fit together to form a larger structure. The tolerance for this can be set in the CAD preferences. This relationship is preserved as a reference between the components by the CAD. When one component snaps to another the coordinates are not simply set equal but is logically attached to that component. When a component is moved, any attached components move with it. Even if components aren't drawn using this mode they can be logically attached. See [Section 4.B.1](#).

3.D.3. Drawing Preferences

The initial properties of a component are controlled by the drawing preferences. Click the **Drawing Preferences** icon in the left toolbar to edit these preferences. By default, the values of these properties are set by the user-defined values set in the Startup window and the CAD defaults. The **3D Structure Type** drawing preference can also be changed in the left CAD toolbar.

It is important to understand that role of the Drawing Preferences: changing the Drawing Preferences will not affect any existing components, only components that will be drawn. Changing the options in the Global Settings will change all components that have properties based on those options.

3.D.4. Try It!

Many techniques for adding components using the mouse have been described in this section. It is best to experiment with these modes in order to better understand their operation.

3.E. Viewing Components

The CAD has several options that control how the components in a design file are displayed. This includes single-pane and multi-pane mode, full and cut mode, zooming, panning, and the display aspect ratio.

3.E.1. Single-Pane Mode

Single-Pane Mode displays a single view of the structure in the layout window and can be accessed via the **1P** button in the view toolbar. The **X**, **Y**, and **Z** buttons in the view toolbar set the axis which the structure should be viewed along. In each view, the axes are clearly labeled in the lower left corner. The **Y** (XZ) view is always used for 2D layouts.

Full & Cut Views:

Each axis view has two modes, Full Mode and Cut Mode. Full Mode displays all the components at once, regardless of their position, and cut mode displays all the components that lie within a specific cut plane. This allows, for example, only components that lie in the Y=100 plane to be shown while in the Y axis mode (viewing the XZ plane).

By default, Full Mode is used and all components will be shown. To enter Cut Mode, click the +/- buttons in the view toolbar. The current cut position is displayed, as well as the Cut Step Size. The user can directly set both these fields. Cut mode is only available in 3D.

3.E.2. Multi-Pane Mode (3D Only)

Multi-Pane mode displays the X, Y, and Z axis views as well as a 3D view simultaneously. It can be accessed via the **4P** button in the view toolbar. The X, Y, and Z axis views can be used in exactly the same way as in single-pane mode. Each view maintains its own viewing window. The currently selected axis view is illustrated in the CAD window by a blue rectangle around the active pane, as well as the state of the buttons in the view toolbar. Each axis view can also be in Full or Cut Mode.

Using the 3D View:

The 3D view shows the structure rendered as 3D polygons and can be manipulated via the mouse:

- *Rotating:* The 3D view can be rotated by clicking on it and moving the mouse.
- *Zooming:* The 3D view can be zoomed in or out by right-clicking on the 3D view and moving the mouse up and down.
- *Panning:* The 3D view can be panned by holding **CTRL** and clicking on the 3D view and moving the mouse.
- *Display Modes:* The 3D view supports several different display modes such as solid, transparent, and wireframe. These modes can be cycled through by double-clicking the mouse on the 3D view.

Additionally, holding **Shift** and double-clicking toggles the display of the axes, holding **Ctrl** and double-clicking resets the view, and right-double-clicking enters an auto-animate mode.

3.E.3. Zooming

A layout window can be zoomed using the mouse, toolbar buttons, or menu items.

- *Zooming via Toolbar Buttons*

There are three zoom buttons on the view toolbar: **Zoom In**, **Zoom Out**, and **Zoom Full**. The zoom percentage is 10% by default but can be set in the CAD preferences.

- *Zooming via the Mouse Wheel*

Holding the `SHIFT` key and using the mouse wheel will zoom in/out. The zoom percentage is 10% by default but can be set in the CAD preferences. This is available for Windows systems only.

- *Zooming via the Mouse*

To zoom with the mouse click the **Zoom Mode** icon in the left toolbar. The mouse will change to a magnifying glass and clicking the left and right mouse buttons will zoom in and out respectively as well as center the display at the mouse position. Holding the `Shift` key while pressing the left button will zoom out. The zoom percentage is 10% by default but can be set in the CAD preferences.

- *Zooming via Menu Items*

It is possible to zoom via the CAD menu with the options under the View menu.

3.E.4. Panning

A layout window can be panned via scroll bars or the mouse.

- *Panning via Scroll Bars*

Each viewing window has scroll bars which can be used to pan the structure view. The scroll bars can be removed if necessary via the CAD preferences described in [Section 6.J](#).

- *Panning via the Mouse*

To pan with the mouse click the **Pan Mode** icon in the left toolbar or hold the `CTRL` and `SHIFT` keys. The structure can then be moved with the mouse. Alternatively, use the mouse wheel to pan vertically, hold the `CTRL` key and use the mouse wheel to pan horizontally.

3.E.5. Display Aspect Ratio:

The display aspect ratio controls how the two axes in the drawing window are scaled. For structures that are much bigger in one direction than another, or for structures where it is critical to see the actual dimensions, it can be useful to change the aspect ratio to better view the structure. The aspect ratio can be set via the view toolbar, or via the View/Set-View-Parameters... menu item. When Auto is chosen from the pulldown menu, the structure will be scaled to best fit the viewing window.

3.E.6. Displaying ‘Simulation’ Objects

The CAD, by default, displays additional objects in the drawing windows:

- *Simulation Domain*

The simulation domain is displayed as a dark solid line and cannot be selected or moved with the mouse. For BandSOLVE and GratingMOD, it is displayed as a dashed line which represents the domain that is used to when displaying material properties. For BandSOLVE, the actual simulation domain is defined by the lattice vectors and can be viewed by clicking the **View Domain** button in the Simulation Properties dialog. For GratingMOD, the actual simulation domain is defined so as to include the entire structure.

- *Launch Fields*

Launch fields are displayed as orange rectangles. Right-clicking on these objects in the layout window will open the Launch Properties. These objects cannot be moved with the mouse.

- *Direction Indicators*

Both launch field and monitor objects have associated directions and can be displayed in the CAD layout window.

The display of these objects can be disabled/enabled via the CAD Preferences window.

3.F. Editing Components

This section discusses how to select, move, scale, flip, and rotate components, as well as standard editing functions such as cut and paste.

3.F.1. Selecting Components

Use the **Select Mode** option in the left toolbar to select components. When in this mode the cursor will appear as an arrow and no drawing is possible.

If reading this section for the first time it might be beneficial to create a simple structure with several components to experiment with.

To select a single component, simply click on the component. The selected component will be highlighted with small black handles which can be used for resizing or scaling the component as discussed below. The component number of the most recently selected component is shown in the CAD status bar.

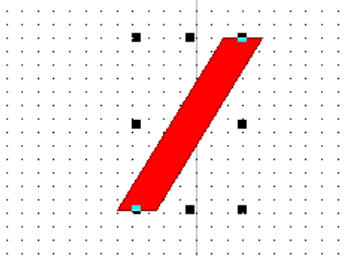


Figure 3-6: A straight component that has been selected by clicking on it.

To deselect component(s), click an unoccupied area of the layout window.

Selecting Components in Slice and Multi-Pane Mode (3D Only)

Some components will be easier to select in different cross-section views. While two components might overlap in one cross-section, they might not in another. Furthermore, it is possible to select segments in both the Full and Cut Modes described in [Section 3.E.1](#).

Selecting Components by Number or Name

It can be convenient to select a component based on its number or name especially when one or more components occupy the same physical space. To do this, use the CAD menu item Edit/Select/Edit-Component and enter the number of the desired component in the dialog. See [Section 6.K](#) for more details on setting a component's name.

Selecting Multiple Components

Multiple components can be edited at once in the CAD by dragging a selection box around the components with the mouse. Additionally, a component can be added/dropped from the current selection by holding the `Shift` button and clicking on the component.

3.F.2. Modifying Components

Once a component or set of components has been selected, they can be operated on in several ways:

Basic Operations

These are simple mouse-based methods to move, resize, flip, and rotate components. Complete control over a component can be achieved by directly setting the component properties in the Component Properties dialog described below.

The snap features described in [Section 3.D.3](#) are not enabled by default when editing existing components. It can be enabled if needed in the CAD Preferences.

- *Moving*

To move the selection, simply drag one of the components in the selection to the desired new position. If the position of the component(s) has been defined by variables in the Component Properties dialog, a dialog will be shown where the user can set an offset which will be added to the current position of the selected components. Both numeric and variable-based expressions can be used. To force this dialog to be shown hold `CTRL` while moving the components.

- *Resizing*

To resize or scale the selection along a particular direction, simply grab the corresponding handle of any one of the selected components and move it to the desired position.

- *Flipping*

The Flip Horizontal and Flip Vertical icons in the top toolbar can be used to flip components around their center. To flip around their top or bottom, hold the left or right `Shift` key while clicking the icon. Alternatively, use the Edit/Flip-Horizontal/ and Edit/Flip-Vertical CAD menu items.

- *Rotating*

The CAD also has a limited capability for rotating components via the Edit/Rotate menu option or the view toolbar . Several limitations on this option exist depending on the component type.

Standard Editing Operations

The CAD layout system possesses the standard editing functions for cut, copy, paste, and delete. These functions are accessed via icons in the top toolbar or through the Edit menu item.

Using the Component Properties Dialog to Edit One or More Components

The most complete method for modifying a component's properties is to directly edit the component's properties dialog which can be opened by right-clicking on the component. For a complete description of this dialog, see [Chapter 4](#).

To edit more than one component at a time, first select the components to be edited. Once a group selection has been made, right-click on any component in the selection (the primary component) to open its properties dialog. Generally, any changes made in this dialog will be applied, if applicable, to all selected components. Some notes:

- If a mixed group of components has been selected (that is a group of components with more than one component type), the user will be presented with a choice to apply changes to all components or just those of the same type of the primary component.
- Special consideration must be given if the selected group of components contains both single-vertex components (lens, circles, monitors, etc) and two-vertex components

(segments). In this case, the single-vertex components are considered to have only a starting vertex. Thus, any changes to the vertex information (refractive index, position, width, and/or height) of a single-vertex component will be applied to all single-vertex components as well as the starting vertex of any selected segments. Any changes to the ending vertex of a segment will only be applied to the ending vertex of any other segments. It is possible to modify this logic using the previous note about editing mixed groups.

- After changes have been made in the properties dialog, the CAD will, for all changed options, check if all selected components had the same initial value as the primary selected component. If they do, the new value will be applied to all selected components. If not, the user will be presented with a choice to apply the change to all selected components (if applicable) or just those components that had the same initial value as the primary selected component.

Changing Component Order

When one component overlaps another component, it can be useful to change the component numbers so that a certain component can be viewed as on top of another. To do this, select one of the options under the CAD menu item Edit/Change-Order to move a component forward, backward, to the front, or to the back. This is done by changing the component number.

By default, the components are drawn in the RSoft CAD by order of the component number (first component is drawn first, etc.), but this procedure can be overridden using the Draw Priority setting described in [Section 6.K.1](#).

3.G. Calculating Material Profiles

It is very useful to view and save plots of material properties such as the actual refractive index profile to be used in a simulation. This option should be used liberally as one of the main causes of inaccurate simulation results is the use of an undesired index distribution.

Using an inaccurate index distribution results in inaccurate simulation results.

3.G.1. Computing a Material Profile

A material profile can be calculated via the **Display Material Profile** button in the left toolbar.

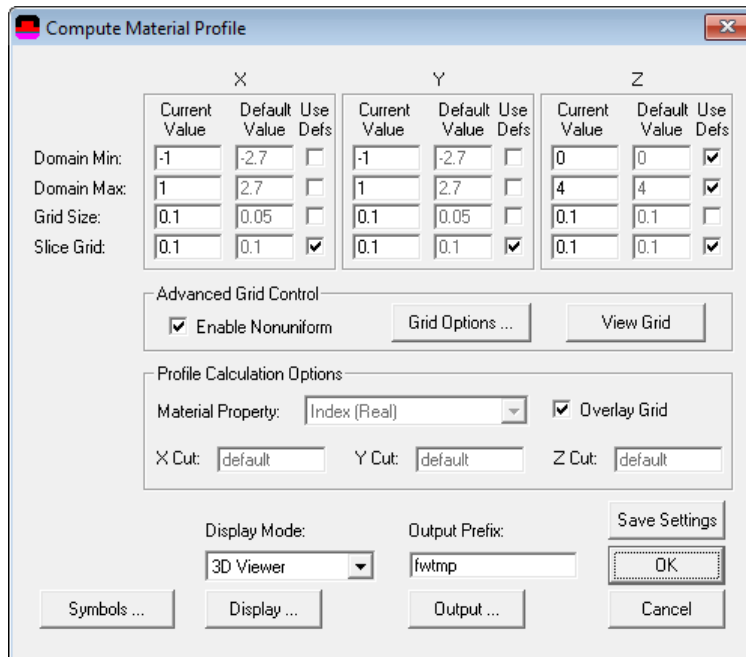


Figure 3-6: The Compute Index Profile window.

The relevant parameters are discussed in the following sections.

Calculation Parameters

The calculation parameters set in this dialog determine the domain and grid that are used to calculate the desired profile. While default values for these parameters are given, there are some guidelines for choosing these parameters:

- *Calculation Domain*

The domain should be large enough to encompass the area of interest in the structure. The default values correspond to the whole structure.

- *Grid Sizes*

The grid sizes should be small enough to resolve the smallest feature in the structure.

- *Slice Grid*

The index information is saved and displayed on the slice grid. For most calculations, it is best to set the slice grid and grid size equal to best view the index.

- *Material Property*

The real or imaginary index profile can be computed, as well as other material parameters for 3D simulations. See [Chapter 8](#) for more information on these material properties. See discussion at the end of this section for information on displaying anisotropic index profiles.

- *Overlay Grid*

This option, enabled by default, overlays the simulation grid on the material profile. Note that this only works with the contormap and 3D viewer display modes.

When the **OK** button is pressed, the desired profile is displayed as a line plot for a 2D profile or a contour plot for a 3D profile.

Choosing the Display Mode

The **Display Mode** field sets the display type to be used for the calculation. Several of the parameters discussed below can be found in the Display Options dialog and are discussed in the next section. The **Display Mode** options are:

- *Default (Fixed Z)* – Computes a cross-sectional profile at the Z value specified in the **Z Cut** field. By default, this corresponds to the **Z Domain Min**.
- *Slices* – Displays the property along Z as slices every value of Z given by the **Z Slice Grid**.
- *3D Slices* – Displays the property as a function of X and Z as 3D slices along Z at every value of Z given by the **Z Slice Grid**. In 3D, the Y position of the slices is specified by the **Slice Position Y**.
- *WireFrame* – Displays the property as a function of X and Z as 3D wireframe graph. In 3D, the Y position of the slices is specified by the **Slice Position Y**.
- *SolidModel* – Displays the property as a function of X and Z as a 3D solid model with shading to portray an illuminated surface. The color of the surface is specified by the field **Surface Color**, and in 3D, the Y position of the slices is specified by the **Slice Position Y**.
- *HeightCoded* – Displays the property as a function of X and Z as a 3D contour graph with color coding to indicate height, or index. The color scale can be set via the options discussed below.
- *ContourMap(XZ)* – Displays the property as a function of X and Z as a 2D color-coded contour map. The color scale can be selected via the options discussed below. In 3D, the Y position of the slices is specified by the **Y Cut**.
- *ContourMap(YZ)* – Only available for 3D calculations. It displays the property as a function of Y and Z. The color scale can be selected via the options discussed below. The X position of the slices is specified by the **X Cut**.
- *ContourMap(XY)* – This option is only available for 3D calculations, similar to the Default (Fixed Z) option.
- *SurfaceRelief* – Displays a surface map for the 3D structure types Rib/Ridge and Multilayer.
- *3D Volume* – This option is only available for 3D calculations and displays a 3D rendering of the property distribution using the MayaVi program. This type of display can be very memory intensive for structures with many grid points, so it is recommended that you increase the slice grid to avoid unnecessarily large memory usage.

- *3D Viewer* – This option is only available for 3D FullWAVE designs. It calculates a 3D refractive index profile, allowing the user to easily browse through different cross-sections. See the FullWAVE manual for more details.

Setting Display Options

The material profile display options are set in the Index Display Options dialog which can be opened via the **Display...** button in the Compute Material Profile window.

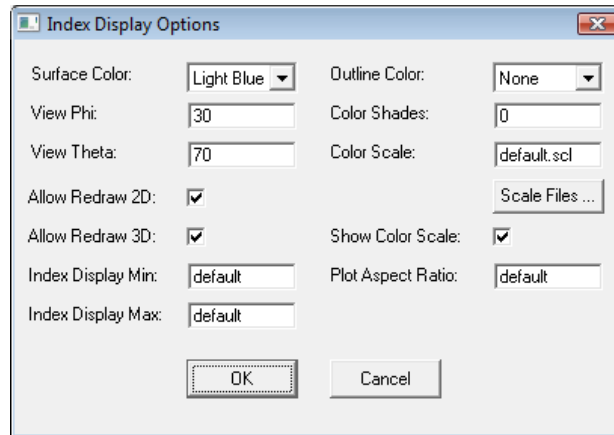


Figure 3-7: The Index Display Options dialog.

The index display options are:

Surface Color

Sets the surface color for plots produced when the **Display Mode** is set to SolidModel.

View Phi and View Theta

Sets the viewing angles Phi and Theta of the plots in the simulation window when the **Display Mode** is set to either SolidModel or HeightCoded

Allow Redraw 2D/ Allow Redraw 3D

Indicates that 2D/3D plots displayed in the simulation window should be stored in memory so that the plots can be redrawn if needed. This option is necessary for computer systems with minimal RAM; most computers will work fine with this option enabled.

Index Display Min and Index Display Max

Sets the minimum and maximum index used when calculating an index profile. Index values which are under this value are displayed in black and white respectively.

Outline Color

This pulldown menu sets the color of the circuit outline to be displayed over the simulated fields.

Color Shades, Color Scale, and Show Color Scale

These options control the colors used to render plots in the simulation window. Color scale files can be chosen via the **Scale Files...** button. See [Appendix F](#) for more on the use of color scales.

Plot Aspect Ratio

This option sets the aspect ratio of the plot in the simulation window.

Setting Output Options

The index profile output options are set in the Index Output Options dialog which can be opened via the **Output...** button in the Display Material Profile dialog.

Output index/epx/mu in complex format

Enables complex output.

Output All index/eps/mu tensor elements

Indicates that all elements on the index tensor should be output. This option was previously called **Output All Index Tensor Elements**.

Notes for Anisotropic Designs

For anisotropic designs, the variable `index_profile_tensor_component` selects the tensor element to be displayed. This variable can have values from 1 to 9 which correspond to the xx, xy, xz, yx, yy, yz, zx, zy, and zz elements respectively. This variable can be used with the options described above if needed.

3.G.2. Saving a Material Profile

In order to save the material property profile simply enter an **Output Prefix..** The real part of the refractive index is saved in the file `<prefix>.ipf`, and a corresponding WinPLOT command file for the graph will be saved in the file `<prefix>.ppf`, where `<prefix>` is the output prefix specified. The analogous files for the loss profile are `<prefix>.lpf` and `<prefix>.ppl`. Please refer to [Appendix B](#) for further information on these files and their formats.

3.G.3. Producing Index Profile Plots from the Command Line

Index profiles can be produced from the command line by simulating the `*.ind` file with the executable for the simulation tool BeamPROP (`bsimw32.exe/xplot` for Windows/Linux) via the variable `index_profile`. The value of any relevant calculation parameters will be taken from the `*.ind` file if not specified in the command. This variable should be set to 1 to compute

the index profile and 2 to compute the loss profile. The default of 0 computes the optical field as usual. Users who are not licensed for BeamPROP can still perform this type of calculation.

3.H. Viewing the Current Value of a Parameter

The current value of any option can be viewed by holding the `CTRL` key and double-clicking on the option. For example holding the `CTRL` key and double-clicking on the input box for the **Component Width** will display the current value of the width.

3.I. Easily Edit Long Expressions

When editing long expression, hold the `SHIFT` key and double-click on the option. A larger pop-up editor will display. After editing, close the window and the option will be automatically updated.

It is also possible to edit the Symbol table in a text editor. Open the Symbol Table, click **Text Editor**, make any necessary changes and/or add new symbols, then close text editor and Symbol Table will be updated.

3.J. Editable Drop-Down Lists

Several drop-down lists in the RSoft CAD and simulation tool dialogs are editable, allowing them to not only be set to one or several pre-defined values, but also to a symbol table expression. This capability further extends the parametric design capabilities of the RSoft CAD, allowing these drop-down lists to be set by symbols. This advanced feature enables these options to be scanned over via MOST and set via the import/export options described in [Section 6.I](#).

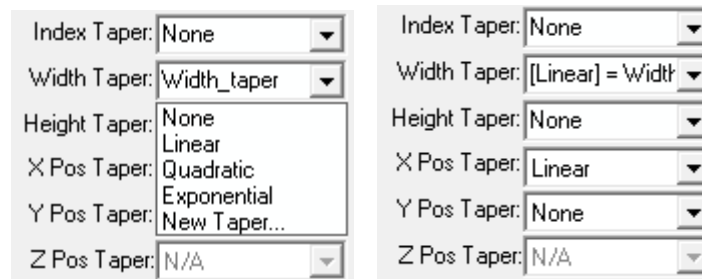


Figure 3-8: An editable drop-down list a) during editing, and b) after it has been set.

Note that these options can only be edited with the mouse on Linux, it is only currently possible to edit these options via the mouse on Windows platforms.

To edit one of these fields, click on the drop-down list and note that the current value is highlighted. Simply type the desired expression in the box as you would with any other text option. Once set, the option will appear as shown in Fig. 3-8a. The current value is shown in square brackets along with the expression.

It is recommended to use values that correspond to the internal values of the drop-down list. The list of built-in values for a particular editable drop-down lists is included in the documentation for that particular option.

As an example, we illustrate a series of symbol table expressions that change the **Width Taper** as a function of **Model Dimension** (i.e. 2D vs. 3D). First, here are the built-in options for the **Width Taper** setting shown in Fig 3-9 are:

Width Taper List Option	Built-In Value
<i>None</i>	TAPER_NONE
<i>Linear</i>	TAPER_LINEAR
<i>Quadratic</i>	TAPER_QUADRATIC
<i>Exponential</i>	TAPER_EXPONENTIAL
<i>User 1</i>	TAPER_USER_1

Here are the expressions that can be used to set the Width Taper as a function of **Model Dimension** (i.e. 2D vs. 3D):

Variable	Value
Is2D	eq(dimension,2)
Width_taper	if(Is2D,TAPER_LINEAR,TAPER_QUADRATIC)

The Is2D variable is a ‘Boolean’ symbol which equals 1 when the design is 2D (dimension = 2) and 0 when the design is 3D (dimension = 3). The Width_taper variable then equals TAPER_LINEAR in 2D and TAPER_QUADRATIC in 3D. Using this variable to define the Width Taper as shown in Fig. 3-9 effectively selects *Linear* for 2D and *Quadratic* for 3D designs.

4

Segment Components

CAD components are the basic building blocks from which a design is created. Any number and combination of different components can be embedded in a background material of a specific refractive index to create the refractive index of the entire structure.

In this chapter the discussion is intentionally limited to the segment component type. Many of the concepts in this chapter apply to other component types; see [Chapter 5](#) for a detailed description of other components and their properties can be found in the next chapter.

A complex design can be achieved by simply adding a combination of components with varying parameters to a design file. Sometimes there is more than one way to create a specific index distribution. In theory, it does not matter how the index distribution was created; however a particular approach might be easier to implement and lend itself to a simpler simulation later on.

4.A. Segment Overview

Segments are the most basic component type used by the CAD. This section provides a background to understand how segments are defined in the RSoft CAD.

4.A.1. Basic Segment Properties

A segment is composed of two vertices, one at the start and another at the end of the segment.

- *2D Layouts:*

In 2D (XZ), a segment has a transverse width and the X position, width, and refractive index can be varied as a function of a normalized coordinate between the two vertices using tapers and the index structure of the segment cross-section can be set via profiles. See [Section 4.E](#) for details.

- *3D Layouts:*

In 3D, a segment has a transverse width and height and the segment X, Y, and Z position, width, height, index, and psi (twist) can be varied as a function of a normalized coordinate between the two vertices using tapers ([Section 4.E](#)). The index structure of the segment cross-section is set by the 3D Structure Type ([Section 4.D](#)) and profiles ([Section 4.E](#)) to achieve fiber, multilayer, custom, etc. segments.

4.A.2. Segment Orientation Overview

Segment Orientation is covered in detail in [Section 4.F](#), but it is helpful to mention here as well. It refers to how segment properties are set relative to the segment axis and results in z-axis segments, seg-axis segments, and extended segments.

As an example of why this is important, consider whether the endfaces of the segment should be perpendicular to the segment axis or along a specific axis. While this may seem trivial, it has important implications for different design areas: Waveguide designers want segments that fit well without gaps and nano-structure designers may want cylinders that are oriented along an arbitrary axis. Both are valid needs and the RSoft CAD provides ways for both designers to get what they need. See [Section 4.F](#) for more details about Segment Orientation.

4.A.3. Setting Segment Properties

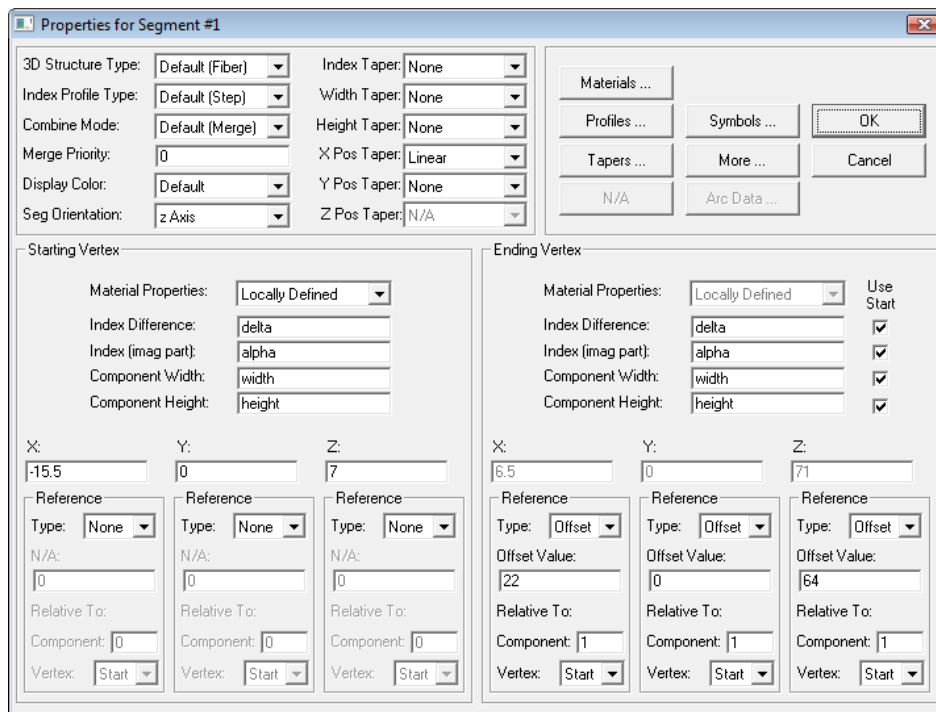


Figure 4-1: The Segment Properties dialog where individual component properties can be set.

By default, segment properties are set either by the user-specified data in the Startup/Global Settings window or program defaults. The CAD is, however, an object oriented design environment and allows the user to individually change these properties as needed.

To access any component's properties dialog right click on the component in the layout window (except in Zoom or Pan Mode). The rest of this chapter will contain a detailed description of the properties shown in Fig. 4-1. Multiple components can be edited at once as described in [Section 6.I](#).

4.A.4. Setting Segment Display Color

Every segment can have a different color. To set the color, use the **Display Color** option in the segment properties dialog. This option is an editable drop-down list which, in addition to being set via the drop-down can also be a Symbol Table expression. See [Section 3.J](#) for details about how to use this feature. The built-in values for this option are:

Display Color List Option

Default

Built-In Value

Unset. If material colors are used, defaults to the material

	color, see Section 8.A.6.
<i>Black</i>	0
<i>Dark Blue</i>	1
<i>Dark Green</i>	2
<i>Dark Cyan</i>	3
<i>Dark Red</i>	4
<i>Dark Magenta</i>	5
<i>Brown</i>	6
<i>Light Gray</i>	7
<i>Dark Gray</i>	8
<i>Light Blue</i>	9
<i>Light Green</i>	10
<i>Light Cyan</i>	11
<i>Light Red</i>	12
<i>Light Magenta</i>	13
<i>Yellow</i>	14
<i>White</i>	15

4.B. Vertices

A vertex is a component property that defines the position, width, height, and refractive index of a component. Segments have two vertices: the properties of the starting vertex are set in the lower left portion of the dialog; the ending vertex in the lower right portion as seen in Fig. 4-1.

Segment properties can tapered, or changed, between the vertices. See [Section 4.E](#) for details.

4.B.1. Setting Position

There are three methods to specify the coordinates of a vertex.

X:	Y:	Z:
4.5	0	84
Reference	Reference	Reference
Type: None	Type: Offset	Type: Angle
N/A:	Offset Value:	Angle Value:
0	0	45
Relative To:	Relative To:	Relative To:
Component: 1	Component: 6	Component: 3
Vertex: Start	Vertex: Start	Vertex: Start

Figure 4-2: An example of the three reference types possible.

The **Reference Type**, which sets the method to be used, is set for each coordinate by a dropdown menu:

- If the **Reference Type** is None, the coordinate is specified absolutely by entering its value in the X, Y, or Z coordinate field. This is the simplest method.
- If the **Reference Type** is Offset, the coordinate value is determined in a relative manner by adding the offset value, which is specified in the **Offset Value** field, to the reference coordinate. The reference coordinate is specified in the **Relative To** fields. This is the most commonly used method. Typically these coordinates are defined within the global design coordinates of the design file, see [Section 6.M](#) for details about using the local coordinate system of the vertex.
- If the **Reference Type** is Angle, the coordinate value is determined such that a line connecting the vertex with the reference vertex specified in the **Relative To** fields makes a specific angle with respect to the Z axis. The angle is measured clockwise from the Z axis and is specified in the **Angle Value** field in units of degrees.

When specifying a reference vertex via **Relative To**, give a component number and whether the vertex is the starting vertex or the ending vertex. For components with only one vertex use the starting vertex. Circuit References can have additional vertices, see [Section 6.F.7](#).

When a component is added to a design file, these settings are filled in appropriately based on the beginning and ending mouse positions. By editing these fields, the user can explicitly control the positioning of components with great flexibility. As an example, consider the portion of a component properties dialog shown in Fig. 4-2 where the X coordinate is set directly, the Y coordinate is defined via an Offset reference, and the Z coordinate is defined via an Angle reference.

4.B.2. Index Definition

There are two methods to define the refractive index of a component:

- Via Materials:

Materials are defined in the Material Editor and can be based on nk data files, etc. To use a Material, select a material already in the project, or add a material by selecting New Material under the **Material** setting for the component. These materials can also include dispersion, non-linearity, anisotropy, and electro-optic, thermo-optic, and stress-optic effects. If a Default Material has been defined in the Global Settings, all components will use that material if they have the default material settings (Locally Defined, delta, alpha).

See [Chapter 8](#) for more details about materials.

- Via an Index Difference:

Components are embedded in a background material and the real index definition is defined as:

$$n(\mathbf{r}) = n_0 + \Delta n f(x, y, z)$$

where n_0 is the **Background Index** and Δn is the local **Index Difference**. The function $f(x, y, z)$ is a profile function ([Section 4.E](#)). The imaginary index of the vertex can be set via the **Index (imag part)** option which defaults to the variable `alpha` (0 by default).

Several notes:

- The **Background Index** and the default value for the Index Difference (the variable `delta`) is set in the Global Settings window.
- By default, the index of the starting and ending vertices are the same. If different, the segment's index will be tapered as described in [Section 4.E](#).
- This definition varies slightly for 3D segments based on the **3D Structure Type**. See [Section 4.D](#) for details on a specific 3D structure type.
- The real refractive index can be specified in absolute units. This mode can be enabled in the CAD Preferences window. When this option is changed in an existing file, the CAD will attempt to convert all existing components to the new mode. It is important to realize that internally the CAD uses the index difference method and so it is the preferred method.

4.B.3. Controlling Geometry

Several vertex properties contribute to the final geometry of a component including the **Component Width** and **Component Height** which set the horizontal (X) and vertical (Y) extent of the component respectively. These properties use the default values in the Global Settings dialog (the variables `width` and `height`). The width and height of the starting and ending vertices are equal; if each vertex has a different value, the segment will be tapered (see [Section 4.E](#)) For 3D segments, the **Profile Type**, and the **3D Structure Type** can also affect component geometry (see [Section 4.D](#)). Typically the Component Width and Component Height are set locally, see [Section 6.M](#) for details about inheriting these settings from a referenced component.

4.C. Basic 2D Segments

Basic 2D segments are very simple to define in the CAD: They consist of only a **Component Width**, **Index Difference**, and a starting and ending vertex. By default, the cross section of the component is assumed to be step-index; however, this can be changed via a profile function as discussed in [Section 4.E](#). Furthermore, a material can be assigned to a simple 2D component as described in [Chapter 8](#).

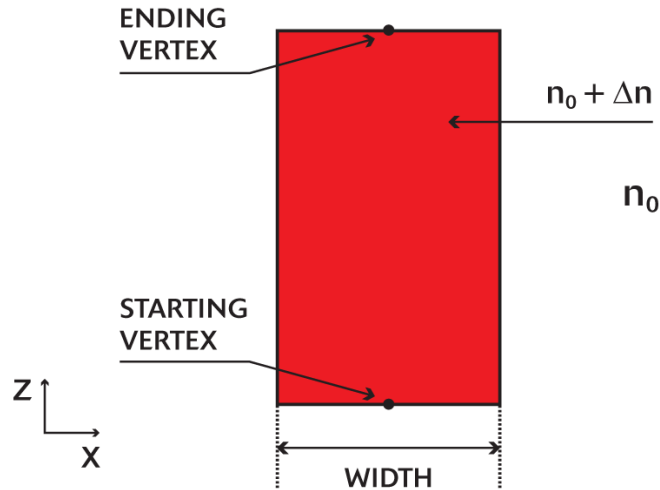


Figure 4-3: The basic 2D component definition.

4.D. 3D Structure Types

The **3D Structure Type** controls the basic transverse profile used for a component. It can be set globally in the Global Settings dialog or individually at the top of a segment properties dialog. When drawing segments, the **Structure Preference** icon in the left CAD toolbar indicates the 3D structure type of the component that will be drawn. See [Section 3.D.3](#) for more information about Drawing Preferences.

Each structure type has a slightly different definition of how basic parameters are interpreted. These parameters include **Background Index**, **Index Difference**, **Material**, **Component Width**, and **Component Height**, as well as other 3D specific parameters. While all these parameters can be set in the Global Settings dialog, some of these parameters are set for segments individually in the segment properties dialog. Additionally, many parameters can be tapered along the segment and profile functions can be used to modify the refractive index (see [Section 4.E](#)).

4.D.1. Fiber

This structure type corresponds to a circular or elliptical core embedded in a uniform background material.

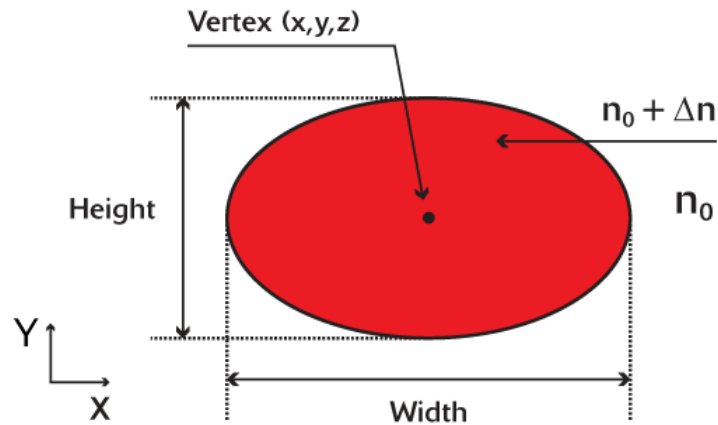


Figure 4-4. Cross-section of the built-in 3D fiber structure.

Fiber Structure Parameters:

The following parameters are relevant for this structure type:

Background Index or Background Material

The refractive index, n_0 , of the background material and is set in the Global Settings dialog.

Index Difference and Index (Imag. part)

The real and imaginary refractive indices of the component. The real part, Δn , is defined as the difference between the real refractive index of the core and the background material.

Alternatively, a material can be used to define the material properties by setting the Material property in the Component Properties dialog. See [Chapter 8](#) for more details on the use of materials.

Component Width and Component Height

The size of the elliptical cross section in the horizontal (X) and vertical (Y) directions respectively.

4.D.2. Channel

This structure type corresponds to a rectangular core embedded in a uniform background material.

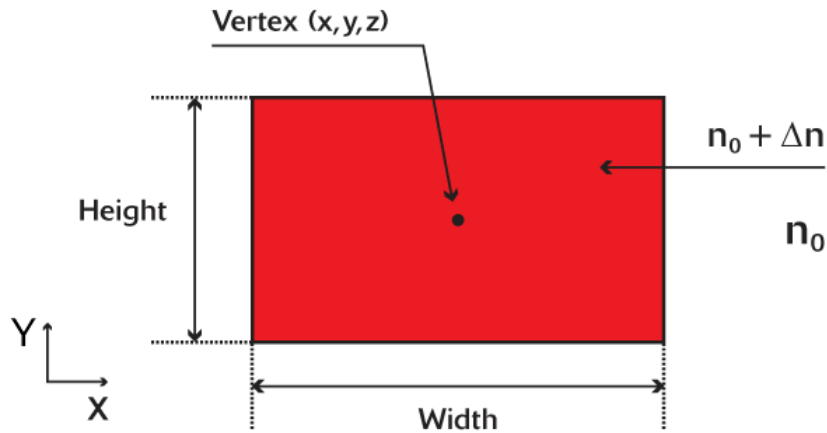


Figure 4-5: Cross-section of the built-in 3D channel structure.

Channel Structure Parameters:

The following parameters are relevant for this structure type:

Background Index or Background Material

The refractive index, n_0 , of the background material and is set in the Global Settings dialog.

Index Difference and Index (Imag. part)

The real and imaginary refractive indices of the component. The real part, Δn , is defined as the difference between the real refractive index of the core and the background material.

Alternatively, a material can be used to define the material properties by setting the Material property in the Component Properties dialog. See [Chapter 8](#) for more details on the use of materials.

Component Width and Component Height

The size of the rectangular cross section in the horizontal (X) and vertical (Y) directions respectively.

Additional Settings

A channel can have a sloped sidewall via the **Sidewall Angle** option. See [Section 6.D.2](#) for details.

4.D.3. Diffused

This structure type corresponds to a diffused structure in a substrate material. As shown in Fig. 4-6, the "background" in this case consists of a semi-infinite region referred to as the

substrate which is defined for $y < 0$, and another semi-infinite region referred to as the cover, which is defined for $y > 0$.

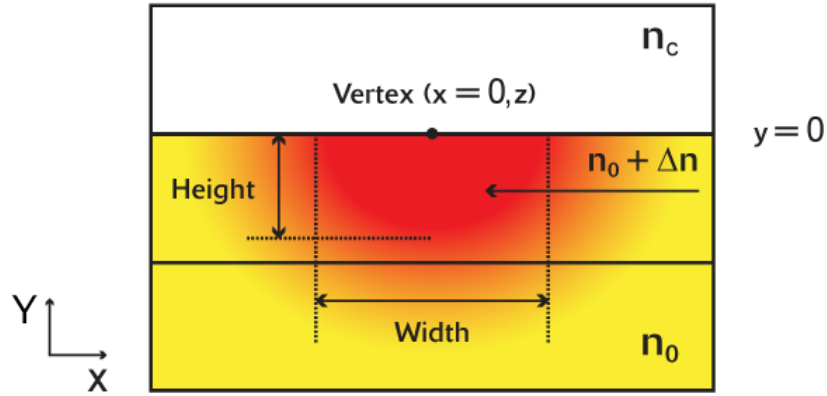


Figure 4-6: Cross-section of the built-in 3D diffused structure.

The position of the component is controlled by specifying the coordinates of the starting and ending vertices. By default, the structure is positioned at $y=0$.

Diffused Structure Parameters

The diffused profile is defined as

$$n(x, y) = n_0 + [\Delta n g(x) f(y)]^\gamma$$

where

$$g(x) = \frac{1}{2} \left\{ \operatorname{erf} \left[\left(\frac{w}{2} + x \right) / h_x \right] + \operatorname{erf} \left[\left(\frac{w}{2} - x \right) / h_x \right] \right\}$$

Here, n_0 is the substrate or background index, Δn is the maximum index change produced by diffusion from an infinitely extended source, w is the width of the source in the horizontal direction, and h_x and h_y are the diffusion lengths in the horizontal and vertical direction respectively ($h_x = h_y$ by default).

The vertical profile $f(y)$ is determined by the variable `diffusion_shape`:

- `diffusion_shape = SHAPE_GAUSSIAN` [default]

$$f(y) = \exp \left[-y^2 / h_y^2 \right]$$

- `diffusion_shape = SHAPE_EXPONENTIAL`

$$f(y) = \exp \left[y / h_y \right]$$

- `diffusion_shape = SHAPE_ERFC`

$$f(y) = \operatorname{erfc}\left[y/h_y\right]$$

The parameter γ is set by the variable `diffusion_gamma`, which equals 1.0 by default. Since it represents the non-linear interaction between the concentration and index, it is applied after the index changes for all components have been accumulated.

The following parameters are relevant for this structure type:

Background Index or Background Material

The refractive index, n_0 , of the background material which is set in the Global Settings dialog.

Cover Index or Cover Material

The refractive index, n_c , of the cover material which is set in the Global Settings dialog.

Index Difference and Index (Imag. part)

The real and imaginary refractive indices of the component. The real part, Δn , is defined as the difference between the real refractive index of the core and the background material. Alternatively, a material can be used to define the material properties by setting the Material property in the Component Properties dialog. See [Chapter 8](#) for more details on the use of materials.

Component Width and Component Height

The width of the diffusion source, w , in the horizontal direction, and the diffusion length in the vertical direction, or, h_y , in the above expressions respectively. The diffusion length in the horizontal direction is given by $h_x = h_y * \text{diffusion_ratio}$, where `diffusion_ratio` is a symbol table variable which defaults to 1.0.

An example 'process file' for a titanium-diffused lithium niobate waveguide is included as `TiLiNbO3.ind` and is documented in `TiLiNbO3.doc`. This file may be imported into another `.ind` file via the Options/Import-Symbols... menu item described in [Section 6.I](#).

4.D.4. Rib/Ridge

This structure type corresponds to what are commonly referred to as rib or ridge structures as shown in Fig. 4-7. The "background" in this case consists first of a semi-infinite region referred to as the substrate, which is defined for $y < 0$. On top of the substrate is a thin film of a given height and index, followed by another semi-infinite region referred to as the cover, which is defined for y values above the top edge of the film. This background forms a one-dimensional three-layer slab waveguide in the vertical direction.

The position of the component is controlled by specifying the coordinates of the starting and ending vertices. In background region, where no structures have been defined, the film layer is

considered to exist up to a height defined by the **Slab Height** option in the Global Settings dialog.

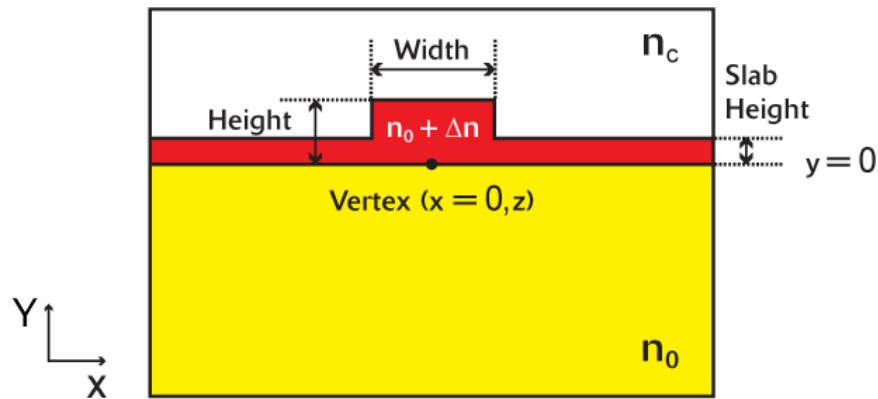


Figure 4-7: Cross-section of the built-in 3D rib/ridge structure.

Rib/Ridge Structure Parameters

The following parameters are relevant for this structure type:

Background Index or Background Material

The refractive index, n_0 , of the background material, and is set in the Global Settings dialog.

Cover Index or Cover Material

The refractive index, n_c , of the cover material and is set in the Global Settings dialog.

Index Difference and Index (Imag. part)

The real and imaginary refractive indices of the component. The real part, Δn , is defined as the difference between the real refractive index of the core and the background material. Alternatively, a material can be used to define the material properties by setting the Material property in the Component Properties dialog. See [Chapter 8](#) for more details on the use of materials.

Slab Index or Slab Material

The refractive index of the thin film material defining the vertical slab in the background. By default, this is defined by a formula such that the **Slab Index** equals the **Background Index** plus the **Index Difference**. This option is set in the Global Settings dialog.

Slab Height

The height of the thin film material defining the vertical slab waveguide which forms the background. This option is set in the Global Settings dialog.

Component Width and Component Height

The size of the rib's cross section in the horizontal (X) and vertical (Y) directions respectively.

4.D.5. Multilayer

This structure type describes a multilayer structure and can be used to define simple rib structures as well as more complicated layered structures.

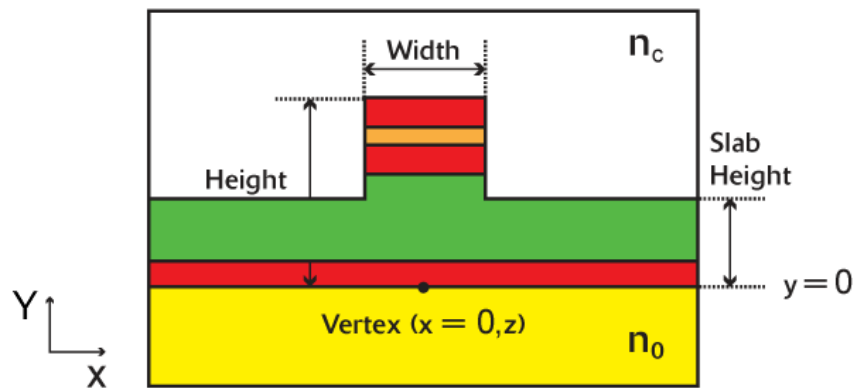


Figure 4-8: Cross-section of the built-in 3D multilayer structure. Each layer can have a unique height and index value.

The position of the component is controlled by specifying the coordinates of the starting and ending vertices. A multilayer structure is defined using the following concepts:

- *Substrate*

The 'background' in this case consists first of a semi-infinite region referred to as the substrate, which is defined for $y < 0$.

- *Layers*

On top of the substrate are one or more layers, or thin films. Each layer is defined by its height and complex-valued refractive index via a Layer Table, which is discussed in the next section. Different components can be assigned different Layer Tables, resulting in a complex arbitrary structure.

- *Cover*

The layer structure is followed by another semi-infinite region referred to as the cover, which is defined for y greater than the top edge of the last layer.

In background region where no structures have been defined the layer structure is considered to exist up to a height defined by the **Slab Height** option in the Global Settings dialog.

Multilayer Structure Parameters:

The following parameters are relevant for this structure type:

Background Index or Background Material

The refractive index, n_0 , of the background material and is set in the Global Settings dialog.

Cover Index or Cover Material

The refractive index, n_c , of the cover material and is set in the Global Settings window.

Layer Table

Sets the layer table to be used. See description below.

Slab Height

The layer structure (defined by Layer Table 0) is considered to exist up to this height in the background where no components are present. This option is useful since the slab structure is usually similar to the core structure, and so only one layer table need be used to define both the slab and core structure. This option is set in the Global Settings window.

Component Width and Component Height

The size of the rib structure in the horizontal (X) and vertical (Y) direction respectively. The layer structure is considered to exist up to the height in the interior of the component. When this value is less than the total height defined in the assigned layer table, the CAD will truncate the layer table at this value. It is not recommended to set the height larger than the total height defined in the assigned layer table as it can cause problems in certain cases.

Defining Layer Tables

The Layer Table Editor can be accessed via the **Edit Layers** button in the Global Settings dialog, via the left toolbar, via the Edit/Tables/Edit-Layers... menu item, or via the **Edit Layers** button in the segment properties dialog. The Layer Table Editor, shown in 4-9, allows the user to create and modify layer tables. A layer table contains film height and refractive index, both real and imaginary, or a material. Once a layer table has been defined, it can then be assigned to a segment.

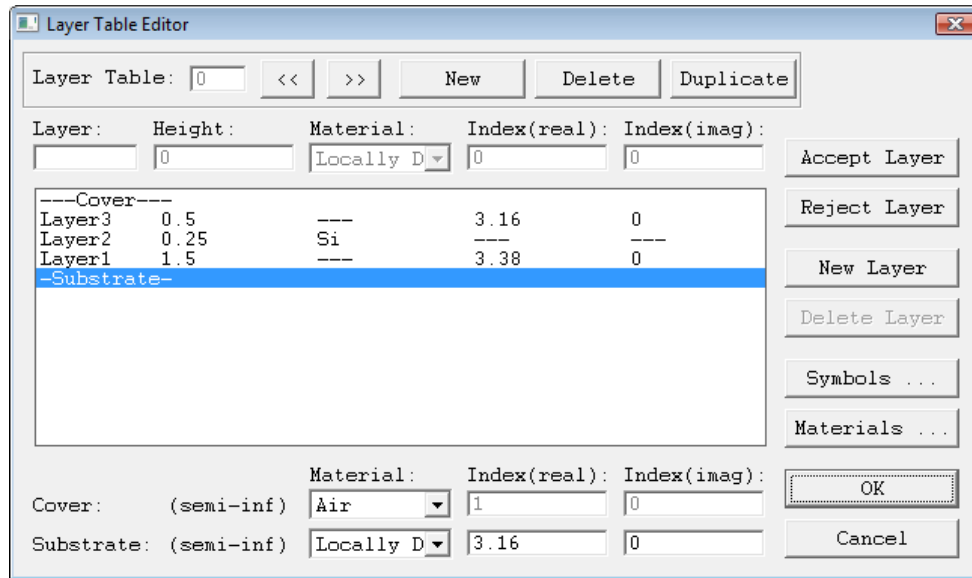


Figure 4-9: The layer table editor dialog, which is used for defining the layer structure for a multilayer component.

The upper portion of the Layer Table Editor selects the layer table to be edited. Layer table 0 defines the background layer structure, or regions where there are no components, as well as the default layer structure within a component unless changed in the component properties dialog. There are also contains buttons for creating a new layer table, deleting a layer table, and duplicating an existing layer table. The right portion of this dialog controls the information for the selected layer table and functions similarly to the symbol table editor described in [Section 3.C](#). The lower portion of the dialog allows the user to define the properties of the semi-infinite substrate and cover, and are the same controls as those used in the Global Settings dialog. Note that when defining a layer table other than the background one, or layer table 0, the user may specify that the index information for a given layer is to be "transparent" to the background. This is done by entering the keyword `transparent` in the corresponding field for the real or imaginary part of the index. To use this feature, the given layer table must be able to merge with the background; that is, there must be a one-to-one correspondence between the number of layers and the layer heights. For example, this allows a component to exist only in a certain layer or group of layers, as well as allowing for vertically coupled components.

Assigning a Layer Table to a Component

Much like the case of using a material to define the properties of a component, the **Index Difference** and **Index (imag part)** fields in the segment properties dialog serve no function and are disabled. They are replaced by the **Layer Table** field, which specify the layer table to be used for the individual component being edited. This allows different components to be assigned different layer structures, since the layer structure of the background is replaced by the specified layer table within the component.

Creating Complicated Multilayer Structures

Multilayer components can be overlapped to achieve complicated multilayer structures. When done, it is recommended that the user use as few layer tables as possible. Additionally, the **Slab Height** and **Component Height** options described above should be used so that layer tables can be ‘reused.’

4.D.6. Polygon Structure Types

This structure type corresponds to a polygon cross-section embedded in a uniform background material as shown in Fig. 4-10. The polygon is inscribed in an ellipse centered at the vertex position.

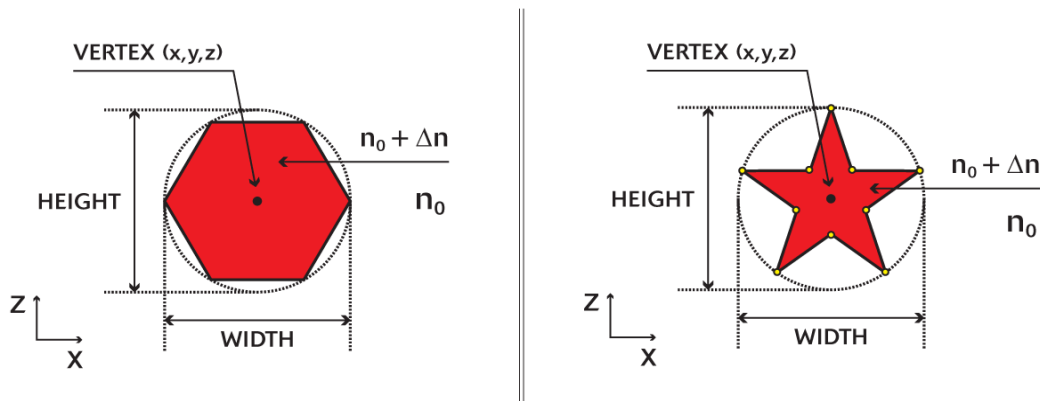


Figure 4-10: Cross-section of two versions of the built-in polygon structure: a) a regular polygon with 6 sides (hexagon), and b) a user-defined polygon. Any number of sides or a custom data file can be used.

Polygon Structure Parameters:

The following parameters are relevant for this structure type:

Background Index or Background Material

The refractive index, n_0 , of the background material which is set in the Global Settings dialog.

Index Difference and Index (Imag. part)

The real and imaginary refractive indices of the component. The real part, Δn , is defined as the difference between the real refractive index of the core and the background material. Alternatively, a material can be used to define the material properties by setting the Material property in the Component Properties dialog. See [Chapter 8](#) for more details on the use of materials.

Component Width and Component Height

The size of the bounding circle in the horizontal (X) and vertical (Y) directions respectively. When these parameters are equal, a regular polygon will be created.

Note that these options set the size of the bounding circle around the polygon. To instead specify the length of a side of the polygon, specify the **Component Width** and **Component Height** using the expression $2 * (\text{sqrt}(S^2 / (2 * (1 - \cos(360 / \text{Num}))))$ where S is the desired side length and Num is the number of sides in the polygon.

Additional Settings

A polygon structure can have a user-specified number of sides or use a data file to define the cross-section shape of the component. These settings can be changed by clicking the **Poly Info...** button in the Segment Properties dialog which opens the Polygon Structure Info dialog.

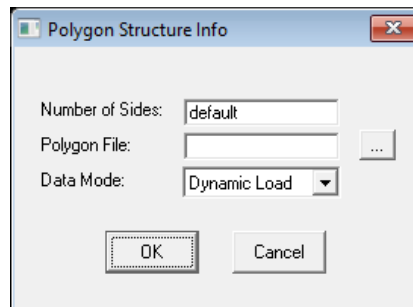


Figure 4-11: The Polygon Structure Info dialog.

Number of Sides

Sets the number of sides for a regular polygon structure. As shown in Fig. 4-10, the polygon is inscribed inside a circle defined by the Component Width and Component Height. When a file is specified, it takes precedence over the number of sides.

Polygon File

Sets the data file that contains the data points for the polygon. The data should be in the polygon format discussed in [Section B.E](#) and is interpreted drawn such that a unit circle will be drawn on the bounding circle shown in Fig. 4-10. If the data in the file should not be scaled, set the **Component Width** and **Component Height** to 2. The polygon data must define a closed surface: the first and last point in the data file must be the same.

It is possible to use a variable to define a polygon file: set the polygon file option to `$<variable>` where `<variable>` is a variable created in the Symbol Table set equal to the name of the desired polygon file.

Date Mode

Sets the load method for the polygon data. A Dynamic Load will load the file from disk every time; a Static Load will embed the polygon data in the .ind file and not reference the file.

4.D.7. Mixing 3D Structure Types

The different structure types available in the CAD can be combined in the same file to produce an extremely wide variety of 3D geometries. When combining structure types, the background definition must be noted. The definition of the background is different for each 3D structure type; for example the background for fiber and channel structures is uniform, while the background for diffused and multilayer structures has a substrate and a cover. The background used is always set by the **3D Structure Type** set in the Global Settings window.

4.E. Profiles & Tapers

By default, most components have a step-index profile such that the refractive index inside the component is the sum of the background index and the index difference and outside is the background index. However, it is possible to change the profile so that other geometries such as graded index profiles, etc, can be achieved. Please see [Section 6.A](#) for a complete discussion of profiles.

Tapers allow a segment's properties to vary between the two vertices. The segment's width, height, index, and position can be tapered independently. Tapers are defined in the top region of the segment properties dialog. Please see [Section 6.B](#) for a complete discussion of tapers.

4.F. Segment Types (z-Axis, Seg-Axis, Extended)

There are three forms of the general segment, each with subtle but important differences. Each has a different interpretation of the width/height specified by the user, how the segment terminates, how tapers are calculated, and how segment profiles are interpreted.

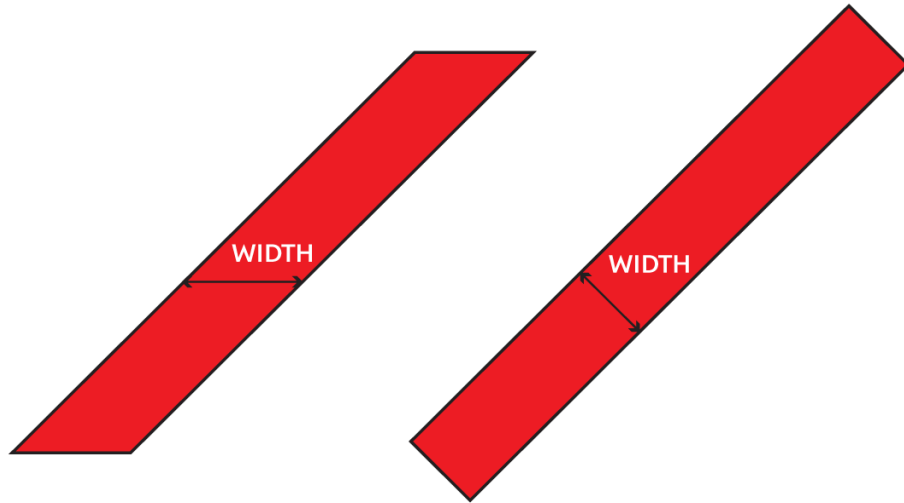


Figure 4-12: 2D Illustration of segment types: a)z-Axis b) Segment-Axis/Extended.

The type of segment is controlled by the **Seg Orientation** option in the segment properties dialog.

Type	Width/Height	Endfaces	Tapers/Profiles	Rotation
z-Axis	Measured relative to z axis	Flush with z axis	Interpreted relative to z axis	N/A
Seg-Axis	Width relative to segment axis, height relative to y axis	Perpendicular to segment axis	Some tapers relative to segment axis, profiles relative to z axis	Rotation in XZ plane
Extended	Width and height relative to segment axis	Perpendicular to segment axis	Tapers and profiles relative to segment axis	Arbitrary rotation possible

Some notes:

- Extended segments are used by default for new segments added to a design.
- When using curved seg-axis or extended segments the resolution at which the curves are resolved can be set via the **Import/Export Resolution** in the CAD Preferences dialog described in [Section 6.E](#).
- Extended segments can easily be added using the **Extended Segments** drawing mode in the left CAD toolbar. See [Section 4.G](#) for more information about these built-in structures.
- Extended segments can be twisted along the segment axis using the **X'Y' Rotation** options in the Additional Segment Properties dialog as described in [Section 6.K](#).

- Extended segments can be rotated using the **Reference Phi**, **Reference Theta**, and **Reference Psi** options in the Additional Segment Properties dialog described in [Section 6.K](#).

Which Type of Segment Should I Use?

Seg-axis or extended segments give more control over segment geometry and are generally preferred. Note that it is possible to produce mismatched width or gaps between components that are not properly matched. Contact RSoft for recommendations for an appropriate solution to a specific problem.

4.G. Drawing Common Extended Segments

Common extended segments can easily be drawn using the **Extended Segments** mode in the left CAD toolbar ([Section 3.A.2](#)). Since all segments are forms of a generalized segment, these segments can easily be modified to produce other customized segments. These extended segments are:

Though pictured in the CAD with a circular cross-section, these extended segments can have any cross-section (**3D Structure Type**).

- Straight*

This mode produces a straight segment.

- Half-taper*

This mode produces a segment with a tapered width and height. The ending width/height is set equal to half the starting width/height; this can be changed by right-clicking on the segment and changing the relevant properties.

- Full-taper*

This mode produces a segment with a full tapered width and height where the ending width/height is set to 0. This can be used to make cones, triangles, etc.

- Arc*

This mode produces 90 degree arc segment. The radius and angular extent of the arc can be set in the Arc Data dialog which can be opened by clicking **Arc Data...** in the Segment Properties dialog.

- Toroid*

This mode produces a toroid segment. The radius can be set in the Arc Data dialog which can be opened by clicking **Arc Data...** in the Segment Properties dialog. The **Arc Reference** setting in that dialog is set to Center so that the position of the starting vertex controls the

center of the toroid. Depending on which plane the toroid is drawn, the reference angles ([Section 6.K.2](#)) will be set to achieve the desired orientation. These can be changed to any value to orient the toroid along any angle.

- *Helix*

This mode produces a helix segment. The radius can be set in the Arc Data dialog which can be opened by clicking **Arc Data...** in the Segment Properties dialog. The **Arc Reference** setting in that dialog is set to Center so that the position of the starting vertex controls the center of the helix. Depending on which plane the toroid is drawn, the reference angles ([Section 6.K.2](#)) will be set to achieve the desired orientation. These can be changed to any value to orient the toroid along any angle. The default helix has 3 turns, this is controls by the Start Angle and can be changed by editing the **Arc Initial Angle** and **Arc Final Angle** from its default values of 0 and 3×360 .

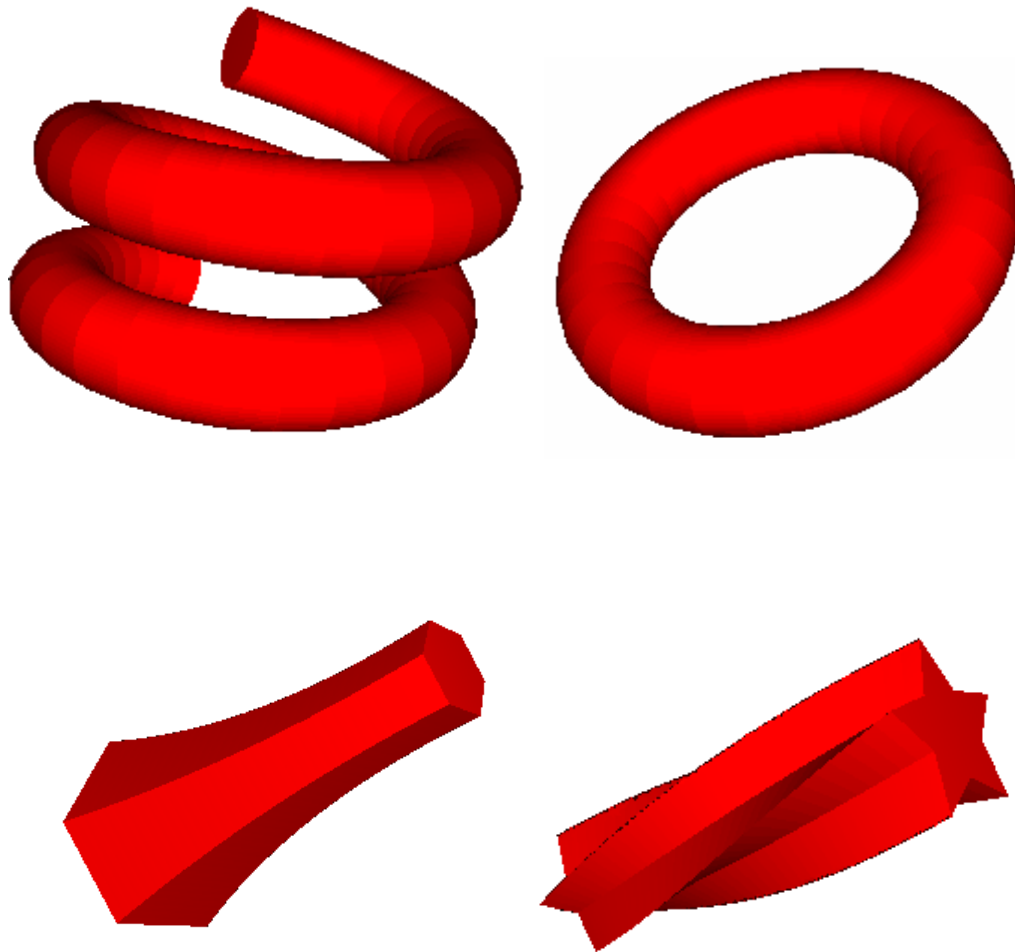


Figure 4-13: Examples of common extended segments: a) helix segment with fiber structure type, toroid segment with fiber structure type, c) straight segment with polygon cross-section (hexagon) with width/height taper, d) straight segment polygon cross-section (data-file) with 'twist' taper.

5

Other Component Types

[Chapter 4](#) described the segment component type. This chapter describes other component types.

5.A. General Overview

In addition to the segment component type, which was described in Chapter 4, the CAD contains several other component types which allow different geometries to be easily created such as lenses, circles, polygons, circuit references, simulation regions, and time monitors. Keep the following in mind when using these component types:

- These component types have one vertex which is specified in exactly the same way as a segment. Rather than repeat the documentation here, see the relevant section in [Section 4.B.1](#).
- These components have only one vertex and so tapers cannot be used; some of these components can, however, have a **3D Structure Type** and **Index Profile Type** defined.

5.B. Lenses

Lenses are segments that are defined by two radii, a center thickness, and a width. Note that while lenses can be used to create circles, cylinders, spheres, and rings, the circle component is easier for that purpose.

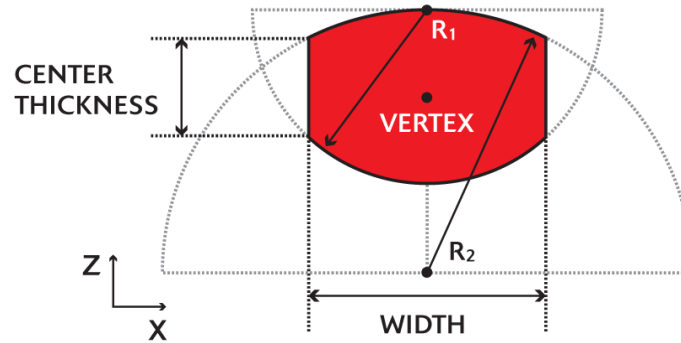


Figure 5-1: A schematic of a lens structure. Two radii, R1 and R2, as well as a center thickness and lens width are shown. In 3D, this structure can be cylinder-like or sphere-like.

Lenses can be added when the **Lens Mode** button in the left toolbar of the CAD has been selected.

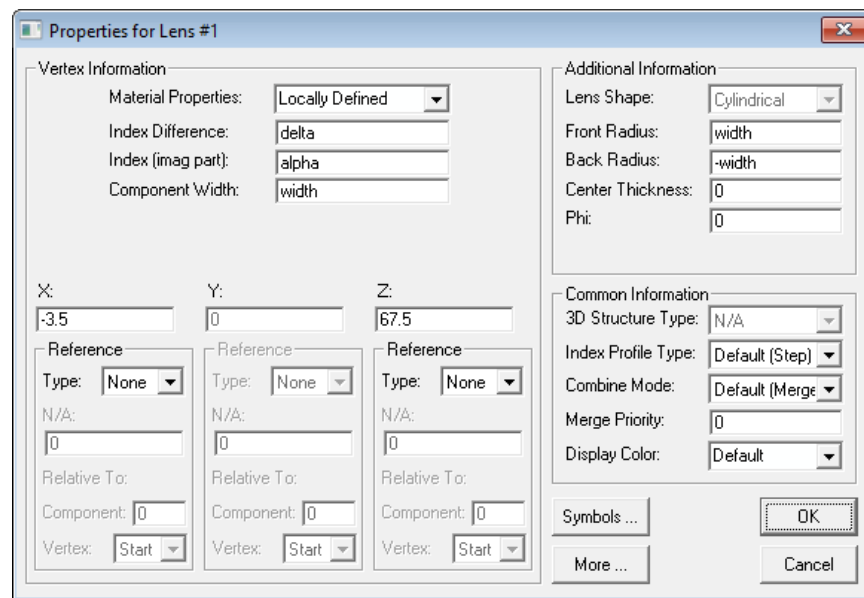


Figure 5-2: Properties dialog for Lens component.

Vertex Settings

The left half of the lens properties dialog is identical to the vertex portion of the segment properties dialog (see [Section 4.B](#)). The vertex defines the center of the lens as illustrated in Fig. 5-1 as well as the lens component's optical properties. The diameter of the lens is set by the smaller of the **Component Width** or twice the smallest radius.

Lens-specific Settings

The right half of the properties dialog defines lens-specific parameters:

Front Radius and Back Radius

The radius of curvature of the front and back radii of the lens, labeled R1 and R2 respectively in Fig. 5-1. The centers of the circles determining the front and back interfaces are defined implicitly such that the circles intersect the front and back faces of the uniform central region. The default value for these radii is `inf` and `-inf` respectively, corresponding to a infinite radius. The sign convention for the radii is the usual one in optics, namely a positive front radius and a negative back radius yields a convex lens.

Center Thickness

The thickness of the uniform central region, if any. This parameter defaults to 0.

Phi

This controls the phi angle of the lens measured from the Z axis. The default values are 0. Typically the **Phi** angle is set locally, see [Section 6.M](#) for details about inheriting this setting from a referenced component.

Lens Shape

This field controls the shape of the lens in 3D. See the next section for more details about its use.

Notes for 3D

In 3D, a lens can have either a cylindrical like-shape, or a sphere-like shape. The vertical extent of the lens along the Y axis is controlled by the **Component Height** field.

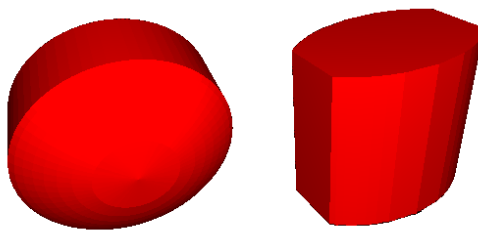


Figure 5-3: Lenses in 3D, a) Spherical version of lens shown in Fig. 5-1, b) Cylindrical version of lens shown in Fig. 5-1.

5.C. Circles

Circles are components that are defined by one radius, and can be used to create circles in 2D and cylinders and spheres in 3D.

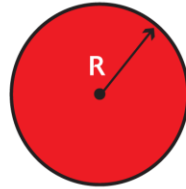


Figure 5-4: The circle component type.

Circles can be added when the **Circle Mode** button in the left toolbar of the CAD has been selected.

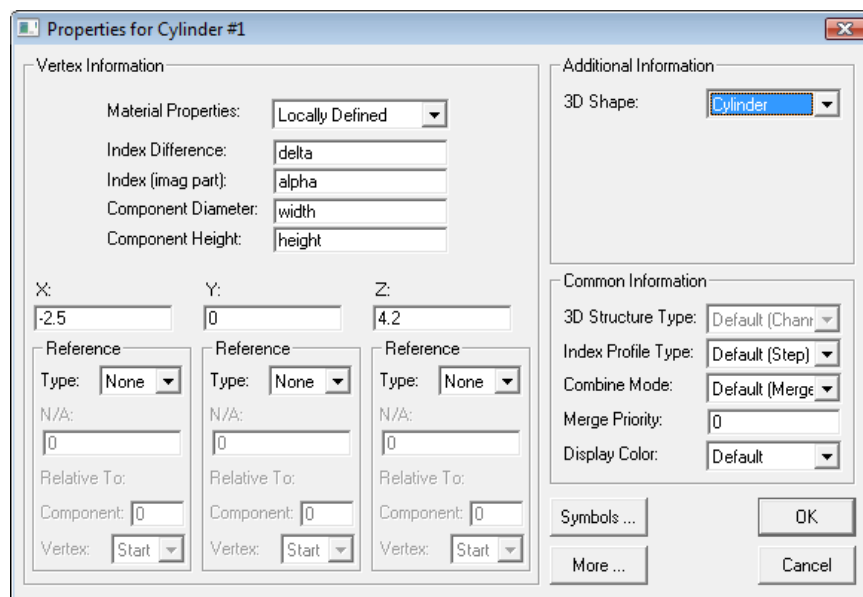


Figure 5-5: Properties dialog for a circle component.

Vertex Settings

The left half of the circle properties dialog is identical to the vertex portion of the segment properties dialog (see [Section 4.B](#)). The vertex defines the center of the circle as illustrated in Fig. 5-4 as well as the circle component's optical properties.

Circle-specific Settings

The right half of the properties dialog defines circle-specific parameters:

Component Diameter

The diameter of the circle.

Component Height

The height of the circle when **3D Shape** is set to Cylinder.

3D Shape

This field controls the shape of the lens in 3D. See the next section for more details about its use.

Common Circle Configurations

In 3D, a circle can have either a cylindrical like-shape, or a sphere-like shape. The vertical extent of the circle along the Y axis is controlled by the **Component Height** field.

Cylinders & Spheres

Cylinders can be created in 3D by setting **3D Shape** to Cylinder, spheres with a **3D Shape** of Sphere.

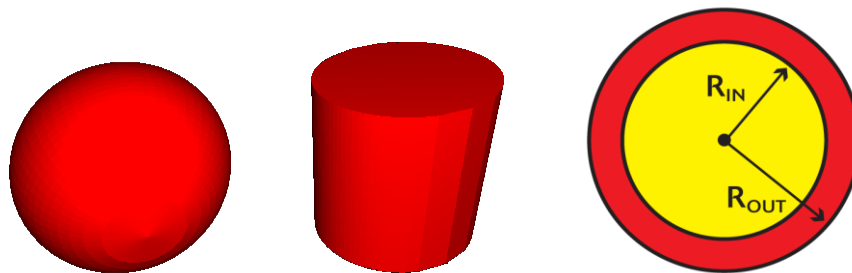


Figure 5-6: A sphere, cylinder, and ring made with circles.

This cylinder is along the Y axis; a cylinder along the Z axis can be made via the 3D fiber structure type.

Rings

One way to make a ring in 2D or 3D is to overlapping two circles/cylinders. The inner segment should have a **Index Difference** of 0 (or some other desired index value) and a higher **Priority** than the outer segment. See [Section 6.C](#) for more details about overlapping segments. These types of objects can also be made with seg-axis or extended segments (see [Section 4.F](#)).

5.D. Polygons

Polygons are segments which are defined via a series of arbitrary vertex points in the XZ plane.

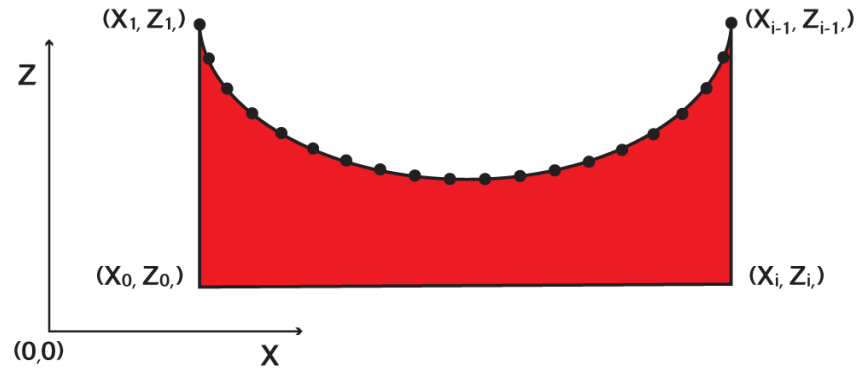


Figure 5-7: Geometry of a polygon component defined by a series of arbitrary vertex points in the XZ plane. For 3D structures, the structure is simply extruded in the Y direction so that it has a height given by the **Component Height** option.

Polygons can be added when the **Polygon Mode** button in the left toolbar of the CAD has been selected.

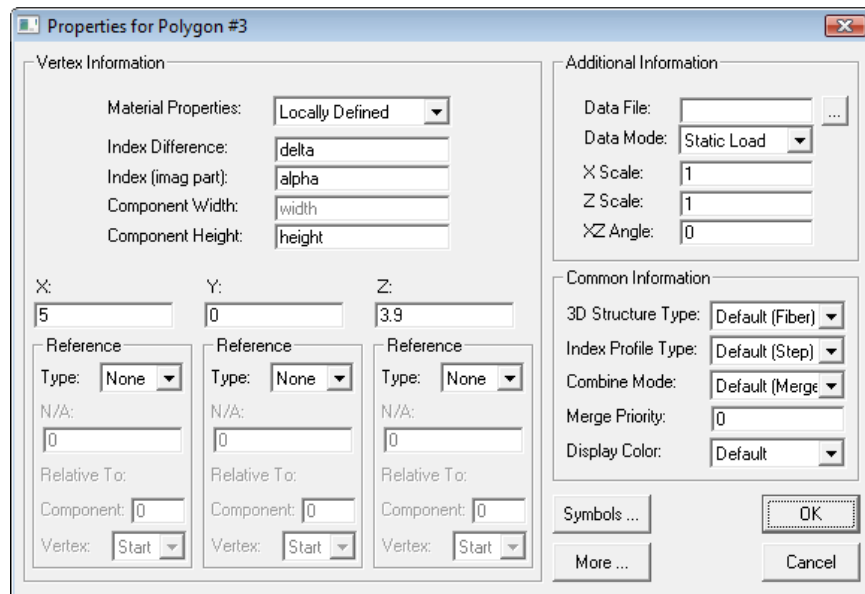


Figure 5-8: Properties dialog for Polygon component.

Vertex Settings

The left half of the Polygon Properties dialog is identical to the vertex portion of the segment properties dialog (see [Section 4.B](#)). The vertex defines the origin of the polygon relative to the design file as well as the polygon component's optical properties.

Polygon-specific Settings

The right half of the dialog defines the following polygon specific properties:

X Scale and Z Scale

A scale factor that can be used to scale the polygon data along the X and Z axes respectively. By default these parameters are 1.

Angle

The angular rotation about the origin (defined by the vertex) to be applied to the polygon data. The default origin and scale factors are chosen so that the polygon coordinates are interpreted as absolute coordinates. However, if the polygon data represents a general shape to be used repeatedly, the scale, angle, and vertex information can be used to size, rotate, and position the component as required, without the user having to generate a different data file for each polygon component to be added. Typically the **Angle** is set locally, see [Section 6.M](#) for details about inheriting this setting from a referenced component.

Specifying the Polygon Data

The data file is selected by pressing the **Data File...** button and selecting the data file that contains the point information. The **Data Mode** controls how the program loads the data: a Static Load copies the contents of the file into the *.ind file, and a Dynamic Load loads the data every time the file is opened. Also, the data file can be stored in a variable as described in [Appendix C](#).

Data Format

The polygon coordinate data must be imported from an ASCII data file containing two columns defining the X and Z coordinates of each point:

```
X(0) Z(0)
... ..
X(i-1) Z(i-1)
X(i) Z(i)
X(0) Z(0)
```

The polygon data must define a closed surface: the first and last point in the data file must be the same. Also, after selecting the file, the coordinate information is stored locally in the *.ind file.

Notes for 3D

For 3D structures, the polygon is simply extruded or extended with a constant thickness along Y. This thickness is given by the **Component Height** field. Polygons can have a **3D Structure Type** as well.

5.E. Registration Marks

Registration marks are useful for mask production and are defined by a width, height, and thickness. This type of segment has no effect on simulations and is not included in the index calculation.

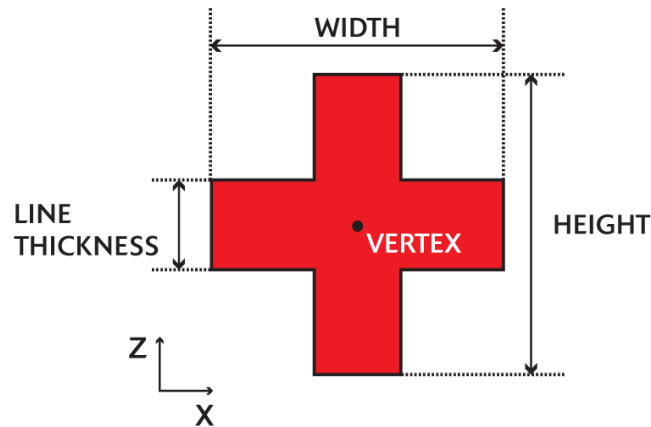


Figure 5-9: The Registration Mark component type.

To add a registration mark to the circuit, select Options/Insert/Mark from the CAD menu. This will open the properties dialog for the new mark.

Vertex Settings

The left half of the mark properties dialog is identical to the vertex portion of the segment properties dialog (see [Section 4.B](#)). The **Width/Length** and **Height** fields set the width and height of the mark. Typically these settings are set locally, see [Section 6.M](#) for details about inheriting this setting from a referenced component.

Mark-specific Settings

The right half of the mark properties dialog specifies mark-specific information:

Mark Type

This field specifies the type of mark to be used.

Line Thickness

This field specifies the line thickness of the mark as illustrated in Fig, 5-8.

5.F. Circuit References, Simulation Regions, Vertices, and Monitors

Circuit references, simulation regions, vertices, and time monitors are advanced components.

- Circuit references are a component that contains another design file. It allows one design file to be nested in another resulting in a hierarchical design. See [Section 6.F](#) for more details.
- Simulation regions allow BeamPROP's simulation parameters to locally be changed. See the BeamPROP manual for more details.
- Monitors make measurements for FullWAVE, DiffractMOD, and ModePROP (see appropriate simulation manual) and can be used to define ports for S-Matrix calculations (see the Custom PDK Utility manual).
- Vertices are control points that can be added to a design, used mostly when defining circuit references. See [Section 6.F.7](#).

6

Advanced CAD Features

This chapter covers advanced RSoft CAD topics. It assumes basic familiarity with the RSoft user interface. If any concept is not clear, please consult the previous chapters.

6.A. Profiles

A component's profile function provides a crucial part of the refractive index of that component:

$$n(\mathbf{r}) = n_0 + \Delta n f(x, y, z)$$

where n_0 is the **Background Index** and Δn is the local **Index Difference**, and $f(x, y, z)$ is a profile function. The profile function is set by the **Index Profile Type** option either globally in the Global Settings window or locally via a component's properties window. By default the profile function is generally step index. There are several built-in profile or users can create a user-defined profile.

6.A.1. Built-in Profile Types

The user profile type is set in the **Index Profile Type** option in the Component Properties dialog. can be set All profile types are not available for all component types; see notes at end of this section. In these equations, w is the component width and x is the distance from the center of the component.

- Default

The default profile is set by the global default in the Global Settings window and is based on the **Model Dimension** and **3D Structure Type**. See table below for the default profile for each structure type.

- Inactive

An inactive component does not contribute to the refractive index distribution of the design file and is useful for defining optical pathways for launch fields and pathway monitors.

- Step Index

A step index profile equals 1 inside the structure and 0 outside. This is the most common profile.

- Gaussian

A Gaussian profile function is defined as:

$$n(x) = n_0 + \Delta n e^{-\frac{x^2}{a^2}}$$

where a is defined to be half the **Component Width**.

- Diffused

A Diffused profile is implemented as a pseudo-effective index profile given by

$$n(x) = n_0 + \Delta n \frac{1}{2} \left\{ \operatorname{erf} \left[\frac{\frac{w}{2} + x}{h} \right] + \operatorname{erf} \left[\frac{\frac{w}{2} - x}{h} \right] \right\}$$

where h represents the diffusion depth and is defined by setting the `height` variable in the symbol table, or via the **Component Height**. For additional information and options, refer to the discussion of **3D Structure Type** of Diffused in [Section 4.D](#).

- New Profile...or User #

These choices allow the user to specify the functional form of the index profile arbitrarily. This feature is further described in [Section 6.A.3](#).

Available Index Profile Types & Defaults

This table summarizes the available profiles for each structure type.

	Inactive	Step Index	Gaussian	Diffused	User
2D Slab	Yes	Default	Yes	Yes	Yes
Fiber	Yes	Default	Yes	Yes	Yes
Channel	Yes	Default	Yes	Yes	Yes

Diffused	Yes	No	No	Default	No
Rib/Ridge	Yes	Default	No	No	Yes
Multilayer	Yes	Default	No	No	Yes
Polygon	Yes	Default	No	No	No

Notes:

- Using a Diffused profile type for a 2D Slab results in the diffused profile described above.
- Using a Diffused profile type for a Fiber or Channel is the same as a Diffused structure type.
- Using a Gaussian profile type for a Fiber or Channel results in a radial Gaussian profile.
- Using a user-defined profile for a Rib/Ridge or Multilayer results in a Height Profile ([Section 6.A.2](#)).

6.A.2. Types of User Profiles

User-defined profiles can be functions of any combination of x, y, and/or z resulting in many possible equation types. This section describes the possible combinations.

Coordinate Normalization

In the following user profile definitions, the normalized coordinates x' , y' , and z' are defined as

$$x' = \frac{2x}{w}, \quad y' = \frac{2y}{h}, \quad z' = \frac{z}{l} \quad (1)$$

where x , y , and z are the actual positions along the segment, and w , h , and l are the local width, height, and length of the segment. Therefore, the normalized coordinates are by default defined to vary from -1 to 1 for x' and y' , and from 0 to 1 for z' .

It is possible to interpret the coordinate and index information in the user-profile absolutely without applying any coordinate or index normalization using the option described in [Section 6.D.5](#). The data will still be translated to the position of the component.

Arbitrary 3D User Profile

This is the most generic user profile function and can be expressed as:

$$n(x', y', z') = n_0 + \Delta n f(x', y', z') \quad (2)$$

where $n(x', y', z')$ is the desired index profile, n_0 is the background index, and Δn is the local index change. Both the channel and fiber 3D structure types accept this type of profile.

Arbitrary 2D User Profile

This is a 2D version of (2), and can be defined in either the X-Z plane for 2D structures or in the X-Y plane for 3D structures:

$$\begin{aligned} n(x', y') &= n_0 + \Delta n f(x', y') \\ n(x', z') &= n_0 + \Delta n f(x', z') \end{aligned} \tag{3}$$

Both the channel and fiber 3D structure types accept this type of profile. Note that in a 2D simulation, even though the profile lies in the XZ plane, the function must be entered as a function of x and y where y represents the z axis and is normalized between 0 and 1.

Arbitrary 1D User Profile

This is a 1D version of (2):

$$n(x') = n_0 + \Delta n f(x') \tag{4}$$

This profile type can be used for all 2D structures as well as in the case of a 3D fiber structure type where it represents a radial profile.

1D/2D Height Profile

This profile type describes the height of a component as a function of x and/or z . This is defined as:

$$\begin{aligned} H(x') &= h f(x') \\ H(x', z') &= h f(x', z') \end{aligned} \tag{5}$$

where h is the local segment height and $H()$ is the actual component height. This profile type is used by both the rib/ridge and multilayer 3D structure types. This functionality enables the use of a surface relief data file to define a surface in the RSoft CAD. The `coatuf` utility described in Appendix E can be used to add coatings to these surfaces.

Other Profile Types

It is possible to achieve other profile types, including anisotropic user-profiles (1D/2D/3D) and user profiles for epsilon (above profiles are for refractive index). Please contact photonics_support@synopsys.com for more information.

6.A.3. Creating & Using User Profiles

User profiles are defined in the User Profile Editor dialog which can be opened by clicking the **Profiles...** in a segment properties dialog or by via a button on the left CAD toolbar. Once a profile has been defined, it can be assigned to a particular component via **Index Profile Type** in the component properties window.

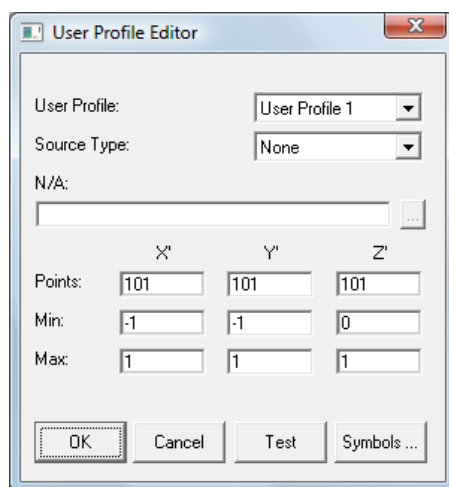


Figure 6-1: The User Profile Editor where User Profiles are defined.

The profile functions in Eqs. 2-5 can either be defined by either an expression or a data file. Once a profile has been defined, it can be viewed in the normalized coordinate system by clicking the **Test** button.

Expression

Set the **Source Type** option shown in Fig. 6-1 to *Expression* to define a profile via a mathematical expression. When $f(x', y', z')$ is defined via an expression, several options exist which allow for further customization of the profile.

- See [Appendix C](#) for a complete discussion of the format and capabilities of RSoft expressions.
- Enter all coordinate variables as x, y, and/or z. The program interprets x as x', etc.
- The expression is interpreted based on the coordinates used. For example, a non-radial 2D profile must have at least one x and y coordinate in the expression; if only x is found the

CAD will interpret it as a radial function. For cases like this, ‘missing’ coordinates can easily be included as $0 \cdot y$, etc such as:

$$f(x', y') = g(x') + 0 \cdot y' \quad (6)$$

- The number of points used to evaluate the expression can be set via the X', Y', and/or Z' **Points** options. This can be useful to resolve rapidly changing functions or periodic functions.
- By default, the variables x' , y' , and z' are defined only within the component. From (1), they are therefore normalized so that they vary between -1 and 1 (x and y) or between 0 and 1 (z). For the case of a 1D radial profile, they vary between 0 and 1. Outside of this domain, the function $f(x', y', z')$ is set to 0.

In some cases, it is useful to define the profile outside of this domain. This can be set by changing the **Min** and/or **Max** values for X', Y', and/or Z'. These settings are in the normalized coordinate system defined in (1).

- For many custom structures such as circles, cylinders, spheres, tori, etc, symmetry along the z axis is required and it is useful for the z' coordinate to vary from -1 to 1 like the x' and y' coordinates. This can be done by replacing z' with $(2z'-1)$ in the user profile. This results in a new definition of the normalized coordinate z' as $z' = 2z/l$.
- It is possible to interpret a user-defined profile absolutely, that is without refractive index or coordinate normalization, by enabling the option discussed in [Section 6.D.5](#). In this case, the profile is interpreted absolutely but it is still translated to the component center.
- Finally, it is recommended that the function $f(x', y', z')$ be normalized to vary between 0 and 1 when x' and y' vary between -1 and 1 and z' between 0 and 1. This additional restriction will ensure that the maximum value of the index difference in the profile will correspond to the local **Index Difference** of the component. While not required, this can simplify the definition of a profile.

Data File

Set the **Source Type** option shown in Fig. 6-1 to Data File to define a profile via a data file,

See [Appendix B](#) for a discussion of data file formats.

When a data file is used to define the function $f(x', y', z')$, the variables x' , y' , and z' are given by (1) and x , y , and z are defined over the range given in the data file.

It can be useful to remove the normalizations in (1) by setting the component height and width equal to 2 and set $\Delta n = 1$ so that (2) reduces to

$$n(x', y', z') = n_0 + f(x, y, z)$$

Because it is usually not possible to set n_0 , the data contained in the file should be altered so that the values are less than their actual amount by n_0 . For more information about generating a file for use as an index profile, please see the example in the next section.

Note that the **Min**, **Max**, and **X**, **Y**, and **Z Points** options in the User Profile window are not used when using a data file to define a profile. The Min/Max values can be used however. The data file can be dynamically loaded at run time by entering the filename as `$name` and setting the variable `name` to the actual filename of the profile.

6.A.4. Example User Profile

As an example, consider the parabolic index profile given by

$$f(x') = \begin{cases} 1 - x'^2 & |x'| \leq 1 \\ 0 & |x'| > 1 \end{cases}$$

Using an Expression

To define this profile via an expression, simply enter the following formula in the appropriate field of the dialog:

$$1 - x^2$$

Note that the variable name x in this expression represents the normalized variable x' which ranges from -1 to +1 over the width of the segment. The domain of the profile can be changed via the **X Min/Max** options shown in Fig 6-1. The range [-1,1] still maps to the width of the segment, however. Also note that outside the domain of definition the function is set to zero.

Using a Data File

To define the above profile via a data file in the standard RSoft file format, first create a text file in the standard 2D output file format for real data, as described in [Appendix B](#). The following data file defines this profile on a coarse grid containing 11 points:

```
/rn,a,b/nx0
11 -1 1 0 OUTPUT_REAL
0.00
0.36
0.64
0.84
0.96
1.00
0.96
0.84
0.64
```

0.36
0.00

During a simulation, the CAD will interpolate between the points in the data file if necessary. This example is illustrated in the example file

<rsoft_dir>\examples\BeamPROP\userprof.ind. More information about the file format for user profile data files can be found in [Appendix B](#).

6.B. Tapers

A taper function allows a segment's width, height, index, and position to be varied or tapered along the segment. The form of a taper function depends on the Segment Type ([Section 4.F](#)):

- *z-Axis/Seg-Axis Segments*: A taper function has the form

$$a(z) = a_0 + (a_1 - a_0) f(z') \quad (8)$$

$$z' = \frac{z - z_0}{z_1 - z_0} \quad (9)$$

where $a(z)$ represents the segment property to be tapered, a_0 and a_1 are the parameter values at the starting and ending vertex, $f(z')$ is the taper function, and z' is a normalized coordinate along z that varies from 0 to 1 along the Z extent of the segment. Z -position tapers are not possible for these segment types; use an extended segment.

As an example, a width taper would be defined as:

$$w(z) = w_0 + (w_1 - w_0) f(z') \quad (10)$$

where $w(z)$ represents the segment width, w_0 and w_1 are the width at the starting and ending vertex, and $f(z')$ is the taper function.

- *Extended Segments*: A taper function have a more general form of the previous definition:

$$a(z) = a_0 + (a_1 - a_0) f(t) \quad (11)$$

where $a(z)$ represents the segment property to be tapered, a_0 and a_1 are the parameter values at the starting and ending vertex, $f(t)$ is the taper function, and t is an arbitrary parametric coordinate that varies from 0 to 1. By using X , Y , and Z tapers, it is possible to define an

arbitrary space curve. Note that while the definition of the taper in Eq. (11) are a function of t , any user-defined tapers should be entered into the GUI as a function of z .

6.B.1. Component Options that Support Tapering

Several options in the RSoft CAD support tapering. This includes the segment properties (Fig 6-2: **Index Taper**, **Width Taper**, **Height Taper**, **X Pos Taper**, **Y Pos Taper**, and **Z Pos Taper**), some Extended Segment Properties ([Section 6.K.2](#)) and the Crystal Axes specifications ([Section 6.K.3](#)).

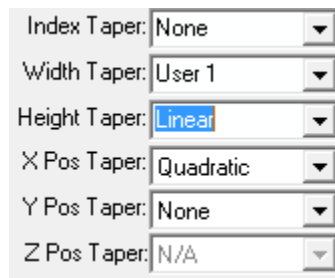


Fig 6-2: The portion of the Segment Properties dialog where segment tapers are set. Tapers are also used elsewhere, including for Extended Segment Properties and Crystal Axes.

The Index Taper, Width Taper, and Height Taper are controlled by editable drop-down lists which allow them to not only be set via the drop-down, but also by a Symbol Table expression. See [Section 3.J](#) for details about how to use this feature. The built-in values for these taper types are:

Taper List Option	Built-In Value
<i>None</i>	TAPER_NONE
<i>Linear</i>	TAPER_LINEAR
<i>Quadratic</i>	TAPER_QUADRATIC
<i>Exponential</i>	TAPER_EXPONENTIAL
<i>User 1</i>	TAPER_USER_1

6.B.2. Built-In Taper Types

The choices for the taper function are:

- None

$$f(z') = 0$$

- Linear

$$f(z') = z'$$

- Quadratic

$$f(z') = z'^2$$

- Exponential

$$f(z') = \frac{e^{Bz'} - 1}{e^B - 1}$$

where B is defined as

$$B = \text{sign}(p_0 - p_1)$$

where p_0 and p_1 are the starting and ending values of the tapered parameter respectively.

- New Taper... and User #

These choices allow the user to specify the form of the taper function arbitrarily and are described in [Section 6.B.4](#).

- Arc (XZ)

This choice indicates that the segment path follows a curved path in the XZ plane and is discussed in [Section 6.B.3](#).

6.B.3. Built-In Arc Taper Types

The properties for an Arc (XZ) taper are set in the Arc Properties dialog which can be opened by clicking the **Arc Data...** button in the segment properties dialog.

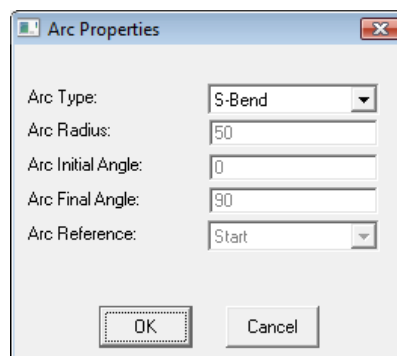


Figure 6-3: The Arc Properties dialog.

The possible **Arc Type** settings are:

- Free

The **Arc Radius**, **Arc Initial Angle**, and **Arc Final Angle** of the circular arc can be arbitrarily set. The program automatically calculates the ending vertex of the segment based on these parameters and the starting vertex.

- Radius

The **Arc Radius** of the circular arc can be set and the program fits the arc to lie through the segment's starting and ending vertices.

- Initial Angle

The **Arc Initial Angle** of the circular arc can be set and the program fits the arc to lie through the segment's starting and ending vertices.

- Final Angle

The **Arc Final Angle** of the circular arc can be set and the program fits the arc to lie through the segment's starting and ending vertices.

- S-Bend

The program fits together two circular arcs to produce an s-bend through the segment's starting and ending vertices.

- Cosine S-Bend

The program uses a cosine taper function for the segment:

$$f(z') = \frac{1}{2}(1 - \cos(\pi z'))$$

- Raised Sine S-Bend

$$f(z') = z' - \frac{\sin(2\pi z')}{2\pi}$$

To recreate these tapers with a user-defined taper, the arguments for the trigonometric functions in the above equations need to be rewritten in terms of degrees.

Arc Reference Point

The **Arc Reference** sets the reference point of the arc to be either at the starting vertex or at the center of the arc. Putting the reference point at the center is recommended for arcs used to define rings, toroids, and helixes so that they can be easily positioned, rotated and translated.

6.B.4. Creating & Using User Tapers

A user taper can be defined in the User Taper Editor window which can be opened by pressing the **Tapers...** in a segment properties window or by via a button on the left CAD toolbar. Once a

taper has been defined, it can be assigned to a particular segment via the pull down menus in the segment properties dialog.

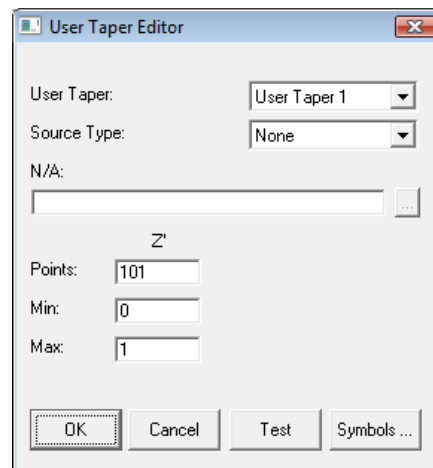


Figure 6-4: The User Taper Editor where User Tapers are defined.

The taper function found in (8) can either be defined by either an expression or a data file.

Expression

To define a taper via a mathematical expression, set the **Source Type** option shown in Fig. 6-4 to *Expression*. When $f(z')$ is defined as an expression, several options exist which allow for further customization of the taper.

- For a complete discussion of the format and capabilities of RSoft expressions, refer to [Appendix C](#).
- Enter all coordinate variables as x , y , and/or z . The program interprets x as x' .
- The number of points used to evaluate the expression can be set via the **Z' Points** options. This can be useful to resolve rapidly changing functions or periodic functions.
- By default, the variable z' is defined only along the length of the segment. From (9), it is therefore normalized so that it varies between 0 and 1.

In some cases, it is useful to define the taper outside of this domain. This can be set by changing the **Min** and/or **Max** values for Z' . These settings are in the normalized coordinate system defined in (1).

- Finally, it is recommended that the function $f(z')$ be normalized to vary between 0 and 1 with z' between 0 and 1. This additional restriction will ensure that values for the starting and ending segment parameters retain their original meaning. While not required, this can simplify the definition of a taper.

Data File

To define a taper via a data file, set the **Source Type** option shown in Fig. 6-4 to Data File. When a data file is used to define the function $f(z')$, the variable z' is given by (9), and is defined over the range given in the data file. For more information on the use of a data file, see the example in the next section. Also, the data file can be dynamically loaded at run time by entering the filename as $\$name$ and setting the variable `name` to the actual filename of the profile.

Grating

This taper type is used to quickly define grating structures for the simulation tool GratingMOD. Its use is discussed in detail in the GratingMOD manual.

6.B.5. An Example User Taper

As an example, consider the parabolic index taper given by

$$f(x') = 1 - z'^2 \quad 0 \leq z' \leq 1$$

Using an Expression

To define this taper via an expression, simply enter the following formula in the appropriate field of the dialog:

$$1 - z^2$$

Using a Data File

To define the above taper via a data file using the standard RSoft file format, first create a data file in the standard 2D output file format for real data, as described in [Appendix B](#).

The following data file defines this taper on a coarse grid containing only 11 points:

```
/rn,a,b/nx0
11 0 1 0 OUTPUT_REAL
0.00
0.36
0.64
0.84
0.96
1.00
0.96
0.84
0.64
0.36
0.00
```

Note that during a simulation, the CAD will interpolate between grid points if necessary.

6.C. Priority (When Components Overlap)

The creation of complicated structures in the CAD sometimes involves overlapping several components.

It is crucial to understand how the CAD treats overlapping components by default and how to override this behavior when necessary.

By default, when one or more components overlap, the program merges the components and uses the index of the component with the higher absolute index difference at each grid point. Several options exist to override this default behavior. They are:

- **Combine Mode** - This option indicates the method in which overlapping components are evaluated.
- **Merge Priority** - This option sets the relative importance between components when a **Combine Mode** of Merge is used.

Both of these settings can be made in a component's properties window.

6.C.1. Combine Mode

The **Combine Mode** of a component has three settings:

- **Default** - This option automatically switches between Merge and Add based on the structure type.
- **Merge** - This is the default setting for all structure types except diffused, and corresponds to the use of a priority scheme in order to determine which index is used. See the next section for more details on priority.
- **Add** - This is the default setting for diffused structures. This option allows the component to act as a local index perturbation: the index is simply added to the background and any other components. For 3D structures, the structure type of the component being added (i.e. that has **Combine Mode** set to Add selected) is restricted to Fiber, Channel, or Diffused structure types. For example, to create a combination of Multilayer and Diffused profiles, the user can select Multilayer as the default structure in the Global Settings, and add Diffused components using **Combine Mode** Add. However, the reverse is not currently possible.

6.C.2. Priority

Component priority is used when **Combine Mode** is set to Merge. The priority scheme used to determine the index at a point where more than one component is defined is:

- If the segments have the same **Merge Priority** setting, the segment with the higher absolute index relative to the background is used.

- If the segments have a different **Merge Priority** setting, the segment with the higher priority is used.

By default, every component has a priority of 0, and so the first rule is used. However, this can be overridden by changing the priority of one of the components that overlap. The actual value of the **Merge Priority** itself does not matter, only the relative priority of a component to other components in a design.

This option is only available for 2D Step-Index and User-Defined profiles, and 3D Fiber, Channel, or Multilayer geometries; if it is needed for other circumstances, please contact RSoft.

6.C.3. General Notes

- Using either Combine Mode Merge or Combine Mode Add to mix structures can, by design, yield very different results. To ensure that the desired structure has been obtained, the user should view the resulting index profile as described in [Section 3.G](#).
- For multilayer structures, the desired geometry might not be realized when using overlapping components. This can usually be fixed by setting the height of the multilayer components to the exact height of the defined structure in assigned the layer table and/or using priority.

6.D. Global Index Generation Options

The CAD has several options for perturbing the index profile to achieve special effects, such as simulating a curved space, truncating the index distribution at a boundary, or incorporating imperfections into the index distribution, such as finite lithographic resolution or roughness. These are controlled through the Global Index Generation Options dialog which can be accessed via the Options/Index Generation... menu item.

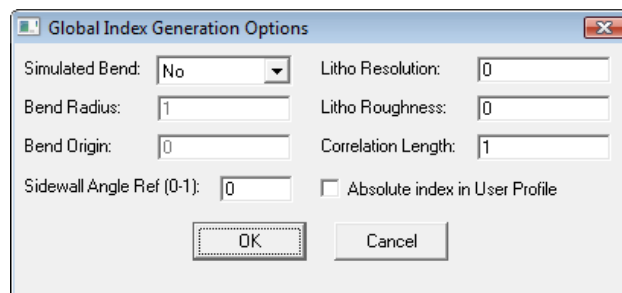


Figure 6-5: The Global Index Generation Options window which contains several options to perturb the index profile to achieve special effects.

6.D.1. Simulating Curved Segments to Large Angles

This feature is primarily used in the simulation tool BeamPROP, and uses a conformal index mapping technique to transform a curved segment geometry into a straight segment geometry. It can also be used to find bending modes in FemSIM. The Global Index Generation Options window sets this feature globally for the entire simulation domain. This feature can also be enabled locally for a segment to handle multiple bends or a series of bends and straight segments via the analogous settings in the Advanced Component Properties dialog which can be opened via the **More...** button in the Component Properties dialog. The method used is based on:

R. Baets and P. E. Lagasse, “Loss calculation and design of arbitrarily curved integrated-optic waveguides”, *J. Opt. Soc. Am.*, **73** (1983)

Motivation

Direct simulation of curved segments to large angles is difficult, even with the Padé-based wide-angle capability available in the simulation tool BeamPROP. This is due to the fact that there are several factors in addition to the obvious paraxiality limitations that must be considered. These include the need for finer grid and step sizes as well as a large computational domain, both of which can increase the computation time significantly. There is, however, an alternate approach which is appropriate for modeling of a single bend or a series of bends (as in an s-bend for example). This technique makes use of a coordinate transformation to map a curved segment, onto a straight segment.

Implementation

The CAD implements the above-mentioned technique, which is accessed via the Simulated Bend fields in the special effects dialog for the case of a single bend; multiple bends are discussed below. When the **Simulated Bend** input field is set to Yes, the entire problem space is transformed to effectively follow a circular arc, with a radius defined by the value in the numeric input field labeled **Simulated Bend Radius**. This is done by transforming the refractive index profile through multiplication by $(1+x/R)$. This method has been demonstrated to be accurate when $w \ll R$, where w is the width of the segment and R is the radius of curvature.

The **Bend Origin** controls how the radius is measured. The center of the bend is located at $x = -R$, and so if the outer edge of a waveguide is to be located at a radius or R , that edge should be positioned at $x = 0$ in the CAD. If the waveguide has a width of w and is centered at $x = 0$, then setting **Bend Origin** to $w/2$ will effectively position the outer edge of the waveguide at R . In this mode, the length of the waveguide becomes the arc length.

For diagnostic purposes it can be useful to see the perturbed index profile. This can be done by setting the variable `index_profile_transform` to 1.

When using this feature globally via the Global Index Generation Options dialog in a design file consisting of a single straight segment located at $X=0$, the Z axis then represents arc length. When using this feature locally for each segment, as would be done to model an s-bend, one

would layout two straight segments in series, and assign the first segment a positive radius indicating a bend curving to the left, while the second segment would be assigned an equal but negative radius corresponding to curving to the right. If desired, one segment can be offset laterally from the other to improve alignment of the curved segment modes, which are generally displaced to the outside of the bend.

6.D.2. Sidewall Angle Control

The sidewalls of a channel or multilayer structure can be tilted using the **Sidewall Angle** option in the Additional Component Properties dialog as discussed in [Section 6.K](#). By default, however, the width of the component is assumed to be constant at the bottom. This can be changed by setting the **Sidewall Angle Ref.** option in the Global Index Generation Options window. A value of 0 corresponds to the bottom and is the default, 1 corresponds to the top. Any value between 0 and 1 can be used.

6.D.3. Simulating the Effects of Finite Lithographic Resolution

The CAD has the ability to simulate the effects of finite lithographic resolution by automatically rounding all component geometry to a grid, as if the circuit were drawn by filling in squares on a piece of graph paper. To enable this feature, enter the desired grid resolution in the **Litho Resolution** field of the Global Index Generation Options window. This feature is also accessed via the `litho_res` variable in the symbol table. The default value of this variable is 0, corresponding to perfect lithographic resolution.

This option only affects the geometry used in the field propagation. The perturbed geometry is not used in the CAD display of the circuit, nor is it used in any of the analysis monitors, although it influences their values through the field.

This feature deals with a systematic perturbation of the index profile; random perturbations, due to processing-induced roughness for example, are discussed in the next section.

6.D.4. Simulating the Effects of Lithographic Roughness

The CAD has the capability to randomly perturb a component's sidewalls in order to simulate the effect of imperfections due to lithographic roughness. There are two parameters, **Litho Roughness** and **Correlation Length**, which control this feature. They are specified in the Global Index Generation Options dialog. This section discusses the definition of these parameters and how the program uses them to calculate the random perturbations.

“Theory of Dielectric Optical Waveguides”, by Dietrich Marcuse, Chapter 4.6 in 2nd edition.

Theoretical Background

The two parameters **Litho Roughness** and **Correlation Length** are used to describe the random perturbation produced by the CAD. To better visualize the definitions of these parameters, an

autocorrelation function will be introduced. These parameters will be defined in the context of the autocorrelation function, and then related to the perturbation function.

An autocorrelation function is used to describe a stationary random process, and measures the correlation, or similarity, between a perturbation function $h(z)$ and the same perturbation function offset a distance u in space. The autocorrelation function $R(u)$ of a function $h(z)$ is defined as:

$$R(u) = \int_{-\infty}^{\infty} h(z)h(z-u) dz \quad (11)$$

This function $R(u)$ has its maximum value at $u=0$. As the value of u is increased, $R(u)$ decreases. For large values of u , $R(u)$ approaches zero since there is no longer any correlation between the function at the two sample points z and $z+u$.

Two parameters are defined to characterize the statistical variation:

Litho Roughness

This parameter can be defined as

$$\bar{\sigma} = \sqrt{R(0)}$$

Since it is assumed that $h(z)$ averages to 0 over large distances, the square of the lithographic roughness is the variance of $h(z)$. The lithographic roughness is also called the rms deviation.

Correlation Length

This is the length $u=D$ at which the correlation function $R(u)$ has decreased to a fraction of its maximum value - usually to $1/e$. It is a measure for the length over which the perturbations are correlated to each other, and is therefore the scale length of the random perturbation. Within this length, $h(z)$ is not changing randomly.

In the context of the autocorrelation function, these two parameters hold intuitive meaning and can easily be visualized. The following autocorrelation function is used as a convenient model for many cases:

$$R(u) = \bar{\sigma}^2 e^{-\frac{|u|}{D}} \quad (12)$$

The Perturbation Functions

In the RSoft CAD, a random perturbation of a component's side walls is created in order to model lithographic roughness. To do this, both the width and position of the component are perturbed, which results in a non-symmetric perturbation. The width and position taken into account during a simulation are:

$$\begin{aligned}x(z) &= x_0(z) + \Delta x(z), \\w(z) &= w_0(z) + \Delta w(z).\end{aligned}\tag{13}$$

The width and position are given as a function of w_0 and x_0 , the unperturbed width and position, and associated perturbations $\Delta x(z)$ and $\Delta w(z)$. Both the width and position perturbations are calculated in the same manner. The program constructs the functions $\Delta x(z)$ and $\Delta w(z)$ such that their resulting autocorrelation functions, as defined in (11), are approximated by (12). In this way, a component with perturbed sidewalls characterized by the **Litho Roughness** and the **Correlation Length**, is created.

This option only affects the geometry used during a simulation and not in the CAD window display of the circuit. Also, a current limitation of this feature is that similar component in a structure at a given z will have the same wall perturbation. This restriction can be relaxed in the future based on user demand.

Additional Comments

This feature can be accessed via the variables `litho_sigma` and `litho_corlen` in the symbol table. The default value of `litho_sigma` is 0, corresponding to no roughness and the default value of `litho_corlen` is undefined. Also, this option only affects the geometry used in the field propagation. The perturbed geometry is not used in the display of the circuit, nor is it used in any of the analysis monitors, although it influences their values through the field.

This feature deals with a random perturbation of the index profile; systematic perturbations, due to finite lithographic resolution for example, are discussed in the previous section. Additionally, additional types of lithographic roughness can be created quite easily. The above formulation is nothing more than a position taper and width taper with a built-in pseudo random function. This effect can be recreated with any functions of the users choosing as described in [Section 6.B](#), thus allowing the user additional control over the type of roughness produced.

6.D.5. Interpreting User Profiles Absolutely

By default, user-defined profiles (see [Section 6.A](#)) normalize both the coordinate and refractive index information to achieve the final profile. While this is useful for many cases, it is also useful to interpret a user-defined profile absolutely. To enable this option, select the **Absolute index in User Profile** option in the Global Index Generation Options window.

When this option is used, it is important to note that the profile is translated from the coordinates defined in the expression/data file to the component center. The easiest way to directly interpret the defined coordinates, simply position the component with the user-defined profile at the origin.

This option sets several advanced sub-options which, if needed, can be controlled separately:

Symbol	Description
<code>user_profile_scale_coords</code>	Set to 1 to control the scaling of the X and Y coordinates.
<code>user_profile_scale_coordsz</code>	Set to 1 to control the scaling of the Z coordinate.
<code>user_profile_offset_coords</code>	Set to 1 to control the offset of the X and Y coordinates.
<code>user_profile_offset_coordsz</code>	Set to 1 to control the offset of the Z coordinate
<code>user_profile_scale_index</code>	Set to 1 to control the scaling of the index by the index difference.
<code>user_profile_offset_index</code>	Set to 1 to control the offset of the index by the background index.

6.E. Geometry Import/Export

The RSoft CAD has the ability to import and export files in both RSoft or other CAD formats. The supported formats are:

- Export: GDS-II, DXF
- Import: GDS-II, DXF, CIF, IND (Synopsys RSoft), TDR (Synopsys TCAD Sentaurus)

6.E.1. Export

The export options are found under the File/Export menu item.

Selecting GDS export will open the Export GDS dialog which sets the output **File Name**, the **Cut Direction** and **Cut Position**, and **Clipping** options. The default clip position corresponds to the simulation domain. Selecting DXF will save a `.dxf` mask file of the XZ structure with the same name as the `.ind` file. This can also be done via the command line utilities described in [Section E.K](#).

See [Section 6.E.3](#) for details about output to a specific layer and other important notes.

6.E.2. Import

There are three methods to import geometry:

- Convert a GDS-II, DXF, or CIF file into an RSoft `.ind` design file:

The options found under the **File/Import** menu item provide a one-time conversion from a GDS-II, DXF, or CIF mask file to an RSoft .ind design file. This can also be done via the command line utilities described in [Section E.K](#).

- Dynamically reference a GDS-II or IND using a Circuit Reference component:

This option embeds a child design file (.gds or .ind) in a parent .ind file via a Circuit Reference. Where appropriate, a specific layer can be imported. See [Section 6.F](#) for more details about Circuit References.

- Use Exact Geometry data in a TCAD Sentaurus TDR file in an RSoft .ind design file:

See the Multi-Physics Utility manual for details about importing TDR geometry data from a Synopsys TCAD Sentaurus simulation.

6.E.3. Import/Export Settings and Important Notes

The primary options controlling related to import/export are contained in the Import/Export tab of the CAD Preferences dialog.

Details for the rest of the CAD preferences can be found in [Section 6.J](#).

Import/Export Resolution

Sets the resolution at which CAD objects are “fractured”, or converted to polygons in the target CAD file. This resolution is defined as the maximum distance between any chord of the fractured object and the true curve. This option is relevant for both import and export. This setting is used when segment-axis segments are used in a simulation and also controls the “fracturing” of curved surfaces. It affects both the CAD display and simulation results. See [Section 4.F](#) for information about Segment Orientation.

Export Mask Layers

Sets the mask layer to be exported, and is used along with the **Mask Layer** option in the Additional Component Properties window. See [Section 6.K](#) for details. When this option is set to 0, components are exported to layers that correspond to the **Mask Layer** for each component. When this option is not set to 0, all components with a **Mask Layer** of 0 will be exported to the value specified in the **Export Mask Layers** field.

Export Polygon Mode

This controls how the CAD exports polygon objects within the CAD. The Raster option causes the CAD to break a polygon into smaller polygons so as to satisfy the 200-point GDS-II limit for polygons. The Raw option exports polygons as they are, without breaking them up.

Notes:

It is important to note several aspects of the import/export process:

- When a mask file is imported into the RSoft CAD, all supported objects are converted to RSoft polygons. Similarly, when the RSoft CAD exports a mask file, all RSoft CAD objects are converted to polygons in the target file format. This avoids possible ambiguities since polygons have a fairly consistent interpretation among different file formats and software packages. As such, only 2D formats can be imported.
- RSoft CAD's imports the following objects from a DXF file: LINE, SOLID, POLYLINE, LWPOLYLINE, ARC, and CIRCLE. The following objects can be imported from a GDS-II file: GDS_BOUNDARY, GDS_BOX.
- Setting the **Mask Layer** of a specific component to a negative number will exclude that component from export.

6.F. Circuit References

Circuit references, or hierarchy, embed one design (the child) within another design (the parent). This nesting can be multiple levels deep. Multiple file formats are supported:

- RSoft `.ind` files, symbols can be passed from the parent to the child for parametric control.
- GDS `.gds` mask files, a specific layer can be specified.
- Python `.py` scripts that generate an RSoft `.ind` file. See [Section 10.A](#) for information about using Python scripts to create `.ind` files. Symbols can be passed from the parent to the child for parametric control.
- TCAD Sentaurus `.tdr` files created by SDE or S-Process.

The figure below shows an example circuit reference shown with a shaded bounding box. Elements in the child circuit are not directly editable but double clicking on the child circuit will open the child file.

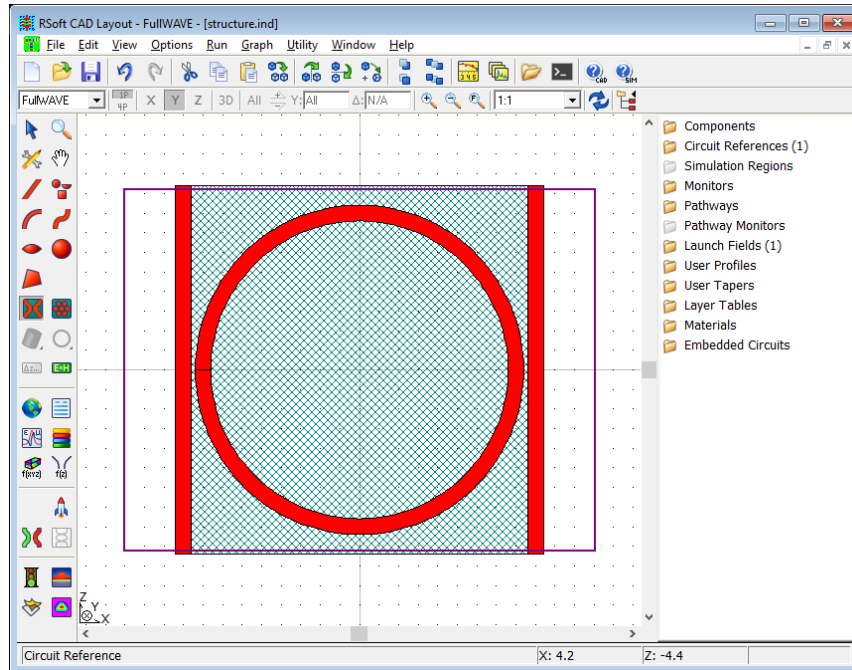


Figure 6-6: Example Circuit Reference in the RSoft CAD. The ring resonator structure is defined in a child circuit. The hatched blue box outlines the circuit reference.

The files can be embedded dynamically (remain as two files) or statically (combine two files into one). Furthermore, this feature allows for child circuits to be periodically repeated in the parent circuit allowing for dynamically sized arrays of objects.

6.F.1. Creating a Circuit Reference

To add a child circuit to a parent circuit, use the **Circuit Reference** button in the left toolbar of the CAD. You will be asked to select the child file (.ind, .gds, or .py) from the current working directory. Note that a special shortcut is created at the top of the directory listing so that you can easily select from RSoft's structure library, select either one of the built-in files or any other file. Once the file is selected, the Circuit Reference properties dialog is shown.

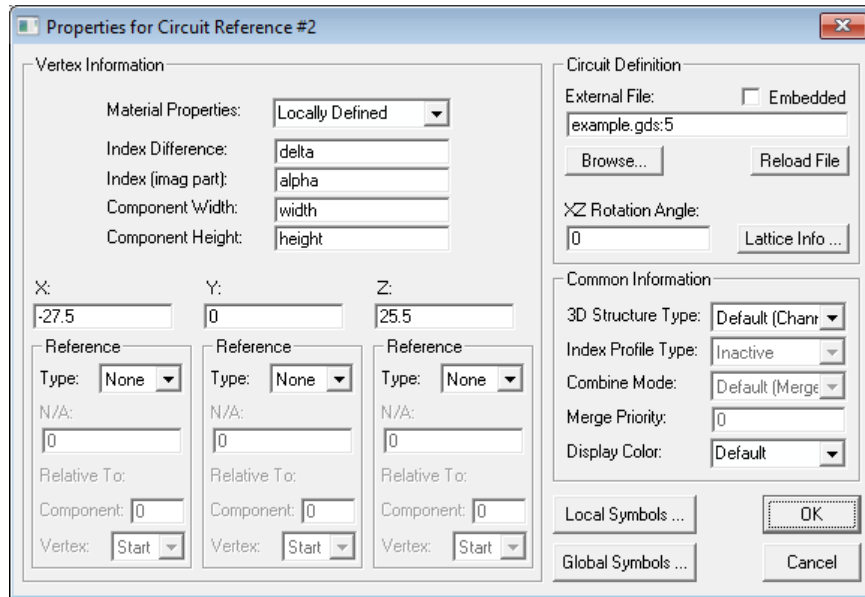


Figure 6-7: The Circuit Reference properties dialog box.

6.F.2. Setting Circuit Reference Properties

There are three main categories of properties: the circuit definition, the position, and the common information.

Circuit Definition:

The circuit definition options are:

Embedded File or External File

This **Embedded File** or **External File** field (the name changes depending on whether the **Embedded** option is selected) specifies the file for the child.

The syntax for the filename depends on the file type:

- *RSoft .ind file:*

Enter the name of the child `.ind` file. The Common Information described below can be used to override the defaults in the child file, symbols/variables in the child file can be set from the parent as described in [Section 6.F.5](#).

- *GDS file:*

Enter the name of the child `.gds` file. The common Information described below can be used to set the material, color, and profile of the components in the child file. See [Section 6.F.8](#) for more options.

- *Python script:*

Enter the name of the child `.py` file. This file will be run using the version of Python included with the RSoft tools to produce an `.ind` file. See [Section 10.A](#) for more details. Note that symbols defined in the Local Symbols table that have names like `ArgN` are passed to the `.py` script as arguments, see [Section 6.F.6](#).

- *TDR Boundary file:*

The Import TDR option described in the Multi-Physics and TCAD Sentaurus Interface manual can be used to automatically import TDR files.

Enter the name of the child `.tdr` file. The file will be dynamically referenced. For 2D TDR files, the syntax `example_bnd.tdr,xy` can be used to orient the TDR file in the XY plane.

Embedded

This option selects if the child file (`.ind` only) is stored within the parent file. You can browse the list of files currently stored in the parent file in the pulldown list in the Circuit Reference dialog, or in the **Embedded Circuits** folder in the tree view on the right of the RSoft CAD. To save a copy of an embedded file as a separate file, double-click on the circuit reference component to open the child file and choose File/Save As from the CAD menu.

XZ Rotation Angle

Sets the angle [degrees] of the child circuit in the XZ plane of the parent circuit, and is measured from the positive Z axis. Typically this angle is set locally, see [Section 6.M](#) for details about inheriting this setting from a referenced component.

Lattice Info...

This control allows for the creation of dynamically sized arrays and is discussed in [Section 6.G](#).

Position & Common Information:

The right hand side of the dialog defines the Starting Vertex. The starting vertex is defined as the position of the child circuit's origin ($X=Y=Z=0$) in the parent circuit. The specification of the starting vertex is similar to the standard component dialog, and is not repeated here. Note that when using the dynamic array option, it is recommended to center the unit cell in the child circuit to simplify the lattice position in the parent circuit. Note that it is also possible to add additional vertices to the circuit reference, useful for connecting components to ports or other locations of interest; see [Section 6.F.7](#).

The left hand side of the dialog defines the Common Information. This includes the **3D Structure Type**, **Index Profile Type**, **Combine Mode**, **Merge Priority**, and **Display Color**. These options override the global default settings in the child file. Note:

- Only components that have default values for these options will be overridden. If a component in the child file has one of the above options specifically set (i.e. **3D Structure Type** set to *Channel*), then it will not be overridden by these settings.
- Note that **Merge Priority** will only be used if all components in the child circuit have a 0 priority (i.e. the default).

6.F.3. Editing a Circuit Reference

Elements in the child circuit are not directly editable, double click on a child circuit to open its child file and edit if appropriate. Child circuits are drawn in the parent circuit with a shaded bounding box, the display properties of this box can be set by the **Circuit Reference Box** preference described in [Section 6.J](#). Child circuits that are defined by scripts will open the script in a default text editor, set the environment variable `RSOFT_EDITOR` to the path of the text editor of your choice.

6.F.4. Controlling Variables in Child File From Parent File

Hierarchy allows for variables to be inherited by the child circuit from the parent circuit. In order to do this, the Circuit Reference dialog has two symbol tables: a Local Symbol table and a Global Symbol table.

Global Symbols...

This button opens the symbol table in the parent file, and behaves as expected. This is the same symbol table that is accessed via the **Edit Symbols** button on the left menu in the CAD interface.

Local Symbols...

This button pulls up a special type of symbol table, in which the variables on the left will override values in the child, while the expressions on the right are evaluated in the context of the parent's symbol table. By default, expressions such as "`width = width`" are defined, which indicate that the child's "`width`" variable (on the left) is assigned the value of the parent's "`width`" variable (on the right). This allows the variables in the parent circuit to be used to control variables in the child circuit. Local symbols can be passed to `.py` child script files, see [Section 6.F.6](#).

Symbols from the child circuit can be exported to this special symbol table using the Hierarchy Export options described in [Section 6.F.5](#). This allows easy control of more common aspects of the child file design to automatically 'bubble' up into the parent file. The square bracketed expression format `[value]` is used to denote that the value from the child file should be used in the parent file. The value between the square brackets will be dynamically updated to show the current value. See [Appendix C](#) for more details on symbol table expressions.

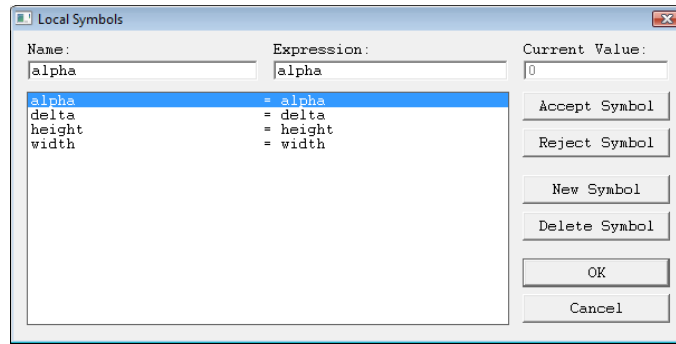


Figure 6-8: Local symbol table showing assignment of the values of parent circuit's variables (right column) to children circuit's variables (left column).

Which Settings Take Precedence?

When a child file is used in a parent file, the following rules are used to determine which settings take precedence:

- Any symbols defined in the Local Symbols table described above will be used in the child file in the final layout.
- In general, any parameters that are set to default values in the child file are overridden by the settings in the parent file. One example of this is the **3D Structure Type**. The most typical configuration (which is the default) of the 3D Structure Type is that it is set in the Global Settings, and individual components are set to this default. If this default is different in the child file and the parent file, then the parent file's value is used in the final layout. For example, if the **3D Structure Type** is set to *Fiber* in the child file, and a component's local **3D Structure Type** is set to *Default* in the child file, and the **3D Structure Type** is set to *Channel* in the parent file, the component will be a channel in the final layout.
- Some parameters in the child file are always overridden by the settings in the parent file out of necessity. These parameters include the **Model Dimension**.
- Some properties of the design file must be defined in both the parent and child file, including materials, tapers, and profiles.

6.F.5. Hierarchy Exports

The Local Symbol table provides an important link between the child and parent files when using hierarchy. Hierarchy exports provides an important capability to this feature, the ability for child files to automatically export or broadcast a list of symbols which will be automatically added to the Local Symbol table. This allows important control variables to be broadcast from the child file to the parent file to facilitate control and allow child files to more easily be reused in different circumstances.

Selecting Symbols to be Exported

The list of symbols that are exported are set at the child circuit level. Open the child circuit in the RSoft CAD and choose the Edit/Edit Hierarchy Exports... option from the RSoft CAD menu.

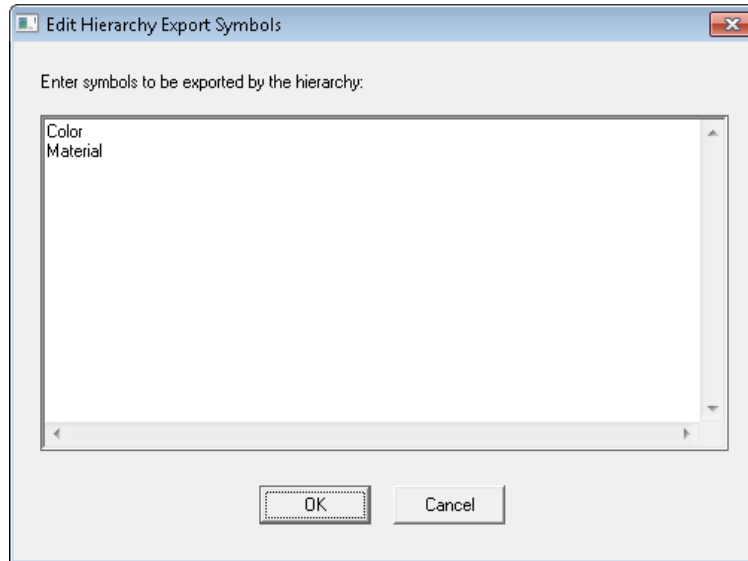


Figure 6-9: The Edit Hierarchy Exports dialog.

Once entered in the Edit Hierarchy Exports dialog, symbols are then exported to a parent file when the child file is used via hierarchy. Multiple symbols can be entered on separate lines as shown in the above figure.

Using Values in Child File for Exported Symbols

In some cases, it can be advantageous to set the value of an exported symbol to the value in the child file. The square bracket notation [xx] can be used in the Local Symbols table to indicate that the value in the child file should be used in the parent file.

Consider the example Local Symbol table shown below. The symbols `Color` and `Material` have been exported from the child to parent file, but, as configured, we wish for the default color and Material to be used in the child file. The value inside the square brackets is automatically updated based on the value set in the child file. In this case, the symbol `Color` is set to `-1` in the child file which indicates that the default color should be used and the symbol `Material` is set to `""` which indicates that the **Material Properties** setting should be set to *Locally Defined*. Note that both of these symbols are used in the child file to set the **Color** and **Material Properties** setting respectively.

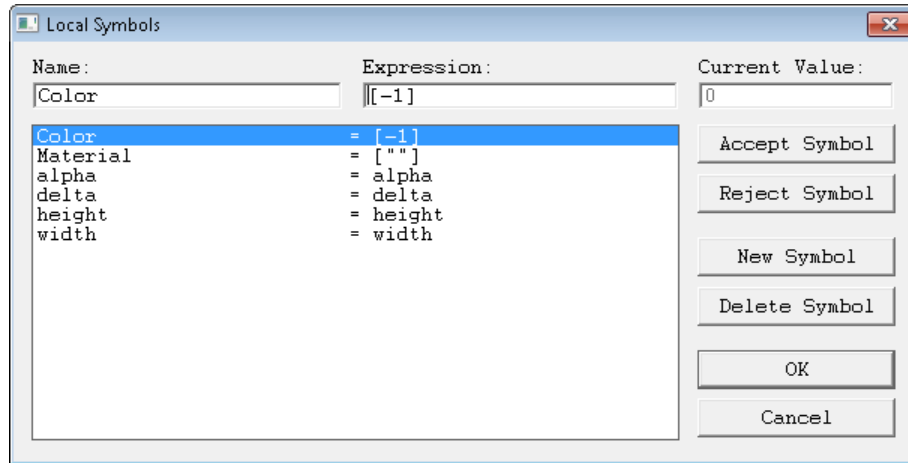


Figure 6-10: Illustration of the square bracket notation.

6.F.6. Creating Python Functions

While it is possible to implement a script that will parse arguments, it is recommended to use the `rsoft_layout()` function. The arguments of this function will automatically be added to the Local Symbol table with the `ArgX` prefix (i.e. `Arg1_name`). The function should return a `RSoftCircuit` object.

Automatically Parse Script Arguments:

The `rsoft_layout()` function can be used to automate argument parsing:

- Define the function `rsoft_layout()` which contains all the code necessary to produce the design file (.ind for a Python script). If present, this function will be automatically executed by the RSoft CAD when running the script for each circuit reference block. Note that the function must compile for this to work.
- The `rsoft_layout()` function should be defined with all arguments that will be passed to the script along with default values.
- Symbols in the Local Symbols table for the circuit reference that begin with the prefix `ArgX` will be automatically created, and then passed to the corresponding arguments in the `rsoft_layout()` function definition along with the design file name. For example, the following `rsoft_layout()` Python function declaration has two arguments:

```
def rsoft_layout(NumRings=int(10),
                 data_file=''):

```

The arguments will be automatically set by Local Symbols that start with `Arg1` and `Arg2`, say symbols `Arg1_N` and `Arg2_file`. Note that if using a string argument as a file name input to the Python API, the file must be present for the script to compile. In these cases it is safer to

use an if statement to check if the file exists (or is not set to '' as shown above) before using the Python API.

- As a note, it is possible to use any function name, so long as you include a comment after the function declaration with `rsoft_layout`, for example:

```
def any_function(NumRings=int(10),    //rsoft_layout
                 data_file=''):

```

Returning a Circuit Object in Python:

The Python script should return a `RSoftCircuit` object like:

```
def rsoft_layout(NumRings=int(10),
                 data_file=''):

    c = RSoftCircuit()
    //script to create circuit c
    return c

```

Example Python Scripts:

Further examples can be found here:

```
<rsoft_dir>\examples\RSoftCAD\scripting\

```

For Python, see [Appendix H](#) for Python support details and [Section 10.A](#) for a detailed description of these examples.

Testing Scripts Outside Circuit Reference Blocks:

Testing scripts outside circuit reference blocks can be done by calling the `rsoft_layout()` function to execute the script:

- You can add a line after the `rsoft_layout()` definition as follows which will save a copy of the circuit as `test.ind` when the parent design file is opened in the RSoft CAD. These lines can be commented out when not needed.

```
def rsoft_layout(NumRings=int(10),
                 data_file=''):

    c = RSoftCircuit()
    //script to create circuit c
    return c

    c2 = rsoft_layout(10,'data_file')
    c2.write('test.ind')

```

- You can create a small wrapper script that imports the layout script, saves the `rsoft_layout()` as a circuit and writes a `.ind` file as described above.

In both cases, the script can also be run from a command line using the command:

```
rspython <script>

```

6.F.7. Adding Vertices to a Circuit Reference

Vertices are a core part of the RSoft CAD and allow components to be easily connected. Circuit references, by default, have one vertex at the local origin of the embedded design file. Some applications benefit from multiple vertices such as multi-port devices where each port can be thought of as a vertex. This allows multi-port objects to be easily connected.

To add a vertex, right-click on the Components folder in the component tree and select New Vertex. The vertex can be positioned as needed in the child file. Once used as a circuit reference in the parent, additional components can be connected to this vertex as usual. Note that vertex 0 of the circuit reference is the default vertex, it is the local origin of the embedded file. There can be any number of additional vertices, these extra vertices start at vertex 1 and up. Also note that vertices must be sequentially numbered starting at 1. This can be automated using the `pv` option described below.

6.F.8. Dynamically Importing GDS files via Circuit References

GDS files can be dynamically imported by setting **External File** to the `.gds` file. There are several options that control how the GDS data is imported (layers, cells, etc), and then several options that control the non-transverse geometry (position, height, material, etc.).

GDS File Options:

At a minimum, the **External File** should be set to the name of the `.gds` file. For example:

```
myGDSfile.gds
```

Several optional options can be added which control how the GDS data is imported:

```
myGDSfile.gds:<layer>/<data>,pv#,n<cell>
```

where `<layer>` is the layer number, `<data>` is the datatype, the `pv#` option automatically adds vertices, and the `n` option selects a particular named cell. Notes:

- Any option supported by the `gds2ind` utility can be used, see the [Mask Conversion Utilities](#) section.
- If no layer/data numbers are provided, all layers in the GDS file will be used.
- If no datatype is set, a datatype of 0 is used.
- The `pv#` option automatically detects ports/vertices and adds vertex objects. See section below for more details.
- The `n` option selects a particular named cell from the GDS. If not specified, the entire GDS is imported.
- (Not shown above) By default, the `.gds` file is positioned in the XZ plane. The `xy` option is a beta option to position it in the XY plane. In this XY mode, the 'Z' length can be set by the

`Length` symbol in the Local Symbols table. This is a beta feature, please contact us with questions or issues.

GDS File Geometry Options:

The GDS file sets the geometry in the transverse plane, the rest of the geometry is set by the vertical position, the layer height, and the layer material. These can be set via the **Y** position (**Z** when the `xy` option is used), the **Component Height**, and the **Component Material**.

GDS Port/Vertex Detection:

The `pv#` option enables the automatic detection of vertices. This is very useful when creating devices that will be connected to other devices, or to place port monitors for S-Matrix calculation with the Custom PDK Utility.

- Circuit reference objects have 1 vertex (vertex 0, the main X,Y,Z position), and so this option will add additional vertices. For example, a structure with 3 ‘ports’ will have 4 vertices: vertex 0 is at the global X,Y,Z position of the component, and vertices 1 through 3 will be at the detected port positions,
- The number argument will translate the structure so that numbered vertex is at the origin. A value of 0 does not move the structure.
- By default, the vertex detection will exclude ‘ports’ with a width > 0.999 μm . This limit can be changed using the `pw#` option.
- By default, vertices are found on all 4 sides of the structure. This can be changed using the `ps` option. The possible sides are `l, r, t, b` for left, right, top, and bottom. For example, setting `pslrt` will look for ports on the left, right, and top sides, excluding the bottom.
- Note that this option only adds vertices, use the **Attach Port Monitors** option to add port monitors, if needed. See the Custom PDK Utility manual for more details.

Example:

To dynamically load layer 88/4 from file `MyGDSfile.gds`, enabling automatic vertex detection for ports on the left and right side, with a maximum width of 1.5 μm , use this setting:

```
myGDSfile.gds:88/4,pv0,pw1.5,pslr
```

6.G. Dynamically-Sized Arrays

The Circuit Reference feature allows child circuits (the unit cell) to be periodically repeated to create a lattice.

6.G.1. Creating a Dynamic Array

There are two ways to create a dynamically-sized array:

- *Using the Dynamic Array button*

The **Dynamic Array** button in the left CAD toolbar provides a convenient way to create a dynamic array. To use this feature, click the **Dynamic Array** button and then click in the CAD drawing pane to add the array. You will be prompted to select the `.ind` design file that represents the unit cell. You can pick one of the built-in files with simple geometries, or select another file. After you have selected the file, the RSoft CAD will automatically create a hierarchy component with the desired unit cell embedded in the parent `.ind` file. You can then edit the Lattice Properties as needed.

- *Converting a Circuit Reference (Hierarchy block) into a Dynamic Array*

You can convert any circuit reference component into a dynamic array by opening the properties dialog for the circuit reference, clicking the **Lattice Info...** button, and setting the options described below. It is recommended to center the unit cell in the child circuit to simplify the lattice position in the parent circuit.

When using dynamic arrays, be sure to use the **Local Symbols...** option described above to logically link options from the parent file to the child file.

6.G.2. Setting Lattice Properties

The lattice properties are set in the Lattice Properties dialog box which can be opened by clicking the **Lattice Info...** button in the Circuit Reference Properties dialog box.

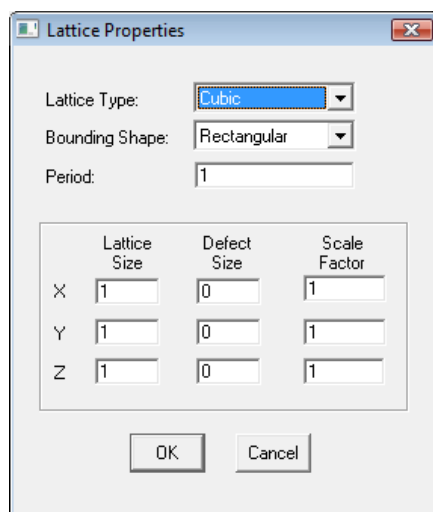


Figure 6-11: The Lattice Properties dialog box.

The lattice properties are:

Lattice Type

Sets the type of lattice including Cubic, Hexagonal (HCP), BCC, FCC, and Diamond lattices.

Bounding Shape

Sets the shape of the bounding box around the lattice. The bounding shape can be Rectangular or Hexagonal.

Period

Sets the period of the lattice.

Lattice Size

Sets the number of periods along the X, Y, and/or Z directions.

Defect Size

Sets the size of a defect measured in periods) at the center of the lattice along the X, Y, and/or Z directions.

Scale Factor

Scales the entire lattice in Cartesian coordinates.

6.H. Defining Pathways

Pathways serve two main purposes within BeamPROP, FullWAVE, and FemSIM:

- To serve as paths through the index structure for BeamPROP and FemSIM monitors.
- To specify which component to launch the incident field into.

This section will cover the creation of a pathway in the CAD. Note that the use of BeamPROP and FemSIM monitors are covered in the BeamPROP and FemSIM manuals respectively, and the definition of a launch field is discussed in the BeamPROP and the FullWAVE manuals.

6.H.1. Creating a Pathway

To define an optical pathway, click on the **Edit Pathways** icon in the left toolbar. The program switches to **Pathway Edit Mode** as shown in Fig. 6-13. The circuit display remains the same, but the toolbar has changed and the status line indicates the new mode.

To define a new pathway, click on the **New** button in the toolbar and the pathway count indicated at the top of the toolbar increments by one. Then select the desired components using the left

mouse button; note each selected component is highlighted in green. To deselect a component, simply click on it again. To select multiple components in one operation, drag a selection box around the components. To define additional pathways, simply click on **New** and repeat the above procedure. When all desired pathways have been defined, click on **OK** to accept the definitions and return to the main layout window mode. Note that when in 4P mode, components can only be added to or removed from a pathway in the pane that was active before entering Pathway Mode.

At any time the user can add new pathways by entering **Pathway Edit Mode** and performing the above steps. To modify or delete existing pathway definitions, click on the << and >> buttons to select the desired pathway, and select or deselect components to modify the definition, or click on the **Delete** button to delete the pathway definition entirely.

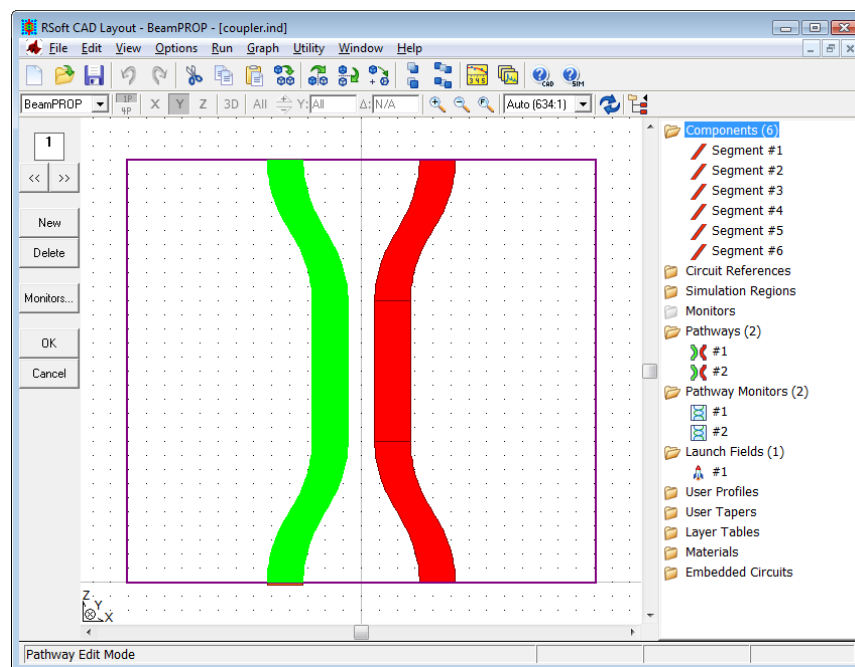


Figure 6-12: Edit Pathway Mode, which is used to define optical pathways for analysis. This example shows a directional coupler with the left pathway selected.

6.H.2. Using Pathways

As stated before, pathways have two uses:

- *For BeamPROP and FemSIM monitors:* Each pathway is essentially a set of components grouped together, usually so that they form a continuous path from one part of the device to another. For example, a directional coupler may consist of two s-bend input arms, two parallel straight sections, and two s-bend output arms. In this case the left set of components forms one natural optical pathway, while the right set forms another. Note that it is possible to define pathways which include *inactive* components. An inactive component as a **Profile**

Type set to Inactive , as is discussed in [Section 6.A](#). This can be useful if it is necessary to monitor the power in a region which does not contain or is otherwise different from a true component region. The use of BeamPROP and FemSIM monitors is discussed in the BeamPROP and FemSIM manuals.

For BeamPROP, pathways are only clearly defined when the components within that pathway do not overlap along the Z axis. If a pathway does have multiple components at a particular Z position, BeamPROP will give a warning during the simulation when it reaches that Z position.

- *For the Launch Field:* The use of a pathway for a launch field is discussed in the BeamPROP, FullWAVE, and ModePROP manuals.

6.I. Importing Symbols or Components from Another Design File

It may be necessary or desirable to import symbols or components from one design file (*.ind file) into another. In this section we describe several CAD features for assisting with each of these tasks.

6.I.1. Importing Symbols

The RSoft CAD provides several features to facilitate the import of symbols between data files. This allows, for example:

- A simple one-time import of symbols from one file to another.
- A pre-selected list of important symbols can be automatically imported from a ‘primary’ design file into a ‘secondary’ design file every time the ‘secondary’ file is opened. This helps keep multiple files synchronized when running simulation scripts.
- Selected symbols can be exported from a child design file to allow easy control from the parent design file. This simplifies the use of circuit references/hierarchy, see [Section 6.F.4](#).

Methods of Import

By default all symbols are imported, see next section for instructions on importing specific symbols.

- *One-Time Import*

Symbols can be imported from one design file to another via the Options/Import-Symbols... menu item. This will copy all of the symbols in the selected *.ind file into the current file. Symbols with the same name will be overwritten by symbols from the imported file.

- *Via Import Symbol File*

The symbol `import_symbol_file` is used to automatically load from a ‘master’ design file. Its use is best illustrated by example: Consider two *.ind files `first.ind` and `second.ind`. Setting `import_symbol_file = first.ind` in the `second.ind` file will automatically load symbols from `first.ind` into `second.ind` every time the `second.ind` file is loaded. This feature can be helpful to keep multiple files synchronized when running simulation scripts. When used, changes to variable values in the primary file are automatically propagated to the secondary files every time the secondary files are loaded. The automatic option can be disabled by setting `import_symbol_file_auto` to 0 even if the `import_symbol_file` option is set.

Specifying Symbols to Import

By default, all symbols are imported when using the options described above. The `import_list` variable can be used to specify the symbols that should be imported. This variable can be used in two ways:

- *Comma-Separated List*

This mode accepts a simple list of comma-separated variables to be imported. Variables not mentioned in the list will not be imported. For example, defining:

```
import_list = width,variable1,variable2,variable3
```

will import only the variables `width`, `variable1`, `variable2`, and `variable3`.

- *Data File*

This mode accepts the name of a data file and is useful when the list of variables to import is long. The ‘@’ character should be used before the data file. For example, defining:

```
import_list = @symbols_to_import.txt
```

will import the variables listed in the data file `symbols_to_import.txt`. If this data file contains the following:

```
width
variable1
variable2
variable3
```

will import only the variables `width`, `variable1`, `variable2`, and `variable3`.

Advanced Symbol Import Options

By default, an unset symbol will use a default value. The sync mode can be used to replicate this behavior. The sync mode will:

- If the symbol is defined in both files or only in the primary file, then it will be copied from the primary to the secondary file.

- If the symbol is not defined in the primary file, it will remove it from the secondary file.

To use the sync mode, add the keyword ‘:sync’ after the variable name. So, for example, when using the comma-separated list:

```
import_list = width,variable1:sync,variable2,variable3
```

or when using a data file:

```
width
variable1:sync
variable2
variable3
```

the symbol `variable1` will be interpreted in sync mode.

6.I.2. Importing Components

Components can be imported from one design file into the current design file by first opening the existing *.ind file, selecting the desired components, and copying them to the clipboard. Then switch back to the current layout via the Window menu, and paste the components into the current layout. If the selection contains formulas, the variables used in those formulas will need to be added to the symbol table of the current layout. For this feature to be useful, the layout of the device in the existing file should be well-designed to make use of references and other features to allow the device to be smoothly connected to the current layout without minimal additional editing.

6.J. CAD Preferences

The CAD Preferences window controls several options within the CAD and can be opened via the Options/Preferences menu item. Changes to some of these options require the CAD to be restarted.

Display Preferences

Draw Interior, Component Color

Display/color of
component interior

Inactive Color, Pathway Color, Rubberband Color

Color of inactive components,
pathways, and rubberbands.

Multi-Pane View

Enables Multi-Pane mode and
sets initial state: [Section 3.E.2](#)

Draw Edge, Edge Color

Display/color of
component edges

Show Simulation Domain, Show Launch Fields

Display simulation domain,
launch field (if applicable)

Show 3D Pane

Enable the 3D pane for Multi-
Pane mode

Draw Center Line,

Direction Indicators

Enable Scroll Bars

Center Color

Display/color of component centerline

Display direction indicators for launch fields and monitors

Enable scroll bars in each drawing pane

Use Material Colors, Show Default Comp as Comp Color, Map File

[Sets the material display color options, See Section 8.A.6.](#)

Start with Tree Expanded

Starts component tree with folders expanded

Circuit Reference Box

Sets display properties of circuit reference boxes

Component Tree

Enables component tree and sets initial state.

Editing Preferences

Attach on Edit

Enables the snap-to feature when editing components

Max Level Undo

Number of undo levels

Zoom Factor (>1.0)

Percentage used for zooming in or out.

Attach Tolerance, Select Tolerance

The pixel distance for 'snap-to' and component selection

Automatically Regrid after Zoom

Display grid is recalculated after zoom

Zoom/Full Affects All Panes

Zoom/Full is be done for all viewing panes at once

Paste Offset

Offset used for pasting objects from clipboard.

Warn when automatically resizing grid

CAD will warn when display grid has been resized

Zoom/Full Uses Simulation Domain

Zoom/Full uses domain rather than structure size

Show All Draw Tools

Display all useful drawing modes in each pane in left CAD toolbar

Zoom/Full Border

Percentage of space around zoom border when using Zoom/Full; must be between 0 and 1

Allow Mixed Types, Allow Mixed Properties

Controls group editing of mixed component types

Units Preferences

The **Index Input Mode** sets the method that users define refractive indices in the CAD. By default, indices are defined via index differences relative to the background index; however in some cases it can be useful to define them absolutely. When this option is changed in an existing file, the CAD will attempt to convert all components to the new mode. Since the CAD uses the index difference method internally it is usually preferred.

Simulation Preferences

Static Color Shades

Default number of static color shades; can be set locally as well

Dynamic Color Shades

Number of dynamic color shades

Color Scale

Default color scale to be used; can be set locally as well

Slice Output

Default state for BeamPROP's slice output: see BeamPROP manual

Use Single Precision

Default state for data type: see FullWAVE, DiffractMOD, ModePROP, or FemSIM manual

MPI Display Whole Domain

Default state for FullWAVE Whole Display option: see FullWAVE

Use Perpendicular Launch Width Convention

Set launch width equal to component width

Enable Auto Clustering

Default state for FullWAVE clustering: see FullWAVE manual

Limit Preferences

These preferences set the limits in the CAD for various items. It is not recommended to increase these values unless needed.

Import/Export Preferences

The top options are discussed in [Section 6.E](#), the bottom option sets the Group Library Path for where materials are stored ([Section 8.A.1](#)). The choice of PDK used by the `pdkinfo()` function described in [Section C.F.10](#) is also chosen here. Note the following:

- Custom PDK parameters can be defined in the `<rsoft_dir>\products\rsoftcad\pdkinfo` directory.
- This setting is global for all new .ind files created in the current user's environment, set the **PDK Info File** option in Global Settings `pdkinfo_file` to the desired PDK file for local control within a specific .ind file.

6.K. Additional Component Properties

The Additional Component Properties dialog can be opened by clicking the **More...** button in a component's properties dialog.

Not all of these options are appropriate/enabled for all simulation tools or components. Consult the appropriate simulation tool manual for information about the use of these options.

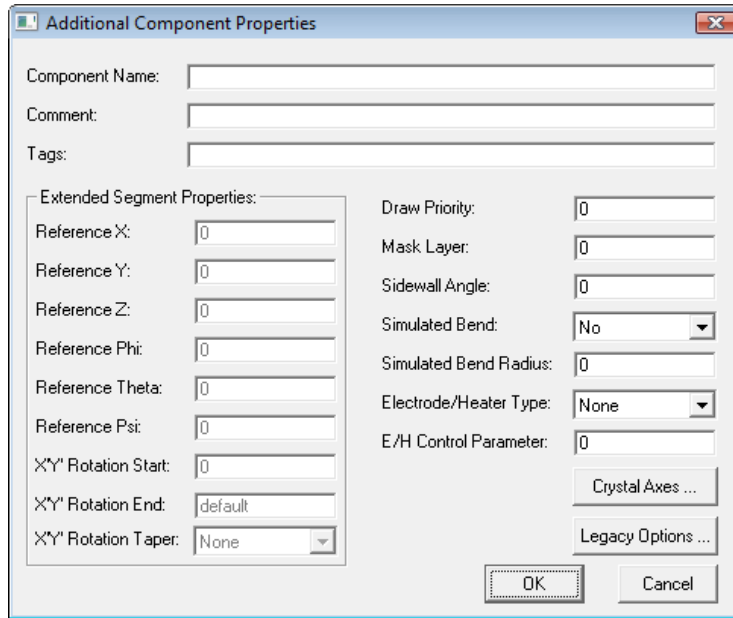


Figure 6-14: The Additional Component Properties dialog which controls infrequently used component properties.

6.K.1. General Properties

Component Name

This name will appear in the titlebar of the component properties dialog. Components can also be selected by name via the Edit/Select/Edit Component menu item. For monitors, the name will be used in some output plots and output file names.

Comment

The comment does not affect the simulation in any way and is solely for the user.

Tag

Tags are used in some cases to pass special instructions to simulation engines. See [Section 6.K.4](#) for more details.

Draw Priority

This option sets the drawing priority for display in the CAD interface. By default, all segments have a priority of 0 which means that component 1 is drawn first, component 2 next, etc., until all components are drawn. If a component has a higher Draw Priority, it will be given precedence and be drawn on top of components with lower priority. It is recommended to use as few draw priority levels around the default of 0 as necessary.

Mask Layer

When exporting the design file to a mask file, this information will be used to place different components in different mask layers. The default mask layer is 0. Setting the layer to -1 for a specific component will exclude that component from export. For more information about the export of mask files, see [Section 6.E](#).

Sidewall Angle

The sidewalls of Channel and Multilayer components can be sloped. This sets the angle in degrees and is measured from the vertical: A positive angle slopes in at the top, while a negative angle slopes out. The angle is maintained along the length of the segment even if the geometry is tapered. By default, the width is assumed to be defined at the bottom of the component, but this can be changed via the **Sidewall Angle Ref.** option in the Global Index Generation Options dialog.

Simulated Bend and Simulated Bend Radius

These fields control a CAD option which applies a coordinate transformation useful for simulating segment bends. This feature is described in [Section 6.D.1](#). These fields apply locally in Z, that is, only over the length of the segment to which they correspond. Alternatively, this option can be globally enabled for the entire simulation domain via the Global Index Generation Options dialog.

Electrode/Heater Type and E/H Control Parameter

These options are used by the Multi-Physics Utility. See the Multi-Physics Utility manual for more information.

6.K.2. Extended Segment Properties

These properties are valid only for extended segments ([Section 4.F](#)).

Reference X, Reference Y, and Reference Z

The reference position offsets the center of the actual object compared to the location of the starting vertex.

Reference Phi, Reference Theta, and Reference Psi

The reference angles specify an extra rotation about the starting vertex. This rotation is done after any reference position settings have been made. The rotation is done in the following order:

- Phi (ϕ) rotation is around Y axis and measured from Z axis.
- Theta (θ) rotation is around X' axis and measured from Z' axis.
- Psi (ψ) rotation is around Z' axis and measured from X' axis.

To transform from the unprimed system to the primed system:

$$X' = (X - X_{ref})(\cos \psi \cos \phi + \sin \theta \sin \phi \sin \psi) - (Y - Y_{ref})(\sin \psi \cos \theta) + (Z - Z_{ref})(\sin \psi \sin \theta \cos \phi - \sin \phi \cos \psi)$$

$$Y' = (X - X_{ref})(\cos \phi \sin \psi - \sin \theta \sin \phi \cos \psi) + (Y - Y_{ref})(\cos \phi \cos \theta) - (Z - Z_{ref})(\sin \psi \sin \phi + \sin \theta \cos \phi \cos \psi)$$

$$Z' = (X - X_{ref})(\cos \theta \sin \phi) + (Y - Y_{ref})(\sin \theta) + (Z - Z_{ref})(\cos \theta \cos \phi)$$

To transform from the primed system to the unprimed system:

$$X = X'(\cos \phi \cos \psi + \sin \theta \sin \phi \sin \psi) + Y'(\cos \phi \sin \psi - \sin \theta \sin \phi \cos \psi) + Z'(\cos \theta \sin \phi) + X_{ref}$$

$$Y = X'(-\sin \psi \cos \theta) + Y'(\cos \theta \cos \psi) + Z'(\sin \theta) + Y_{ref}$$

$$Z = X'(\sin \theta \cos \phi \sin \psi - \sin \phi \cos \psi) - Y'(\sin \phi \sin \psi + \sin \theta \cos \phi \cos \psi) + Z'(\cos \theta \cos \phi) + Z_{ref}$$

where X_{ref} , Y_{ref} , and Z_{ref} are the **Reference X**, **Reference Y**, and **Reference Z** settings.

X'Y' Rotation Start, X'Y' Rotation End, and X'Y' Rotation Taper

This rotation ‘twists’ the component along its local axis. By default, the ending angle is set equal to the starting angle. If they are not equal, the ‘twist’ can be tapered along the length of the component. See [Section 6.B](#) for more information on tapers.

6.K.3. Crystal Axes

The Crystal Axes Specification dialog, which can be opened via the **Crystal Axes** button in the Additional Component Properties dialog controls the local crystal axes of the component material. This option can only be used when an Anisotropic material has been used, and with simulators that support full anisotropy. See [Section 8.B.3](#) for more information about defining anisotropic materials and simulator manuals for full anisotropy support.

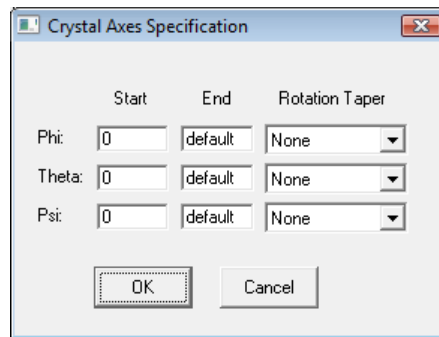


Figure: 6-15: The Crystal Axes Specification dialog.

Simple Crystal Axes Rotation

To simply rotate the crystal axes relative to the orientation definition in the Material Editor, set via the **Start** values for **Phi**, **Theta**, and **Psi**:

- A **Phi** rotation is around Y axis and measured from Z axis.
- A **Theta** rotation is around X' axis and measured from Z' axis.
- A **Psi** rotation is around Z' axis and measured from X' axis.

This type of crystal axes rotation is available for all types of components.

Tapered Crystal Axes Rotation for Segments

For segments (i.e. two vertex components), it is possible to taper, or change, the crystal axes along the segment. The taper has the form:

$$\theta(z) = \theta_0 + (\theta_1 - \theta_0) f(z')$$

$$z' = \frac{z - z_0}{z_1 - z_0}$$

where $\theta(z)$ represents the crystal axis to be tapered, θ_0 and θ_1 are the **Start** and **End** values specified in the Crystal Axes Specification dialog for that crystal axis, $f(z')$ is the taper function, and z' is a normalized coordinate along Z that varies from 0 to 1 along the Z extent of the segment. Individual control over all three angles is possible, and setting **End** to the keyword `default` effectively disables tapering and uses a simple rotation as described above. Several built-in tapers can be used, as well as user-defined tapers. See [Section 6.B](#) for more information about using tapers.

When using an extended segment, the variable `crystal_axis_prime` can be set to 1 to align the crystal axis along the segment Z' axis.

6.K.4. Component Tags

Tags are used to pass special instructions to simulation engines and are set in the Additional Component Properties dialog. They should not be set unless needed. Tags are discussed in appropriate simulation manuals and include:

- `skip_est`: This tag, when applied to a component, removes it from the default parameter estimation including domain and grid sizes.
- Several grid-related tags, see individual simulation manuals as well as Section 9.C and the simulation manuals for more details.
- Several monitor-related tags, see individual simulations manuals for more details.

Multiple tags should be separated by spaces, a tag setting of:

```
skip_est grid_size=Dx
```

enables the `skip_est` tag and sets the local grid size to the value of the `Dx` symbol.

Settings Tags

Tag may also be set in the Edit Default Tags dialog. This method allows tags to be assigned to segments that are defined by a specific material or when importing geometries from TCAD Sentaurus. The Edit Default Tags dialog can be opened via the Edit menu item in the RSoft CAD. The syntax for these tags is:

```
component <name> = tag1=value tag2=value  
region <name> = tag1=value tag2=value  
material <name> = tag1=value tag2=value
```

where the keywords `component` and `region` are synonymous and allow tags to be assigned to particular components in a design and the `material` tag allows tags to be assigned to components that are defined by the material specified.

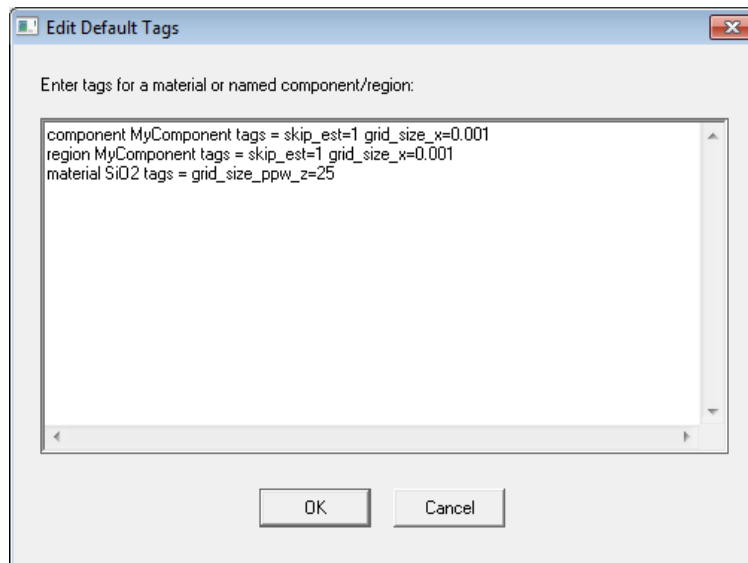


Figure 6-13: The Default Tags dialog where tag assignments can be specified.

Tag Precedence

The order of precedence for tags is, in order of importance:

- Tags set locally for a particular component
- Tags set via the 'Edit Default Tags' dialog for a particular component/region
- Tags set via the 'Edit Default Tags' dialog for a particular material

Example Tags

The above figure shows an example that assigns the following tags:

- The tags ‘skip_est=1 and grid_size_x=0.001’ to the component named `MyComponent`. Note that while both `component` and `region` keywords are used here, only one need to be used since they are synonymous.
- The tags ‘grid_size_ppw_z=25’ to components that are defined via the SiO2 material.

6.L. Using EIM to Simulate 3D Structures in 2D (2.5D)

The Effective Index Method (EIM) can be used to greatly reduce the computation time and memory requirements of a simulation by converting a 3D structure into an approximate 2D structure. This is sometimes called ‘2.5D’. This option only affects the simulation; the definition of the original 3D structure is unaltered allowing this feature to be safely turned on and off without changing the defined structure. This feature is enabled by the **Effective Index Calculation** option in the Global Settings dialog.

The use of EIM requires the following:

- Use one of the supported RSoft tools, including FullWAVE, BeamPROP, ModePROP, or DiffractMOD. See the manuals for these products for additional information and/or examples.
- Use only components with only one of the supported **3D Structure Types**: *Rib/Ridge*, *Channel*, or *Diffused*. The Y position of these segments must be:
 - *Rib/Ridge* or *Diffused* components must be positioned at Y=0
 - Channel components must be positioned either at the same Y position or such that their bottom edge is at the same Y position.

Note the following when using EIM:

- The definition of polarization when using EIM is the same as is for 3D simulations allowing the feature to be turned on and off in a consistent way. When using EIM, the polarization option in the simulation dialog is displayed as **Pol (3D EIM)** and should be set using the standard 3D polarization convention. This polarization is what is used to calculate the effective 2D indices, the appropriate polarization will be used when actually simulating the structure in 2D.
- For simulations that contain multiple wavelengths in a single simulation (i.e. FullWAVE ‘pulse’ simulations), the material properties of the effective material are automatically fit to a dispersive model to account for dispersion.

6.M. Expanded Referencing of Vertices

One of the core aspects of the RSoft CAD is that a vertex position can be defined relative to another vertex, allowing for components to be logically connected/positioned within a design. This feature can be extended to include the width/height of a component as well as the angle/orientation of the vertex.

Expanded referencing can be enabled by setting:

- The **X**, **Y** (for 3D), and
- The **Z Reference** of the starting vertex must have a **Type** of *Offset* and be **Relative To** the same vertex

Once enabled, the width, height, or angle can be referenced:

- *Width/Height Referencing:*

The **Component Width** and **Component Height** can be referenced by setting these options to the keyword `default` which will inherit the width/height value from the referenced vertex. If expanded referencing is not enabled, it will be set the values set in the Global Settings dialog.

- *Angle Referencing:*

The angle (see details below for supported angles) can be referenced by settings this option to the keyword `default` which will inherit the angle value from the referenced vertex. If expanded referencing is not enabled, it will be set to 0.

Expanded references are supported by the following components:

- *Segments:*

Segments ([Chapter 4](#)) support expanded referencing for the **Component Width**, **Component Height**, and the **Reference Phi** angle (in Additional Segment Properties dialog) options. Note that **Segment Orientation** must be set to *Extended*.

- *Polygons:*

Polygon components ([Section 5.D](#)) support expanded referencing for the **Component Width**, **Component Height** and **XZ Angle** options.

- *Lenses:*

Lens components ([Section 5.B](#)) support expanded referencing for the **Component Height** and **XZ Angle** options.

- *Circles:*

Circle components ([Section 5.C](#)) support expanded referencing for the **Component Diameter**, **Component Height** and **XZ Angle** options.

- *Monitors and Port Monitors:*

Monitor components (see [Section 5.F](#) and simulation tool manuals) support expanded referencing for the **Component Width**, **Component Height**, and **Phi** options.

- *Marks and Vertices:*

Mask ([Section 5.E](#)) and vertices ([Section 5.F](#)) support expanded referencing for the **Width/Length**, **Height**, and **XZ Angle** options.

- *Circuit References:*

Circuit References ([Section 6.F](#)) support expanded referencing for the **Component Width**, **Component Height**, and **XZ Rotation Angle** options.

6.N. Warning Options

The CAD and simulation tools will give warnings in some cases. They can be disabled in a design file through the Warning Preferences window. These warnings should only be disabled if necessary.

6.O. Component Filters

Component filters allow you to create custom list(s) of components based on user-defined searches/filters. This helps to organize and group components, simplifying views, and

Filters help to organize and group components, can be used to produce custom views in the drawing pans, and generally help to find specific components.

For example, a filter can:

- Show a list of all components that have a Material of ‘Si’
- Show a list all monitors
- Show a list of all components with a Merge Priority of 2
- In the drawing pane(s), hide all components that have a material of ‘Si’ and a Display Color of Red.

6.O.1. Defining Filters

Filers are defined in the Component Filter folder in the tree view on the left of the RSoft CAD.

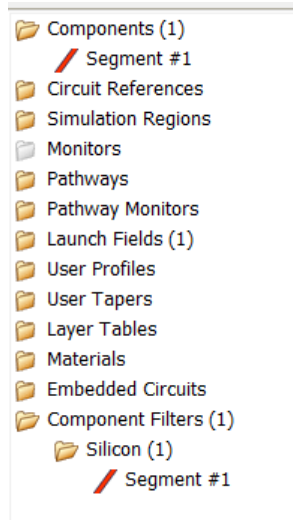


Fig 6-14: The CAD tree view, showing the Component Filter folder with one filter defined.

Right-clicking on the Component Filter folder gives the option to create a new filter, opening the Filter Creation dialog.

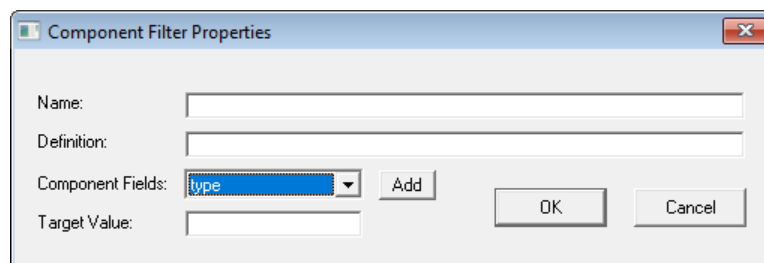


Figure 6-15: The Component Filter Properties dialog.

To define a component filter, give it a **Name** and **Definition**. You can use the **Component Fields** and **Target Value** fields to help define the definition. The syntax is `field=target`, with boolean AND or OR operators to combine multiple fields. Consider this example definition which returns all components that have a 'Si' material and a Merge Priority of 1:

```
Material=Si AND priority=1
```

Here are the supported search fields, along with special rules for matching:

Component Field	Description
type	Matches against any of the following (case-insensitive): - segment – matches segments in the Components tree folder - polygon – matches polygon components in the Components tree folder

- `lens` – matches lens components in the Components tree folder
- `mark` – matches mark components in the Components tree folder
- `monitor` – matches monitors in the Monitors tree folder
- `simregion` – matches simulation regions in the Simulation Regions folder
- `hierarchy` – matches circuit references in the Circuit Reference tree folder
- `launch` – matches launch fields in the Launch Fields tree folder

`structure`

Matches the **3D Structure Type** of a component (case-insensitive):

- `fiber`
- `channel`
- `diffused`
- `ribbridge`
- `multilayer`
- `polygon`

`priority`

Matches the **Merge Priority** of a component.

`draw_priority`

Matches the **Draw Priority** of a component.

`mask_layer`

Matches the **Mask Layer** of a component.

`color`

Matches the **Display Color** of the component (case insensitive):

- `black`
- `dkblue`
- `dkgreen`
- `dkcyan`
- `dkred`
- `dkmagenta`
- `brown`
- `ltgray`
- `dkgray`
- `blue`
- `green`
- `cyan`

	• red • magenta • yellow • white
material	Matches the Material of a component (case-sensitive).
name	Matches the Name of a component (case-sensitive)
tags	Matches the Tag of a component. Note that some tags have values, some do not: • To match a tag that does not have a value, use <code>tags=abc</code> syntax. • To match a tag that has a value, use <code>tags:var=abc</code> syntax.
groups	Matches the Group of a component

6.O.2. Using Filters

Once a filter is defined, it appears in the Component Filters folder in the CAD tree view. You can use the filters to simply sort components, or use them to show/hide the group, edit the group properties, etc. Right-click on the filter to select one of these options:

- **Edit Filter:** Edit the filter settings.
- **Delete Filter:** Delete the filter
- **Select All:** Select all components in the filter
- **Hide Group:** Hide the components in the filter
- **Show Group:** Show the components in the filter
- **Delete Group:** Delete all components in the filter
- **Group Properties:** Edit the group properties of all components in the filter

6.P. Simulation Log Files

Each simulator saves information in a log file. By default, this file is named `log.txt` and appended to each time a simulation is run in a particular directory. There are several options to control this:

- To create a new file each time the simulation runs (i.e.. delete the old file), use the Menu option in the RSoft CAD View/Clear Log/Always.

- The variable `rsoft_logbyprefix` can be used to name the log file using the simulation prefix. A value of 0 uses the default `log.txt` name, a value of 1 uses the simulation prefix (i.e. `<prefix>_log.txt`) and appends data for subsequent simulations, and a value of 2 uses the simulation prefix and creates a new file for each simulation

7

Layout Generation Utilities

Some designs call for complicated structures which can be very tedious to manually create. For this reason, the CAD includes automatic layout utilities for certain common problems. These can be accessed via the Utility menu item:

- *Array Layout Generator*

This utility is used to create periodic arrays of segments and will be described in this chapter.

- *WDM Router Layout tool*

This utility is used to create layouts for simulation and mask creation for arrayed-waveguide-grating routers (or AWG's). This utility is described in detail in the AWG Utility manual.

- *GratingMOD Grating Layout tool*

This utility is used create grating layouts to be used with RSoft's grating simulation tool GratingMOD and is described in detail in the GratingMOD manual

The rest of this chapter will describe the usage of the Array Layout utility.

7.A. Motivation

Dynamically-sized arrays can be created using the hierarchy feature discussed in [Section 6.F](#).

The RSoft CAD system is very powerful and allows expressions to define aspects of the geometry and component references to define logical relationships between components. Once a properly designed layout is complete, these features make it very easy to vary the structure and perform design studies. The Array Layout utility allows the user to quickly create structured

composed of periodic arrays. It can be opened via the Utility menu item in the CAD window (Fig. 7-1). The Array Layout Utility can be divided into three sections: Unit cell Parameters, Lattice Parameters, and Creation Options. These three sections are shown in Fig. 7-1 and are described in the next sections.

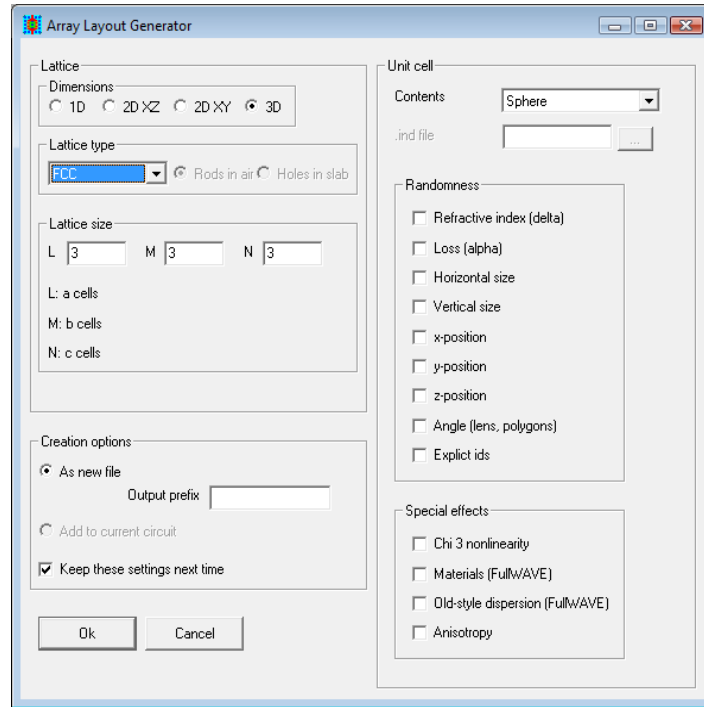


Figure 7-1: The Array Layout Generator interface.

7.B. Lattice Parameters

The lattice parameters define the type of periodic lattice to create from the defined unit cell. This section will cover the lattice parameters; the next the unit cell parameters.

Dimensions

This sets the dimensions of the lattice to be created. Possible choices are *1D*, *2D (XY)*, *2D (XZ)*, and *3D*. Each dimension has different possible lattice types which are described below.

Do not confuse this setting with the dimension of the simulation to be performed. This only defines the periodicity to use. For example, a PBG slab is periodic in two dimensions, but is finite in the third and thus requires a 3D simulation.

Lattice Size

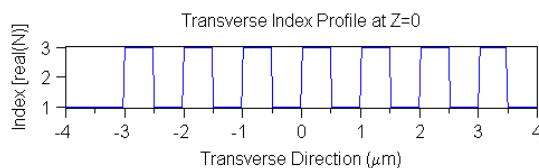
The lattice size controls the size of the lattice produced. The definition of L, M, and/or N will change depending on the dimension and lattice type chosen, and is shown in the GUI. For a general cubic case, L corresponds to the number of elements along the X axis, M along the Y axis, and N along the Z axis. However, other lattice types have different definitions.

7.A.1. 1D Lattice Types

The available 1D lattices are:

- *1D X, 1D Y, 1D Z*

These options create a 1D lattice along the specified coordinate. One such lattice with a rectangular unit cell is show here:



The **Lattice Size** settings correspond to the number of unit cells in the a direction.

7.B.2. 2D Lattice (XZ and XY) Types

The available 2D lattices are shown in Fig. 7-2, and described below.

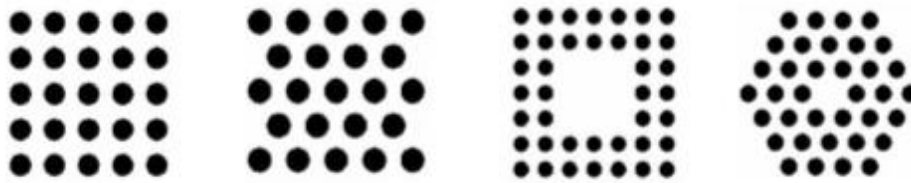


Figure 7-2: 2D lattice types a) cubic, b) hexagonal, c) cubic rings, and d) hexagonal rings.

- *Cubic*

This options creates a 2D cubic lattice, and the **Lattice Size** settings correspond to the number of unit cells in the a and b directions.

- *Hexagonal*

This option creates a 2D hexagonal lattice, and the **Lattice Size** settings correspond to the number of unit cells in the a and b directions. The lattice may be rotated using the symbol `PhiPattern` to achieve the other typical hexagonal lattice orientation.

- *Cubic Rings*

This option creates a 2D lattice formed by cubic rings, and the **Lattice Size** settings correspond to the number of outside and inside rings. The example lattice shown above has 3 outside rings and 2 inside rings.

- *Hexagonal Rings*

This option creates a 2D lattice formed by hexagonal lattice, and the **Lattice Size** settings correspond to the number of outside and inside rings. The example lattice shown above has 3 outside rings and 1 inside ring.

7.B.3. 3D Lattice Types

The available 3D lattices are shown in Fig. 7-3, and are described below.

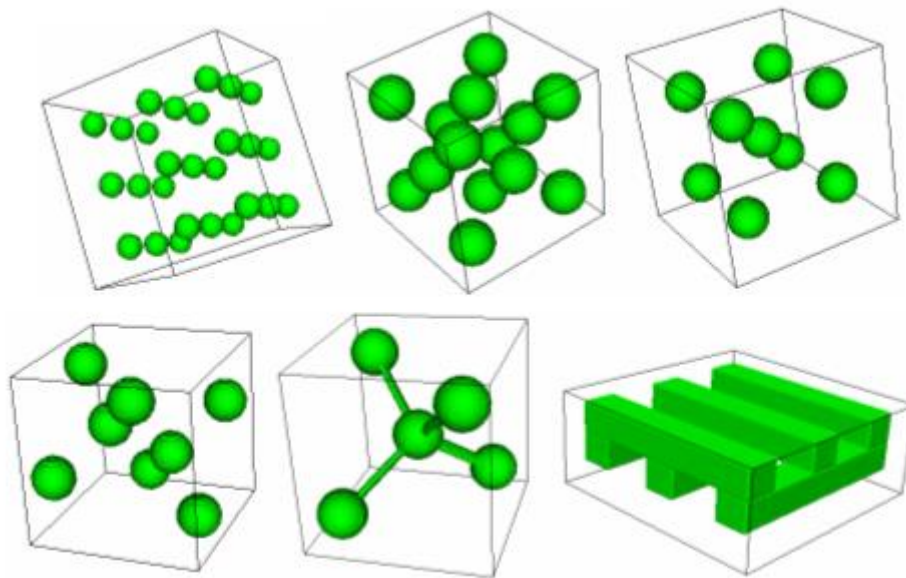


Figure 7-3: Some of the available 3D lattice types (clockwise from top right) a) cubic, b) FCC, c) BCC, d) woodpile, e) diamond logs, and f) diamond.

- *Cubic*

This option creates a 3D cubic lattice and the **Lattice Size** settings correspond to the number of unit cells in the a, b, and c directions.

- *FCC*

This option creates a 3D FCC (face centered cubic) lattice and the **Lattice Size** settings correspond to the number of unit cells in the a, b, and c directions.

- *BCC*

This option creates a 3D BCC (body centered cubic) lattice and the **Lattice Size** settings correspond to the number of unit cells in the a, b, and c directions.

- *Diamond*

This option creates a 3D diamond lattice, and the **Lattice Size** settings correspond to the number of unit cells in the a, b, and c directions.

- *Diamond Logs*

This option creates a 3D diamond lattice with connecting logs, and the **Lattice Size** settings correspond to the number of unit cells in the a, b, and c directions.

- *Woodpile*

This option creates a 3D woodpile lattice, and the **Lattice Size** settings correspond to the number of unit cells in the a, b, and c directions.

7.C. Unit Cell Parameters

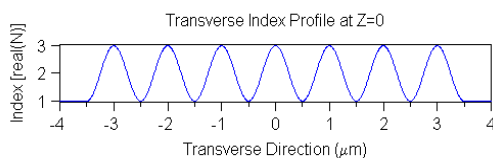
This section of the Array Layout Utility window allows the user to define the contents of the unit cell to be periodically repeated. The contents of the unit cell are set by the pulldown menu labeled **Contents**. The options are:

7.C.1. 1D Unit Cells

The available 1D unit cells are

- *Sine Wave*

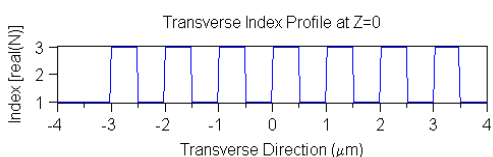
This option creates a 1D unit cell composed of a sine wave such as is shown here:



The *.ind file created has one segment with either a user profile or taper depending on whether the variation is along X, Y, or Z.

- *Square Wave*

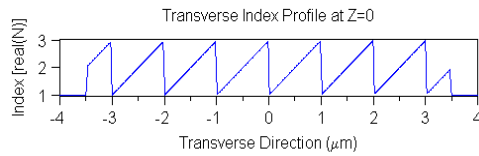
This option creates a 1D unit cell composed of a square wave such as is shown here:



The *.ind file created has one segment with either a user profile or taper depending on whether the variable is along X, Y, or Z.

- *Sawtooth Wave*

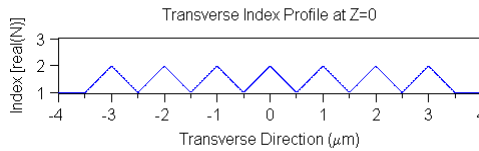
This option creates a 1D unit cell composed of a square wave such as is shown here:



The *.ind file created has one segment with either a user profile or taper depending on whether the variable is along X, Y, or Z.

- *Triangle Wave*

This option creates a 1D unit cell composed of a square wave such as is shown here:



The *.ind file created has one segment with either a user profile or taper depending on whether the variable is along X, Y, or Z.

7.C.2. 2D Unit Cells (XZ and XY)

The available 2D lattices are shown in Fig. 7-4, and are described below:

Note that all options are not available for all lattice dimensions.



Figure 7-4: Some of the available 2D unit cells: a) square, b) hexagon, c) circle, and d) ellipse.

- *Square*

This option creates a 2D unit cell composed of a singular square object. The segments used in the *.ind file to create the lattice are standard waveguide segments for both XZ and XY layouts, and can be easily modified to produce rectangular segments if necessary.

- *Hexagon*

This option creates a 2D unit cell composed of a singular hexagonal object. The segments used in the *.ind file to create the lattice are polygons. This option is only available for XZ layouts.

- *Circle*

This option creates a 2D unit cell composed of a single circular object. The segments used in the *.ind file to create the lattice are lenses for XZ layouts and 3D fibers for XY layouts.

- *Ellipse*

This option creates a 2D unit cell composed of a single elliptical object. The segments used in the *.ind file to create the lattice are segments with width tapers for XZ layouts and 3D fiber objects for XY layouts.

- *.ind file*

This options uses hierarchy ([Section 6.F](#)) to periodically repeat a ‘child’ design file to create a custom lattice in a ‘parent’ file. The contents of this unit cell are defined by whatever is in the *.ind file used. The *.ind file to be used is specified in the field **.ind file**.

7.C.3. 3D Unit Cells

The available 3D lattices are shown in Fig. 7-5, and are described below:

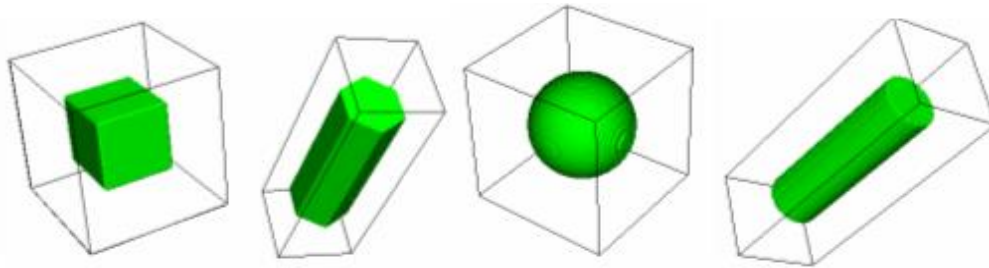


Figure 7-5: Selected 3D unit cells: a) cube, b) hexagonal prism, c) sphere, and d) cylinder.

- *Cube*

This option creates a 3D unit cell composed of a single cubic object. The segments used in the *.ind file to create the lattice are simply polygons and can be easily modified to produce rectangular segments if necessary.

- *Hexagonal Prism*

This option creates a 3D unit cell composed of a single hexagonal prism object. The segments used in the *.ind file to create the lattice are polygon segments.

- *Sphere*

This option creates a 3D unit cell composed of a single spherical object. The segments used in the *.ind file to create the lattice are 3D lens objects.

- *Cylinder*

This option creates a 3D unit cell composed of a single cylindrical object. The segments used in the *.ind file to create the lattice are 3D lens objects.

- *.ind file*

This options uses hierarchy ([Section 6.F](#)) to periodically repeat a ‘child’ design file to create a custom lattice in a ‘parent’ file. The contents of this unit cell are defined by whatever is in the *.ind file used. The *.ind file to be used is specified in the field **.ind file**.

- *Rectangular Rods*

This option applies to the *Woodpile Lattice Type* only and creates a woodpile lattice composed of rectangular rods. The segments used in the *.ind file are channel segments.

- *Elliptical Rods*

This option applies to the *Woodpile Lattice Type* only and creates a woodpile lattice composed of elliptical rods. The segments used in the *.ind file are fiber segments.

7.C.4. Additional Options

These options further control the definition of the unit cell.

Randomness

It can be useful to incorporate perturbed lattice parameters for each point in the lattice to study non-ideal crystals. All selected parameters will be defined using the `ranseq` function. A seed variable is also created to allow the user to control the set of random numbers used. The perturbation is defined as

$$\langle \text{parameter} \rangle + \text{sig_} \langle \text{parameter} \rangle * \text{ranseq}(\text{seed_} \langle \text{parameter} \rangle, \$id)$$

where `<parameter>` is the parameter to be varied. For example, if a random Refractive Index, or `delta`, is used, the Index Difference of each object in the array will have an Index Difference defined as

$$\text{delta} + \text{sig_delta} * \text{ranseq}(\text{seed_delta}, \$id)$$

where the following table explains the parts of the equation:

Parameter	Description
-----------	-------------

<code>delta</code>	The standard Index Difference defined in the Global Settings dialog.
<code>sig_delta</code>	The “amplitude” of the randomness, controls the min/max perturbation.
<code>ranseq</code>	This function returns a random number based on the two seed variables described below. The value it returns lies in the range (-0.5,0.5). See Appendix C for more details on this function.
<code>seed_delta</code>	The first seed variable is the same for each segment. Changing this variable in the symbol table will choose a different “random” set of perturbations for the design.
<code>\$id</code>	The second seed variable is set to a value of <code>\$id</code> , which is a special notation for the segment number. This allows each segment to have a different random number for a given value of the <code>seed_delta</code> parameter.

Special Effects

These fields allow the user to add additional capability to the design file to use materials.

7.D. Creation Options

The final section of the Array Layout Generator dialog allows the user to set the name of the output file and to save lattice creation settings.

Output Prefix

The output prefix sets the name of the `*.ind` file to be created and is required.

Keep these Settings Next Time

This option saves the settings in the Array Layout Generator to for the next time a lattice is created.

Advanced Materials

RSoft's Material Editor provides a convenient database of complex material models that can be used in simulations. These materials can be simple dielectrics, but can also incorporate dispersion, non-linearity, and anisotropy. A built-in material library is provided; users can also create specific materials as needed. Materials can be shared between different designs and users. This chapter will discuss how to define these material properties and assign them to specific objects.

Since the CAD is shared by several simulation tools, all possible material properties will not have meaning for all simulation tools. It is critical to understand the underlying simulation mechanisms and if various properties are relevant.

The first section outlines the usage of the Material Editor; the rest of the chapter is devoted to documenting specific material parameters.

8.A. Using the Material Editor

The Material Editor dialog, shown in Fig. 8-1, can be opened by clicking the corresponding button in the left toolbar of the CAD, by clicking the **Materials...** button in component properties dialogs, or by selecting New Material in the **Material Properties** field in a component properties dialog. This dialog has a material tree control where the user can browse the materials available for use within the current project, as well as any material libraries that have been defined. When selected, a material's detailed information is displayed over several tabs.

8.A.1. The Material Tree

The Material Tree allows users to quickly navigate through materials which have been defined.

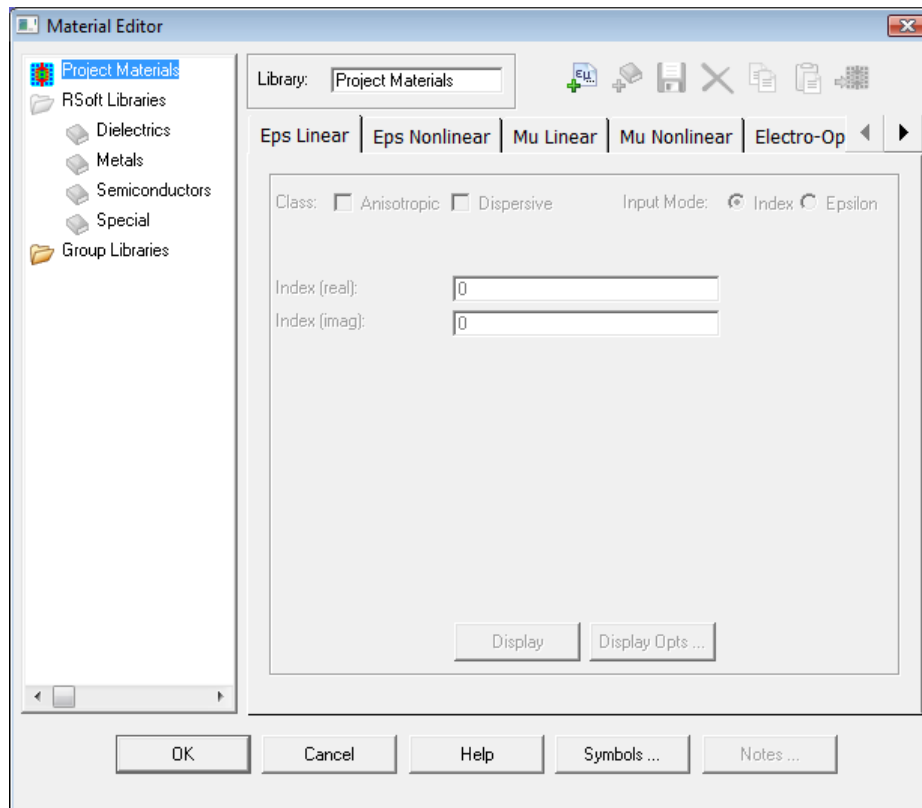


Figure 8-1: The Material Editor dialog. The material tree is located on the left, the toolbar is on top, and the material property tabs are displayed on the right.

The tree is divided into several main categories:

- *Project Materials*

These materials are available to use and are stored within the local design file.

- *RSoft Library*

This built-in, write-protected library is provided by RSoft. It contains typical dielectric, metallic, semiconductor, and special materials such as Graphene (see Section 8.K) and PEC (Perfect Electric Conductor). Materials in this library cannot be edited; if changes need to be made to one of these materials, copy it to a user-defined library make the changes there.

- *RSoft CAD Info Folder(s)*

Materials from one or more RSoft CAD Info folders can be used in a design file, see the Custom PDK Utility manual for more information on RSoft CAD Info folders.

- *Group Libraries*

This category holds user-created libraries and materials that are available to multiple users on a single computer. Users can use this library to ‘share’ materials between .ind files: materials from the Project Materials group to this group, and copy materials from this group to the Project Materials. Libraries and materials in this category are stored in a user-defined directory which should be a shared directory outside the RSoft installation directory. There are two ways to define this directory:

- *Via a Preference*

This directory can be easily set by right-clicking on the **Group Libraries** folder and selecting New Library option. Click **Yes** and then select a location to create the new folder. Alternatively, it can be set via the Group Library Path option in on the Import/Export tab of the CAD Preferences dialog described in [Section 6.J](#).

- *Via an Environment Variable*

The environment variable `RSOFT_MATLIB_GROUP_PATH` can be used to select the location of this folder. Note that if the ‘Preference’ method described above is used, it will override this setting. Please contact your system administrator for information on setting this environment variable.

Users with only read-access to this directory will not be able to create new or modify existing libraries/materials. Users with write-access must take care to avoid inadvertent changes to materials as this will permanently change the database for other users. See notes in [Section 8.A.7](#) for using data files in materials defined in this group.

8.A.2. The Material Tabs

The material tabs can be accessed when a material is selected in the material tree. Each tab corresponds to a different material property such as linear epsilon or electro-optic parameters. These tabs are described in Sections 8.B and following in this chapter. Not all material properties can be used by all simulation engines, nor do all properties have to be set.

8.A.3. Creating and Organizing Materials in the Material Editor

The user can create and organize materials in the Material Editor using the buttons on the top toolbar, or by right-clicking in the tree area. These options are only available when it is appropriate to use them.

These options are:



New Material

Create a new material in selected library. The name of the material can be changed at the top of the dialog.



Copy Material

Copy selected material to the material clipboard.



New Library

Create a new library in the User Libraries category. The name of the library can be changed at the top of the dialog.



Paste Material

Paste contents of material clipboard into selected library.



Save Library

Save library after changes have been made. If not saved, the CAD will prompt to do when exiting the Material Editor.



Use Material (Add to Project)

Duplicates selected material in the Project Materials category for use in local project (design file).



Delete Material or Library

Deletes the selected material or library.

8.A.4. Using Materials in a Project (Local Design File)

It is critical to understand that in order to use a material in a project (the local design file), a copy of the material *must* be placed within the Project Materials category of the materials tree. To do this, either copy/paste the material or use the **Add to Project** button described in the previous section. The benefit of this approach is twofold: to keep the design file consistent even when materials in the Material Editor change, and to simplify the sharing of design files.

Materials must be in the Project Materials category to be used within the design file.

Once in the Project Materials category, the material can be used within the RSoft CAD interface in places such as the **Material Properties** setting in a Component Properties dialog, the Layer Table editor, or the Global Settings dialog.

Using a Symbol to Define Assign a Material

The **Material Properties** setting can be set via a symbol if desired. To do this, click on the text part of the **Material Properties** control and type the symbol name preceded by the \$ character, i.e. \$Material_name. The symbol should contain the actual name of the desired material in quotes, i.e. "Ag". The empty but still quoted value of "" can be used to select the Locally Defined option and effectively disable the use of materials.

This can be useful in many circumstances but is especially useful when using hierarchy as a means to control materials in a child file from the parent file. See [Section 6.F.5](#) for details.

8.A.5. Displaying Material Definition

It can be beneficial to test specific material definition to ensure that accuracy. This can easily be done via the **Display** button which will open a plot of the current material property. Several options which control the format of the output can be set in the Material Display Options window which can be opened by clicking the **Display Opts...** button. These options are:

Display Axes

Sets the axes used to display the material data. Both the X and Y axes can be set.

Plot Range/Resolution

Sets the minimum and maximum values on the X axis, as well as the grid spacing used to display the material properties. Separate controls are given for wavelength and frequency.

Measurement Data

Allows a data file to be plotted with the computed material data. This is useful when trying to match a particular data file. The data file must have the same units as the chosen Display Axes for a direct comparison. This feature is for comparison purposes only.

Output Prefix

Sets the name of the output file used.

8.A.6. Material Properties

The material properties are set in the General Material Properties dialog, which can be opened by clicking the Properties button in the Material Editor. Here you can set the material color and set several notes.

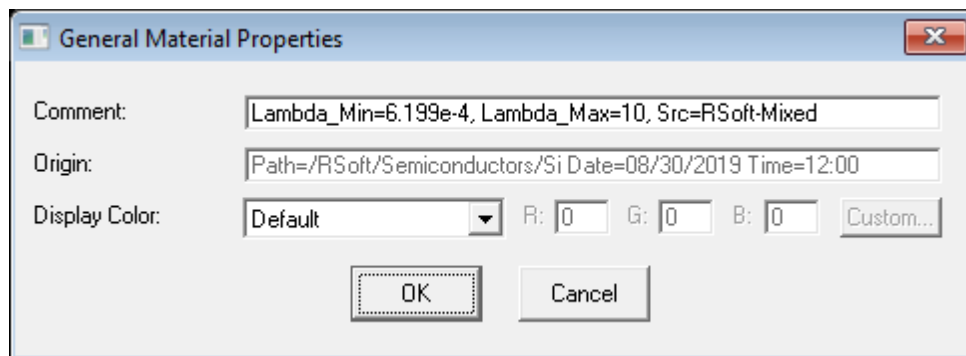


Fig 8-2: The General Material Properties dialog.

The options in this dialog are:

Comment

Sets a comment for the material. This information is not used anywhere in the software, it is strictly for record keeping purposes.

Origin

Displays the origin of the material. Materials in the RSoft Material library include the wavelength range and/or references if applicable. This information is not used anywhere in the software, it is strictly for record keeping purposes.

Display Color

This option is used in specific circumstances only, see notes below.

Sets the display color of components to which the material is assigned. You can select from one of the standard colors or define RGB values for a custom color. The options are:

- Default: Uses the default color, see options below.
- A standard Color: Select the desired color from the dropdown list, i.e. Red, Dark Red, etc.
- Custom: Select an RGB value for the color.

Note on Display Colors

There are various places to set the color, here is the order of precedence for the options:

- Global Preference Color Map File: The Color Map file set in the Preferences dialog (See [Section 6.J](#)) can be used to set material colors at a group level. This is intended for situations such as multiple users who work on the same file(s) and want a standard color scheme across multiple users, projects, and files.

The color map file is a text file, each line sets the default color for a material. Color codes or hex color values can be used, for example:

```
Glass = COLOR_RED  
SiO2 = COLOR_RED  
SiN = #0045234
```

- Users can change a material color for specific materials in a project using the material Display Color option described above. This overrides the global color map file.
- The color of specific components can be directly set by the Display Color option in the component properties dialog. This overrides any other setting.

8.A.7. Tips & Traps

Keep the following in mind when using the Material Editor:

- All of the toolbar actions described in [Section 8.A.3](#) can also be done with the mouse. Right-clicking on a library or material gives a menu of the possible actions, and materials can be copied and pasted into other libraries as well as dragging them to the Project Materials category. Keyboard shortcuts are also available for most actions. These shortcuts are displayed in the menu shown when right-clicking on a library or material.
- Materials from one or more RSoft CAD Info folders can be used in a design file, see the Custom PDK Utility manual for more information on RSoft CAD Info folders.
- The Symbol Table button on the bottom of the Material Editor dialog opens the Symbol Table for the local design file. It is recommended that users not use symbols when defining materials that will be copied to the Group Library category. This is because these symbols must not only be defined in any design file that the material will be used in, but also have the correct values. The one exception to this rule is the built-in symbol `free_space_wavelength` which is defined in all design files. This exception allows for the creation of simple wavelength-dependent material properties such as those described in the notes in [Section 8.B.1](#).
- If using an nk data file via the `userreal()` and `userimag()` functions, the wavelength range in the data file will be displayed in the Material Editor. A warning will also be displayed if a simulation is run at a wavelength outside the range. If ignored, the nk data at the end of the range will be used.
- Synopsys periodically updates material data for materials in the RSoft Material Library. Archives of older material data, sorted by version, can be found in this folder:

```
<rsoft_dir>\products\rsoftcad\materials\rsoft\archives
```
- It is possible to use user-created data files to define material properties as a function of wavelength via the `userdata()` function (see the notes in [Section 8.B.1](#) for an example). These data files must be in specific locations so that the CAD can find them:
 - For materials defined in the Project Materials category, no path information is necessary if the files are in the same directory as the design file. The appropriate expression would be:

```
userdata("material_property.dat", free_space_wavelength)
```
 - For materials in the Group Library category, any associated files should be located in the directory defined by the `RSOFT_MATLIB_GROUP_PATH` environment variable. The special tag `<groupmat>` can be used in any expressions to refer to this path. The appropriate expression would be:

```
userdata("<groupmat>\material_property.dat", free_space_wavelength)
```
- Each built-in material has notes which can be viewed by clicking the Notes button in the bottom of the editor. These notes contain important information about the material model. Users are encouraged to create these notes for user-defined materials as well.

- It is critical to make sure that the properties of any material(s) used correspond to the actual physical conditions you wish to simulate. This is especially important when using the built-in materials since each model is valid only for certain conditions such as wavelength, temperature, pressure, etc.. Also, they may not be useful for FullWAVE simulations with a single wavelength, or for multiple wavelengths at a time. Use the display options liberally, check the material notes, and read through the notes in [Section 8.B.1](#) to ensure that any material properties are valid for your specific needs. RSoft does not guarantee the accuracy of any supplied materials and may change them without notice in future releases.

8.B. Linear Epsilon Properties

The linear epsilon, or relative permittivity, properties can be defined in the **Eps Linear** tab. It can be either a simple real or complex number (i.e. isotropic/uniaxial and non-dispersive), or either anisotropic or dispersive.

8.B.1. Linear Epsilon Overview

The Material Editor allows users to create materials with the following definition for linear epsilon:

$$\mathbf{D} = \epsilon_0 \mathbf{E} + \mathbf{P}$$

where

$$\mathbf{P} = \epsilon_0 \left[\chi(\omega) \mathbf{E} + \text{non-linear terms} \right]$$

and $\chi(\omega)$ can be a frequency dependent linear dispersion term. For the rest of this section, we will assume that there are no non-linear terms; see [Section 8.B.2](#) for more on non-linear epsilon.

Notes on Dispersive Linear Epsilon

Material dispersion, or the change of refractive index as a function of wavelength, is not automatically included by any of RSoft's simulation tools. The user needs to define how the refractive index varies with wavelength. Some materials, such as Silicon, have additional properties: see Section 8.I.

For simulation tools that perform monochromatic simulations only, a parameter scan over wavelength is required to study dispersive effects.

There are several ways to define dispersive materials in the CAD.

- *Via Simple Linear Epsilon*

This method sets a component's complex refractive index to a single value either via the Component Properties dialog (when **Material Properties** is set to Locally Defined) or the Simple Linear Epsilon properties in the Material Editor. This value for real or imaginary epsilon can be defined as a function of wavelength via an expression or via a data file. One example of using an expression could be

```
1+(B*free_space_wavelength^2)/(free_space_wavelength^2-C)
```

where **B** and **C** are coefficients in the symbol table and `free_space_wavelength` is the simulation wavelength, and one example of using a data file could be

```
userdata("tungsten_real.dat", free_space_wavelength)
```

where the file `tungsten_real.dat` contains refractive index data in the standard RSoft format and `lambda` is the simulation wavelength. See the notes in [Section 8.A.7](#) for more information on using symbols and data files in materials, and see [Appendix C](#) for more details of this and other functions that can be used in the symbol table and [Appendix B](#) for more details on file formats.

This method is valid for single simulations at one wavelength, and if defined as a function of wavelength, for parameter scans over wavelength.

- *Via Dispersive-Class Materials in the Material Editor*

This method uses a dispersion-class material in the Material Editor to define a comprehensive dispersive relation for a material. This method is valid for single simulations at one wavelength, scans over wavelength and for FullWAVE simulations where multiple wavelengths are present in a single simulation such as a pulsed excitation.

8.B.2. Simple Linear Epsilon

A simple linear epsilon can be defined when **Anisotropic** and **Dispersive** are not selected. This is equivalent to:

$$\mathbf{D} = \epsilon \mathbf{E}$$

Dispersive materials can be defined via this method if epsilon (or index) is defined as a function of the simulation wavelength as noted in [Section 8.B.1](#). All simulation tools support this material property.

The options for this type of material are:

Input Mode

Sets the way in which the entered data is interpreted. When Epsilon is selected, the user can input epsilon directly. When Index is selected, the user can input the refractive index, or the square root of epsilon. This does not automatically convert any entered data between these two formats.

Index (real) or Epsilon (real)

Sets the real part of the linear epsilon (or index).

Index (imag) or Epsilon (imag)

Sets the imaginary part of the linear epsilon (or index).

Import NK Data...

This button selects a data file that contains n/k data and automatically sets the **Index/Epsilon** options so that they can read in the data from the data file. Specifically, these options are set to:

```
userreal("<data_file>", free_space_wavelength)
userimag("<data_file>", free_space_wavelength)
```

where <data_file> is the name of the selected file. The data file should have three columns, the first should be wavelength [μm], the second should be the real index (n), and the third should be the imaginary index (k). The filename should not contain spaces. Alternatively, the data can contain real/imaginary epsilon. The data will be interpreted according to the **Input Mode**. The data can have the standard RSoft 1D real/imag header or no header. See [Appendix A](#) for a description of supported file formats and [Appendix C](#) for more information on the `userreal()` and `userimag()` functions.

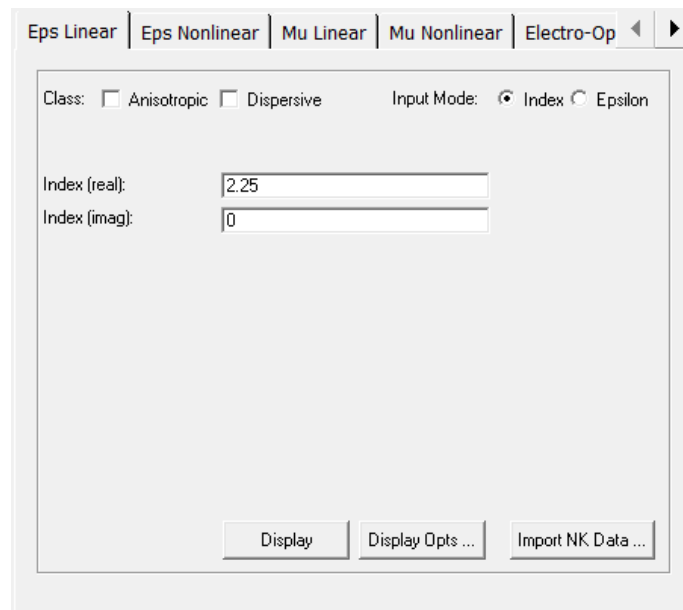


Figure 8-3: The portion of the Linear Eps tab in the Material Editor where a simple linear epsilon is defined.

8.B.3. Anisotropic Linear Epsilon

An anisotropic linear epsilon can be defined when **Anisotropic** is selected. This is equivalent to:

$$\mathbf{D} = \begin{pmatrix} \epsilon_{xx} & \epsilon_{xy} & \epsilon_{xz} \\ \epsilon_{yx} & \epsilon_{yy} & \epsilon_{yz} \\ \epsilon_{zx} & \epsilon_{zy} & \epsilon_{zz} \end{pmatrix} \mathbf{E}$$

Dispersive materials can be defined via this method if the tensor elements are defined to be a function of the wavelength as noted in [Section 8.B.1](#). Not all simulation tools support the use of anisotropic materials or the use of all tensor elements; consult the relevant simulation tool manual for more information. Instructions on viewing material profiles for specific tensor elements can be found in the ‘Notes for Anisotropic Designs’ section in [Section 3.G.1](#).

The options for this type of material are:

Input Mode

Sets the way in which the entered data is interpreted. When Epsilon is selected, the user can input the epsilon tensor directly. When Index is selected, the user can input the refractive index, or the square root of epsilon. This square root is done on an element by element basis like $n_{ij}^2 = \epsilon_{ij}$ where i and j are the tensor indices.

xx(re), xx(im), et al.

Sets the real and imaginary parts of epsilon (or index) for the xxth element in the epsilon (or index) tensor. The other elements are set in similar fashion. The crystal axes of the material can be explicitly defined via these controls or rotated for a particular CAD component using the **Crystal Axes** options described in [Section 6.K.3](#).

The screenshot shows the 'Linear Eps' tab in the Material Editor. The 'Class' section has 'Anisotropic' checked and 'Dispersive' unchecked. The 'Input Mode' section has 'Index' selected and 'Epsilon' unselected. The 'Index Tensor' section contains a 3x3 grid of input fields for the real (re) and imaginary (im) parts of the permittivity components. The values are: xx(re)=1.54, xy(re)=0, xz(re)=0; xy(im)=0, yy(re)=1.53, yz(re)=0; xz(im)=0, zy(re)=0, zz(re)=1.54. All imaginary parts are set to 0.

Figure 8-4: The portion of the Linear Eps tab in the Material Editor where an anisotropic linear epsilon is defined.

8.B.4. Dispersive Linear Epsilon

A dispersive linear epsilon and/or a linear gain with saturation can be defined when **Dispersive** is selected. This is equivalent to:

$$\mathbf{D} = [\varepsilon(\omega) + G(\omega)] \mathbf{E}$$

$$\varepsilon(\omega) = \varepsilon_{\infty} + \sum_k \frac{\Delta \varepsilon_k}{-a_k \omega^2 - b_k (i\omega) + c_k}$$

where $G(\omega)$ is a frequency-dependant gain term (for FullWAVE only) to be discussed later, ε_{∞} is the value of epsilon in the limit of infinite frequency, $\Delta \varepsilon_k$ is the strength of each resonance, a_k , b_k , and c_k are fitting coefficients, and ω is measured in units of radians/ μm . The choice of these parameters sets the type of resonance used.

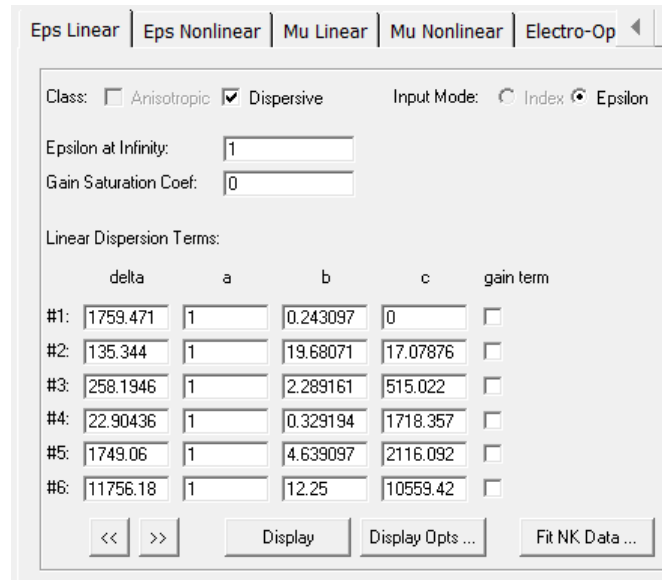


Figure 8-5: The portion of the Linear Eps tab in the Material Editor where a dispersive linear epsilon is defined.

All simulation tools support the basic use of this material property, and will compute the dispersive properties at the simulation wavelength. Furthermore, this method provides a true dispersive environment for FullWAVE simulations with multiple simulation wavelengths (such as in the case of a pulsed excitation). To use this feature, the **Dispersion/Nonlinearity** option in the FullWAVE Simulation Parameters window must be enabled. Also, when using this method in the context of a FullWAVE CW simulation, note that an optimized form of the dispersion parameters are used since one wavelength is primarily present. This optimization can be disabled by setting `fdtd_dispersion_opt` to 0.

Standard linear dispersion models, such as Lorentz, Drude, and Debye models can be used by choosing the fitting parameters a_k , b_k , c_k as follows:

	a_k	b_k	c_k
Lorentz	1	free	free
Drude	1	free	0
Debye	0	free	1

The free parameters above, plus the coefficient $\Delta\epsilon_k$ (delta in the graphical interface), can be used to match any single dispersion resonance. Multiple resonances can be combined to realize complicated dispersion relations; to view additional resonances, use the `<</>>` buttons. The **Gain Term** checkbox should not be checked unless the gain feature described later in this section is used.

Fitting Data Files to Lorentzian Coefficients

N/k data fits are only used for pulsed FullWAVE simulations. All other simulation types are at a single wavelength and a data fit is not necessary.

The RSoft Material Editor can automatically fit a data file to Lorentzian coefficients. There are two types of fits:

- *Auto-fit for Each Simulation:*

Click the **AutoFit** button in the Linear Epsilon tab of the Material Editor and select the auto-fit options. The n/k data will be automatically fit before each simulation. The **AutoFit Method** selects the type of fit: A Simple fit matches the value and slope of the n/k data at the **Free Space Wavelength**; a Lorentzian fit uses a multiple Lorentzian model to the data. Click the **Try Fit** button to view the current fit (note that the actual fit will be calculated before each simulation).

- *One-time Fit:*

To fit the data, open the Material Editor, create a new material, and click the **Dispersive** button on the **Linear Eps** tab. Click the **Fit NK Data...** button to open the Fit NK Data Options dialog. Select the n/k data file and click **Try Fit**. The fit will be performed, and then a comparison between the original data and data fit will be displayed. The fit looks good, but if it wasn't you can change the fitting parameters and retry the fit. Click **Accept Fit** to copy the calculated coefficients into the material.

In both cases, the fit is done with the `fitdisp` utility, see [Appendix E](#) for more details about this utility.

Note the following:

- The data file should contain complex index (n/k) data vs. wavelength [μm]. This can be from any source including experiment or many online sources. The data file should have three columns like:

```
wavelength1  n1  k1
wavelength2  n2  k2
wavelength3  n3  k3
etc...
```

No header is required, though other formats (see [Appendix B](#)) can be used.

- If the fit is not adequate, vary the fit options (see the documentation for the `fitdisp` utility in [Appendix E](#) for more details about these options) and click the **Try Fit** button again. Repeat until a good fit is found.

Option	Description
Max # Lorentzians	Sets the maximum number of Lorentzian resonances

	used for the fit.
Fit Tol	Sets the fitting tolerance.
Lambda Min	Sets the range of wavelengths to be fit. If a good fit cannot be found, try reducing the wavelength range.
Lambda Max	When using the auto-fit option, the wavelength range is automatically determined at run-time based on the simulation settings.

Fitting Standard Linear Dispersive Models

Assuming the following forms for typical resonances, the fitting parameters for each resonance type is given below.

These equations represent typical forms of these resonances. Results for similar forms can easily be found.

- *Lorentzian Resonance*

$$\varepsilon(\omega) = \varepsilon_{\infty} + \sum_k \frac{A_k \omega_k^2}{-\omega^2 - 2i\omega\delta_k + \omega_k^2}$$

where A_k is the resonator strength, δ_k is the damping factor, and ω_k is the resonant frequency.

- *Drude Resonance*

$$\varepsilon(\omega) = \varepsilon_{\infty} + \frac{\omega_p^2}{-2i\omega\nu_c - \omega^2}$$

where ω_p is the plasma frequency and ν_c is the collision frequency.

- *Debye Resonance*

$$\varepsilon(\omega) = \varepsilon_{\infty} + \frac{A_k}{1 - i\omega t_k}$$

where A_k is the resonator strength and t_k is the Debye relaxation time constant.

- *Sellmeier Resonance*

$$\varepsilon(\lambda) = \varepsilon_{\infty} + \sum_k \frac{A_k \lambda^2}{\lambda^2 - \lambda_k^2 - i\lambda\delta_{Sk}}$$

where λ_k is the resonant wavelength, A_k is the resonator strength, and δ_{Sk} is the Sellmeier damping factor.

The following chart outlines the appropriate choice of fitting parameters to realize a Lorentz resonance:

Parameter	Lorentz Value	Drude Value	Debye Value	Sellmeier Value
$\Delta\epsilon_k$	$A_k w k^2$	w_p^2	A_k	$A_k (2\pi/\lambda_k)^2$
a_k	1	1	0	1
b_k	$2\delta_k$	$2\nu_c$	t_k	$(2\pi/\lambda_k)^2 \delta_{Sk}$
c_k	w_k^2	0	1	$(2\pi/\lambda_k)^2$

Dispersive Linear Gain for Epsilon

As previously mentioned above, FullWAVE supports an optical gain term that is defined through a saturable, frequency dependent conductivity, and is defined as:

$$G(\omega) = \frac{1}{i\omega} \left[\frac{1}{1 + g_{sat} I} \right] \sum_k \sigma_k(\omega)$$

$$\sigma_k(\omega) = \frac{\Delta\sigma_{0k} \left(\frac{b_k^2}{4} - \frac{i\omega b_k}{2} \right)}{-a_k \omega^2 - i\omega b_k + c_k}$$

where $\Delta\sigma_{0k}$ is related to the peak strength of each resonance, a_k , b_k , and c_k are fitting coefficients, and ω is measured in units of radians/ μm . These fitting coefficients can be chosen for standard resonance types in the same exact way as described above for non-gain terms.

It is possible to define a material that has standard dispersion and dispersive gain. In order to define a resonance as a gain term, check the **Gain Term** box next to the resonance in the Material Editor. If this box is not checked, the resonance will be applied to the standard dispersion relation. Also, multiple gain resonances can be defined to realize a complicated dispersion relation.

8.C. Non-Linear Epsilon Properties

The non-linear epsilon, or relative permittivity, properties can be defined on the **Eps Nonlinear** tab and allow control over both second-order and third-order non-linearities.

Overview

The Material Editor allows users to create materials with the following definition for non-linear epsilon:

$$\mathbf{D} = \varepsilon_0 \mathbf{E} + \mathbf{P}$$

$$\mathbf{P} = \varepsilon_0 \left[\text{linear term} + \chi^2 \mathbf{E}^2 + \chi^3(\omega) |\mathbf{E}|^2 \mathbf{E} \right]$$

where χ^2 is the second-order non-linear susceptibility, and $\chi^3(\omega)$ is the saturable dispersive third-order non-linear susceptibility and is defined as a linear sum of multiple resonance terms multiplied by a saturation term:

$$\chi^3(\omega) = \left[\chi_\infty^3 + \sum_k \chi_k^3(\omega) \right] \left[\frac{1}{1 + c_{sat} \mathbf{E}^2} \right]$$

where χ_∞^3 is the value of the susceptibility in the limit of infinite frequency and $\chi_k^3(\omega)$ are dispersion terms. Each dispersion term is defined according to the following:

$$\chi_k^3(\omega) = \frac{\Delta \chi_k^3}{a_k (i\omega)^2 - b_k (i\omega) + c_k}$$

where $\Delta \chi_k^3$ is the strength of each resonance, a_k , b_k , and c_k are fitting coefficients, and ω is measured in units of radians/ μm . The choice of these parameters sets the type of resonance used.

Only BeamPROP and FullWAVE support the use of this material property. Specifically, BeamPROP supports third-order non-linearity, and computes the susceptibility at the simulation wavelength. FullWAVE supports both second- and third-order non-linearities and computes the susceptibility for any wavelengths present in the simulation. To use this feature in FullWAVE, the **Dispersion/Nonlinearity** option in the FullWAVE Simulation Parameters window must be enabled. For both BeamPROP and FullWAVE, the parameter `nl_iterations` (default = 1) sets the number of iterations used to compute the non-linear effects.

Setting Non-Linear Epsilon Properties

The options for this material property are:

Chi2

This sets the χ^2 susceptibility described above.

Chi3

This sets the χ^3 susceptibility described above, and corresponds to the value in the limit of infinite frequency.

Chi3 Saturation Coef:

This sets the saturation coefficient described above for χ^3 .

Eps Linear | **Eps Nonlinear** | Mu Linear | Mu Nonlinear | Electro-Op

Chi2:
 Chi3:
 Chi3 Saturation Coef:

Chi3 Dispersion Terms:

	delta	a	b	c
#1:	0	0	0	0
#2:	0	0	0	0
#3:	0	0	0	0
#4:	0	0	0	0
#5:	0	0	0	0
#6:	0	0	0	0

Figure 8-6: The portion of the Non-Linear Eps tab in the Material Editor where non-linear epsilon is defined.

Fitting Standard Non-Linear Models

Standard nonlinear models, such as Raman and Kerr models can be used by choosing the fitting parameters a_k , b_k , and c_k as follows:

	a_k	b_k	c_k
Lorentz	1	free	free
Drude	1	free	0

The free parameters above, plus the coefficient $\Delta\chi_k^3$, can be used to match any single resonance. Multiple resonances can be combined to realize complicated dispersion relations.

A Note on Units for Non-Linear Parameters in FullWAVE:

Since most problems are linear, the field units are essentially irrelevant since monitor results are normalized to the input power: all results are clearly interpreted as relative. However, when using nonlinear materials, units become important: the software must be used in a consistent way to get the correct answer:

This section provides a simple interpretation of units. A detailed approach for FullWAVE can be found in the appendices of the FullWAVE manual.

- *Using field values without units:*

It is sometimes easier, to keep E unitless and use dimensionless χ^2 and χ^3 parameters. This is most easily done by setting the launch **Normalization** to be None or Unit Peak, leaving the **Launch Power** set to 1, and using dimensionless values for these parameters as follows:

$$\chi_{\text{norm}}^2 = \chi^2 [\mu\text{m}/\text{V}] * E_0 [\text{V}/\mu\text{m}]$$

$$\chi_{\text{norm}}^3 = \chi^3 [(\mu\text{m}/\text{V})^2] * E_0^2 [(V/\mu\text{m})^2]$$

where E_0 is the true peak field amplitude in the launch. This approach has the advantage that the fields are of the order of unity, avoiding difficulties in the graphics display. Similar normalizations can be done with Unit Power, but are not described here.

- *Using field units:*

The easiest approach is to assume that all field components have units of V/ μm . This choice requires that: 1) while the unit of E is the traditional MKS unit, all units of length must be assumed to be μm , and 2) the unit of H is the same as the unit of E.

If the field has units of V/ μm , then the nonlinear parameters χ^2 and χ^3 must have units of $\mu\text{m}/\text{V}$ and $(\mu\text{m}/\text{V})^2$ respectively.

This approach also requires a consistent interpretation of the power contained in the launch field(s) based on the launch **Normalization**:

Type	Launch Power
None	Sets E_0^2 where the electric field E_0 [V/ μm] multiples the unnormalized launch field.
Unit Peak	Sets E_0^2 where the electric field E_0 [V/ μm] is the peak electric field
Unit Power	Sets the power of the launch field: By default, the power is in V^2 . Setting <code>fdtd_launch_power_unit = 1</code> forces the Launch Power to be interpreted in Watts.

8.D. Linear Mu Properties

The linear mu, or relative permeability, properties can be defined on the **Mu Linear** tab.

Overview

The Material Editor allows users to create materials with the following definition for linear mu:

$$\mathbf{B} = \mu_0 \mathbf{H} + \mathbf{M}$$

$$\mathbf{M} = \mu_0 \left[\chi_m(\omega) \mathbf{H} + \text{non-linear terms} \right]$$

where $\chi_m(\omega)$ is a frequency dependent linear dispersion term. For the rest of this section, we will assume that there are no non-linear terms.

Only FullWAVE supports the use of this material property. To enable its use in a simulation, select the **Use Magnetic Materials** option in the Advanced Parameters dialog.

Setting Linear Mu Properties

The equations and options for linear mu are exactly analogous to those for dispersive linear epsilon (except that gain is not supported), and so are not repeated here. See [Section 8.C](#) for this information.

8.E. Non-Linear Mu Properties

The non-linear mu, or relative permeability, properties can be defined on the **Mu Nonlinear** tab.

Overview

The Material Editor allows users to create materials with the following definition for non-linear mu:

$$\mathbf{B} = \mu_0 \mathbf{H} + \mathbf{M}$$

$$\mathbf{M} = \mu_0 \left[\text{linear term} + \chi_m^2 \mathbf{H}^2 + \chi_m^3(\omega) |\mathbf{H}|^2 \mathbf{H} \right]$$

where χ_m is the second-order non-linear susceptibility, and $\chi_m^3(\omega)$ is the saturable dispersive third-order non-linear susceptibility

Only FullWAVE supports the use of this material property. To enable its use in a simulation, select the **Use Magnetic Materials** option in the Advanced Parameters dialog.

Setting Non-Linear Mu Properties

The equations and options for non-linear mu are exactly analogous to those for non-linear epsilon, and so are not repeated here. See [Section 8.D](#) for this information.

8.F. Electro-Optic Properties

The electro-optic properties can be defined on the **Electro-Optic** tab of the Material Editor.

Overview

The Material Editor allows users to create materials whose refractive index is perturbed by the presence of electrodes in the simulation domain. This perturbation is added to the normal refractive index of the material. For more information on the use of electrodes and the Multi-Physics Utility, see the Multi-Physics Utility manual.

Setting Electro-Optic Properties

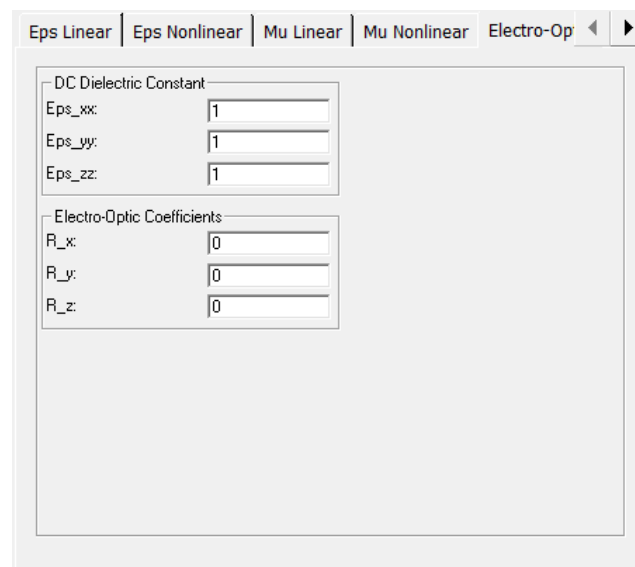
The options for this material property are:

Eps_xx, **Eps_yy**, and **Eps_zz**

Sets the diagonal elements of the DC dielectric constant tensor ϵ (also known as the RF dielectric constant) used to compute the spatially dependent DC electric field from any defined electrodes via the Poisson equation. The zz value of this quantity is currently not used and is provided for compatibility with a future release.

R_x, **R_y**, and **R_z**

Sets the electro-optic coefficients R_x and R_y used to compute the index perturbation from any defined electrodes. The units of these parameters are $\mu\text{m}/\text{V}$. The z value of this quantity is currently not used and is provided for compatibility with a future release.



The screenshot shows the Material Editor interface with the 'Electro-Optic' tab selected. The tab bar at the top includes 'Eps Linear', 'Eps Nonlinear', 'Mu Linear', 'Mu Nonlinear', and 'Electro-Optic'. The main panel is divided into two sections: 'DC Dielectric Constant' and 'Electro-Optic Coefficients'. The 'DC Dielectric Constant' section contains three input fields: 'Eps_xx' with a value of 1, 'Eps_yy' with a value of 1, and 'Eps_zz' with a value of 1. The 'Electro-Optic Coefficients' section contains three input fields: 'R_x' with a value of 0, 'R_y' with a value of 0, and 'R_z' with a value of 0.

Figure 8-7: The Electro-Optic tab in the Material Editor where electro-optic properties are defined.

8.G. Thermo-Optic Properties

The thermo-optic properties can be defined on the **Thermo-Optic** tab of the Material Editor.

Overview

The Material Editor allows users to create materials whose refractive index is perturbed by the presence of heaters in the simulation domain. This perturbation is added to the normal refractive index of the material. For more information on the use of heaters and the Multi-Physics Utility, see the Multi-Physics Utility manual.

Setting Thermo-Optic Properties

The options for this material property are:

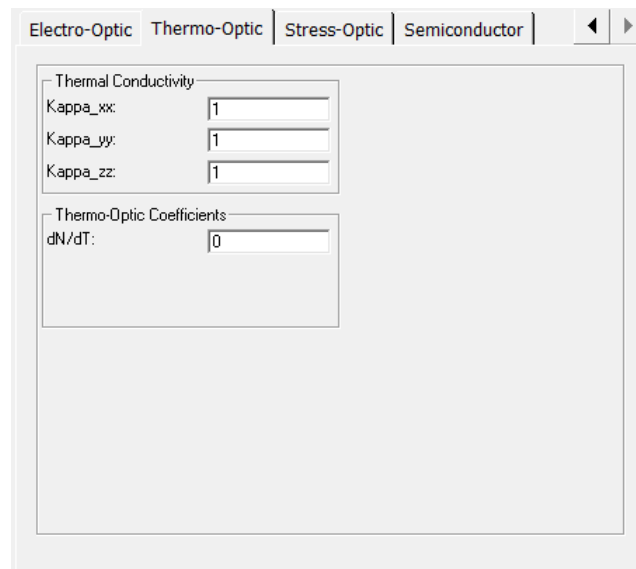


Figure 8-8: The Thermo-Optic tab in the Material Editor where thermo-optic properties are defined.

Kappa_xx, **Kappa_yy**, and **Kappa_zz**

Sets the diagonal elements of the thermal conductivity tensor κ used to compute the spatially dependent temperature distribution from any defined heaters via the Poisson equation. The units of these parameters are $W/(\mu m \cdot K)$.

dN/dT

Sets the temperature coefficient dn/dT for index change in units of K^{-1} .

8.H. Stress-Optic Properties

The stress-optic properties can be defined on the **Stress-Optic** tab of the Material Editor. The use of this option is licensed separately; contact us with questions.

Overview

The Material Editor allows users to create materials whose refractive index is perturbed by the stress created during the cooling from the fabrication temperature. This perturbation is added to the normal refractive index of the material.

When a simulation that incorporates stress is initiated, the displacement is calculated using force-balanced equations as a function of the temperature differential between operating temperature (ΔT) and fabrication temperature, which is defined in the Multi-Physics Utility, and the stress properties defined in the material editor. The spatially-dependent strain distribution ϵ is then computed from the displacement. The spatially-dependent stress $\sigma(\mathbf{x})$ is then computed from this quantity as:

$$\begin{pmatrix} \epsilon_x \\ \epsilon_y \\ \epsilon_z \end{pmatrix} = \frac{1}{E} \begin{pmatrix} 1 & -\nu & -\nu \\ -\nu & 1 & -\nu \\ -\nu & -\nu & 1 \end{pmatrix} \begin{pmatrix} \sigma_x \\ \sigma_y \\ \sigma_z \end{pmatrix} + \alpha \Delta T$$

where ϵ [unitless] is the strain, E is Young's Modulus, ν [unitless] is Poisson's Ratio, $\alpha [K^{-1}]$ is the thermal expansion coefficient, and $\Delta T [K]$ is the temperature change. The units of E and the computed $\sigma(\mathbf{x})$ values are in inverse units, for example if E is in GPa then $\sigma(\mathbf{x})$ would be in GPa^{-1} . It is important to use the same units for E for all materials in a simulation.

The local index perturbation used in a simulation is computed from the strain ϵ as:

$$\Delta \begin{pmatrix} n_x \\ n_y \\ n_z \end{pmatrix} = \begin{pmatrix} P_1 & P_2 & P_2 \\ P_2 & P_1 & P_2 \\ P_2 & P_2 & P_1 \end{pmatrix} \begin{pmatrix} \epsilon_x \\ \epsilon_y \\ \epsilon_z \end{pmatrix}$$

where P_1 and P_2 [unitless] are defined in Material Editor and ϵ is the strain. For more information on the use of the Multi-Physics Utility, see the Multi-Physics Utility manual.

This material property is supported by all simulation tools. Special care should be given when using stress with simulation tools that require periodic structures since the resulting perturbation may render the structure unperiodic. The stress distribution can be viewed using the multi-physics utilities discussed in the Multi-Physics Utility manual.

Setting Stress-Optic Properties

The options for this material property are:

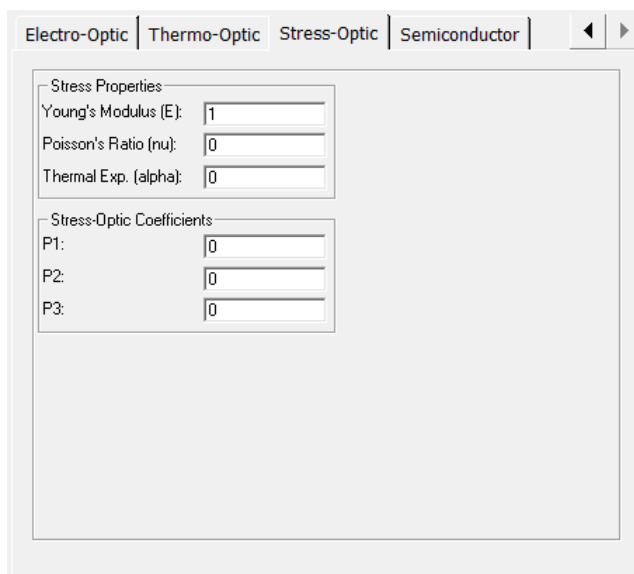
The image shows a software window titled 'Material Editor' with four tabs: 'Electro-Optic', 'Thermo-Optic', 'Stress-Optic', and 'Semiconductor'. The 'Stress-Optic' tab is selected. It contains two sections. The first section, 'Stress Properties', has three input fields: 'Young's Modulus (E):' with a value of '1', 'Poisson's Ratio (nu):' with a value of '0', and 'Thermal Exp. (alpha):' with a value of '0'. The second section, 'Stress-Optic Coefficients', has three input fields: 'P1:' with a value of '0', 'P2:' with a value of '0', and 'P3:' with a value of '0'. The background of the window is a light gray.

Figure 8-9: The Stress-Optic tab in the Material Editor where stress-optic properties are defined.

Young's Modulus (E), Poisson's Ratio (nu), and Thermal Exp. (alpha)

Sets E , ν , and α as defined in the previous section. These parameters are used to compute the spatially-dependant stress/strain distribution. The output values of the stress will be in the same units as Young's modulus.

P1, P2, P3

Sets the strain optic coefficients used to compute the index perturbation as described in the previous section.

8.1. Semiconductor Properties

The semiconductor properties can be defined on the **Semiconductor** tab of the Material Editor. This tab allows users to access the LaserMOD material library and to define certain other parameters that are required when the effects of electronic transport need to be considered. Editing the properties in this tab will most likely be necessary only when using RSoft's Solar Cell Utility, Tapered Laser Utility, or the Multi-Physics Utility, or when the optical properties of compound materials are required.

Setting Optical Semiconductor Properties

The optical properties of a semiconductor are defined on the **Eps Linear** tab of the Material Editor. While the options on this tab are described in detail in [Section 8.B](#), it is worth noting several usage scenarios here.

The refractive index of ternary or quarternary semiconductor material systems, at a specific alloy composition, can be interpolated between those of their constituent binaries. Interpolation functions (`nkinterp2()`, `nkinterp4()`, etc. described in [Section C.F.7](#)) and for extracting the complex index are available for use in the **Eps Linear** Tab of the Material Editor. The ‘x’ and ‘y’ arguments for these functions are set by the **Alloy X** and **Alloy Y** parameters in the Semiconductor Tab, respectively. See notes in the next section for the definition of **Alloy X** and **Alloy Y**.

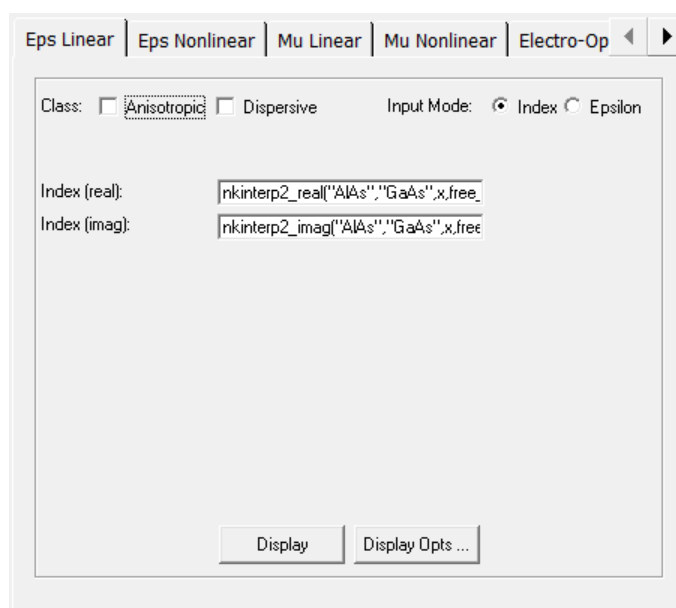


Figure 8-10: The Eps Linear tab in the Material Editor for the AlGaAs (ternary) material system. The complex index is interpolated between those of AlAs and GaAs according to the “Alloy X” parameter in the Semiconductor Tab, and at the wavelength specified by the `free_space_wavelength` symbol.

Setting Semiconductor Properties

For each semiconductor (and certain other materials) available in the RSoft CAD material library, there is a corresponding LaserMOD material library entry. This entry is selected via the **Material System** parameter on the Semiconductor Tab.

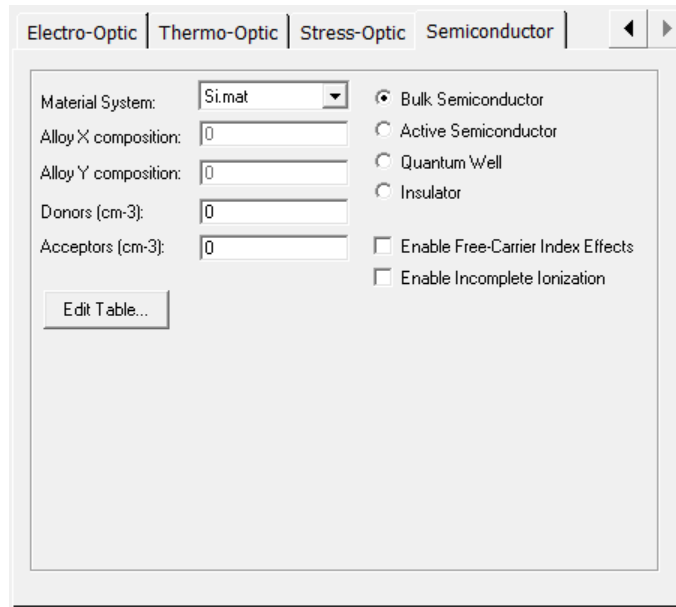


Figure 8-11: The Semiconductor tab in the Material Editor.

As stated in the previous section, the **X Alloy** and **Y Alloy** composition must also be selected for most semiconductor systems:

- *Ternary Material Systems (X only):*

The composition is $A(x)B(1-x)C$. For example, AlGaAs is $Al_xGa_{1-x}As$.

- *Quarternary Material Systems (X and Y):*

The composition is $A(x)B(1-x)C(y)D(1-y)$. For example, GaInAsP is $Ga_xIn_{1-x}As_yP_{1-y}$.

Furthermore, the doping concentration in a particular region of the device must be specified for nearly all electronic simulations; donors for n-type regions and acceptors for p-type regions. *There must be a project material in the problem file for each unique combination of material system, alloy composition, and doping.*

The Semiconductor Tab also allows for the specification of how a material region will be treated by the active simulation; as a (non-optically active) bulk semiconductor, as an optically active bulk semiconductor, as a quantum well structure, or as an insulator. Lastly, the Edit Table button allows access to a local symbol table that permits overriding of any of the parameters in the LaserMOD Material Library.

Including Free Carrier Effects in Silicon

Two options further control the refractive index of Silicon based on the doping levels:

Do not use these options if these effects have already been accounted for in the material model that you are using (i.e. if the material model has come from a TCAD Sentaurus simulation that

already includes free-carrier effects). Using these options in such a scenario will result in a double counting of this effect.

- The complex index perturbation due to free carriers can be included by selecting the **Enable Free-Carrier Index Effects** option. This effect requires that a doping concentration be specified via the **Donors (cm-3)** or **(Acceptors cm-3)** option (otherwise the free-carrier concentration is zero).
- The complex index perturbation is calculated using these equations:

$$\begin{aligned}d_n &= fcn_elcoef * (n^{fcn_elexp}) + fcn_hlcoef * (p^{fcn_hlexp}) \\d_a &= fca_elcoef * (n^{fca_elexp}) + fca_hlcoef * (p^{fca_hlexp})\end{aligned}$$

where n and p are the doping concentrations in cm^{-3} . Note that d_n and d_a are scaled by $(\lambda / \lambda_0)^{fca_lamexp}$, and are in dimensionless and cm^{-1} units respectively. The 'fca' coefficients and exponents can be defined in several ways:

- *Using Published Data:*

There are two sets of published data available taken from Ref 1 and Ref 2 below. Set `fca_method = 1` to use the data from Ref 1 (default), set `fca_method = 2` to use the data from Ref 2. These symbols should be set in the symbol table in the Semiconductor tab.

- *Using Custom Data:*

To use custom coefficients/exponents, create a simple text file with the desired parameters and values. Then, set `fca_file` to the name of the file in the Symbol table in the Semiconductor tab. Do not set the `fca_method` symbol, it takes precedence. An example file could be:

```
fcn_elcoef = 0.0
fcn_hlcoef = 0.0
fca_elcoef = 0.0
fca_hlcoef = 0.0
fcn_elexp = 1.0
fcn_hlexp = 1.0
fca_elexp = 1.0
fca_hlexp = 1.0
fca_lam0 = 1.0
fca_lamexp = 0.0
```

Note that this example shows the default value for each parameter. If a parameter does not appear in the file, its default value will be used.

- Furthermore, the number of free carrier can be impacted by incomplete ionization. This model is enabled by the **Enable Incomplete Ionization** option.

Using a Custom Doping Profile

It is possible to define the doping profile via data files allowing for custom profiles. The following symbols control this feature:

Variable	Description
<code>dope_file_import</code>	Set to 1 to enable the use of custom doping profiles.
<code>dope_file_donor</code>	Sets the donor profile.
<code>dope_file_acceptor</code>	Sets the acceptor profile.
<code>dope_file_net</code>	Sets the net doping profile.

Note that negative values indicate acceptors, while positive values indicate donors. Units are always cm^{-3} . Also, all profiles should have the same coordinate system as the simulation and will be applied to the entire domain.

References:

- 1 Soref, R. et al, IEEE Journal of Quantum Electronics, **QE-23** (1987)
- 2 Nedeljkovic et al, IEEE Photonics Journal **3** (2011)

8.K. Special Materials

Some materials have additional options and/or require additional configuration. These include:

- *Graphene*
GrapheneCX
GrapheneCY
GrapheneCZ

The ‘Graphene’ material is isotropic and the GrapheneCX, GrapheneCY, and GrapheneCZ materials are anisotropic. The name of the anisotropic material denotes the out-of-plane or stacking direction: for example, GrapheneCY is oriented so that the Graphene layer is in the XZ plane. In general the isotropic Graphene model is preferred, but depending on your polarization and Graphene orientation it may be necessary to use one of the anisotropic models. Each design file can contain multiple Graphene models, each with different properties.

These Graphene models require additional material properties that are set in the special symbol table in the semiconductor tab described in [Section 8.I](#). Click **Edit Table** on the Semiconductor tab to change these values. Note that these material models are based on the `graphene_real()` and `graphene_imag()` functions described in [Section C.F.7](#): see this

section for a complete description of the mathematical model used and how these parameters are used to calculate the Graphene properties.

While the built-in material uses reasonable default values for these parameters, it is critical that you use values that correspond to your specific situation.

Parameter	Description & Notes
U_	Sets the chemical potential [eV]
G1_	Sets the intraband Γ_1 [eV]. Note: Γ [eV] = $\hbar \Gamma$ [1/sec] To convert a time constant τ , typically $\Gamma = 1/(2\tau)$
G2_	The interband Γ_2 [eV] See notes above for Γ units and conversions Setting G2_ to -1 makes $G2_ = G1_$.
T_	Sets the temperature [K]
H_	Sets the Graphene thickness H [μm], typically set to 0.0007
EpsC_	Reference Eps (ϵ_{ref}) for in-plane calculation described in Section C.F.7
E_	Reference Eps (ϵ_{ref}) for out-of-plane direction, only used for anisotropic Graphene models

Non-Uniform Grids

The RSoft CAD uses an intelligent grid generation algorithm to produce a simulation grid based on the structure(s) defined in the CAD environment, as well as a variety of grid options described in detail later in this chapter. The grid generation algorithm automatically calculates a uniform or non-uniform grid best suited for the defined structure. The grid is composed of rectangular shaped cells, or elements, of varying sizes, which cover the entire computational domain, including any PML. Additionally, FemSIM supports a hybrid triangular and rectangular grid which is described in more detail in the FemSIM manual.

Please note that not all options are available for all simulation programs. BeamPROP and FemSIM's capabilities are described in this chapter, FullWAVE's non-uniform capabilities are described in the FullWAVE manual.

9.A. Grid Basics

The entire grid is built on a base unit called *grid cells*. The definition of a grid cell differs for BeamPROP and FemSIM, specifically how fields are calculated for each cell:

- **BeamPROP:** A finite-difference algorithm is used to represent field values at specific grid points. These grid points are located at the center of each grid cell as shown in Fig. 9-1a.
- **FemSIM:** A finite element method is used which represents the field over the entire cell as a function of pre-determined basis functions as shown in Fig. 9-1a. Furthermore, FemSIM

supports a subdivision of cells into triangular elements. This option is described in further detail in the FemSIM manual.

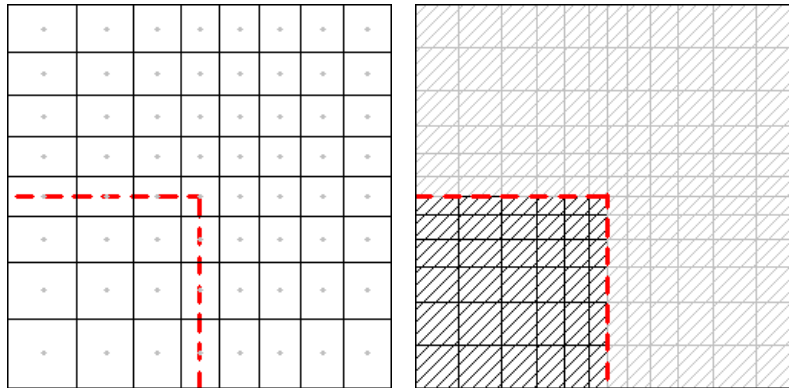


Figure 9-1: The grid cell definition for a) BeamPROP and b) FemSIM. The dashed red line represents a material boundary, the solid black lines represent cell boundaries, and the gray dots represent the grid points. The FemSIM cells are shaded to indicate that the field is defined over the entire cell.

9.B. Advanced Grid Controls

The main spatial domain and grid controls are located in the Simulation Parameters dialog. The **Enable Nonuniform** option in the Simulation Parameters dialog controls whether a non-uniform grid is used. This non-uniform grid is based on the structure defined in the RSoft CAD, the materials used, the simulation wavelength, and several other options. The grid options are set in the Advanced Grid Controls dialog (shown in Fig. 9-2) which is opened by clicking the **Grid Options...** button in the Simulation Parameters window. Each coordinate has separate grid controls and are described in this section. Note that it is possible to overspecify the grid size, the final simulation grid used is the union of all grid settings such that the smallest grid specified at any location is used.

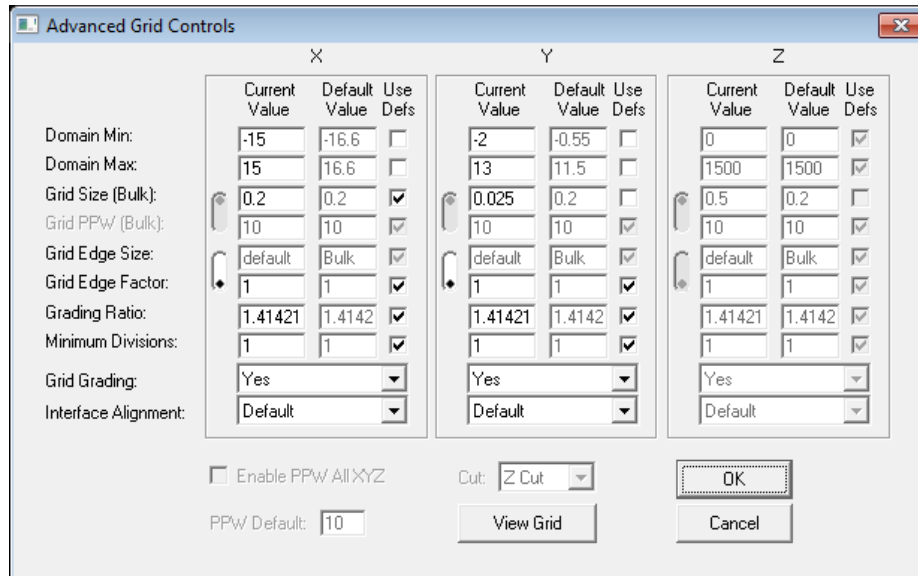


Fig 9-2: The BeamPROP Advanced Grid Controls dialog where advanced grid options are set. Note that FemSIM contains several additional options which are addressed in the FemSIM manual.

RSoft's grid generation algorithm is capable of creating a non-uniform rectangular grid based on the structure defined in the CAD and other grid options that allow user control over the size and position of grid cells. This section describes these grid options, which include: interface alignment, grid size at interfaces, grid grading, and required minimum divisions within a region.

Spatial Domain:

The **X**, **Y**, and **Z Domain Min** and **Domain Max** options set the simulation domain and are duplicates of the same options in the main Simulation Parameters dialog. See the BeamPROP and FemSIM manuals for a description of these options.

Grid Sizes:

The simulation grid can be defined in several ways, choose the method that is best for your specific application. The simplest method to use a non-uniform grid is to set the **Grid Size** and the **Enable Nonuniform** options. It is possible to over specify the grid size, the final simulation grid used is the union of all grid settings such that the smallest grid specified at any location is used.

When using a non-uniform grid, the simulation grid is smoothly varied between regions using the **Grid Grading** option described below. If needed, the grid within a particular component or material can be set directly via the tags described in [Section 9.C](#).

- **Grid Size (Bulk)**

This is the primary control for grid generation and are duplicates of the similarly named **Grid Size** option in the main Simulation Parameters dialog. See the BeamPROP and FemSIM manuals for a description of this parameter.

- **Grid Edge Size or Grid Edge Factor**

These options help accurately model field variations at material interfaces which are typically larger than in bulk regions. **Grid Edge Size** sets the grid size at the edge of a material interface in spatial units [μm], whereas **Grid Edge Factor** sets the edge grid relative to the ‘bulk’ grid. Only one method may be used per spatial coordinate, use the slider control to set which option is used. **Grid Edge Factor** is used by default, use the radio buttons to select **Grid Edge Size** if needed. Note that **Grid Edge Factor** should be between 0 and 1.

- **Minimum Divisions**

This option sets the minimum number of grid cells used between two material interfaces and has a default value of 1. This option should be set to an integer (1 or greater); it is recommended to set it equal to the thickness of the region of interest divided by the desired “average” grid size.

Grid Grading:

The **Grid Grading** option automatically grades, or varies, the grid size between regions. The **Grading Ratio** sets the rate at which adjacent grid sizes vary and defaults to $\sqrt{2}$. A recommended guideline is to set this parameter to a value between 1.1 and 2 depending on the structure. A value that is too small increases the computation burden and a value that is too large can create accuracy issues.

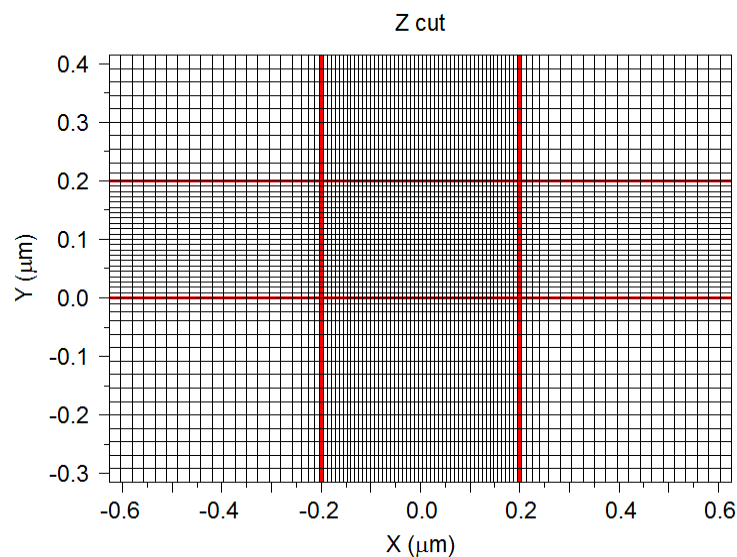


Fig 9-3: A sample non-uniform grid that uses grid grading between the small grid in the structure core and the background regions.

Additional Grid Options:

The **Interface Alignment** option controls how grid cells are aligned to interfaces between material regions. This option has several possible choices, including Default, None, Centered Cells, or Straddled Cells. The default is *Centered Cells*, setting to *None* can be useful to approximate a uniform grid along a particular axis when **Enable Nonuniform** is enabled.

9.C. Overriding Grid Settings for Specific Components or Materials

While the **Grid Size** and **Grid PPW** options give adequate control over the grid for most cases, it can sometimes be advantageous to set the desired grid size(s) within a particular component or material. This allows, for example, more/less grid points to be used in a particular region of interest.

All grid options described above can be enabled/disabled for particular components or materials using tags. Tags can be assigned to a particular component directly or via the **Edit Default Tags** option in the Edit menu item in the RSoft CAD. Multiple tags can be set, use spaces between tags. The final grid will be the union of the global grid settings and any tags set (the tags take precedence). Note that a non-uniform grid must be used to enable these options.

The tags that control grid related parameters are:

See sections above for details about what each option does, including recommended usage.

Tag	Description
grid_size_x grid_size_y grid_size_z	These tags correspond to the X, Y, and Z Grid Size options which sets the grid size in μm units.
grid_ppw_x grid_ppw_y grid_ppw_z	These tags correspond to the X, Y, and Z Grid PPW options which sets the grid size in points-per-wavelength units.
grid_edge_x grid_edge_y grid_edge_z	These tags correspond to the X, Y, and Z Grid Edge options which sets the grid edge in μm .
grid_edge_factor_x grid_edge_factor_y grid_edge_factor_z	These tags correspond to the X, Y, and Z Grid Edge Factor options which sets the grid edge factor.
grid_mindiv_x grid_mindiv_y grid_mindiv_z	These tags correspond to the X, Y, and Z Grid Minimum Divisions options.
grid_bulk_nonuniform_x	These tags correspond to the X, Y, and Z Grid Grading

grid_bulk_nonuniform_y
grid_bulk_nonuniform_z

options which enable/disable grid grading. Set to 0 to disable, 1 to enable.

grid_ratio_x
grid_ratio_y
grid_ratio_z

These tags correspond to the **X**, **Y**, and **Z Grid Ratio** options which sets the grid grading ratio

Grid Tag Example:

To set the X grid size to 35 points-per-wavelength and the Y grid size to 1 nm within a particular component in the RSoft CAD, set the **Tags** option in the Additional Component Properties dialog to:

```
grid_ppw_x=35 grid_size_y=0.001
```

9.D. Viewing the Grid

The simulation mesh can be viewed by pressing the **View Grid** button in either the Simulation Parameters window or within the Advanced Grid Controls window. For 3D simulation, the cross-section of the mesh can be selected via the **Cut** option. Note that PML will be displayed with the mesh.

Note that the grid can also be viewed on top of the refractive index using the **Overlay Grid** option.

10

Scripting & Scanning

The RSoft device simulation tools share a comprehensive set of scripting features for layout, simulation, parameter scanning/optimization, and data manipulation. Layout scripting is done via a Python API, simulations scripting is done via the user's operating system (ANY language), and data manipulation is done via a built-in set of utilities and/or a Python API. Additionally, MOST can be used to automate tedious scanning and optimization problems.

10.A. Layout Scripts

There are two main ways to script device layouts:

- Using scripts within the RSoft CAD
- Generating design files completely via script

The method you use will depend on your exact need. Each method has benefits, and the methods can also be combined as needed.

10.A.1. Using Scripts within the RSoft CAD

The hierarchical approach of using scripts with the RSoft CAD is beneficial for many situations. The control file can pass relevant design parameters to the child file(s). Child files can be RSoft .ind files, GDS files, or Python scripts. This modular approach allows portions of the design to easily change without having to continuously regenerate the entire design. See [Appendix H](#) for a description of the Python API to generate design files, including supported Python versions.

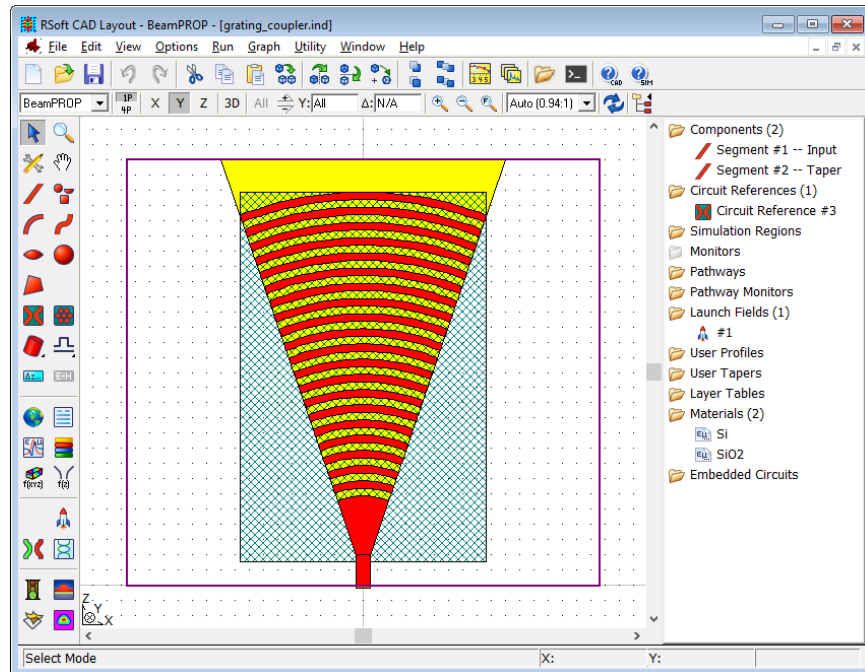


Figure 10-1: A fiber-grating coupler shown in the RSoft CAD. The description of the structure is below. The colors are chosen to show the etch depth, red is a standard waveguide, yellow can be either a partial or full-etch.

As an example, consider a standard fiber-grating coupler shown above. This design consists of:

- A series of structure properties, materials, and simulation settings set via symbols such as the input waveguide width, number of arc segments, grid sizes, etc. (not shown)
- Two static components (straight red waveguide at bottom and tapered yellow waveguide) that do not need to be scripted, their settings can easily be taken from the symbols in the control file.
- A launch field and several monitors for measurement purposes that do not need to be scripted, their properties can again be easily set by the symbols in the control file.
- A series of dynamic components (arc segments shown in red) that need to be dynamically produced based on the number of rings setting in the control file.

For this example, we define everything except the yellow arc segments in the control file and use a Circuit Reference block to dynamically call the Python API to produce the arc segments.

See [Appendix H](#) for a description of the Python API to generate design files, including supported Python versions.

Passing Parameters to a Python Script

There are several ways to pass settings and/or symbols from the control file to the Python script. The method you choose depends on the parameter or setting type as well as the which method is more efficient. There are two types of parameters:

While any parameter can be passed as an argument, it is usually better to use the other methods if possible to keep the Python code simple, clean, and reusable.

- Parameters that, when changed, require a new design file to be generated. These parameters typically control how many components are in the design file or something similar. This type of parameters must be passed to the script as script arguments. See [Section 6.F.6](#) for more details about passing script arguments to the Python script as well as the examples below.
- Parameters that can be changed through Symbols. These parameters typically control the properties of components. This type of parameters can be passed to the `.ind` file generated by the script through the options in the circuit reference dialog and/or the Local Symbols table for the circuit reference. See [Section 6.F.2](#) for details about setting options in the circuit reference dialog, see [Section 6.F.6](#) more details about using hierarchy exports with the Python API, and the examples below.

Example 1-1: Grating Coupler

We begin by going through the example described above in more detail. The files for this example are located here:

```
<rsoft_dir>\examples\RSoftCAD\scripting\Python\grating_coupler\
```

Open the file `grating_coupler.ind`. This is the master control `.ind` file that uses the Python script `grating_coupler.py` which generates the series of yellow arcs.

We will not cover the creation of the full master control file `grating_coupler.ind` here, instead we will focus on the circuit reference component that uses the Python API to draw the yellow arc segments.

Creating a Circuit Reference using a Python script

Right-click on the yellow arcs to open the Circuit Reference dialog to see how the Python script is loaded and how the necessary parameters are passed.

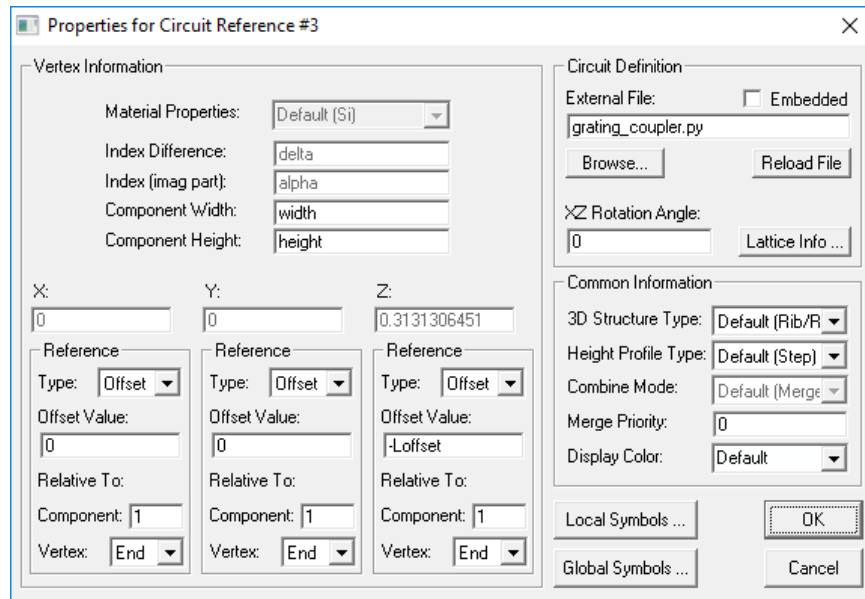


Figure 10-2: The Circuit Reference dialog used to call the Python script.

Note that the **External File** is set to `grating_coupler.py` which will call the Python script to generate the layout. This script (which we will go through below) generates an `.ind` file containing the arc segments.

Passing Parameters to the Script and/or Generated `.ind` File

Before looking at the script, let's consider the symbols and settings that are needed to define the grating arcs:

Symbol/Property	Description
N	Sets the number of rings
Period	Sets the period/spacing of the rings
Angle	Sets the half-angle of the rings
Rmin	Sets the minimum radius of the rings
Material Properties	Sets the material of the rings
Period	Sets the periodicity of the rings
Duty	Sets the duty cycle of the rings
Depth	Sets the height of a ring
Merge Priority	Sets the merge priority of the ring, used because the rings have a lower index than the Si waveguide
Display Color	Sets the color of the rings

There are several ways to pass settings and/or symbols from the control file to the Python script:

- *Circuit Reference Component Settings:* Many settings can be simply set in the properties dialog for the Circuit Reference. As shown in Fig. 10-2, we have set the **Material Properties** of the rings to **SiO2** which matches the upper cladding, we have set the **Component Width** to **Period*Duty**, and **Component Height** to **Depth**. These symbols are evaluated in the parent file and will be automatically passed to the `.ind` file created by the Python script. Also set here is the **Merge Priority** and **Display Color**.
- *Hierarchy Exports:* Other settings that can be simply passed to the `.ind` file created in the Python script are `Period`, `Duty`, `Rmin`, and `Angle`. These symbols have been set in the Local Symbols table (click **Local Symbols** to open) shown below. They have also been set as Hierarchy Exports in the Python script so that they automatically appear in the Local Symbols table.

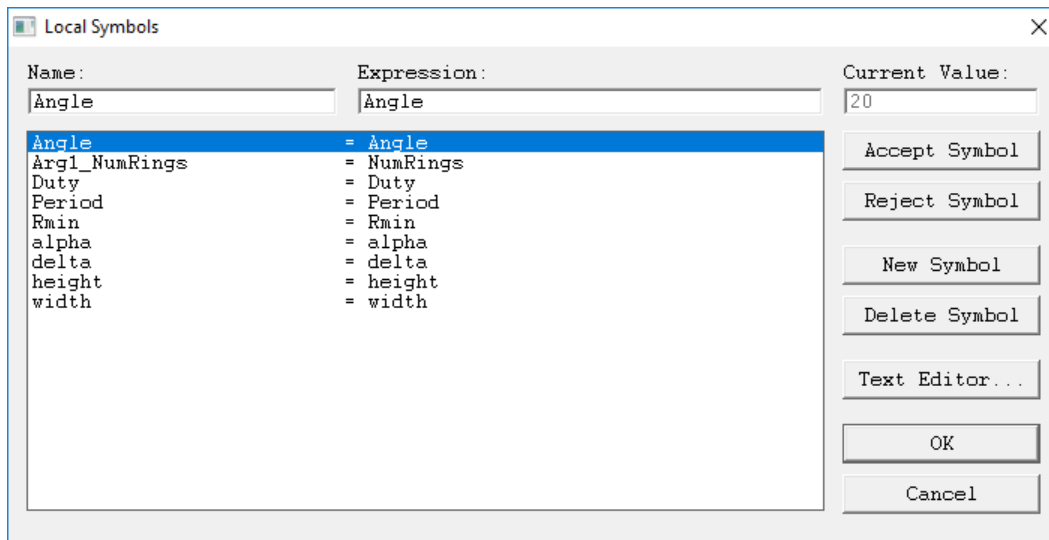


Figure 10-3: The Local Symbols table for the Circuit Reference.

- *Script Arguments:* This method is required for properties that cannot be simply passed as symbols. The only property of the grating coupler that this is true for is `N`, the number of arcs. Script arguments are set in the Local Symbol table with names that start with the prefix `ArgX`. Here we have one symbol `Arg1_NumRings` which is set to `NumRings`. Had there been more arguments, they should start with names like `Arg2_`, `Arg3_`, etc. We'll note this later, but the script should have default values for arguments in case they are not defined in the Local Symbols table.

The Python Script

Open the file `grating_coupler.py` in the text editor of your choice or by double-clicking on the red arcs in the RSoft CAD. We will go through this script line by line.

You can select the editor used when opening from the CAD by setting the environment variable `RSOFT_EDITOR` to the text editor of your choice.

```
1
2 from rstools import RSoftCircuit
3
4 def rsoft_layout(NumRings=int(10)):
5
6     # create a circuit
7     c=RSoftCircuit()
8
9     # override default symbols as needed
10    c.set_symbol('height',1)
11
12    # add symbols for dynamic control of layout and export as needed
13    c.set_symbol('Period',0.5);           #Period
14    c.set_symbol('Duty',0.5);             #Minimum Radius
15    c.set_symbol('Angle',10);             #Half-Angle
16    c.set_symbol('Rmin',4);                #Minimum Radius
17    c.set_symbol('W','Period*Duty')
18    c.set_exports('Period Duty Angle Rmin')
19
20    # add components
21    for i in range(1,NumRings+1):
22        x = '-(Rmin+%s*Period)*sin(Angle)\n' % (i-1)
23        z = '(Rmin+%s*Period)*cos(Angle)\n' % (i-1)
24        r = '(Rmin+%s*Period)\n' % (i-1)
25        ia = '90-Angle'
26        fa = '90+Angle'
27        w = 'W'
28        c.add_arc((x,0,z),(r,ia,fa),(w))
29
30    # add initial sector
31    draw_sector=True
32    if draw_sector:
33        x = '-0.5*(Rmin+W/2)*sin(Angle)\n'
34        z = '0.5*(Rmin+W/2)*cos(Angle)\n'
35        r = '0.5*(Rmin+W/2)\n'
36        ia = '90-Angle'
37        fa = '90+Angle'
38        w = 'Rmin-2*Period+W/2'
39        c.add_arc((x,0,z),(r,ia,fa),(w))
40
41    # return the circuit
42    return c
43
```

Figure 10-4: The Python script for the grating coupler.

See [Appendix H](#) for a description of the Python API to generate design files, including supported Python versions.

- *Line 2:* Loads the RSoft API

- *Line 4:* Define the `rsoft_layout()` function, the recommended function that will be automatically run by the hierarchy block to build the `.ind` file. Note that the arguments to the `rsoft_layout()` function will be automatically parsed from the arguments used when calling the script, automatically set by the Local Symbols that start with the prefix `ArgX` as described above. In this case there is just one argument `NumRings` which has a default value of 10. It is recommended to always define a default value so that the script will run even if the argument symbols are not defined in Local Symbols.
- *Line 7:* Creates the `RSoftCircuit()` object `c`.
- *Lines 10-18:* Sets various symbols and values, and then sets four of these symbols as hierarchy exports so that they appear in the Local Symbols table.
- *Lines 20-28:* Defines a for loop to create the arc segments. The starting (x,z) value, radius, initial and final angle, and width are calculated and then the arc is created using the `add_arc()` function.
- *Lines 30-39:* Defines the starting sector or wedge based on `Rmin`.
- *Line 42:* Returns the `RSoftCircuit` object

Putting it All Together:

Once defined, and once the various parameters and symbols are defined, the grating coupler can be used just as any other design file. Changing the options/symbols in the master file `grating_coupler.ind` will control the entire design. For example, changing `N` in the symbol table of the master file will change the number of rings produced by the Python script. Note that the script is not run every time the display is refreshed for efficiency purposes, you can force the script to be re-run by selecting View/Refresh Hierarchy from the CAD menu. Also note that the design is always refreshed before running a simulation.

Example 1-2: Binary Phase Mask

This example uses a data file to define a patterned binary phase mask element. The data file defines the local thickness of the structure for one period (unit cell). Specifically, a 0 value in the data file corresponds to fully etching that position, a value of 1 corresponds to not etching that position. The final grating is shown below.

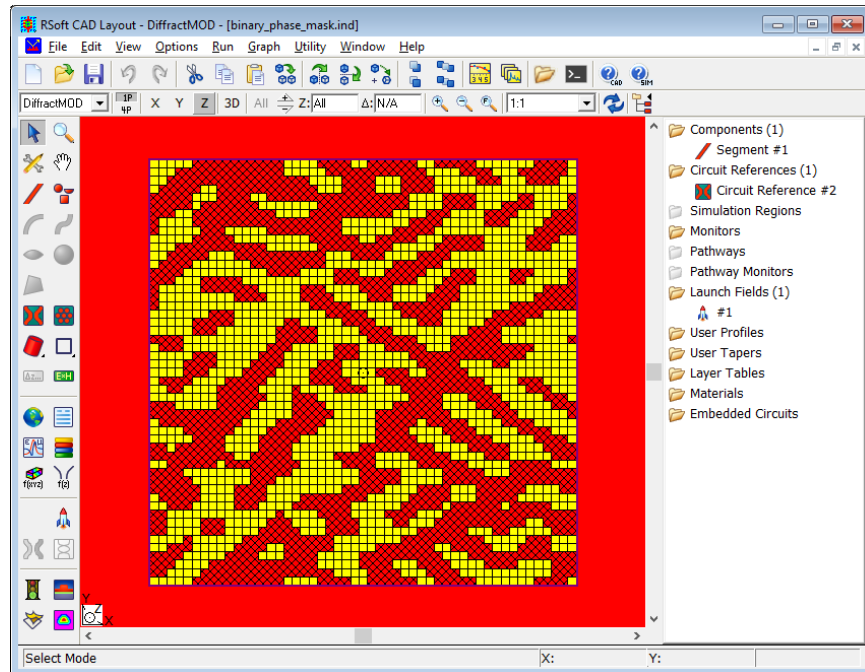


Figure 10-5: Top-view of an example binary phase mask.

This example uses both Python APIs (`RSoftUserFunction()` and `RSoftCircuit()`). We begin by going through the example described above in more detail. The files for this example are located here:

```
<rsoft_dir>\examples\RSoftCAD\scripting\Python\binary_phase_mask
```

Open the file `binary_phase_mask.ind`. This is the master control `.ind` file that uses the Python script `binary_phase_mask.py` which generates the yellow boxes based on the data in the file `binary_phase_mask.dat`.

We will not cover the creation of the full master control file `binary_phase_mask.ind` here, instead we will focus on the circuit reference component that uses the Python API to draw the yellow box segments. The only input is the data file `binary_phase_mask.dat`, the script automatically scales the data to fit within the a single unit cell (defined by the symbols `PeriodX` and `PeriodY` variables) in the `binary_phase_mask.ind` file. The data file is passed as an argument in the Local Symbol table for the circuit reference.

The Python Script:

Open the file `binary_phase_mask.py` in the text editor of your choice or by double-clicking on the yellow boxes in the RSoft CAD. We will go through this script line by line.

```

1
2 from rstools import RSoftCircuit
3 from rstools import RSoftUserFunction
4
5 def rsoft_layout(data_file=''):
6
7     # create a circuit
8     c = RSoftCircuit()
9
10    # add symbols for dynamic control of layout and export as needed
11    c.set_symbol('L', 1)
12    c.set_exports('L')
13
14    # read data file
15    # if successful, add components as needed to represent the binary mask
16    data = RSoftUserFunction()
17    if data.read(data_file):
18        (x,y,f) = data.get_arrays()
19
20        # add symbols
21        c.set_symbol('NX', len(x))
22        c.set_symbol('NY', len(y))
23        c.set_symbol('DX', 'width/NX')
24        c.set_symbol('DY', 'height/NY')
25
26        # add components
27        for i in range(len(x)):
28            for j in range(len(y)):
29                if f[i][j] >= 0.5:
30                    X = '(%s+0.5)*DX'%i
31                    Y = '(%s+0.5)*DY'%j
32                    c.add_segment((X,Y,0),(0,0,'L'),('DX','DY'))
33
34    # return the circuit
35    return c
36

```

Figure 10-6: The python script to generate the binary phase mask.

See [Appendix H](#) for a description of the Python APIs to read data files and generate design files, including supported Python versions.

- *Lines 2-3:* Loads the RSoft APIs.
- *Line 5:* Define the `rsoft_layout()` function, There is one argument, `data_file`. This will be set by the symbol `Arg1_data_file` in the Local Symbol table of the circuit reference.
- *Line 8:* Creates the `RSoftCircuit()` object `c`.
- *Lines 16-18:* Reads data from the file using the `RSoftUserFunction()` class into three arrays: the x coordinates, y coordinates, and 2D data. Note that an if function is used to protect against the case of a missing file

- *Lines 20-24:* Define several symbols, including `DX` and `DY` which represent the X and Y size of each of the pixels.
- *Lines 27-32:* Loop through all elements in the data array and, if set to 1, add a segment at that position.
- *Line 35:* Return the `RSoftCircuit` object

Example 1-3: Metalens Design

This example uses a Python script to draw a simple Metalens design. The design is characterized by a data file which contains hole radius as a function of lens radius. The final structure is shown below.

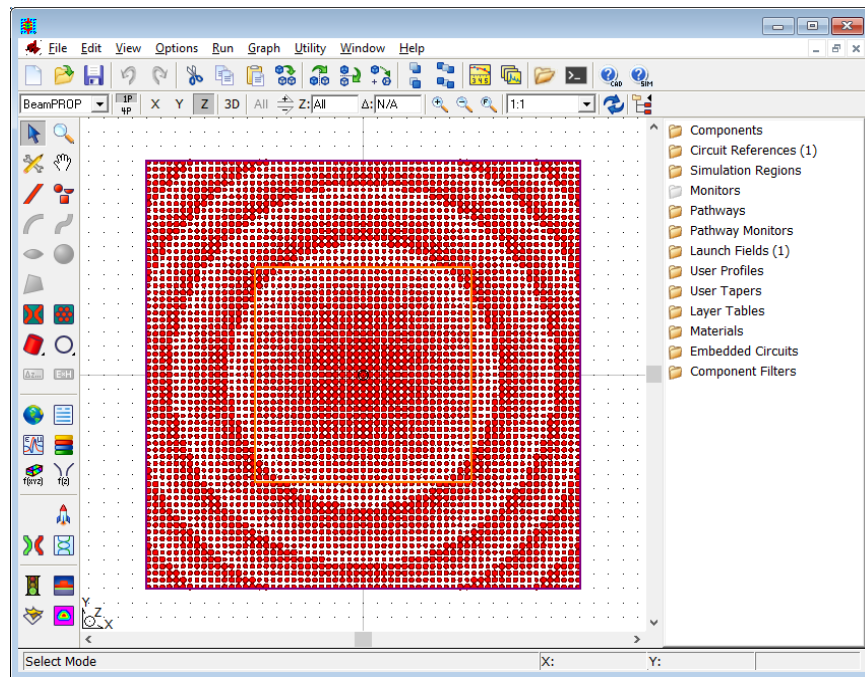


Figure 10-7: The simple metalens design shown in the RSoft CAD.

This example uses both Python APIs (`RSoftUserFunction()` and `RSoftCircuit()`). We begin by going through the example described above in more detail. The files for this example are located here:

```
<rsoft_dir>\examples\RSoftCAD\scripting\Python\metalens
```

Open the file `metalens.ind`. This is the master control `.ind` file that uses the Python script `metalens.py` which generates the yellow boxes based on the data in the file `diameter_ratio.dat`.

We will not cover the creation of the full master control file `metalens.ind` here, instead we will focus on the circuit reference component that uses the Python API to draw metalens.

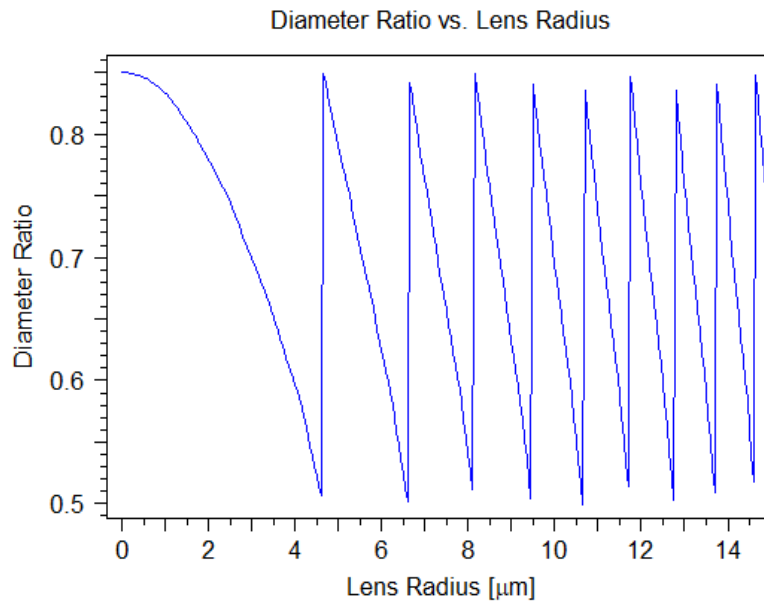


Figure 10-8: The data file used to defined hole radius as a function of lens radius.

The Python Script

Open the file `metalens.py` in the text editor of your choice or by double-clicking on the red arcs in the RSoft CAD. We will go through this script line by line.


```

1
2 from rstools import *
3 from math import *
4
5 def rsoft_layout(diameter_ratio_file='',
6                 Period=0.24,
7                 D=10):
8
9     # create a circuit
10    c=RSoftCircuit()
11
12    # set symbols
13    c.set_symbol('Thickness', 1)
14    c.set_exports('Thickness')
15
16    # read data file
17    diameter_ratio_data=RSoftUserFunction()
18    diameter_ratio_data.read(diameter_ratio_file)
19    ..
20    # calculate array dimension
21    N=2*int((D/2)/Period)+1
22
23    # build metalens
24    for i in range(N):
25        for j in range(N):
26            x=(i-(N-1)/2)*Period
27            y=(j-(N-1)/2)*Period
28            r=sqrt(x**2+y**2)
29            ratio=diameter_ratio_data.eval(r)
30            c.add_segment((x,y,0),(0,0,'Thickness'),(ratio*Period,ratio*Period))
31
32    # return the circuit
33    return c

```

Figure 10-9: The Python script used to draw the simple metalens.

See [Appendix H](#) for a description of the Python API to generate design files, including supported Python versions.

- *Line 2:* Loads the RSoft API
- *Line 3:* Loads the math library so we can use the `sqrt()` function
- *Line 4:* Define the `rsoft_layout()` function, the recommended function that will be automatically run by the hierarchy block to build the `.ind` file. Note that the arguments to the `rsoft_layout()` function will be automatically parsed from the arguments used when calling the script, automatically set by the Local Symbols that start with the prefix `ArgX` as described above. Here we set the data file, Period, and Diameter.
- *Lines 10-14:* Create a new circuit and add symbols and exports.
- *Lines 17-18:* Read in data file
- *Line 21:* Calculate the array dimension based on the period and diameter
- *Lines 24-30:* Draw the array, using the data from the file.
- *Lines 33:* Return the circuit object.

10.A.2. Generating Design Files Completely via Scripts

There are some situations where it is beneficial to generate design files completely via scripting and not invoke the RSoft CAD. Note that all functionalities may not be available when operating in this mode, please contact technical support with questions. Also note that the designer will have to make choices such as grid sizes and perform device optimization as usual.

Example 2-1: 1x2 MMI

As an example, consider a 1x2 MMI device. Here we will create this device completely via the Python API.

```
<rsoft_dir>\examples\RSoftCAD\scripting\Python\mmi_1x2\
```

The Python file is `make_mmi_1x2.py` and can be run by moving to the directory above and running this command:

```
rspython make_mmi_1x2.py
```

The file `mmi_1x2.ind` will be generated.

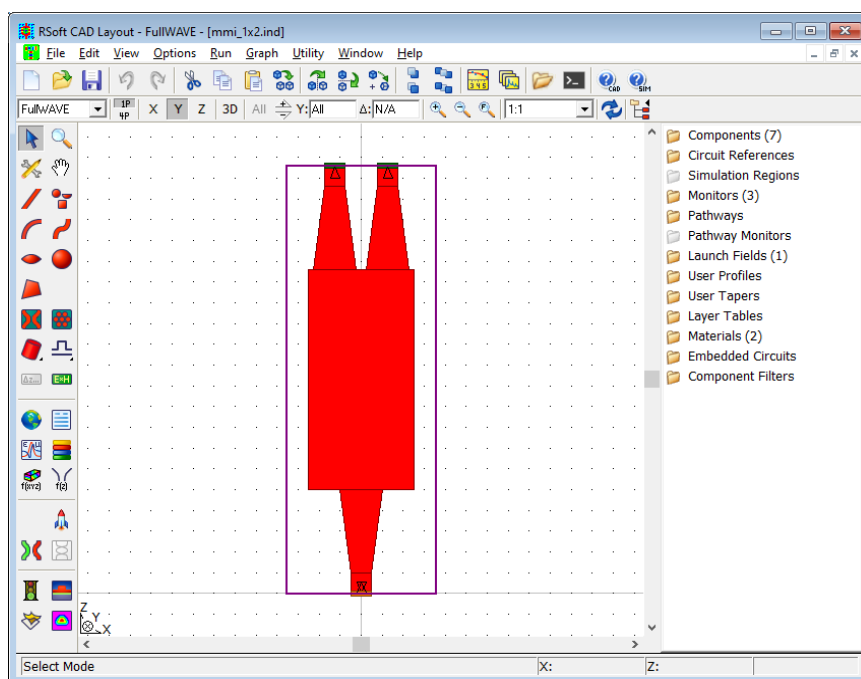


Figure 10-13: The 1x2 MMI shown in the RSoft CAD.

The Python Script:

Open the file `make_mmi1x2.py` in the text editor of your choice, here is a description of each line:

See [Appendix H](#) for a description of the Python API to generate design files, including supported Python versions.

- *Line 1:* Loads the RSoft API
- *Line 4:* Defines the name of the file to produce.
- *Lines 10-17:* Defines the properties of the 1x2 MMI, these properties will become symbols. Note that here we use a Python dictionary to store the options as an illustration, there are many ways to create this type of script
- *Lines 20-44:* Sets various simulation options, these properties will become symbols. Again we use the same Python dictionary.
- *Lines 47-50:* Creates the `RSoftCircuit` object, loads a standard Silicon settings file, and sets the symbols defined in previous lines.
- *Lines 53-59:* Adds components in to draw the MMI. Note that the position of the components is not explicitly defined, these components will be attached together to form the structure in the next section.
- *Lines 61-66:* The components are attached to form the 1x2 MMI.
- *Lines 69-71:* Port monitors are created and attached.
- *Line 78:* Write the `.ind` file.

Example 2-2: Waveguide Crossing from GDS

Another design flow is to load the structure from a GDS file. Each layer has a corresponding position, height, and material based on the foundry stack data.

```
<rsoft_dir>\examples\RSoftCAD\scripting\Python\  
waveguide_crossing_from_gds\
```

The structure is a waveguide crossing contained in the file `roWaveguideCrossing.gds`, and contains two layers: layer 88/0 which corresponds to a standard Si rib waveguide, and layer 89/0 which corresponds to a partial-etch Silicon waveguide.

The Python script is `make_WaveguideCrossing_from_gds.py` and can be run by moving to the directory above and running this command:

```
rspython make_WaveguideCrossing_from_gds.py
```

The file `roWGCrossing.ind` will be generated.

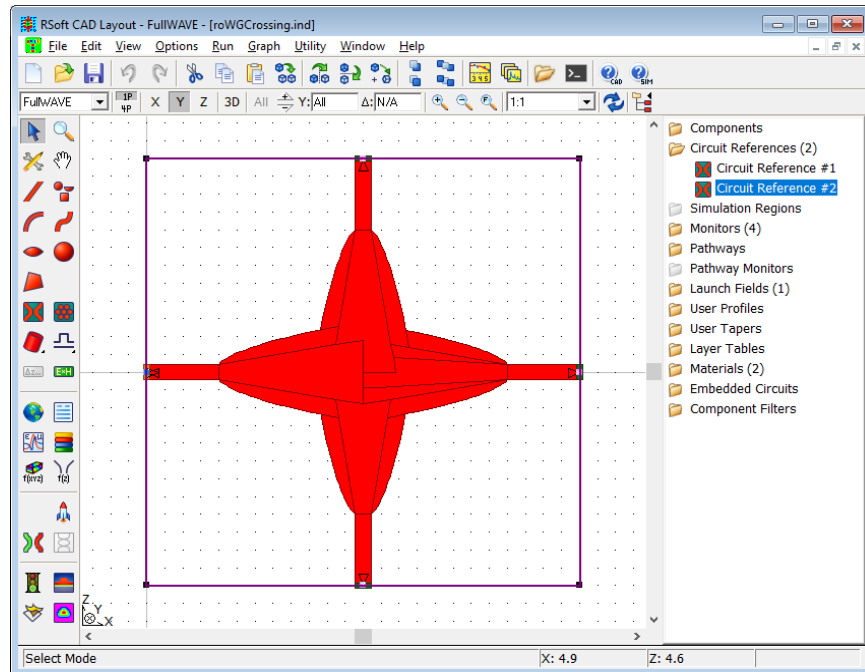


Figure 10-14: The waveguide crossing shown in the RSoft CAD.

The Python Script:

Open the file `make_WaveguideCrossing_from_gds.py` in the text editor of your choice, here is a description of each line:

Note that in this example we hardcode the layer properties (vertical position, height, and material), see the next example for how load stack data from a particular PDK. Also, see [Appendix H](#) for a description of the Python API to generate design files, including supported Python versions.

- *Line 1:* Loads the RSoft API
- *Line 4:* Defines the name of the file to produce.
- *Line 10:* Defines the GDS file to use
- *Lines 13-14:* Defines the layers to import, and the layer where ports should be added.
- *Lines 17-19:* Defines the layer properties such as the base (Y position), height, and material. These arrays are tied to the layers defined above.
- *Lines 21-50:* Defines the simulation parameters for the waveguide crossing. Note that we use a Python dictionary to store the options as an illustration, there are many ways to create this type of script. Also, note that the transverse geometry is set by the GDS file.

- *Lines 53-56:* Creates the `RSoftCircuit` object, loads a standard Silicon settings file, and sets the symbols defined in previous lines.
- *Lines 59-79:* Adds layers, setting the position, height, and material. The ports are added to the correct layer.
- *Line 64:* Write the `.ind` file.

Example 2-3: Waveguide Crossing from GDS using PDK Stack Data

This example is an extension of Example 2-2 above, using stack data directly from the referenceOpto PDK. This concept can be generalized to any PDK.

Note that you will need access to the referenceOpto PDK to run this example.

The files associated with this tutorial can be found here:

```
<rsoft_dir>\examples\RSoftCAD\scripting\Python\
waveguide_crossing_from_gds_refopto\
```

The Python script is `make_WaveguideCrossing_from_gds_refopto.py` and can be run by moving to the directory above, ensuring that your environment points to the refOpto PDK, and running this command:

```
rpython make_WaveguideCrossing_from_gds_refopto.py
```

The file `roWGCrossing_refopto.ind` will be generated.

The Python Script:

Open the file `make_WaveguideCrossing_from_gds_refopto.py` in the text editor of your choice. The script is the same as Example 2-2 with the following differences:

- The layers info is not set at the top of the script, this data will come from the `pdkinf` file (`refopto` stack info).
- A settings file from the PDK is used.
- The code to set the layer properties is slightly different. Instead of accessing the layer info arrays, the settings are made via the `pdkinf()` function. The data in the `refOpto` `pdkinf` file are named by layer numbers, for example `L_88_0_material = "Si"` and `L_89_0_height = 0.22`. The script builds strings based on the layer number, automatically loading the stack data when the `.ind` file is opened or run.

10.B. Parameter Scanning and Optimization

The RSoft CAD allows users to automatically scan over design parameters to find an optimal design. This is done by using RSoft's MOST package. Details about the usage of MOST can be found in the MOST manual.

10.C. Data Manipulation

The RSoft tools produce many types of data, and in many cases this data is enough for the designer's needs. There are cases, however, where it can be useful to manipulate the data for post-processing or other purposes. This can be done through WinPLOT, command line utilities, and/or the `RSoftUserFunction()` Python API.

- *Manipulating data with WinPLOT:*

WinPLOT, the plotting software included with the RSoft tools, provides options to scale, translate, transform, and perform other operations on data before plotting. Note that this is for display purposes only.

As an example, the `/powy2` command will square data before display, the `/sx4` command will scale the X data by 4 before display, and the `/try6` command will translate the Y data by 6 before display. See the WinPLOT manual for more details about these and other supported commands.

- *Using Command Line Utilities:*

The RSoft tools include several command line utilities which are documented in [Appendix E](#). These utilities can be used to calculate overlap integrals (`bdutil -i`), far-field transformations (`bdutil -f`), Cropping/re-gridding/slicing data files (`bdconv`), bitmap image to RSoft data format (`bmp2ind`), conversion to/from ray-tracing formats for Synopsys' CODE V, LightTools, and many other functions.

- *Using the `RSoftUserFunction()` Python API:*

This API allows users to pragmatically interact with RSoft data files. All standard RSoft file formats described in Appendix B are supported, including 1D, 2D, 3D, real, complex, uniform, and non-uniform data formats. Note that files in the TCAD Sentaurus TDR format are not currently supported. See the example in [Section 10.A.2](#) for an example, see [Appendix H](#) for a complete description of this API, including supported Python versions.

10.D. Simulation Scripts

Simulation scripts are built around the fact that all of the RSoft tools can be run with complete functionality from the command line. This section will explore this functionality, and then provide several examples of such scripts in different programming languages.

The use of simulation scripts for most applications can be automated with RSoft's MOST package.

10.D.1. Running the Simulation Programs from the Command Line

The ability to run the RSoft simulation tools from the command line forms the basis for all simulation scripts. In this section, we will illustrate this concept and give examples for each product.

Syntax

The basic syntax for running the simulation programs from the command line is:

```
SIM_PROGRAM index-file-name [name=value name=value ...] @<symfile>
```

Where `SIM_PROGRAM` is the name of the simulation program (the names are listed in Conventions section in the Preface), the `index-file-name` is the name of the `*.ind` file to be simulated, the `name/value` entries are symbol table variables and values to be applied to the `*.ind` file, and `<symfile>` is an optional file that contains a list of additional symbols. For example, to run the FullWAVE example file `wg.ind`, which is located in the directory `<rsoft_dir>\examples\fullwave`, with an output prefix of 'run1', issue the following command from the directory where the `*.ind` file is located:

```
fullwave wg.ind prefix=run1
```

For Linux systems, the command will be:

```
xfullwave wg.ind prefix=run1
```

Examples for other tools are below. If you get errors that the system can not find the program or application, read through RSoft Installation Guide to ensure that you have set up the `PATH` properly.

Controlling how Simulation Windows Behave

Several options exist can be used to control how simulation windows behave. To indicate that the simulation window should start minimized, use the `-min` switch. Similarly, to indicate that the

simulation window should be run as a background task, use the `-hide` switch. For example, to run FullWAVE in the background, use a command like:

```
fullwave -hide wg.ind prefix=run1
```

When using scripts it can be very useful to close the simulation window automatically after a simulation has completed so the control is returned to the script and the next item can be run. To do this, set the variable `wait` to 0 like:

```
fullwave -hide wg.ind prefix=run1 wait=0
```

The `wait` option can be set directly in the symbol table, but since it is only really useful when scripting it makes most sense to add it to the command line. While this option will work for all the RSoft simulation tools, MOST uses a slightly different syntax:

```
rsmost -nowait design.ind
```

Try It!

The best way to understand this concept is to try it. To open a command prompt on Windows, either choose the ‘Command Prompt’ option from the start menu, or choose Run from the Start menu and give the command `cmd`. To open a command prompt on a Linux system, open a shell window.

Please choose the appropriate product(s) from the list below. Also, note that the following commands are for Windows systems. Linux users should consult [Section 1.C](#) for the appropriate program name.

In all these cases, a simulation should open and save the results with a prefix of `run1`.

- *BeamPROP*

Move to the directory `<rsoft_dir>\examples\beamprop` and give the command:

```
bsimw32 coupler.ind prefix=run1
```

- *FullWAVE*

Move to the directory `<rsoft_dir>\examples\fullwave` and give the command:

```
fullwave ring-res.ind prefix=run1
```

- *BandSOLVE*

Move to the directory `<rsoft_dir>\examples\bandsolve` and give the command:

```
bandsolve pbgheh.ind prefix=run1
```

- *GratingMOD*

Move to the directory `<rsoft_dir>\examples\gratingmod\Tutorial\TutA1` and give the command:

```
grmod grating1.ind prefix=run1
```


- *DiffRACTMOD*

Move to the directory `<rsoft_dir>\examples\diffRACTmod` and issue the command:

```
dfmod checkerboard3d.ind prefix=run1
```

- *FemSIM*

Move to the directory `<rsoft_dir>\examples\femsim` and issue the command:

```
femsim rib.ind prefix=run1
```

- *ModePROP*

Move to the directory `<rsoft_dir>\examples\modeprop\plasmon` and issue the command:

```
modeprop plasmon.ind prefix=run1
```

10.D.2. Creating a Simulation Script

An example of a basic simulation script would be a simple parameter scan. This section will present the same script in two different languages: as a batch file and in Perl.

The scripts in this section are for BeamPROP but the concepts are the same for other engines.

Using a Batch File

A batch file is simply a text file which contains a list of commands to be run sequentially from the command line. This example illustrates the process of finding an optimal gap for a directional coupler using BeamPROP. We will use the file `coupler.ind` located in

`<rsoft_dir>\examples\beamprop`. Several notes about this script:

- This script will perform simulations with values of the variable separation from 1 μm to a value of 10 μm with a step of 1 μm .
- Data will be stored with a prefix of runXX where XX corresponds to the simulation number.
- This script will *not* produce a scan result like MOST. To do this, this script would have to be modified using the `scan_output`, `scan_prefix`, and `scan_variable` variables used in the Perl script below. For a simple case like this, it would be easier to use MOST; this example is only meant to illustrate basic scanning concepts.

The batch file is:

```
bsimw32 coupler.ind wait=0 prefix=run01 separation=1
bsimw32 coupler.ind wait=0 prefix=run02 separation=2
bsimw32 coupler.ind wait=0 prefix=run03 separation=3
bsimw32 coupler.ind wait=0 prefix=run04 separation=4
bsimw32 coupler.ind wait=0 prefix=run05 separation=5
bsimw32 coupler.ind wait=0 prefix=run06 separation=6
bsimw32 coupler.ind wait=0 prefix=run07 separation=7
bsimw32 coupler.ind wait=0 prefix=run08 separation=8
```

```
bsimw32 coupler.ind wait=0 prefix=run09 separation=9  
bsimw32 coupler.ind wait=0 prefix=run10 separation=10
```

To run this script, save it in a text file with extension *.bat and run it from the command line.

10.D.3. Using Schedulers

The RSoft tools can be integrated into a scheduler using the command line syntax described above. See manual for tools that support clustering (FullWAVE FDTD and MOST) for more specific information about integrating schedulers.

11

Transverse Mode Solving

RSoft's Passive Device Design Suite contains several mode solvers that can be used to compute transverse modes of a structure with an arbitrary index cross section. These modes can then be used as a launch file or as needed. Two of these methods are based on the Beam Propagation Method (BPM), another (TmmSIM) is based on the Transfer Matrix Method (TMM), and another (FemSIM) is based on the Finite Element Method (FEM).

The mode solvers based on BPM and TMM can be used by **all** users who are licensed for any of RSoft's passive device simulation tools. The FEM-based mode solver (FemSIM) is **licensed separately**.

11.A. BPM-Based Mode Solvers

All users who use the RSoft CAD can use these mode solvers; a license for BeamPROP is not needed. This section provides a brief overview of these mode solvers; a full discussion: a technical background can be found in Chapter 2 (including many references) and usage details in Chapter 6 of the BeamPROP manual. An online form of this manual can be opened via the Help menu item in the CAD window and a pdf format can be found in the `<rsoft_dir>/docs` directory.

The RSoft CAD includes two mode solvers that are based on the Beam Propagation Method (BPM), the same algorithm that is used by RSoft's BeamPROP simulation engine. The two BPM-based mode solvers are the iterative method and the correlation method.

11.A.1 Basic Theory of BPM-Based Mode Solving

The BPM algorithm is an approximate numerical approach to solving the exact wave equation for monochromatic waves, and is based on the scalar steady-state Helmholtz equation. Assuming that most of the variation in the field is due to the phase variation along the propagation axis, and applying the slowly varying envelope approximation results in the basic BPM equation:

$$\frac{\partial u}{\partial z} = \frac{i}{2\bar{k}} \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + (k^2 - \bar{k}^2)u \right)$$

where $\bar{k}$ is the reference wavenumber and is a constant number chosen to represent the average phase variation. This equation is solved numerically by first specifying a launch field at $z=z_0$, and then integrating in z to obtain the field for $z>z_0$.

During a mode calculation, it should be assumed that the structure is uniform along z , and so the propagation can be described in terms of the modes of the structure and their propagation constants. Since we are only going to illustrate the concepts of mode solving, we will consider a 2D propagation; the concepts are exactly the same in 3D.

Iterative Method

Propagation of an electromagnetic field through a structure can be expressed via a simple phase variation based on the propagation constants, β_m , of each mode $[\phi_m(x)]$ of the structure. The iterative method is based on imaginary-distance BPM, and propagates along an imaginary z axis by replacing the longitudinal coordinate z by $z'=iz$ so that each mode experiences an artificial exponential increase:

$$\phi(x, z) = \sum_m c_m \phi_m(x) \exp[\beta_m z]$$

This implies that each mode will exponentially grow with a growth rate equal to its real propagation constant β_m . The mode with the highest propagation constant, which is conveniently the fundamental mode ($m=0$), will therefore grow faster than all other modes in the structure causing the simulation results to converge to the field profile of the fundamental mode. The propagation constant can then be obtained using a variational-type expression.

Higher order modes can be obtained by using an orthogonalization procedure to subtract contributions from lower order modes while performing the propagation. Of course, it is important to launch an arbitrary field which excites all the interesting modes in the structure. Issues such as optimal choice of launch field, reference wavenumber, grid size(s), etc., are discussed in the BeamPROP manual. This method is not valid for leaky or lossy modes.

Correlation Method

The correlation method launches an arbitrary field into the structure and propagates via normal BPM. During the propagation the following correlation function between the input field and the propagating field is computed:

$$P(z) = \int \phi_{in}^*(x) \phi(x, z) dx$$

This function can also be expressed, using an expansion of the propagating field in the modes to be found, in a form such that a Fourier transform of this function produces a mode spectrum with peaks at the modal propagation constants. To obtain the mode profiles, a second BPM propagation is performed in which the known propagation constants are beat against the propagating field.

Additional information on this technique can be found in the BeamPROP manual. While the correlation method is generally slower than imaginary-distance BPM, it can sometimes be advantageous because it can be applicable to problems that are difficult or impossible for imaginary distance BPM, such as leaky or radiating modes.

11.A.2. Using the BPM Mode Solvers

Mode solving via BPM can be thought of as a four step process. These steps are:

Step 1: Create the index structure in the CAD.

In order to solve for the mode of a structure, the structure has to be created. The structure should be uniform in Z, and have the desired index profile. Use the **Display Material Profile** option to check that the profile is indeed correct.

Step 2: Choose the appropriate mode solver and related options.

As discussed above, BeamPROP contains two mode solvers which can be accessed by clicking the **Compute Modes** icon in the left toolbar of the CAD. The one you choose depends on the type of modes you want to find. Generally:

- The iterative method is the default choice and is recommended for most standard waveguide problems. The calculation will stop when the desired modes are found.
- The correlation method can have advantages for problems such as antiguiding, leaky, lossy, or radiating modes, or when large numbers of modes need to be calculated. The calculation consists of two full BPM simulations.

The desired mode solver, along with several related options such as the modes you want to solve for, can be set in the Mode Options dialog which can be opened by clicking the **Mode Options...** button in the Mode Calculation Parameters dialog.

Step 3: Choose the mode calculation parameters

Once the mode solver and related parameters have been chosen, the mode calculation parameters need to be chosen. The Mode Calculation Parameters dialog, shown in Fig. 12-1, allows the user to specify simulation domain and numerical grid values for the mode calculation.

Generally, these parameters should be chosen such that the domain includes the entire structure, the domain is large enough to let the mode field be sufficiently close enough to 0 at the boundaries, the transverse grid sizes need to be small enough to resolve the structure all fields, the Z step size has a special meaning in the case of the iterative method (see the BeamPROP manual), and the launch field should be set up so that it excites all the modes you wish to find. Generally, a simple Gaussian launched off-axis will satisfy this need.

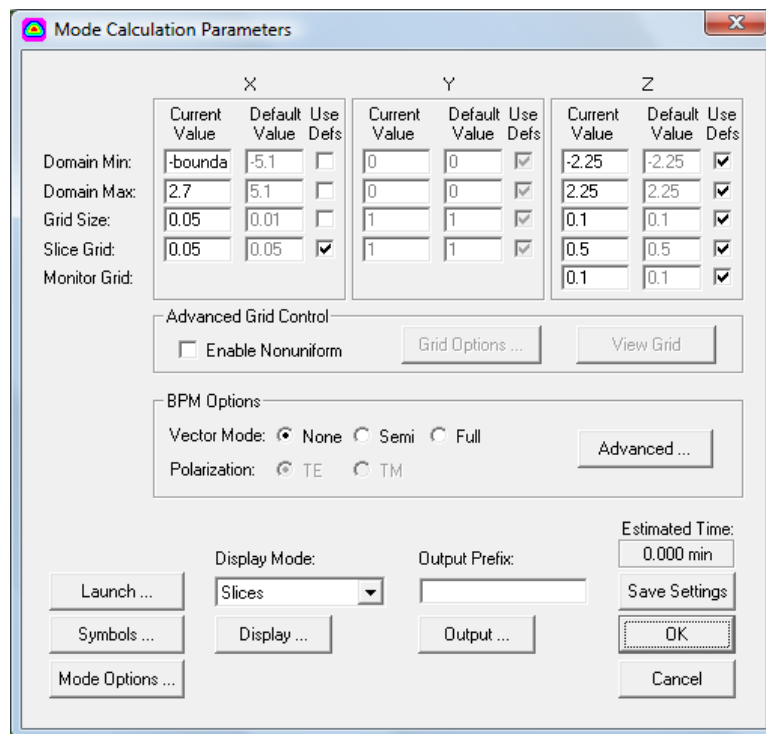


Figure 12-1: The Compute Modes dialog box where the mode calculation parameters are set.

Step 4: Perform the mode calculation.

After the mode calculation parameters have been chosen, enter an **Output Prefix** and click the **OK** button. The mode files will be saved to the current working directory. Each mode found will have two files: the mode data is contained in the file <prefix>.m##, and has a corresponding WinPLOT command file <prefix>.p##, where ## is the mode number. Also, a *.nef file is created which contains a list of the effective indices are found. The format of these files are discussed further in [Appendix B](#).

11.B. TMM-Based Mode Solver (TmmSIM)

This section outlines the use of the TMM-based 1D mode solver TmmSIM. This mode solver can calculate the modes of step-index structures with an arbitrary number of layers (either slab or radial geometry).

TmmSIM can be used by all users who are licensed for any of RSoft's passive device simulation tools.

To open TmmSIM, choose the appropriate option from the Utility menu item in the RSoft CAD interface.

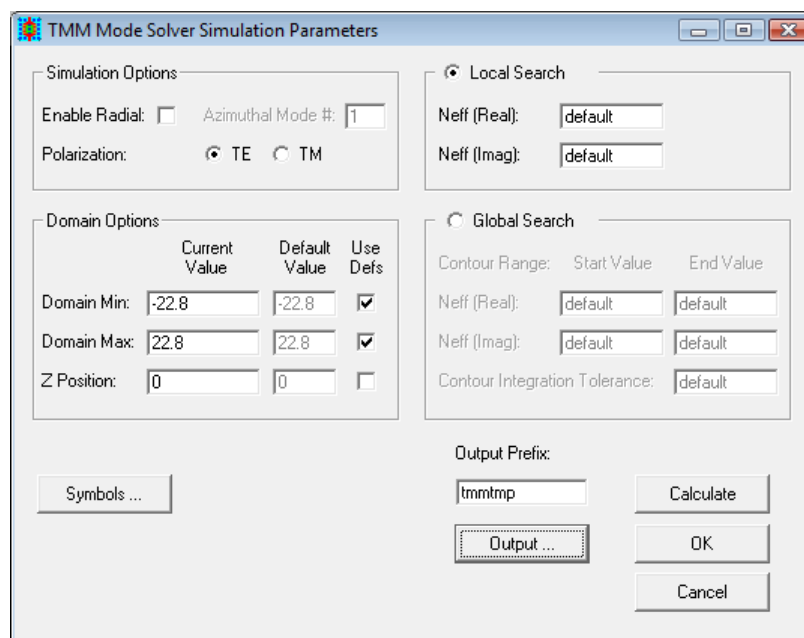


Figure 12-2: The TmmSIM simulation dialog.

11.B.1. Using TmmSIM

To use TmmSIM, first define the structure of interest in the CAD as normal. Then, open the TmmSIM simulation parameters dialog shown above and set the following:

- *Simulation Options*

The simulation options control the **Polarization** (TE or TM) and whether the geometry is a slab or radial. If the structure has radial symmetry, check the **Enable Radial** option and enter the desired **Azimuthal Mode #**. This mode number has the same definition as RSoft's other tools, namely that it sets the m in the HE_{mn} type designation.

- *Simulation Domain*

The simulation domain is defined to be along the X axis (at Y=0), and the **Domain Min** and **Domain Max** can be set by the user. Additionally, the **Z Position** can also be set.

- *Mode Searching Options*

Modes can be found via a Local Search, where the user can specify a guess for an effective index (via **Neff (Real)** and **Neff (imag)**), and the program tries to find the closest mode. When using a Global Search, the user can specify a range in the complex plane to search for modes, and the program tries to find all the modes within that range.

- *The Output Prefix*

The **Output Prefix** sets the name to be used for all output files created.

- *Finding Leaky Modes*

TmmSIM will usually automatically detect if the structure supports leaky modes and then find them. However, in some cases if this auto detection does not work, TmmSIM can be forced to find leaky modes by setting `tmm_leaky` to 1.

11.B.2. Output Options

TmmSIM outputs modes exactly the same as the BPM-based mode solvers and FemSIM. The modes are numbered up from 0 (the fundamental mode), and are output as usual. Also, a `*.nef` file is created which contains a list of the modes found.

Also, any modes found in a radial calculation can be converted to an XY format automatically using the option in the Output Options dialog. Note that this is not step is only necessary for visual purposes, and is not needed if launching these files into a 3D simulation as they will be automatically converted.

11.C. FEM-based Mode Solver (FemSIM)

FemSIM is based on the Finite Element Method, and allows users to compute both propagating and cavity modes. Its use is fully described in the FemSIM manual; an online form can be opened via the Help menu item in the CAD window and a pdf format can be found in the `<rsoft_dir>/docs` directory.

FemSIM is licensed separately and is mentioned here for completeness.

Appendix A

Tips and Traps

This appendix contains some advice on good habits and sources of confusion for novices and experienced RSoft CAD users alike.

A.A. Common RSoft CAD mistakes

- *Misuse of overlapped segments.*

Overlapped segments can be used to achieve complicated geometries. However, it is very important to understand the logic used by the CAD when segments overlap. Please read through [Section 6.C](#).

- *Using the wrong units.*

It is important to use the correct units when describing a structure in the CAD. All units of length are given in μm , angular units are given in degrees, and the units of imaginary index values are:

$$n_{\text{imag}} = \frac{\gamma\lambda}{4\pi}$$

where λ is the wavelength and γ is the usual exponential loss coefficient defined such that the power decays as $e^{-\gamma z}$, and is given in units of μm^{-1} .

A.B. Some Good RSoft CAD habits to learn

Developing the following habits will save you trouble and time in the long run.

- *Use the **Display Material Profile** option frequently*

It is easy to make mistakes with geometry and refractive index information. Many apparently bizarre results are simply the result of simulating the wrong structure.

- *Get Full Usage out of the Symbol Table*

The Symbol Table is a powerful tool which provides a simple way to parameterize a design. Virtually all fields in the CAD, including those in the Segment Properties box, can accept variables and function of variables. This allows the user to create complicated designs in which each parameter, including waveguide coordinates, simulation specific parameters, and virtually every program option is logically tied to other parameters in the circuit. When designing a circuit, it is highly recommended that you utilize the Symbol Table as much as possible. As tempting as it may be to enter exact coordinates for waveguide vertices or refractive index values, it is usually better to create variables in the Symbol Table which equal the desired quantities, and then enter these variable names into the appropriate fields in the CAD interface. This allows for a design file which is very easy to modify, and easier to understand.

- *Use Referenced Coordinates*

The coordinate specification by logical references described in [Section 4.B.1](#) is a unique and powerful CAD feature that can be used to layout circuits with specific geometric relationships which can remain constant while various dimensions (lengths, displacements, separations, etc.) are changed. Coupled with the programmability feature described above that enables formulas to be used to define coordinates or offset/angle parameter values, this feature allows the user to define a logical arrangement of waveguide components whose specific dimensions can be changed by simply altering a few values in the symbol table. Illustrations of this approach are given in several of the example circuits found in the `<rsoft_dir>examples` directory.

- *Learn to use the command line utilities included with the RSoft Photonic Suite*

You can achieve a lot of tedious work by writing a single command. You may even find uses for these utilities in your other work. These utilities are documented in [Appendix E](#).

- *Learn the scripting capabilities of the RSoft Photonic Suite*

While the RSoft CAD is designed to be used via a graphical interface (GUI), complicated structures can be quite tedious to create in this fashion. You can create a layout script which automatically creates a desired design file. Additionally, simulation scripts can be created to automate tedious simulations. See [Chapter 10](#).

- *Read the WinPLOT manual*

WinPLOT's rather terse command syntax can be daunting at first, but allows complex plotting features to be described quickly in just a few lines. Being able to quickly launch plots with old command files from the command line can be a huge time saver.

- *User color scales to produce attractive plots.*

WinPLOT is capable of producing many types of plots beyond the standard output usually displayed. Color scale files can be found in the directory, `<rsoft_dir>RPLOTDIR` and are used to change the default scale used when displaying contour plots. These files are easy to create, and provide a quick way to customize the display of data for presentation purposes. More information on the use and creation of color scales can be found in [Appendix F](#).

Appendix B

File Formats

This appendix documents file formats used both by the RSoft CAD. Note that some file formats are also used by simulation engines as well. For example, the Standard RSoft File Format is used by every simulation engine for a variety of outputs. Except where noted, all files are in ASCII text format. For further information, contact RSoft.

B.A. Standard RSoft File Format

This is the standard ASCII file format which is used to represent functions of one, two, or three variables. This is the most common format used by RSoft software and is used by several different input and output file types such as field profiles, mode profiles, material property profiles (refractive index, etc), user-defined tapers, user-defined profiles, or with the `userdata()` functions described in [Appendix C](#). When using this format, the extension of the file (i.e. `.dat`, `.fld`, etc.) does not matter.

Note that the TDR file format described in [Section B.B](#) can also be used as input virtually anywhere in the software that the Standard RSoft File format can be used. In addition, TDR files may be output as described in [Section B.B.2](#).

B.A.1. Syntax Parameters

Before discussing the actual syntax, we first define several parameters used in the syntax.

While these types of files describe a function of X, Y, and/or Z, these coordinates are local to the data file and do not have to correspond to actual coordinates. This is especially true when using the `userdata()` functions where X, Y, and/or Z represent the value of symbols.

Parameter	Description
Nx, Ny, Nz	Number of data points in the X, Y, and Z directions.
X0, Y0, Z0	The X, Y, and Z coordinate of the first data point.
Xn, Yn, Zn	The X, Y, and Z coordinate of the last data point.
Zpos	The Z coordinate. The actual numerical value does not affect the field data in any way and is included for bookkeeping purposes. For a 3D data file, this should be set to the keyword <code>Z_DEPENDENT</code> .
Optional_Data	These parameters are not required, but are used in special cases. For instance, if the field is a result of a mode calculation, there are two optional fields containing the real and imaginary parts of the mode effective index. Also, the wavelength and refractive index of the plane where the data is defined can be specified here.
Output_Type	The sub-format of the data. This field can be one of the following keywords: Each output type consists of a pair of keywords: the first is used for data files with one coordinate, the second (<code>_3D</code>) for data files with two coordinates. OUTPUT_AMPLITUDE and OUTPUT_AMPLITUDE_3D Indicates that the data corresponds to the amplitude of a field. OUTPUT_REAL and OUTPUT_REAL_3D Indicates that the data corresponds to the real part of a field. OUTPUT_IMAG and OUTPUT_IMAG_3D Indicates that the data corresponds to the imaginary part of a field. OUTPUT_REAL_IMAG and OUTPUT_REAL_IMAG_3D Indicates that the data corresponds to both the real and imaginary parts of a field. The data is presented in two columns; the first contains the real part of the field, and the second the imaginary part of the field. Note that for data files of two coordinates (<code>_3D</code>), there should be two columns of data per Y value, one for the real and one for the imaginary. OUTPUT_AMP_PHASE and OUTPUT_AMP_PHASE_3D Indicates that the data contained in the file corresponds to both the

amplitude and phase of a field. The data is presented in two columns; the first contains the amplitude of the field, and the second the phase. Note that for data files of two coordinates ($_3D$), there should be two columns of data per Y value, one for the amplitude and one for the phase.

B.A.2. File Syntax

Since this type of file is used to describe a wide variety of data types, there are several syntax variations. This section describes the major formats that will be encountered when using the software. While some variation is possible, it is best to use the formats described here when manually creating a file.

The keywords used in these definitions are described in the previous section. For example of each of these file types, see the next section.

1D Uniform Grid Syntax:

The syntax for a 1D data file with a uniform X grid is:

```
/rn,a,b/nx0
Nx X0 Xn Zpos Output_Type Optional_Data
Data(Xmin)
...
Data(Xmax)
```

where there are two header lines: one ‘/’ line that indicates to the software how to interpret the header and one line that contains the X coordinate information and additional information such as the data type. The data is specified in the remaining rows of the file, one row for each point.

1D Non-Uniform Grid Syntax:

The syntax for a 1D data file with a non-uniform X grid is:

```
/rn
Nx X0 Xn Zpos Output_Type Optional_Data
X0      Data(X0)
...
Xn      Data(Xn)
```

where there are two header lines: one ‘/’ line that indicates to the software how to interpret the header and one line that contains the X coordinate data and additional information such as the data type. The data is specified along with the X values in the remaining rows of the file.

1D Non-Uniform Grid Syntax with No Header:

The syntax for a 1D data file with a non-uniform X grid and no header is:

```

X0      Data (X0)
...
Xn      Data (Xn)

```

where there are no header lines, and the `Data (X)` entries can be complex. This format is useful for defining the refractive index information for materials where the X data represents wavelength and the `Data (X)` entries are the real and imaginary parts of the refractive index.

2D Uniform Grid Syntax:

The syntax for a 2D data file with a uniform X and Y grid is:

```

/rn,a,b/nx0
/rn,qa,qb
Nx X0 Xn Zpos Output_Type Optional_Data
Ny Y0 Yn
Data (X0,Y0) ...      Data (X0,Yn)
...
Data (Xn,Y0) ...      Data (Xn,Yn)

```

where there are four header lines: two `‘/’` lines that indicate to the software how to interpret the header and two lines that contain the X coordinate data, Y coordinate data, and additional information such as the data type. Note that the data at each X and Y value are defined as a matrix, each row and column representing a single X and Y value respectively.

2D Non-Uniform Grid Syntax:

The syntax for a 2D data file with a non-uniform X and Y grid is:

```

/rn
/r
/rq
Nx X0 Xn Zpos Output_Type Optional_Data
Ny Y0 Yn
0      Y0      ...      Yn
X0      Data (X0,Y0) ...      Data (X0,Yn)
...
Xn      Data (Xn,Y0) ...      Data (Xn,Yn)

```

where there are five header lines: three `‘/’` lines that indicates to the software how to interpret the header and two lines that contain the X coordinate data, Y coordinate data, and additional information such as the data type. Note that the X values are now defined explicitly in the first column and the Y values are defined explicitly in the first data row. The first 0 in the Y coordinate row is required as a spacer only as the actual value does not matter. The data itself is a matrix as in the 2D uniform grid syntax.

3D File Syntax

The syntax for a 3D data file with a uniform X and Y grid is:

```

/rn,a,b/nx0
/r,qa,qb
/r
Nx X0 Xn Z_DEPENDENT Output_Type Optional_Data
Ny Y0 Yn
Nz Z0 Zn
Data(X0,Y0,Z0)      ...      Data(X0,Yn,Z0)
...
Data(Xn,Y0,Z0)      ...      Data(Xn,Yn,Z0)
Data(X0,Y0,Z1)      ...      Data(X0,Yn,Z1)
...
Data(Xn,Y0,Z1)      ...      Data(Xn,Yn,Z1)
...
Data(X0,Y0,Zn)      ...      Data(X0,Yn,Zn)
...
Data(Xn,Y0,Zn)      ...      Data(Xn,Yn,Zn)

```

where there are six header lines: three ‘/’ lines that indicates to the software how to interpret the header and three lines that contain the X coordinate data, Y coordinate data, Z coordinate data, and additional information such as the data type. For a 3D file, the Z position (z_{pos}) field should be set to the keyword `Z_DEPENDENT` as shown. The data is a series of matrices each corresponding to multiple Z cross-sections.

A Note about Headers

Various outputs will create data files that have some additional items in the first header line. These are primarily plotting commands that effect how WinPLOT will plot the file. For example, the command `/ls1` indicates that the first line style (solid) should be used to plot all data series in the file. When creating files, it is generally not necessary to include these types of options in the data file.

An important and useful exception to this rule is the `/xy` command, which will transpose the x and y coordinates. While this does effect plotting in WinPLOT, this command can also be used to transpose the data before using the file in the CAD or in a simulation tool as a user-profile, launch field, etc.

B.A.3. Example Files

In order to better illustrate the field file format, several examples are given. In each case, a function will be given, and then a corresponding data file.

Example 1:

The following file contains the function $f(x) = x^2$ over the X range [0,1] with an X grid of 0.1:

```

/rn,a,b/nx0
11 0 1 0 OUTPUT_REAL
0.00
0.01

```



```

0.04
0.09
0.16
0.25
0.36
0.49
0.64
0.89
1.00

```

Example 2:

The following file contains the function $f(x) = x^2 + ix^3$ over the X range [0,1] with an X grid of 0.1:

In this type of format, the second column contains imaginary/phase data.

```

/rn,a,b/nx0
11 0 1 0 OUTPUT_REAL_IMAG
0.00 0.000
0.01 0.001
0.04 0.008
0.09 0.027
0.16 0.064
0.25 0.125
0.36 0.216
0.49 0.343
0.64 0.512
0.81 0.729
1.00 1.000

```

Example 3:

The following file contains the function $f(x) = x^2$ over the X range [0,1] with an X grid of 0.1 and with no header.

```

0.00 0.000
0.10 0.010
0.20 0.040
0.30 0.090
0.40 0.160
0.50 0.250
0.60 0.360
0.70 0.490
0.80 0.640
0.90 0.810
1.00 1.000

```

Example 4:

The following file contains the function $f(x) = x^2 + ix^3$ over the X range [0,1] with an X grid of 0.1 and no header:

In this type of format, the third column represents imaginary/phase data.

```
0.00  0.000  0.000
0.10  0.010  0.001
0.20  0.040  0.008
0.30  0.090  0.027
0.40  0.16  0.064
0.50  0.25  0.125
0.60  0.36  0.216
0.70  0.49  0.343
0.80  0.64  0.512
0.90  0.81  0.729
1.00  0.00  1.000
```

Example 5:

The following file contains the function $f(x,y) = xy$ over the X range [1,2] and Y range of [0,1] with an x grid of 0.25 and a y grid of 0.5:

```
/rn,a,b/nx0
/r,qa,qb
5 1 2 0 OUTPUT_REAL_3D
3 0 1
0.000 0.500 1.000
0.000 0.625 1.250
0.000 0.750 1.500
0.000 0.875 1.750
0.000 1.000 2.000
```

Example 6:

The following file contains the function $f(x,y) = x+iy$ over the X range of [1,2] and Y range of [0,1] with an X grid of 0.25 and a Y grid of =0.5:

In this type of format, every other column represents imaginary/phase data.

```
/rn,a,b/nx0
/r,qa,qb
5 1 2 0 OUTPUT_REAL_IMAG_3D
3 0 1
1.00  0.0  1.00  0.5  1.00  1.0
1.25  0.0  1.25  0.5  1.25  1.0
1.50  0.0  1.50  0.5  1.50  1.0
```

```

1.75  0.0   1.75  0.5   1.75  1.0
2.00  0.0   2.00  0.5   2.00  1.0

```

Example 7:

The following file contains the function $f(x) = x^2$ on a non-uniform x grid with 5 points (0.00, 0.10, 0.50, 0.55, and 1.00):

In a 1D non-uniform grid format, the first column contains the X values and the second contains the data.

```

/rn
5 0 1 0 OUTPUT_REAL
0.00  0.0000
0.10  0.0100
0.50  0.2500
0.55  0.3025
1.00  1.0000

```

Example 8:

The following file contains the function $f(x,y) = 2x + i3y$ on a non-uniform x grid with 5 points (1.00, 1.10, 1.50, 1.60, and 2.00) and a non-uniform y grid with points (100, 125, 200):

In a 2D non-uniform grid format, the first column contains the X values and the first data row contains the Y values. Also, since this is a complex-valued function, every other column (excluding the first) contains imaginary data.

```

/rn
/r
/rq
5 1 2 0 OUTPUT_REAL_IMAG_3D
3 1 2
0      1.00  1.25  2.00
1.00  2.00  3.00  2.00  3.75  2.00  6.00
1.10  2.20  3.00  2.20  3.75  2.20  6.00
1.50  3.00  3.00  3.00  3.75  3.00  6.00
1.60  3.20  3.00  3.20  3.75  3.20  6.00
2.00  4.00  3.00  4.00  3.75  4.00  6.00

```

Example 9:

The following file contains the function $f(x,y,z) = x + y + z$ over the x range [-1,1], y range of [-1,1], and z range [0,1] with an x grid of 1, a y grid of 1, and a z grid of 0.5

This file contains three slices, each 3x3, which contain the field data at each Z point.

```

/rn,a,b/nx0
/r,qa,qb
/r
3 -1 1 Z_DEPENDENT OUTPUT_REAL_3D
3 -1 1
3 0 1
-2.000      1.000      0.000
-1.000      0.000      1.000
 0.000      1.000      2.000
-1.500     -0.500      0.500
-0.500      0.500      1.500
 0.500      1.500      2.500
-1.000      0.000      1.000
 0.000      1.000      2.000
 1.000      2.000      3.000

```

B.B. TDR File Format

Note that the Standard RSoft File format described in [Section B.A](#) can also be used as input virtually anywhere in the software that the TDR File format can be used.

TDR files are the basic input and output used by Synopsys' TCAD Sentaurus product. This section only covers the relevant information for using TDR files in the RSoft software. See the TCAD Sentaurus documentation for a complete description of the TDR format.

B.B.1. Using TDR Files as Inputs to the RSoft Software

TDR files can be used as inputs in the software anywhere a data file can be entered. This includes user profiles, launch fields, etc. The syntax is:

```
<tdr-file>[:<quantity>]
```

where `<tdr-file>` is the name of the TDR file and `quantity` is optional (and case-sensitive!) and sets the name of the desired quantity within the TDR to be used. For example, entering

```
run1.tdr:doping
```

will use the `doping` quantity from the `run1.tdr` file. If no quantity is specified, the first quantity in the file will be used.

Manipulating TDR Files for Use as RSoft Input

In some cases the coordinate system and how the structure has been drawn in TCAD Sentaurus will be different than the RSoft CAD. In these cases it will be necessary to manipulate the TDR data in order to move between the two systems.

The required manipulation typically consists of one or more data transformations including scaling, flipping, or transposing one or more coordinates. These operations, which can also be performed on data in the RSoft Standard File format described in [Section B.A](#), are:

More information about the `bdutil` utility can be found in [Appendix E](#). Note that some of these options are documented as part of `bdconv`, but `bdutil` acts like `bdconv` if no function is specified. Note that the `tdrutil` utility can also be used to work with TDR files.

- *Translating TDR Data:*

To translate data along an axis, use the `-trx#` or `-try#` options. For example, to translate the X axis in the file `in.tdr` by 4.2 and save it as `out.tdr`, use this command:

```
bdutil -trx4.2 in.tdr out.tdr
```

- *Scaling TDR Data:*

To scale data along an axis, use the `-scx#` or `-sxy#` options. For example, to scale the X axis in the file `in.tdr` by 2.5 and save it as `out.tdr`, use this command:

```
bdutil -scx2.5 in.tdr out.tdr
```

- *Flipping TDR Data:*

To flip data along an axis, just scale by -1. For example, to flip the file `in.tdr` along the Y axis and save it as `out.tdr`, use this command:

```
bdutil -scy-1 in.tdr out.tdr
```

- *Transposing TDR Data:*

To transpose the X/Y data use the `-t` option. For example, to transpose the X/Y data in the `in.tdr` file and save it to the file `out.tdr`, use this command:

```
bdutil -t in.tdr out.tdr
```

B.B.2. Enabling TDR File Output from the RSoft Software

By default, when any input to the RSoft software is in the TDR format, all data outputs will also be in a TDR format. To explicitly enable the output of TDR file, set the variable `output_as_tdr` to 1. Note that any TDR output will use the same filenames as non-TDR output but the file format will of course be different.

B.C. User-Profile Functions

A user-profile can be defined by either an expression or a data file. Data files should be in the standard file format described in [Section B.A](#). User profile files can be in 1D, 2D, and/or 3D formats as described in [Section 6.A.3](#).

B.D. User-Taper Functions

A user-taper can be defined by either an expression or a data file. Data files should be in the standard file format described in [Section B.A](#). User taper files should be in a 1D format as described in [Section 6.B.4](#).

B.E. Polygon Files

Polygons provide a customizable approach to creating components. A polygon is defined by a series of data points in the (X,Z) plane contained in a data file. These points should be on separate lines, and the first and last points should be the same.

For example, a square could be defined by the file:

```
0,0  
0,1  
1,1  
1,0  
0,0
```

Polygons can be scaled in both the X and Z coordinates, as well as rotated at an arbitrary degree. Also, these files can either be statically loaded once, or dynamically loaded at runtime. See [Section 5.D](#).

Appendix C

RSoft Expressions

Virtually any numeric input field in any RSoft product can accept an analytical expression involving pre-defined and user-defined variables that are defined in the symbol table. This appendix discusses the form that these expressions can take, the allowed operators, and the built-in functions that are available.

Expressions are written in the common form accepted by most programming languages (e.g. C, Fortran, etc.), and can include numbers, variables, arithmetic operators, and function calls.

C.A. Arithmetic Operators & Order of Operations

Valid operators for RSoft expressions, listed in order of precedence, are:

Operator	Description
$\wedge$	exponentiation
$*$	multiplication
$/$	division
$+$	addition
$-$	subtraction

Parenthesis can be used as usual to group operations and override this precedence.

C.B. Built-in Constants

RSoft supports several built-in constants. These variables may be used in the symbol table, or in any numeric field within the software. These variable names should be avoided when creating your own variables to avoid conflict.

Variable	Value
----------	-------

inf	infinity
-----	----------

c	2.9979246e014 $\mu\text{m/s}$
---	-------------------------------

e	2.718281828
---	-------------

pi	3.141592653
----	-------------

true	1
------	---

false	0
-------	---

yes	1
-----	---

no	0
----	---

\$id	This parameter equals the segment number of the segment it is used from (if used in Segment 1, its value will be 1. This is useful to provide different seed numbers for multiple segments when using the random number generator functions described below.
------	--

C.C. Using File Names in Variables/Expressions

The CAD allows file-name fields to contain a variable which represent the actual file name. This can be useful when performing scans that use a different data file for each step. If the text in the field begins with a \$ character, the remaining text is interpreted as a symbol that is defined as the file name. It is possible to use these parameters for any data file in the CAD, such as for files for launch conditions, monitor files, polygon files, user tapers, user profiles, material names etc. Using MOST, these variables can be scanned easily.

C.D. Defining Default Value for Variable when used in Expression

In most cases, variables that are used in an expression are defined in the symbol table. In some cases, especially for built-in symbols, a variable may not be defined in the Symbol Table if it is set to its default value. In these cases, it is possible to define a default value for such a variable in the expression where it is used using the 'variable:value' syntax.

For example, consider the following expression based on the variable `dimension`, the built-in variable corresponding to the **Model Dimension** setting that controls whether the CAD design is 2D or 3D:

```
My_symbol = if(eq(dimension:3,2), Value2D, Value3D)
```

This expression sets the symbol `My_symbol` to `Value2D` for 2D (`dimension` equals 2), and `Value3D` for 3D (`dimension` not equal 2). If `dimension` is not defined in the Symbol Table, this expression uses a default value of `dimension=3` by using the syntax '`dimension:3`'.

C.E. Square Bracket Notation for Hierarchy Export Symbols

The Hierarchy Export option described in [Section 6.F.5](#) uses a square bracket notation `[xx]` to denote that the value of the symbol should be taken from the child file instead of defined in the parent file. The value displayed inside the square brackets is dynamically updated based on the value in the child file.

C.F. Built-In Functions

RSoft expressions support the standard function calls found in programming languages as well as several other functions useful for optical design.

C.F.1. Basic Functions

Function	Description
<code>abs(x)</code>	absolute value
<code>exp(x)</code>	exponential
<code>log(x)</code>	natural logarithm
<code>log2(x)</code>	logarithm in base 2
<code>log10(x)</code>	logarithm in base 10
<code>sqrt(x)</code>	square root
<code>cos(x)</code>	cosine
<code>sin(x)</code>	sine
<code>tan(x)</code>	tangent
<code>acos(x)</code>	inverse cosine

<code>asin(x)</code>	inverse sine
<code>atan(x)</code>	inverse tangent
<code>atan2(y,x)</code>	inverse tangent
<code>sinh(x)</code>	hyperbolic sine
<code>cosh(x)</code>	hyperbolic cosine
<code>tanh(x)</code>	hyperbolic tangent
<code>sinc(x)</code>	sinc function $[\sin(x)/x]$
<code>erf(x)</code>	error function
<code>erfc(x)</code>	complementary error function
<code>ceil(x)</code>	truncate up to next higher integer
<code>floor(x)</code>	truncate down to next lower integer
<code>round(x[,y])</code>	When used with one argument, this function rounds x to nearest integer (y=1); when used with two arguments, it rounds x to the nearest multiple of y
<code>min(x1,x2,...)</code>	minimum of arguments in list
<code>max(x1,x2,...)</code>	maximum of arguments in list
<code>hypot(x,y)</code>	length of hypotenuse of right triangle with sides x and y
<code>sign(x)</code>	sign of argument (-1 or +1)
<code>clip(x)</code>	clip function defined as 0 for x less than 0, 1 for x greater than 1, and x otherwise.
<code>list(x,y,...)</code>	function to pass a list of parameters

C.F.2. Comparison Functions

These functions allow two arguments to be compared:

Function/Definition	Description
$eq(x,y) = \begin{cases} 1 & x = y \\ 0 & x \neq y \end{cases}$	Determine if two arguments are equal.
$eq(x,y,z) = \begin{cases} 1 & x - y \leq z \\ 0 & x - y > z \end{cases}$	Determine if two arguments (x,y) are equal within a tolerance (z).

$$ne(x, y) = \begin{cases} 1 & x \neq y \\ 0 & x = y \end{cases}$$

Determine if two arguments are unequal.

$$ne(x, y, z) = \begin{cases} 1 & |x - y| > z \\ 0 & |x - y| \leq z \end{cases}$$

Determine if two arguments (x,y) are unequal within a tolerance (z).

$$gt(x, y) = \begin{cases} 1 & x > y \\ 0 & x \leq y \end{cases}$$

Determine if one argument (x) is greater than another (y).

$$ge(x, y) = \begin{cases} 1 & x \geq y \\ 0 & x < y \end{cases}$$

Determine if one argument (x) is greater than or equal to another (y).

$$lt(x, y) = \begin{cases} 1 & x < y \\ 0 & x \geq y \end{cases}$$

Determine if one argument (x) is less than another (y).

$$le(x, y) = \begin{cases} 1 & x \leq y \\ 0 & x > y \end{cases}$$

Determine if one argument (x) is less than or equal to another (y).

C.F.3. Logical Functions:

These logical functions allow arguments to be logically compared:

Function/Definition	Description
$if(x, a, b) = \begin{cases} a & x \geq 0.5 \\ b & x < 0.5 \end{cases}$	This is the equivalent of a standard logical 'if' function.
$not(x) = \begin{cases} 1 & x < 0.5 \\ 0 & x \geq 0.5 \end{cases}$	This is the equivalent of a standard logical 'not' function.
$and(x, y) = \begin{cases} 1 & x \geq 0.5 \text{ and } y \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$	This is the equivalent of a standard logical 'and' function.
$or(x, y) = \begin{cases} 1 & x \geq 0.5 \text{ or } y \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$	This is the equivalent of a standard logical 'or' function.
$case(x, v1, v2, ...) = \begin{cases} v1 & x = 1 \\ v2 & x = 2 \\ \dots & x = \dots \end{cases}$	This is the equivalent of a standard 'switch' statement where the first argument is the index and the remaining arguments are the potential values. For example, <code>case(2, 4, 8, 4)</code> will return 8.
$defined(x) = \begin{cases} 0 & \text{if 'x' does not exist} \\ 1 & \text{if 'x' exists} \end{cases}$	This function checks if a symbol is defined and returns 0 (undefined) or 1 (defined).

C.F.4. Step Functions

These functions are step functions that are especially useful for defining tapers:

Function/Definition	Description
$u(x) = \begin{cases} 0 & x < 0 \\ 0.5 & x = 0 \\ 1 & x > 0 \end{cases}$	This is a standard non-repeating step function that equals 0.5 at the step.
$um(x) = \begin{cases} 0 & x < 0 \\ 0 & x = 0 \\ 1 & x > 0 \end{cases}$	This is a variation of the function $u(x)$ that equals 0 at the step.
$up(x) = \begin{cases} 0 & x < 0 \\ 1 & x = 0 \\ 1 & x > 0 \end{cases}$	This is a variation of the function $u(x)$ that equals 1 at the step.
$step(x) = \begin{cases} 1 & 0 \leq x < 0.5 \\ 0 & 0.5 \leq x < 1 \end{cases} \quad \{x \in [0,1)\}$	This is a periodic step function that is periodically extended outside the x range $[0,1)$.
$step2(x,a) = \begin{cases} 1 & 0 \leq x < a \\ 0 & a \leq x < 1 \end{cases} \quad \{x \in [0,1)\}$	This is a variation of the function $step(x)$ with a variable duty cycle controlled by the second argument a (which must be between 0 and 1).

C.F.5. Random Functions

These deterministic functions allow the generation a random number or a random function of a continuous variable.

Random Numbers [ranseq()]

The deterministic function `ranseq()` returns a “random” result for a given pair of arguments. Its syntax is:

```
ranseq(int seed, int i)
```

where `seed` is the random seed, and `i` is an integer useful for correlating several “random” parameters together. The distribution is linear between -0.5 and 0.5, and several special seeds are recognized: 0 returns 0.0, -1 returns 0.5, and -2 returns -0.5.

As an example, consider a case where several parameters within a component are to have random values. To do this, define one or more `seed` variables in the symbol table, and then use the same index variable `i` within the component. In this way two or more fields within the component are correlated. Changing the seeds within the symbol table will result in different

random numbers. If this correlation is unnecessary, both of the arguments can be treated as separate seed parameters.

Examples:

Example	Value	Description
<code>ranseq(5,4)</code>	0.3715	Returns a random number between -0.5 and 0.5.
<code>0.5+ranseq(1,1)</code>	0.3964	Returns a random number between 0 and 1.
<code>0.5+ranseq(1317,1942)</code>	0.1452	Returns another random number between 0 and 1.

Random Function [`ranline()`]

The deterministic function `ranline()` returns a “random” function of a continuous variable. Its syntax is:

```
ranline(int seed, double corlen, double z)
```

where `seed` is the random seed, `corlen` is the correlation length of the random function, and `z` is the continuous variable. The seed variable behaves the same as the function `ranseq()` described above and is not repeated here. This function has the same statistical behavior as the built-in roughness option described in [Section 6.D.4](#), and may be used in tapers. When using this function, be careful with the normalizations of tapers with respect to the units of `z` and `corlen`. For example, if `L` is the true physical length of the taper, and `CorLen` is the true physical correlation length, then the taper function should use `ranline(iseed, Corlen/L, z)`.

C.F.6. Data-File Functions

These functions allow data contained in data files to be used in an expression. Three functions (`userdata()`, `userreal()`, and `userimag()`) allow the value of an expression to be defined via an ASCII data file as a function of up to three other variables and one function (`userderiv()`) allows the value of an expression to be defined via the derivative of data contained in an ASCII data file as a function of one variable. The names of data files used with these functions should not contain spaces.

While these functions can be used anywhere an expression allowed, it is better to employ them only in the symbol table and assign their result to another symbol to be used anywhere since their evaluation can be time-consuming.

- *Standard Data-File Functions*

The `userdata()` and `userreal()` functions are synonymous, and return the real part of values in a data file (which can be real or complex). The `userimag()` function returns the imaginary part (or 0 if the value is real). All functions have the same syntax; only `userdata()` is described here.

The syntax for these functions is:

```
userdata("filename", var1)
userdata("filename", var1, var2)
userdata("filename", var1, var2, var3)
```

where the first argument is the file name in quotes, and the remaining arguments (up to three) are the variables the data is a function of and must be defined in the Symbol Table. This function will evaluate the variable(s), then look up the corresponding value in the data file.

- *User Derivative Function*

The `userderiv()` function returns the first derivative of data contained in an ASCII data file. The syntax for this function is:

```
userderic("filename", var)
```

where the first argument is the file name in quotes and the second argument is the variable that the data is a function of and must be defined in the Symbol Table. This function will evaluate the variable(s), then look up the corresponding first-derivative based on the data in the file.

Additional Notes:

- When using these functions to define materials stored in the Group Library discussed in [Section 8.A.1](#), you can use the keyword `<groupmat>` to refer to the PATH defined. See note in [Section 8.A.7](#).

- You can use a symbol to specify the filename using the '\$' prefix, for example:

```
filename = data.dat
userdata($filename, free_space_wavelength)
```

- The format of the file is currently limited to the formats described in [Appendix B](#), see a simple example below.
- The RSoft CAD will not give an error if a blank string is used for the filename. This is useful when using the ':' syntax to define default values. Consider the following series of symbols:

```
V_exp = 7
V_file = userdata($F:, free_space_wavelength)
Parameter = if(defined(F), V_exp, V_file)
F = data.dat
```

In the above example, if the symbol `F` is not defined, then the `V_file` symbol will be set to a null value (set via the ':' default option described in [Section C.D](#)). In this case, the `Parameter` value will be set to the value `V_exp`.

Example 1:

As an example, consider the case of defining the background index as a function of wavelength. Assume the refractive index information is contained in a data file named SiO2.dat and has the following contents:

```
/rn,a,b/nx0/ls1
5 1.50 1.60 0 OUTPUT_REAL
1.4430
1.4435
1.4440
1.4445
1.4450
```

This data file defines the refractive index over a wavelength range of 1.50 to 1.60 μm using 5 points. See [Appendix B](#) for more details about this file format. It is also possible to use a header-less format like this:

```
1.500 1.4430
1.525 1.4435
1.550 1.4440
1.575 1.4445
1.600 1.4450
```

with two columns containing the wavelength and real index data. A three column file with the imaginary index can also be used.

The proper syntax to define the background index as a function of the wavelength would then be:

```
background_index = userdata("SiO2.dat",free_space_wavelength)
```

The second and third numeric arguments could be used to conveniently add dependence on other variables, such as temperature or concentration, if the data file contained this information. In this case, setting the variable `free_space_wavelength` to 1.55 will result in a `background_index` value of 1.4440. Results are interpolated from the data in the data file if needed.

Example 2: Using a Data File to Define Refractive Index in the RSoft CAD

It is important to use refractive index data that corresponds to the actual materials being simulated to ensure meaningful simulation results. Refractive index data, which can be found in reference materials, on the Internet, or from experiment, can be used to create a new material in the RSoft CAD's Material Library. This new material can then be used with any of RSoft's passive device simulation tools.

- The refractive index data should be in a simple text format: the first column contains the wavelength, the second column contains the real index, and the third optional column contains the imaginary index. It is important that the file contains data for the entire wavelength range to be simulated. The data can be at non-uniformly spaced wavelengths and no header is required, though a simple text header can be used.

- For this example, the data file will be put in the same directory as the `.ind` design file in which the material will be used. If needed, you can create a centrally stored Group Library of materials which can be shared among design files; see [Section 8.A](#) for details.
- Open the Material Editor with the left CAD toolbar, create a new material, and give the material a descriptive name.
- Click the **Import NK Data** button and select the material data file. Expressions using the `userreal()` and `userimag()` functions will automatically be used to load the data into the material.
- You can use the CTRL-doubleclick feature of the RSoft CAD to see the current value of the index. Hold the CTRL key and double-click the mouse on the **Index (real)** text field to see the value which should correspond to the value in the data file at the current wavelength.

C.F.7. Material Functions

There are two types of material functions supported by the RSoft CAD: functions that can return the refractive indices of materials that are defined in the Material Library, and functions that return the refractive index of special materials such as Graphene. This section describes both types of material functions.

Functions to Obtain Refractive Index of Defined Materials

There are six functions that allow for the refractive indices of materials to be used in symbol table expressions. The first two functions lookup the real and imaginary values for the refractive index at the global wavelength (`free_space_wavelength`) defined in the `.ind` file:

```
nreal("material")
nimag("material")
```

where `material` is the name of the material as defined in the Material Editor. To calculate the refractive index at a specific wavelength use the form:

```
nreal("material", lambda)
nimag("material", lambda)
```

where `lambda` is the wavelength [μm]. The other four functions lookup the real and imaginary values of the refractive index at a specific wavelength for ternary and quaternary materials:

```
nkinterp2_real("materialA","materialB",x,lambda,dir)
nkinterp2_imag("materialA","materialB",x,lambda,dir)
nkinterp4_real("materialA","materialB","materialC","materialD",x,y,lambda,dir)
nkinterp4_imag("materialA","materialB","materialC","materialD",x,y,lambda,dir)
```

where `material[A|B|C|D]` is the name of the material(s) as defined in the Material Editor, `x` and `y` are the ternary and quaternary alloy parameters (set in 'Semiconductor Tab' in the Material Editor), `lambda` is the wavelength [μm], and `dir` is the directory code for the material file

location. The reason the directory is specified is so that the basis material(s) need not be in the Project Materials portion of the Material Editor. Possible directory codes are 0 (the working directory), 1 (RSoft Material library), or 2 (Group Material Library). These functions return the following value:

$$\begin{aligned} \text{nkinterp2_}[\text{real}|\text{imag}]() &= x*A + (1-x)*B \\ \text{nkinterp4_}[\text{real}|\text{imag}]() &= x*y*A + x*(1-y)*B + (1-x)*y*C + (1-x)*(1-y)*D \end{aligned}$$

where A, B, C, D are the n (real) or k (imag) values from the respective materials evaluated at λ .

Special Material Functions

The RSoft CAD supports functions that return the refractive index of special materials:

- *Graphene*

The properties of Graphene depend on the material properties and environment. The `graphene_real` and `graphene_imag` functions calculate the real and imaginary refractive index of Graphene as a function of several parameters:

$$\begin{aligned} \text{graphene_real}(\lambda, u, g1, g2, t, h, e) \\ \text{graphene_imag}(\lambda, u, g1, g2, t, h, e) \end{aligned}$$

where

Parameter	Description and Notes
λ	The wavelength [μm]
u	The chemical potential μ [eV]
$g1$	The intraband Γ_1 [eV]. Note: • Γ [eV] = $\hbar \Gamma$ [1/sec] • To convert a time constant τ, typically $\Gamma = 1/(2\tau)$
$g2$	The interband Γ_2 [eV] • See notes above for Γ units and conversions • Setting $g2$ to -1 makes $g2 = g1$.
t	The temperature T [K]
h	The Graphene thickness H [μm], typically set to 0.0007
e	The reference epsilon ϵ_{ref}

The `graphene_real()` and `graphene_imag()` functions are used by the built-in Graphene models in the Material Library but can also be used in any expression throughout the software. When using these functions make sure to set the appropriate parameters that

correspond to your exact situation. Note that older versions of the software had fewer arguments and need to be manually updated to the form described above.

The model used by these functions calculates the real and imaginary refractive index as follow:

$$n(\omega) = \sqrt{\varepsilon(\omega)}$$

$$\varepsilon(\omega) = \varepsilon_{ref} + \frac{i\sigma(\omega, \mu, \Gamma_1, \Gamma_2, T)}{\varepsilon_0 \omega H}$$

where ε_0 is the permittivity of free space and $\sigma(\omega, \mu, \Gamma_1, \Gamma_2, T)$ is calculated as described in Eqn. 1 of the reference below. Note the following exceptions:

- Γ_1 is used for the intraband term and Γ_2 is used for the interband term
- The sign convention for ‘i’ is changed (i is replaced with $-i$)
- The integral is solved exactly for all cases, the approximation used in Eqn. 4 is not used

Reference: Hanson et al, “Dyadic Green’s functions and guided surface waves for a surface conductivity model of graphene”, *Journal of Applied Physics* **103**, (2008).

C.F.8. Effective Index Functions

There are several functions that can be used to analytically compute effective and group indices of certain 2D and 3D geometries.

- *Effective index of a 3-layer slab waveguide mode* - `slabneff()`

The function `slabneff()` computes the effective index of a 3-layer slab waveguide mode; it’s syntax is:

```
slabneff(p,m,lambda,nc,ns,nf,h[,g])
```

where `p` is the polarization (0=TE, 1=TM), `m` is the mode number (0 is fundamental), `lambda` is the free space wavelength in μm , `nc` is the cover index, `ns` is the substrate index, `nf` is the film or guiding layer index, and `h` is the thickness of the film or guiding layer in μm . The last argument `g` is optional and, if set to 1, calculates the group index instead of the effective index.

- *Effective index of rib waveguide geometries* – `eimneff()`

The function `eimneff()` computes the effective index via the effective index method for rib waveguide geometries; it’s syntax is:

```
eimneff(p,mx,my,lambda,nc,ns,nf,w,h,hside[,g])
```

where `p` is the polarization (0=TE, 1=TM), `mx` and `my` are the X and Y mode numbers (0 is fundamental), `lambda` is the free space wavelength in μm , `nc` is the cover index, `ns` is the

substrate index, `nf` is the film or guiding layer index, `w` and `h` is the width and height of the rib in μm , and `hside` is the slab height. The last argument `g` is optional and, if set to 1, calculates the group index instead of the effective index.

- *Effective index of LP fiber modes* – `lpneff()`

The function `lpneff()` computes the effective index of LP fiber modes; it's syntax is:

```
lpneff(l,m,lambda,nclad,ncore,d[,g])
```

where `l` and `m` are the mode numbers, `lambda` is the free space wavelength in μm , `nclad` is the cladding index, `ncore` is the core index, and `d` is the core diameter in μm . The last argument `g` is optional and, if set to 1, calculates the group index instead of the effective index.

C.F.9. Polygon Functions

These functions can be used to define a 2D polygon function which can be used to make custom user-profiles, etc:

```
polygon(n, x, y)
polygon("file", x, y)
```

The first form generates a regular polygon with `n` sides. The second form takes an arbitrary set of points contained in a file which must be in quotes. The file should be in the polygon format in [Appendix B.E](#).

C.F.10. PDK Functions

The `pdkinfo()` function returns design parameters from the PDK currently set by the **PDK Info File** option in the Global Settings dialog. A generic PDK is included and used by default, PDKs from specific foundries can be installed if you have permission from the foundry (contact support for details). User-defined custom PDKs can be added. The syntax is:

Function/Syntax	Description
<code>pdkinfo("parameter")</code>	Returns a numeric value from a pdkinfo file.
<code>\$pdkinfo("parameter")</code>	Returns a string value from a pdkinfo file.

As an example, consider the following sample pdkinfo file, and then example function calls.

```
WG_width = 0.5
WG_height = 0.22
WG_material = "Si"
```

Example usage would be `pdkinfo("WG_width")` which would return 0.5, and `$pdkinfo("WG_material")` which would return "Si". Note that it can be useful in some cases to also define the parameters using the layer number, see discussion in the Custom PDK Utility manual.

C.F.11. String Functions

There are several functions that help manipulate and use strings:

Function	Description
<code>strlen(x)</code>	Returns the length of string <code>x</code> .
<code>\$cat(x,y,...)</code>	Concatenates string arguments. For example, <code>cat("AA","BB")</code> returns "AABB".
<code>\$itoa(x)</code>	Returns a string from an integer argument <code>x</code> .
<code>\$ftoa(x)</code>	Returns a string from a float argument <code>x</code> .
<code>atoi()</code>	Returns an integer from a string argument <code>x</code> .
<code>atof()</code>	Returns a float from a string argument <code>x</code> .

C.F.12. Unit Conversion Functions

There are several functions that help convert units:

Function	Description
<code>eng2sci(x)</code>	Converts engineering format to scientific format. It accepts one string argument. For example, <code>eng2sci("2u")</code> returns <code>2e-6</code>. The accepted codes and factors are: • <code>Y</code>, a factor of <code>1e24</code> • <code>Z</code>, a factor of <code>1e21</code> • <code>E</code>, a factor of <code>1e18</code> • <code>P</code>, a factor of <code>1e15</code> • <code>T</code>, a factor of <code>1e12</code> • <code>G</code>, a factor of <code>1e9</code> • <code>M</code>, a factor of <code>1e6</code> • <code>K</code>, a factor of <code>1e3</code> • <code>M</code>, a factor of <code>1e-3</code> • <code>u</code>, a factor of <code>1e-6</code> • <code>n</code>, a factor of <code>1e-9</code> • <code>p</code>, a factor of <code>1e-12</code> • <code>f</code>, a factor of <code>1e-15</code>

- a, a factor of 1e-18
- z, a factor of 1e-21
- y, a factor of 1e-24

`lm2um(x)` Returns a length metric in microns, and returns `1e-6*eng2sci(x)`. The `x` argument is a string, but if a number argument is used, that number is returned.

C.F.12. Sentaurus Workbench (TCAD) Functions

There are functions that enable and simplify the use of the RSoft Photonic Device Tools within TCAD Sentaurus. The `swb()` and `$swb()` functions enable seamlessly switching between a default value before Sentaurus Workbench pre-processes and the actual parameter value after pre-processing. The `$swb()` function should be used with string values. The syntax is:

```
swb("@text_to_be_preprocessed@", default_value)
$swb("@text_to_be_preprocessed@", "default_value")
```

Before SWB pre-processes, the second argument is returned. When the text is pre-processed, then the first argument is returned. See the TCAD Sentaurus documentation for more information on pre-processing, including supported syntax and options.

Example

```
swb("@wavelength@", 1.55)
```

```
swb("1.31", 1.55)
```

```
$swb("n@node@_fps.tdr", "saved_dnk.tdr")
```

```
$swb("n8_fps.tdr", "saved_dnk.tdr")
```

Value Returned

1.55

1.31

"saved_dnk.tdr"

"n8_fps.tdr"

Appendix D

Symbol Table Variables

In this appendix we document the symbol table variable names which control the internal operation of the RSoft CAD. Most of these variables correspond to dialog fields which have already been documented.

The following sections list the variable names corresponding to these dialog fields:

D.A. Startup Window/Global Settings

The following variables correspond to the fields found in the Startup/Global Settings dialog box:

Field Name	Variable Name
Waveguide Model Dimension	dimension
Radial Calculation	radial
Effective Index Calculation	eim
Polarization	polarization
Simulation Tool	sim_tool
Free Space Wavelength	free_space_wavelength
Background Index	background_index

Background Material	background_material
Index Difference	delta
Component Width	width
Component Height	height
Profile Type	profile_type
3D Structure Type	structure
Cover Index	cover_index
Cover Material	cover_material
Slab Index	slab_index
Slab Height	slab_height
Slab Material	slab_material

D.B. Special Effects

The following variables correspond to the fields in the Index Generation Options dialog box:

Field Name	Variable
Simulated Bend	simulated_bend
Simulated Bend Radius	simulated_bend_radius
Litho Resolution	litho_res
Litho Roughness	litho_sigma
Litho Correlation Length	litho_corlen
Sidewall Angle Ref (0-1)	sidewall_reference
Absolute User Profile	user_profile_absolute

D.C. Advanced Grid Parameters

The following variables correspond to the fields found in the Advanced Grid Parameters dialog:

Field Name	Variable Name
X Domain Min	boundary_min
X Domain Max	boundary_max

X Grid Size (Bulk)	<code>grid_size</code>
X Grid Size (Edge)	<code>grid_edge_x</code>
X Griding Ratio	<code>grid_ratio_x</code>
X Minimum Divisions	<code>grid_mindiv_x</code>
X Grid Grading	<code>grid_bulk_nonuniform_x</code>
X Interface Alignment	<code>grid_align_x</code>
Y Domain Min	<code>boundary_min_y</code>
Y Domain Max	<code>boundary_max_y</code>
Y Grid Size (Bulk)	<code>grid_size_y</code>
Y Grid Size (Edge)	<code>grid_edge_y</code>
Y Griding Ratio	<code>grid_ratio_y</code>
Y Minimum Divisions	<code>grid_mindiv_y</code>
Y Grid Grading	<code>grid_bulk_nonuniform_y</code>
Y Interface Alignment	<code>grid_align_y</code>
Z Domain Min	<code>domain_min</code>
Z Domain Max	<code>domain_max</code>
Z Grid Size (Bulk)	<code>step_size</code>
Z Grid Size (Edge)	<code>grid_edge_z</code>
Z Griding Ratio	<code>grid_ratio_z</code>
Z Minimum Divisions	<code>grid_mindiv_z</code>
Z Grid Grading	<code>grid_bulk_nonuniform_z</code>
Z Interface Alignment	<code>grid_align_z</code>

D.D. Advanced Features

The following variables correspond to advanced features discussed throughout this manual:

Variable	Description
<code>alpha</code>	Sets the default imaginary index for waveguide segments.
<code>background_alpha</code>	Sets the imaginary index for the background.
<code>cover_alpha</code>	Sets the imaginary index of the cover for rib/ridge/multilayer structure types.

default_material	Sets the default material for components that have Material Properties set to <i>Locally Defined</i> , Index Difference set to <code>delta</code> , and Index (Imag part) set to <code>alpha</code> .
diffusion_shape	Sets the diffusion shape for diffused waveguides.
diffusion_gamma	Sets the nonlinear relation between the diffusion concentration and the index.
diffusion_ratio	Sets the diffusion width/height ratio.
change_profile	Enables the use of a Z-dependent user profile.
index_profile_all	Outputs all index tensors for anisotropic calculation when computing the index profile.
import_symbol_file	Controls symbol import options.
import_symbol_file_auto	
import_list	

D.E. Display Material Profile

The following variables correspond to Material Profile Calculations:

Variable	Description
boundary_min	X Domain Min
boundary_max	X Domain Max
grid_size	X Grid Size
slice_grid_size	X Slice Grid
boundary_min_y	Y Domain Min
boundary_max_y	Y Domain Max
grid_size_y	Y Grid Size
slice_grid_size_y	Y Slice Grid
domain_min	Z Domain Min
domain_max	Z Domain Max
step_size	Z Grid Size
slice_step_size	Z Slice Grid
slice_display_mode	Display Mode
prefix	Output File Prefix

<code>index_profile = 1</code>	Compute Index Profile
<code>index_profile = 2</code>	Compute Loss Profile

D.F. CAD Preferences:

The options controlled from the CAD Preferences dialog box are not set via variables in the Symbol Table, but rather through the initialization file `bcadw32.ini`. This file can be found in your users directory.

Preferences File Options

The `bcadw32.ini` file location is `HOMEDRIVE+HOMEPATH` on Windows, and `HOME` on Linux, where `HOMEDRIVE`, `HOMEPATH`, and `HOME` are standard environment variables. The `RSOFT_INI_PATH` environment variable enables the use of a customer `bcadw32.ini` file. Set it to the folder that contains the custom file.

Appendix E

Utilities

RSoft software includes a number of utilities that can be used for a variety of tasks, including, but not limited to:

These utilities are documented in this appendix in alphabetical order.

- **Simple Data File Operations**
 - Cropping/regridding/slicing data files (`bdconv`)
 - Reordering data columns (`shufflemat`)
 - Arithmetic operations with data columns/files (`mathmat` and `bdutil -o`)
- **Simple Post-Processing**
 - Far-field calculation (`bdutil -f`)
 - Overlap integrals (`bdutil -i`)
 - Spot size calculations (`bdutil -s`)
 - Dispersion parameter calculation (`disperse`)
 - Peak-extraction from spectra (`fhakit`)
 - Analysis of FullWAVE state files (`fwmodekit`)
 - Combine results from individual frequencies (`rscombine`)
- **Data Conversion**

- Bare-matrix to RSoft format (`mat2bp`)
- Complex matrix conversion (`bdconv -v`)
- Bitmap image to RSoft data format (`bmp2ind`)
- Conversion of NK material data file to Lorentzian coefficients (`fitdisp`)
- **Customized Simulation Output**
 - Specialized simulation outputs that are not automatically created by a simulation can be calculating using these utilities.
- **Field Conversion Interfaces**
 - Convert to Synopsys' CODE V software (Ray-Tracing Converter, `bp2codev`, `codev2bp`)
 - Convert to Synopsys' LightTools software (`bdutil -ray`, BSDF Utilities)
- **Field Scattering (BSDF) Interfaces**
 - Use non-periodic excitation for periodic DiffractMOD and FullWAVE simulations (BSDF Utilities)
 - Interfaces with Synopsys' LightTools and CODE V products (BSDF Utilities)
- **Interfaces with Other Tools**
 - Many tools have text inputs and outputs. Many of these utilities can be used to convert data to/from the RSoft format.
- **Custom Post-Processing**
 - Combining several of these utilities to create custom output!

These utilities provide a convenient way to perform and automate common tasks. Many of these utilities can also be spawned directly from MOST, the RSoft scanning tool, making scan post-processing essentially transparent.

bdconv – Matrix Manipulation

This utility performs matrix transformations on data files. These transformations include extracting cross-sections (or “slices”) from a data file, changing the grid sizes in a file, and changing the file format of an arbitrary data file. This utility is designed to work file formats discussed in [Appendix B.A](#) but can be used with any arbitrary data file as well.

Note that `bdconv` and `bdutil` share the same options, either utility name can be used.

Syntax:

`bdconv` supports the following syntax and options:

usage: `bdconv` [options] input-file-name output-file-name(s)

options:

-?	help - prints this message
-x<range>	select range in x (<range> = # or #,#)
-y<range>	select range in y (<range> = # or #,#)
-dx#	set grid size in x
-dy#	set grid size in y
-rp#	rotate in phi
-rt#	rotate in theta
-tx#	translate in x (shift data - keep domain)
-ty#	translate in y (shift data - keep domain)
-sc{x y z}#	scale x,y,z axes
-tr{x y z}#	translate x,y,z axes
-sym{x y z}[#]	mirror about axis (+-1=min, +-2=max, <0=antisym, def=1)
-sc#	scale data values
-tr#	translate data values
-c	conjugate data values
-unwrap	unwrap phase of data values
-v<type>	select output value type (<type> = a,r,i,z,q,n,p)
-t	transpose matrix (3D only)
-a	alternate output fmt with explicit x/y coordinates (2D/3D)
-abare	alternate bare matrix output format (3D only)

Options:

-x<range>	These two options are used to extract data either from a certain range in the data file or at a given value of one coordinate. This is useful to either reduce the domain of a 2D file, or to take a cross section from a 2D file and thus create a 1D file.
-y<range>	

eg. Change the domain of a file `input.m00` from `[xmin, xmax, ymin, ymax] = [-5,5,-7,7]` to `[-2,3,-7,7]` and save the output as `output.dat`:

```
bdconv -x-2,3 input.m00 output.dat
```

eg. Calculate the cross section from `input.m00` at `Y=2` to create a file `output.dat` with a single column of data:

```
bdconv -y2 input.m00 output.dat
```

-dx#
-dy#

These two options are used to set the grid size in either the X or Y directions to a desired setting. The points are interpolated between given data points so that the file can be set to either have more or less data points than before.

eg. To change the x grid size of `input.m00` from `0.1 μm` to `0.5 μm` in order to decrease the size of the file:

```
bdconv -dx.5 input.m00 output.dat
```

The created file, `output.dat`, contains less grid points than the file `input.m00`, and therefore will be smaller and have a decreased resolution. The grid size can be decreased, and thus the size of the file would be increased. This option cannot create any new data, but it can be useful to increase or decrease the number of grid points.

-rp#
-rt#
-tx#
-ty#

These switches either rotate a field or translate the center coordinate of a field.

The `-rp` and `-rt` commands tilt the input field by the angle `phi` given in degrees so that it will propagate at an angle of `phi` in the XZ plane. `Phi` is measured from the Z axis.

The `-tx` and `-ty` commands translate the center of the input field in X or Y by the given number but maintains the same data domain.

eg. To tilt the file `mode.m00` in the XZ plane by 30 degrees and save the result in the file `mode_tile.m00`:

```
bdutil -rp30 mode.m00 mode_tile.m00
```

eg. To translate the file `mode.m00` by 3 microns in the X direction, and 2 microns in the Y direction and save the result in the file `mode2.m00`:

```
bdutil -trx3 -try2 mode.m00 mode2.m00
```

<code>-sc{x y z}#</code>	This command scales the x, y, or z axes.
<code>-tr{x y z}#</code>	This command translates the x, y, or z axes.
<code>-sym{x y z} [#]</code>	This option mirrors data around the axis specified to construct ‘full’ data from data calculated with symmetry. The x, y, or z axis can be selected along with an optional # number: • The number # is 1 corresponds to the minimum side along the selected axis, 2 for the maximum. • A positive # indicates symmetry, negative indicates anti-symmetry. • The default # is 1 (symmetry on the minimum side). eg. To mirror a file <code>data.dat</code> which contains the positive half of the data along the X axis: <pre>bdconv -symx data.dat data.dat</pre> eg. To mirror a file <code>data.dat</code> which contains the first quadrant of data (positive X/Y): <pre>bdconv -symx data.dat data.dat</pre>
<code>-sc#</code>	This command scales the data values.
<code>-tr#</code>	This command translates the data values
<code>-c</code>	This option takes the conjugate of the input data file and saves it to another data file. This is the same as manually taking a data file of type Real/Imaginary, and changing the sign of the columns of imaginary data.
<code>-unwrap</code>	This command outputs a ‘real’ data file which contains unwrapped phase. If the input file is real, it assumes the phase is in the real part in degrees. If the file is complex, it extracts the phase.
<code>-v<type></code>	This option allows the user to convert a data file in one of the RSoft data formats into another RSoft format. The types are: a amplitude

r real
i imaginary
z real and imaginary
q amplitude and phase
n normalized
p phase

The utility will take the input file, extract the data, and convert it to the requested format so the user can get the desired information.

eg. To convert `input.m00` from a data format of Real/Imaginary (3D) to Amplitude (3D):

```
bdconv -va input.m00 output.dat
```

-t

This option will transpose the data in the input file and save it in the output file. This is useful when performing data manipulation or matrix calculations with the data file.

-a

This option will output a file that has the absolute coordinates in x and y of the data points. Typically, a file using the RSoft file formats has several lines on the top which dictate the domain min and max as well as the number of points that are in the file for each row or column. This option will convert a file so that the absolute coordinates are given.

-abare

This command will strip the header information from a data file in an RSoft file format. By using this, all domain and number of point information will be removed from the data file.

bdutil – Overlap Integrals, Far Fields, Spot Sizes, and LightTools Ray Files

The `bdutil` utility performs several calculations within several RSoft programs and is included so the user can perform these calculations separately. The utility can calculate the overlap integral between two fields, grating coefficients, the field size, the far field projection, simple file arithmetic, and convert a far-field into a LightTools ray file. This utility is designed to work with the file formats as outlined in [Appendix B](#).

Depending on the type of calculation to be performed, the utility accepts one or more data files, calculates the requested information, and then outputs the new data either as a set of numbers or a new data file. The options for each switch will be examined, and then several examples will be given to illustrate these options.

Note that `bdconv` and `bdutil` share the same options, either utility name can be used.

Syntax

`bdutil` supports the following syntax and options:

```
usage: bdutil -i [options] input-file-name1 [input-file-name2 [weight-file]]
       bdutil -k [options] mode-file1 mode-file2 index-file1 index-file2
       bdutil -s [options] input-file-name
       bdutil -f [options] input-file-name(s) output-file-name
       bdutil -ray [options] input-file-name output-file-name
       bdutil -o$ [options] input-file-name1 input-file-name2 output-file-name
       bdutil without a function operates as bdconv.
```

options:

```
-?          help - prints this message
-i          power or overlap integral calculation (1 or 2 files)
-wx<range>  select window in x (<range> = #,#)
-wy<range>  select window in y (<range> = #,#)
-xz<range>  select window in z (<range> = #,#)
-sq1        square field 1 in overlap integral
-k          coupling coefficient calculation
-s          field size calculation
-f#         far field calculation: #=distance
-fa         far field calculation: intensity versus angle
-fp         far field calculation: periodic
-f          far field calculation: k-space
-fi         far field calculation: k-space (inverse)
-flens#     field image through ideal lens (#=focal length in um)
-r          automatically resize domain
```

```

-da#          override default angular spacing for above
-l#           set wavelength          (default=1.0)
-nu#          set frequency           (default=1.0)
-n#           set far field refractive index (default=1.0)
-nn#          set near field refractive index (default=auto)
-nl           set launch angle for periodic far field
-za           zero order angle for periodic far field
-zt           zero order theta for periodic far field
-ray          ray output from far field (angles only)
-rayp         ray output from far field (angles and positions)
              must specify extent with /wx /wy /wz
-nrays#       approximate # of rays (default is # of angles in far field)
-nraysx       force nrays to be exact
-o$           perform operation on data files ($=+|-|*|/)
-v<type>      Select output value (<type> = a,r,u,z,q,n,p)

```

Overlap Integrals: -i

The `-i` switch indicates that an overlap should be performed. The syntax for this option is:

```
bduutil -i [options] input-file1-name1 [input-file-name2 [weight-file]]
```

Note that the `-ir` switch can be used to calculate overlap integrals in radial coordinates.

Options:

The following options control how an overlap integral is calculated:

```

-wx<range>    These options control the limits of integration. By default, the utility
               uses the limits of integration which are given in the data files as
               coordinate ranges. If the files do not lie in the same range, the utility
               calculates the integral over the area where the files overlap.

-sq1          This option squares the first field before calculating an overlap
               integral. See example below for details.

-bare         This option restricts output to only numbers which can help simplify
               scripting and other tasks.

-ir           Enables radial overlap integrals. The -irout=<filename> option will
-irout        output the radial integral to a file <filename>_theta.dat.

```

Usage:

There are several combinations of input data files possible and the utility performs different calculations in each case:

These examples assume that the files contain only one coordinate. The definitions can be generalized for files which contain two coordinates. Note that these integrals are normalized.

One Input File:

If one data file is given which contains a function $f(x)$, the utility calculates three integrals of the form:

$$S = \int_{x_1}^{x_2} h(x) dx$$

where $h(x)$ is equal to $f(x)$, $|f(x)|^2$, and $|f(x)|^4$. Since $f(x)$ can be a complex function, the value of the integral of $h(x) = f(x)$ contains both a real and imaginary part.

Note that using the switches `-i1`, `-i2`, or `-i3` will output only the numerical result without any accompanying text which can aid in using the overlap results in scripts. The switch `-i1` outputs the $f(x)$ result, `-i2` outputs $|f(x)|^2$, and `-i3` outputs $|f(x)|^4$.

Two Input Files:

If two data files are given which contain functions $f(x)$ and $g(x)$ respectively, the utility calculates the overlap integral of these functions as:

$$S = \frac{\int_{x_1}^{x_2} f(x) g^*(x) dx}{\left(\int_{x_1}^{x_2} f(x) f^*(x) dx \right)^{1/2} \left(\int_{x_1}^{x_2} g(x) g^*(x) dx \right)^{1/2}}$$

where the asterisk denotes the complex conjugate. The program returns values for both S and $|S|^2$. Since a complex conjugate is taken the order in which the files are input is important. Note that using the `-iu` switch will calculate the overlap integral without the normalization, i.e. just the numerator shown in the equation above.

Two Input Files and a Weight Function:

If a weight function $\sigma(x)$ is given in addition to two data files, the utility calculates the overlap integral using the form:

$$S = \frac{\int_{x_1}^{x_2} f(x) \sigma(x) g^*(x) dx}{\left(\int_{x_1}^{x_2} f(x) f^*(x) dx \right)^{1/2} \left(\int_{x_1}^{x_2} g(x) g^*(x) dx \right)^{1/2}}$$

where the asterisk denotes the complex conjugate. The program returns values for both S and $|S|^2$. Since a complex conjugate is taken the order in which the files are input is important. Note that using the `-iu` switch will calculate the overlap integral without the normalization, i.e. just the numerator shown in the equation above.

Using the `-sq1` Option:

As an example of using the `-sq1` option, consider an overlap with two data files:

$$S = \frac{\int_{x_1}^{x_2} (f(x))^2 g^*(x) dx}{\left(\int_{x_1}^{x_2} f(x) f^*(x) dx \right) \left(\int_{x_1}^{x_2} g(x) g^*(x) dx \right)^{1/2}}$$

This is the same form as shown above except the first field $f(x)$ is squared in the numerator and the normalization term in the denominator for the first field does not have a square root (and therefore is effectively squared). The `-sq1` option can also be used with the weight function as needed.

Examples:

To calculate the integral of a function $f(x)$, contained in a file `field.fld` over a range of (-3,2):

```
bduutil -i -wx-3,2 field.fld
```

To calculate the overlap integral between two data files, `field1.fld` and `field2.fld`, with the default limits of integration and also squaring the first field:

```
bduutil -i -sq1 field1.fld field2.fld
```

To calculate the overlap integral between two data files, `field1.fld` and `field2.fld`, with a weight file, `weight.fld`, over the range (-2.3, 6.1), use the command:

```
bduutil -i -wx-2.3,6.1 field1.fld field2.fld weight.fld
```

Coupling Coefficients: -k

The `-k` switch calculates the coupling coefficient, kappa, between two modes traveling in opposite directions in a grating. The correct syntax for this calculation is:

```
bduutil -k [options] mode-file1 mode-file2 index-file1 index-file2
```

This switch requires four files to be input: two index profiles corresponding to the peak and mean index perturbation of the grating and two mode files which correspond to the two modes being studied. These files need to be in the file formats specified in [Appendix B](#). They can be produced directly with RSoft software or through any other means as long as the file format is correct. The `-bare` option can be used to restrict the output to only numbers which can help simplify scripting and other tasks.

Usage:

The utility will produce both the real and imaginary parts of kappa, as well as the effective index of the grating. Kappa is defined as:

$$\kappa = \frac{k_0}{4n_{eff,\mu}} \frac{\int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} (n_2^2 - n_1^2) \xi_{\nu t} \xi_{\mu t}^* dx dy}{\int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} \xi_{\mu t} \xi_{\mu t}^* dx dy}$$

where n_1 is the mean index and n_2 is the peak index. The calculated number is a dimensionless form of the above expression in which the k_0 factor is not present.

This is consistent with the book: Raman Kashyap, “Fiber Bragg Gratings,” Academic press, 1998, pp.143, equation (4.3.6). It is also consistent with the paper: P. Correc, “Coupling Coefficients for Trapezoidal Gratings,” IEEE Journal of Quantum Electronics, Vol. QE-24, No. 1, 1988, pp.8, equation (1), (2) and (3).

Example:

To calculate the coupling coefficient between a forward mode, `mode1.m00`, and a backward mode, `mode2.m00`, in a grating with a peak index profile of `index1.ipf` and a mean index profile of `index2.ipf`:

```
bdutil -k mode1.m00 mode2.m00 index1.ipf index2.ipf
```

Field Size Calculation: -s

The `-s` switch calculates the 1/e height and width of a field in microns. Its syntax is

```
bdutil -s [options] input-file-name
```

For 2D data files, the 1/e width is clearly defined. For 3D data files, `bdutil` calculates the effective width/height. For example, to calculate the width of a field $a(x,y)$, `bdutil` first calculates the average amplitude $b(x)$ by integrating $a(x,y)$ along y , then finds the 1/e width of $b(x)$.

Example:

For example, the 1/e width and height of a mode file `mode.m00` can be found using the command:

```
bdutil -s mode.m00
```

Far Field Calculations: -f

The `-f`, `-f#`, `-fa[u]`, `-fp`, `-fi`, and `-flens#` switches calculate the spatial-far field projection of a field file in a homogeneous dielectric region (free space) via a Fourier transform. The syntax for this option is (`-f` is used as an example but any of the previous switches can be used):

```
bdutil -f [options] input-file-name(s) output-file-name
```

If multiple input files are specified, they will be interpreted as vector components of the same field (i.e. E_x , E_y , and/or E_z). Note that when non-uniform grids are used, the near-field is converted to a uniform grid (with average grid size of non-uniform grid) before the far-field is calculated.

Usage:

To calculate a far-field with `bdutil` you must use one of these switches:

`-fa` The `-fa` switch produces a plot of intensity vs. angle along with values for the beam divergence of the full angle at both the $1/e$ as well as the half intensity. This computation starts with the spatial near-field, $E(x)$ and computes the Fourier transform, $F(k_x)$. Next, $F(k_x)$ is normalized such that

$$\int F(x) dx = \int F(k_x) dk_x$$

The angular distribution of the far field $I(\theta_x)$ is then computed as

$$I(\theta_x) = \left| \cos(\theta_x) F\left(\frac{2\pi}{\lambda} \sin(\theta_x)\right) \right|^2$$

and normalized by its maximum value, resulting in a cumulative form of:

$$\frac{I(\theta_x)}{I_0} = \frac{|E(\theta_x)|^2}{|E_0|^2} = \frac{\cos^2(\theta_x) \left| \int_{-\infty}^{\infty} E(x) \exp(j \sin(\theta_x) k_0 x) dx \right|^2}{\left| \int_{-\infty}^{\infty} E_0(x) dx \right|^2}$$

This derivation illustrates a 1D version of the calculation; the 2D is directly analogous.

By default, the angular resolution of the Fourier transform is calculated directly from the range of data in the input file. To override this angular resolution, the option `-da#` can be used to specify a custom angular spacing. The normalization can be skipped using the `-fpu` option.

The files output by this option are functions of the angles ϕ and θ (in 3D) and are meant to be plotted in polar coordinates (use the `/polar` WinPLOT option).

`-f` The `-f` switch calculates the intensity of the far-field in k-space via a Fourier integral. This computation is equivalent to the first steps of the method to compute the far field as a function of angle described above to compute the normalized value of $F(k_x)$.

`-f#` The `-f#` switch calculates the field at a distance of # microns. This calculation is performed via the FFT-BPM method and propagates for one step which is only valid for homogenous or free-space regions. See the BeamPROP manual for further discussion of this method. The option `-r` can be used to increase the boundary size if the field diverges outside of the boundary specified in the input file.

`-fp` The `-fp` switch calculations the far-field projection of a periodic field from a single plane wave launch. This method assumes that the data file contains one period of the field, decomposes it into plane waves, and then plots the amplitude in each diffraction order. The options `-za` and `-zp` can be used to set the launch angles of the plane wave and the `-nl` option can be used to set the index of the launch material.

`-fi` The `-fi` switch calculates the far-field in inverse k-space.

`-flens#` The `-flens#` option calculates the field image through an ideal lens where # is the focal length in μm .

Options:

These options can be used to further control the far-field calculation:

- | | |
|-----------|--|
| -da | This option overrides the default angular spacing for the -fa switch which calculates the far-field intensity vs. angle. |
| -l
-nu | By default, <code>bdutil</code> will use the wavelength defined in the header of the data file(s) used to calculate the far-field. These options allow the user to specify the wavelength (or frequency) that should be used in the calculation: only one of these options should be specified at once. The option -l sets the wavelength [μm] and defaults to a value of 1 μm and the option -nu# sets the frequency [μm^{-1}] and defaults to a value of 1 μm^{-1} . |
| -n | This sets the far-field refractive index that the field is propagated through. By default, it is assumed the field propagates through air, and therefore this value defaults to 1. A value of -1 indicates that the near-field index should be extended to the far-field. This option is useful, for example, when computing the farfield in a substrate or another material than air. |
| -nn | This sets the near-field refractive index. This should be set to the index where the near-field was measured. By default, this value is taken from the header of the data file(s) used. If not specified in the data file, the far-field index value (-n) is used. |
| -nl | This sets the launch index when using the -fp switch. |
| -r | This option can be used to automatically resize the domain when using the -f# option. |
| -v<type> | This option allows the user to set the format of the far-field output. The options are the same as the <code>bdconv</code> utility. |
| -za | This option sets the launch angle ϕ for the -fp option (1D and 2D data files). |

`-zt` This option sets the launch angle θ for the `-fp` option (2D data files only).

Examples:

The far field projection of a field file `field.fld` given as intensity as a function of angle with an angular resolution of 0.1 degrees at a wavelength of 1.55 μm in a material with a refractive index of 1.5 can be saved in a file `field2.fld`:

```
bdutil -fa -da0.1 -l1.55 -n1.5 field.fld field2.fld
```

The field profile for the above case after a distance of 100 μm :

```
bdutil -f100 -r -l1.55 -n1.5 field.fld field2.fld
```

The far-field from a periodic near-field `field.fld` calculated with a plane wave launch incident in a material with refractive index of 1.5, at a launch angle of 10 degrees, at a wavelength of 1.55 μm , and calculated into a material with a refractive index of 2 can be calculated with this command:

```
bdutil -fp -l1.55 -nn1.5 -n2 -za10 field.fld field2.fld
```

LightTools Interface

The `-rays` switch converts RSoft far-field into a LightTools ray file for use as a Ray Data File in LightTools. Its syntax is:

```
bdutil -ray [options] input-file-name output-file-name
```

where the filenames correspond to the RSoft far-field file and the desired LightTools ray data file. This process can be automated when using the LED Utility by using the options described in the LED Utility manual. Note that the polarization data in the RSoft BSDF file will be averaged when converted.

Options:

`-ray[p]` Required for ray data file conversion. The `-ray` option outputs a point source ray data file, the `-rayp` option outputs the ray data file over an extended plane.

`-nrays#` This sets the number of rays to use. The default is based on the field resolution in the far-field data file.

`-nraysx` If the number of rays requested is more than the resolution the data file supports, this option will force the converter to use the number of rays requested.

`-wx#, #`
`-wy#, #`
`-wz#, #` Sets the dimensions of the extended plane.

Example:

To convert RSoft far-field `run1_ffoutput.dat` into the LightTools ray data file `lt_rdf.txt` with approximately 4000 rays, use the command:

```
bdutil -ray -nrays4000 run1_ffoutput.dat lt_rdf.txt
```

Simple File Arithmetic and Data Transformations: `-o`, `-sc`, `-tr`

The `-o` switch perform simple mathematical operations on one or two fields and outputs the result. Its syntax is:

```
bdutil -o$ [options] input-file-name1 input-file-name2 output-file
```

where `$` can be one of four mathematical operations: `+`, `-`, `/`, or `*` for addition, subtraction, division, and multiplication. Note that the `-sc` and `-tr` options can be used to scale and translate the data or axes by a factor (see `bdconv` documentation). Note that the `-sc` and `-tr` options can be used together, the scale operation will be done first.

Example:

The file `file1.dat` and `file2.dat` can be added together and saved as `file3.dat` using the command:

```
bdutil -o+ file1.dat file2.dat file3.dat
```

The file `file1.dat` can be scaled by a factor of 2.4 and saved as `file2.dat` using the command:

```
bdutil -sc2.4 file1.dat file2.dat
```

The X data in the file `file1.dat` can be translated by 5 and saved to the file `file2.dat` using the command:

```
bdutil -tx5 file1.dat file2.dat
```

bmp2ind – Convert Bitmap Image to a User Profile

The `bmp2ind` utility converts an indexed-color bitmap image file into an RSoft data file. This allows, for example, the use of an SEM image to define the cross-section of a waveguide or fiber via a user-defined index profile. The utility supports monochrome (1-bit), 16-color (4-bit), or 256-color (8-bit) indexed-color bitmaps.

This utility does not support 24-bit bitmaps. See ‘Background’ section for details about converting bitmaps to a supported format.

Syntax:

`bmp2ind` supports the following syntax and options:

```
usage: bmp2ind [options] <input-file> [<output-file>]
```

options:

```
-?      help - print this message
-w#     the unit width of the output profile
-h#     the unit height of the output profile
-b#     the lower index that the profile will scale to
-f#     the upper index that the profile will scale to
-s#     0-std format, 1-scientific format
-m=$    specifies the index map to be used
```

Usage:

`-w#` Sets the width (`-w`) and height (`-h`) of the output profile. The default
`-h#` value for both of these options is 2.

`-b#` Sets the data value range that will be mapped to the color palette in the
`-f#` bitmap image:

- The `-b` option sets data value assigned to the lowest color index in the color palette. The default value for this parameter is 0.
- The `-f` option sets the data value assigned to the highest color index in the bitmap file. The default value for this parameter is 1.

The data value for each pixel is calculated as:

$$f(x, y) = \frac{c_i}{c_m} (v_f - v_b) + v_b$$

Where v_b is the data value set by the `-b` option, v_f is data value set by the `-f` option, c_i is the color index of the pixel, and c_m is the maximum color index in the color palette.

The default data range of (0,1) is designed to create normalized index profile functions as described in [Section 6.A](#). These options, when used with a grayscale bitmap, use the gray level to set the data value. Typically, this means that black (color index of 0) maps to the minimum data value, and white (color index of 255) maps to the maximum data value.

The use of these options is illustrated in Example 2 below. Note that the use of the `-m` option supersedes these options.

`-s#` Controls the use of scientific notation in the output data file: a value of 1 uses it, a value of 0 does not.

`-m=$` Selects an optional index map file. This file allows you to map specific color indices in the palette to specific data values. The use of this option supersedes the `-b` and `-f` options.

The index map file is a simple text file that contains multiple lines, each line which maps a color index or range of color indices to a data value. For example, a 16-color bitmap might have an index map file like:

```
0 = 1.0
1-5 = 3.5
6-7 = 1.5
8-9 = 1.8
10-13 = 2.4
14-15 = 1.3333
```

where, for example, any pixels in the bitmap with a color index of 11 will be assigned a refractive index of 2.4. The use of this feature is illustrated in Example 1 below.

Indexed-Color Bitmap Background

Note that our use of the term ‘color’ here refers to an arbitrary RGB (red/green/blue) value, including white, black, shades of gray, etc.

The `bmp2ind` utility supports monochrome (1-bit), 16-color (4-bit), or 256-color (8-bit) indexed-color bitmap image files. An indexed-color bitmap file is a data matrix where each pixel, or matrix value, is assigned a color index from a color palette, or lookup table, as illustrated in Fig. 1. There are several programs that can be used to save these types of bitmaps as well as view/modify the color palette. These include Adobe® Photoshop® (www.adobe.com), Microsoft Paint (saving bmp files only), and IrfanView (free for non-commercial use, www.irfanview.com).

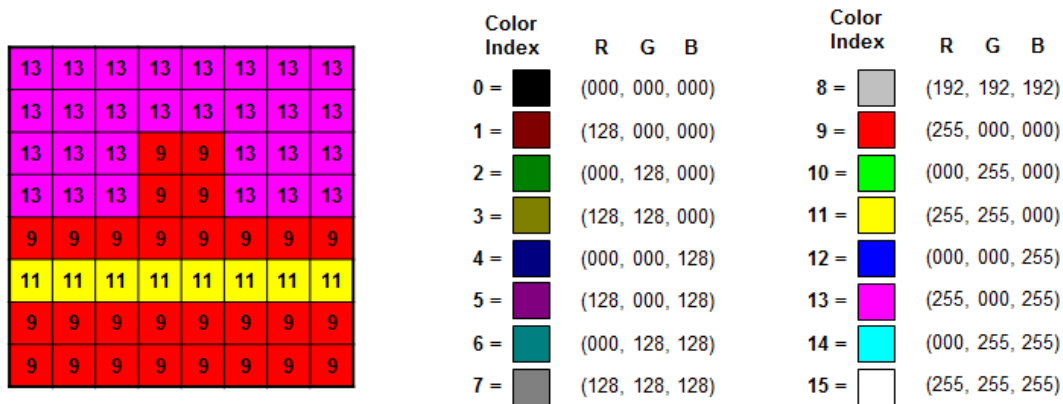


Figure 1: Example 16-color bitmap. a) The bitmap as it would appear on-screen with color indexes for each point. b) The color palette used to display this bitmap. Each color index corresponds to a RGB color, for example 9 = red = RGB(255, 000, 000). This is the standard Microsoft Windows default 16-color palette.

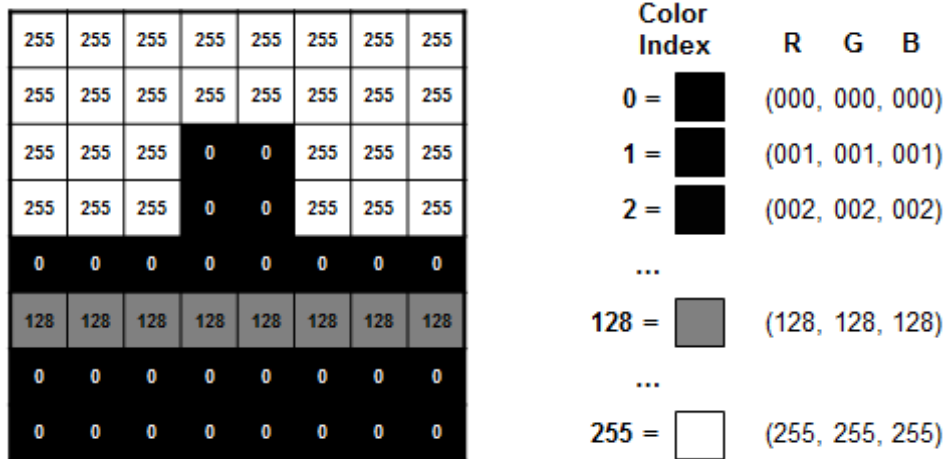


Figure 2: Example 8-bit grayscale (256-color) bitmap. a) The bitmap as it would appear on screen with color indexes for each point. b) The 256-level grayscale color palette used.

The `bmp2ind` utility converts the color index of each data point, or pixel, to a refractive index data value. While it is necessary to understand the relationship between color index and RGB color to have a meaningful data conversion, the actual color map used does not matter. When used with the `-m` option, the utility will convert each color index to a particular data value, and when used with the `-f` and `-m` options, the color index will be scaled between the defined min/max values.

Examples

Consider the two bitmaps illustrated in Figs. 1 and 2; we will convert these into index profile data files to illustrate the use of the `bmp2ind` utility:

Note that these bitmaps are very low resolution for illustrative purposes, when converting your own images it is better to use high-resolution bitmap images.

Example 1: 16-color Bitmap

The bitmap in Fig. 1 represents the cross-section of a typical SOI (Silicon-on-Insular) structure. For the purposes of this example, assume that the bitmap image represents a 5 μm by 5 μm region, and that the colors map to actual refractive index values as:

Color	Color Index	Desired Material	Desired Refractive Index
Red	9	Silicon	3.5
Yellow	11	Silica	1.5

Magenta

13

Air

1.0

Based on the information in the above table, we create a simple text file that maps the color indexes to the desired refractive indexes. One such file is:

```
0-9    = 3.5
10-11  = 1.5
12-15  = 1.0
```

Assuming that the bitmap is named `image.bmp` and the index map is named `map.txt`, the conversion can be done with the following command:

```
bmp2ind -w5 -h5 -m=map.txt image.bmp data.dat
```

where `-w5` and `-h5` set the dataset's dimensions, and `data.dat` is the output file name. The resulting data file is shown in Fig. 3 and should be used with the **Absolute index in User Profile** option described in [Section 6.D.5](#).

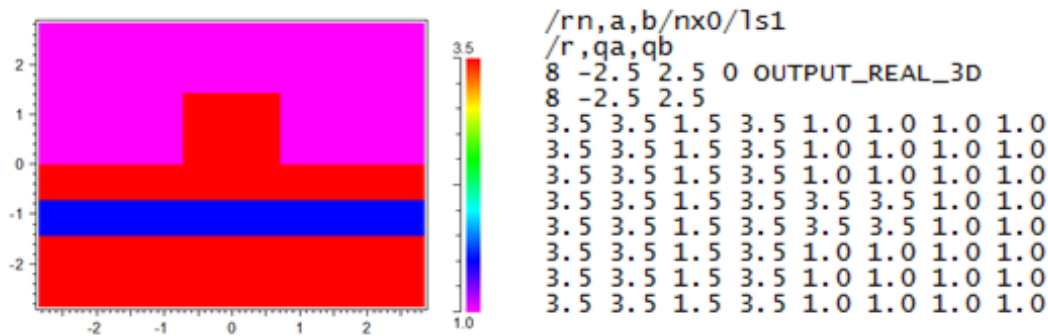


Figure 3: Data file a) as plotted in WinPLOT (with interpolation disabled), and b) the converted data as seen in a text editor (produced with the `-s0` option and slightly edited for visual clarity).

Example 2: 8-bit Grayscale Bitmap

The bitmap in Fig. 2 also represents the cross-section of a typical SOI (Silicon-on-Insular) structure. For this example we will use the default data range (0, 1) and the default X and Y coordinate range (-1, 1) so that it can more easily be used as an index profile in the RSoft CAD.

Assuming the bitmap is named `image.bmp`, the conversion can be done with the following command:

```
bmp2ind image.bmp data.dat
```

where `data.dat` is the output file name. The insulator region, which has a color index of 128, will, according to the equation $f(x,y) = (c_i/c_m) \cdot (v_f - v_b) + v_b$, have a data value of $(128/255) \cdot (1 - 0) + 0$, or 0.501961. The resulting data file is shown in Fig. 4.

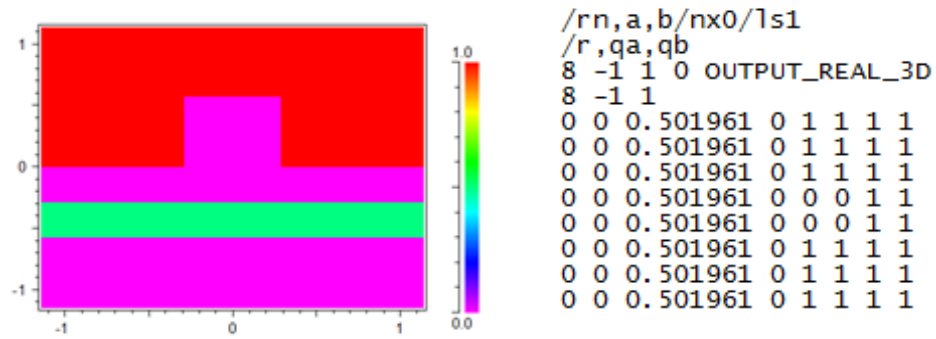


Figure 4: Data file a) as plotted in WinPLOT (with interpolation disabled), and b) the converted data as seen in a text editor (produced with the `-s0` option for visual clarity).

bp2codev and codev2bp – Ray Tracing Converter for CODE V

The `bp2codev` and `codev2bp` utilities convert RSoft field data to Synopsys' CODE V data and vice versa.

Converting from RSoft to CODE V (bp2codev)

The `bp2codev` utility can be used to convert RSoft field files to the CODE V format.

Syntax:

`bp2codev` supports the following syntax and options:

usage:

```
bp2codev      [options] input-file output-file
bp2codev -V [options] input-file [input-file2] [input-file3] output-file
bp2codev -I [options] input-file output-file [output-file2]
```

options:

```
-?      help - prints this message
-R      CODEV "Real"    output format (default is "Complex")
-V      CODEV "Vector" output format (Ex and/or Ey and/or Ez input)
-I      CODEV "INT"     output format (outfile1=FIL [outfile2=WFR])
-L#     wavelength (um) [N/A for INT format]
-N#     refractive index [N/A for INT format]
```

Usage:

- ? This option prints the help for this utility.

- R This option enables the output of a “Real” formatted CODE V file. The default, if this option is unset, is “Complex”.

- V This option enables the output of a “Complex” formatted CODE V file. In this mode, 1, 2, or 3 RSoft field files for different vector components can be used.

- I This option enables the output of the CODE V “INT” format. In this mode, the first output file is the “FIL” file, and the second optional output file is the “WFR” file.

- L# This option sets the wavelength in microns of the RSoft field files. The default takes the wavelength value from the header of the data files, and if not provided in the header, uses a value of 1.

- N# This option sets the refractive index of the plane where the RSoft field files are defined. The default takes the refractive index value from the header of the data files, and if not provided in the header, uses a value of 1.

Examples:

To convert the vector RSoft field contained in the files `field_ex.dat`, `field_ey.dat`, `field_ez.dat`, where the wavelength is defined in the file header, to the CODE V file `codev.dat` use this command:

```
bp2codev field_ex.dat field_ey.dat field_ez.dat codev.dat
```

To convert the scalar RSoft field contained in the file `field.dat` where the wavelength (1.55 μm) is not defined in the file header, to the CODE V file `codev.dat` use this command:

```
bp2codev -1.55 field.dat codev.dat
```

Note that the phase in the WFR output file is always wrapped.

Converting from CODE V to RSoft (codev2bp)

The `codev2bp` utility can be used to convert CODE V files to the RSoft field format.

Syntax:

`codev2bp` supports the following syntax and options:

usage:

```
codev2bp input-file-name output-file-name ("Complex" input)
codev2bp FIL-file-name [WFR-file-name] output-file-name ("INT" input)
```

options:

- ? help - prints this message
- A interpret FIL data as amplitude instead of intensity
- P# output plot scripts (1=write script, 2=spawn script)

Usage:

- ? This option prints the help for this utility.

- A Interprets the FIL dataset as amplitude instead of intensity.

- P This option enables the output of WinPLOT plot files.

Examples:

To convert the vector CODEV field contained in the file `codev.dat` to the RSoft file format with a prefix of `field`, use this command:

```
codev2bp codev.dat field
```

BSDF Utilities (`bsdfgen` and `bsdfutil`)

The RSoft BSDF Utilities `bsdfgen` and `bsdfutil` can be used to generate and use BSDF (Bi-Directional Scattering Function) files.

These utilities are fully documented in the RSoft BSDF Utilities manual.

- The BSDF Generation Utility uses an RSoft product (FullWAVE or DiffractMOD) to generate an RSoft BSDF file that contains the scattering information of a periodic structure as a function of input angle(s), polarization, and/or wavelength. The simulations only require one unit cell of the periodic structure.
- The BSDF Viewer can be used to visualize the contents of an RSoft BSDF file.
- RSoft BSDF files can be used via the RSoft BSDF UDOP can be used to define surface properties in LightTools.
- The `bsdfutil` utility can be used to ‘pass’ an arbitrary finite optical field ‘through’ the BSDF database and reconstruct the transmitted and reflected near-field. This effectively allows the use of an arbitrary finite optical field with single unit-cell FullWAVE or DiffractMOD simulations. This would allow, for example, a finite Gaussian beam to be launched at a periodic surface grating.
- These utilities, along with the Ray Tracing Converter Utility, enable interfaces with ray-tracing tools such as Synopsys’ CODE V package.

Custom PDK Utility (smatgen)

The generation of custom PDK models, including s-matrix, effective index, or other data, is a common task for the optical designer. The RSoft tools include the Custom PDK Utility, see the Custom PDK Utility manual. The utility automates the calculation of simulation data, mask files, and creates several PDK-related files including a compact model that can be simulated in Synopsys' PIC Solution tools.

coatuf – Create Coating for User Profile

When used as a height profile on a multilayer component, a user profile data file can be used to define a surface in the RSoft CAD. This data file can be obtained from Atomic Force Microscopy (AFM) or other means. The `coatuf` utility allows users to add a coating on top of this surface by creating a second user profile data file that can then be used in the RSoft CAD to define the coating.

Syntax:

`coatuf` supports the following syntax and options:

```
usage: coatuf [options] <userprofile-file> <output-file>
options:
  -?      help - prints this message
  -t#     set thickness in all directions
  -tx#    set thickness in X
  -ty#    set thickness in Y
  -tf#    set thickness in F (function value)
  -sx#    set scale in X
  -sy#    set scale in Y
  -sf#    set scale in F (function value)
```

Options:

- | | |
|--|---|
| -? | This option display help for this utility. |
| -t# | Sets the coating thickness. The <code>-t</code> option can be used to set all directions equally, alternatively the individual options can also be used. |
| -tx# | |
| -ty# | |
| -tf# | |
| -s# | Sets the scale factor for applying the coating. In many cases the data in the user function is normalized and will be scaled in one or more ways in the RSoft CAD. These options set the scale between the coordinate system in the data file and the final desired coordinate system. If these options are not set correctly, the thickness will not have the correct thickness. The data in the file typically has coordinates in the header to specify the X/Y extent of the data and the data will have its own extent. |
| -sx# | |
| -sy# | |
| -sf# | |
| | |
| There are many cases, here are two simplest: | |
| <ul style="list-style-type: none">• For the case where the data in the file is in real coordinates, i.e. | |

the X/Y coordinates in the header and the data values all represent the actual data coordinates, then the scale factor should be 1 for all axes.

- For the case where the data in the header is normalized from -1 to 1 for both X and Y, and the data values are normalized from 0 to 1, the X and Y scale factors should be half of their respective size in real coordinates and the function scale coordinate should be the actual total thickness of the uncoated surface.

Example:

To add a thickness of 0.3 μm in all directions to the user profile contained in the file `profile.dat` and save it in the file `profile_coat.dat` where the data in the profile uses the normalized coordinate system (X/Y from -1 to 1, data from 0 to 1) but will have a final size of 10 μm in X, 4.5 μm in Y, and a uncoated thickness of 2.3 μm , use this command:

```
coatuf -t0.3 -sx5 -sy2.25 -sf2.3 profile.dat profile_coat.dat
```

The `profile_coat.dat` file can then be used as a height profile for a second segment in the CAD that is positioned at the same location as the uncoated segment.

disperse – Calculate Dispersion Relations

The `disperse` utility takes a file containing effective index values and produces a dispersion relation.

Syntax

`disperse` supports the following syntax and options:

```
usage: disperse [options] neffdatafile
options:
  -v      view derivative data in Notepad
  -p      plot derivative data in WinPlot
  -b      compute derivatives of beta vs k instead of neff vs x
  -g      compute group velocity and dispersion vs lambda
  -x#     x data is (0=lambda,1=1/lambda,2=k) (default=0)
  -c      y data is complex (i.e. "nk" data)
  -k      output nimag if present
  -fp#    enable polynomial fit (default=disabled, # is order
[default=3])
```

The input file `neffdatafile` must contain effective index data as a series of columns, with the first column containing wavelength/frequency data as indicated by the `-x` option.

dxf2ind – Convert DXF Mask to .ind File

See Mask [Conversion Utilities](#) section.

ffutil – Far-Field Utility

This program calculates multi-plane far-fields.

Syntax

ffutil supports the following syntax and options:

```
usage: ffutil [options] [<ind-file>]

options:
--prefix $      prefix for near field files
--nu#           set frequency                (default=1.0)
--nphi#         set number of phi points     (default=72)
--ntheta#       set number of theta points  (default=36)
--nf-index#     index of near field media    (default=1.0)
--ff-use-planes$ near field planes to use    (default=xp,xm,yp,ym,zp,zm)
--ff-preserve   preserve near field files    (default=0)
```

Usage

ffutil calculates multi-plane far-field with these options:

--prefix \$	Sets the Output Prefix of the near-field files.
-nu#	Sets the frequency in μm^{-1} , set to 1 by default.
-nphi# -ntheta#	Sets the number of phi and theta points, defaults to 72 and 23 by default.
-nf-index#	Sets the near-field index, defaults to 1.0.
-ff-use-planes\$	Select the planes to use, by default all 6 are used.
--ff-preserve	Save the near field files after the calculation.

fhakit – Fast Harmonic Analysis of Complex Exponential Time Series

The analysis of cavity modes and Q -factors by FullWAVE's Q-Finder tool is based on the Fast Harmonic Analysis (FHA) technique discussed in Chapter 9 of the FullWAVE manual. Given a time (or distance) series that is expected to be a sum of complex exponential terms (or approximately so):

$$f(t) = \sum_k c_k \exp(i2\pi\tilde{\nu}_k t) + \text{c.c.}$$

FHA can efficiently extract the complex frequency and amplitude of those terms which fall within a narrow frequency range.

This analysis can be performed directly at the command line using the utility `fhakit`. This can be useful when you need to analyze a series that was not produced by an automated Q-Finder calculation, or if you wish to analyze a signal in a different way to that performed by Q-Finder. For example, from a single propagation, you could extract the complex frequencies of cavity modes that occur in quite different frequency bands by running `fhakit` several times on the same `.tmn` file. In contrast, because it searches dynamically for a resonance, Q-Finder focuses on a single frequency band in each run.

Syntax

`fhakit` supports the following syntax and options:

- Extract complex frequencies and amplitudes using Fast Harmonic Analysis method:

```
fhakit -fha [options] filename prefix
```

- Perform FFT and extract real peak frequencies:

```
fhakit -fft [options] filename prefix
```

options:	
-help	[show this message]
-v <int>	[verbosity (default=0)]
-q	[quiet mode (default=0). Don't write to standard output.]
-dt <float>	[time (or distance) step. [default=value from column zero]]
-cols <string>	[comma-separated list of columns to analyze [default=column one]]
-complex	[interpret columns as complex pairs [default=no]]
-J <int>	[number of local basis states for FHA expansion]

<code>-numin <float></code>	[minimum search frequency (required for FHA)]
<code>-numax <float></code>	[maximum search frequency (required for FHA)]
<code>-toff <int></code>	[time offset (number of points to drop at start of time series). [default=0]]
<code>-tmax <int></code>	[maximum number of time steps of time series to use [default=all]]
<code>-n</code>	[don't suppress negative frequencies]
<code>-t <float></code>	[threshold: fraction of total energy to accept a peak [default=1e-5]]
<code>-rt <float></code>	[relative threshold: fraction of energy of maximum accepted peak to accept peak [default=1e-4]]
<code>-tol <float></code>	[tolerance for consistency check of FHA peaks [default=1e-3]]
<code>-spec</code>	[interpolate spectrum [default=0]]
<code>-si</code>	[sort in order of decreasing intensity instead of increasing frequency]

Usage:

`fhakit` can be used in two modes, FHA or FFT, which are selected by the flag `-fha` or `-fft`. In each case, `fhakit` reads data from a file `<filename>` and writes a list of detected peaks and properties to a file named `<prefix>_m.nufha` or `<prefix>_m.nufft`, where `m` is an integer from 0 indicating the particular set of columns being analyzed. Intermediate output is written to a file named `<prefix>_fha.log`.

In each mode, `fhakit` looks for all resonances in the bandwidth between `numin` and `numax` whose amplitudes satisfy certain criteria discussed in the explanation of options below.

The output files contain the following data: resonance index, complex angular frequency, complex frequency, complex amplitude, quality factor, an internal consistency error, the mean “energy” of the resonance, and the fraction of total energy in the signal represented by this resonance. The mean energy is simply the mean value of the norm of the function f :

$$E = \frac{1}{N} \sum_{j=0}^{N-1} |f_j(t)|^2$$

In FFT mode, the imaginary parts of the frequency and the Q factor are not available and are recorded as zeros. Further, the values of peak and fractional energy are not very meaningful. However, in FFT mode only, the total energy in the band between `numin` and `numax` is recorded in the header as the `Bandwidth energy`.

By default, `fhakit` interprets the zeroth column of a data file as the time variable (which must be equally spaced) and reads real signal data from the first column. However, multiple columns including the zeroth column may be analyzed using the `-cols` option and interpreted as complex

data using the `-complex` flag. If the zeroth column is used as signal data, then the time step must be specified using the `-dt` option.

Options:

<code>-help</code>	This option prints the help for this utility.
<code>-v <int></code>	Verbosity. A value of 0 means low output, 1 means verbose output. In particular, in verbose mode, a list of candidate eigenvalues is output to the screen. The candidate values list all detected resonances that lie within the requested frequency band regardless of their strength. This list of candidates is then culled according to the <code>-tol</code> , <code>-t</code> and <code>-rt</code> options. Thus this mode can be useful for identifying small peaks which are not showing up in the final answer.
<code>-q</code>	Sets quiet mode. In this mode, all output is written only to log files. There is no text written to standard output.
<code>-dt <float></code>	Sets the value of the increment between time steps. This option is required if the <code>-cols</code> option specifies that the zeroth column is to be interpreted as data. Otherwise, it is optional and can be used to scale frequencies differently to the default time step extracted from the zeroth column.
<code>-cols <string></code>	This option takes a comma-separated list of positive integers that specifies which columns of the file (counted from zero,) should be analyzed. The first requested column produces data in the file <code><prefix>_0.nufha</code> , the second column produces <code><prefix>_1.nufha</code> , etc. If the <code>-complex</code> option is set, the columns are treated as real and imaginary pairs and there must be an even number of columns specified.
<code>-complex</code>	Indicates that the input file columns should be interpreted as real/imaginary pairs. If the <code>-cols</code> option is not specified, then columns 1 and 2 are analyzed as a complex pair.

<code>-J <int></code>	Number of basis states used for FHA calculation. Values above 400 or so will run increasingly slowly. Normally this should not need to be set.
<code>-nummin <float></code> <code>-nummax <float></code>	Minimum/maximum value of frequency range in which to look for peaks. The argument is a plain frequency, not an angular frequency.
<code>-toff <int></code>	Number of time steps to ignore at the start of the time series. The default is 0. This can be useful if the start of the time series is noisier than the rest of the signal and gives rise to spurious weak resonant peaks.
<code>-tmax <int></code>	Maximum number of time steps to include. This can be used to truncate a signal which degenerates into noise at large values of time. If both <code>-toff</code> and <code>-tmax</code> options are specified the range of time steps sampled is <code>[toff, tmax+toff)</code> .
<code>-n</code>	Include negative frequency peaks in analysis. By default, <code>fhakit</code> ignores negative frequencies. For a “clean” real signal, every positive frequency peak should be accompanied by a negative frequency of the same amplitude. For complex signals, the positive and negative frequency peaks are in general different. With this option, the allowed frequency range is effectively the union of <code>[-numax, -numin]</code> and <code>[numin, numax]</code> .
<code>-t <float></code>	Specifies the threshold peak energy as a fraction of total energy in the signal. Any peak accounting for less than this fraction of energy is discarded. The default is 10^{-5} . Use a smaller value if you find peaks of interest are being ignored.
<code>-et <float></code>	Specifies the relative threshold peak energy as fraction of the energy of the largest accepted peak. Any peak accounting for less than this fraction of the energy of the largest peak in the frequency search range is discarded. The default is 10^{-3} . Use a smaller value if you find peaks of interest are being ignored.

<code>-tol <float></code>	Specifies the tolerance of the eigenvalue consistency error. A consistency check is performed on each candidate peak to detect spurious peaks. The acceptable error can vary greatly depending on the quality of the signal. The default value is $10e-3$, but for a low-noise signal, the achievable consistency error may be much lower. On the other hand, for a noisy signal, it may be necessary to set values as large as 10 or more. Use the following strategy to set this option. At first, use the default value. If fhakit fails to find the expected peaks, run it in verbose mode and check the consistency error reported in the list of candidate resonances. If all these errors are larger than the default $10e-3$, then use this option to set a suitable value.
<code>-spec</code>	In FHA mode, generates a plot named <code><prefix>_spec_fha.pcs</code> that represents the complex spectrum corresponding to the detected resonances. In FFT mode, generates a plot named <code><prefix>_spec_fft.pcs</code> that contains the complex FFT in the specified frequency range.
<code>-si</code>	Causes resonant peaks to be sorted in order of decreasing peak intensity instead of increasing real frequency.

findpks – Find Peaks Utility

The `findpks` utility can be used to find peaks in a data file.

Syntax:

`findpks` supports the following syntax and options:

```
usage: findpks [options] <infile >outfile
ptions:

-n#       set maximum number of peaks (default=5)
-t#       set threshold (fraction of maximum, default=0.01)
-m#       only use first # peaks for determining maximum (default=all)
-mx#      only use peaks with x<# for determining maximum (default=all)
-minx#    output only peaks with x>=# (default=all)
-maxx#    output only peaks with x<=# (default=all)
-x        exclude peak at x=0
-s#       set scale factor by which to multiply peak positions
-v#       set optional value to output before peaks
-c<cols>  set col number(s) to search for peaks (default=1,noargs=all)
          (<cols> = comma-separated list of #'s or #-# ranges)
-cr#      set relative tol for multicolumn search (default=0.5*dx)
-ca#      set absolute tol for multicolumn search (default=N/A)
```

Options:

-n#	Sets the maximum number of peaks to find. By default, 5 peaks are found.
-t#	Sets the threshold used to define a peak. This is set as a fraction of the maximum peak value in the file, by default it is 0.01.
-m#	Sets the number of peaks to use when determining the maximum peak value.
-mx#	Use only peaks with $x > \#$ when determining the maximum peak value.
-minx# -maxx#	Output peaks with $x > \text{min}$ or $x < \text{max}$
-x	Exclude peak at $x = 0$.
-s#	Sets a scale factor by which to multiple peak positions.
-v#	Sets optional value to output before peak, useful for building data files to be plotted.

-c<cols>	Sets the columns within which to search for peaks. Default = all.
-cr#	Sets relative tolerance for multi-column search (default = 0.5*dx)
-ca#	Sets absolute tolerance for multi-column search.

fitdisp – Dispersion Fitting Utility

The `fitdisp` utility can be used via a graphical interface in the Material Editor as described in [Section 8.B.4](#).

The `fitdisp` utility allows users to create Lorentzian dispersion models from data files that can be used with the Material Editor in the RSoft CAD. The data file should have two or three columns: the first with frequency or wavelength data, the second with the real refractive index data (epsilon or index), and a third optional column with the imaginary index data (epsilon or index).

The `fitdisp` utility will display the coefficients for the material fit as well as create a file name `out.ind` which will have the material defined in the Material Editor. This file can be opened in the RSoft CAD, and using the Group Library described in [Section 8.A.1](#), this material can be transferred to other `.ind` design files.

The `fitdisp` utility is primarily useful for FullWAVE users who want to use a Pulse simulation which requires the use of Lorentzian dispersion model. Users of other tools such as DiffractMOD, BeamPROP, and/or FemSIM, of FullWAVE users using CW simulations, can use the raw data directly in the Material Editor without having to use a curve fit.

Syntax:

`fitdisp` supports the following syntax and options:

```
usage: fitdisp [options] input-file
       (input-file is 'wavelength(um) n [k]' by default)
```

```
options:
  -?                help - prints this message
  -xtype#           x data type, 0=Frequency(1/um), 1=Wavelength(um) (default=1)
  -ytype#           y data type, 0=Eps, 1=Index (default=1)
  -xytype#          sets xtype and ytype together
  -o#               order, i.e. # of Lorentzians to be used (default=1)
  -epsinf#          epsilon at infinite frequency (default=1)
  -drude            force first term to be Drude (c=0)
  -x#, #            set x range to fit (default=all data)
  -v#               verbosity (default=3)
  -p#               plot results, 0=none 1=.pcs 2=.pcs+spawn (default=1)
  -out=$            prefix for output files (default="out")
```

Usage:

- `-?` This option prints the help for this utility.
- `-xtype#` This option sets the type of X data in the data file. A value of 0 indicates frequency data in μm^{-1} , and a value of 1 (the default) indicates wavelength data in μm .
- `-ytype#` This option sets the type of Y data in the data file. A value of 0 indicates epsilon data, and a value of 1 (the default) indicates index data.
- `-xytype#` This option provides a convenient way to set the X and Y data types together.
- `-o#` This option sets the number of Lorentzians to use for the curve-fit. By default, one Lorentzian is used.
- `-epsinf#` This option sets the epsilon value at infinite frequency and defaults to a value of 1.
- `-x#, #` This option sets the X data range to use for the fit. By default, all the actual data in the file is used.
- `-v` This sets the verbosity. A value of 3 is used by default.
- `-p` This option controls the output of a WinPLOT plot file. A value of 0 disables plot output, a value of 1 creates a WinPLOT plot file, and a value of 2 creates a WinPLOT plot file and opens it.
- `-out=$` This option sets the output prefix. Note that you must use an equal sign for this option, e.g. `-out=run3`.

Example:

To attempt a 4 Lorentzian curve-fit of the refractive index data contained in a file `index.dat`, (which has three columns, wavelength, real index, and imaginary index), use this command:

```
fitdisp -o4 -out=fit index.dat
```

To attempt a 4 Lorentzian curve-fit of the refractive index data contained in a file `index.dat`, (which has three columns, frequency, real epsilon, and imaginary epsilon), use this command:

```
fitdisp -o4 -xytype0 -out=fit index.dat
```

Another example of material data curve fitting can be found in Tutorial 5 in the FullWAVE manual.

fwmodekit – Extract Data From FullWAVE State File

fwmodekit allows the user to extract data from state files created via the **Continue Simulation** option. This utility is unsupported and its behavior is subject to change.

Syntax

fwmodekit supports the following syntax and options:

- fwmodekit (-h|--help)

Prints help message.

- fwmodekit --version

Prints version.

- fwmodekit (-s|--slice) (-x x0|-y y0|-z z0) --prefix <prefix> [--nobcs] (ex|ey|ez|hx|hy|hz|eden|hden|tden) <fstfile> <indfile>

For a 3D simulation, extracts and plots a 2D slice of field component to a plot file.

For a 2D simulation, extracts and plots the entire domain to a plot file.

- fwmodekit (-f|--fslice) (-x x0|-y y0|-z z0) --prefix <prefix> [--nobcs] [--lightcone <lightcone k-radius>] (ex|ey|ez|hx|hy|hz|eden|hden|tden) <fstfile> <indfile>

Extracts and plots fourier transform of slice of field component to a plot file.

- fwmodekit (--view3d) --prefix <prefix> (ex|ey|ez|hx|hy|hz|eden|hden|tden) <fstfile> <indfile>

For a 3D simulation, extracts and plots a 3D view of the field components. No symmetry boundary processing is applied.

- fwmodekit (--fileslice) --prefix <prefix> (-x x0|-y y0|-z z0) <slicefile> <indfile>

Takes an existing slice file on disk and uses the boundary conditions to extend the file to the full domain. The slice direction option (-x, -y or -z) is only required for slices taken from 3D calculations and the value (x0, y0, z0) is not important and can be set to 0.

- fwmodekit (-c|--compare) <file1>.fst <file2>.fst

Compares fields in two fst files and writes their fractional difference to standard output.

- fwmodekit (-a|--analyze) <file1>.fst

Summarizes properties of .fst file.

- fwmodekit (-v|--volume) --prefix <prefix> [--nu nu0] <fstfile> <indfile>

Finds the mode volume of the mode stored in the .fst file. The mode volume is defined as

$$V = \frac{\int U_E(\mathbf{x}) d\mathbf{x}^3}{\max[U_E(\mathbf{x})]}$$

where $U_E(\mathbf{x})$ is the electric energy density. The resonant frequency can be specified. This helps to determine the required simulation length and is recommended though not required. When `fwmodekit` detects a symmetry-based subdomain calculation, the volume is multiplied by 2 for each direction in which symmetry conditions have been applied. This can be suppressed using the option `--nobcs`.

Options:

<code>-q</code>	This option enables ‘Quiet mode’ which suppresses display of the generated file in a plot window.
<code>-g</code>	Run FullWAVE calls in a GUI window.
<code>--nobcs</code>	Suppress use of boundary conditions to unwrap field profiles on subdomains.
<code>--np <int></code>	Number of machines required for cluster calculation.
<code>--machinefile <machfile></code>	File of machine names required for cluster calculation.
<code>--timesteps <nsteps></code>	Number of timesteps before output (default=1).
<code>--nu</code>	Resonant frequency of mode under analysis.

ind2gds, ind2dxf – Convert .ind File to Mask

See [Mask Conversion Utilities](#) section.

gds2ind – Convert GDS Mask to .ind File

See [Mask Conversion Utilities](#) section.

killall – Utility to stop processes

The `killall` utility can be used to stop processes with a particular name.

WARNING: This utility can be used to terminate any running process, not just Synopsys processes. Use at your own risk.

Note that this utility is only included on Windows operating systems. Linux systems typically have a similar utility. Note that the Linux syntax may be different what is documented here, please check the `killall` man page on your system.

Syntax

`fwmodekit` supports the following syntax and options:

```
killall <process_name>
```

where `<process_name>` is the name of the process.

Example:

To stop all RSoft CAD processes on your system, use this command:

```
killall bcadw32
```

Mask Conversion Utilities

The RSoft tools include various utilities to convert to and from common mask formats:

GDS Utilities

RSoft `.ind` files can be converted to and from GDS masks via the `gds2ind` and `ind2gds` utilities.

GDS to .ind File: gds2ind

This utility creates a `.ind` file from a GDS mask file. Note that these options can also be used when loading a GDS file via a circuit reference as described in [Section 6.F.2](#).

Syntax:

`gds2ind` supports the following syntax and options:

```
usage: gds2ind [options] gds-input-file-name [ind-output-file-name]
options:
-?          help - prints this message
-i#         show structure info only (#=hierarchy-level, default=1)
-n$         set structure name to process (default=last)
-l$         set layer name to process (default=all)
-h#         set default layer thickness (default=1)
-xy         import to XY plane
```

Options:

<code>-i#</code>	Display structure information, # sets hierarchy level.
<code>-n#</code>	Sets the structure or cell name to process.
<code>-l\$</code>	Select the layer to process.
<code>-h#</code>	Sets the default layer thickness (can be changed in <code>.ind</code> file later).
<code>-xy</code>	Import to the XY plane instead of the XZ plane.
<code>-pv, -pw, -ps</code>	Enables the automatic placement of vertices at 'port' locations. See Section 6.F.8 for more details.

Ind file to GDS: ind2gds

This utility creates a `.gds` file from an RSoft design file (`.ind`). Note that this utility is the same used by the GDS export option described in [Section 6.E.1](#).

Syntax:

`ind2gds` supports the following syntax and options:

```
usage: ind2gds [options] ind-file-name [name=value ...]
options:
-?          help - prints this message
-o=$       set output file (default=<infile>.gds)
-r#        set resolution (CAD preferences set default)
-l#        map layer number 0 to #
-d#        map data type 0 to #
```

Options:

<code>-o=\$</code>	Sets the output file, by default the input file name will be used.
<code>-r#</code>	Sets the output resolution, defaults to the CAD preference.
<code>-l\$</code>	Maps layer number 0 to #.
<code>-d#</code>	Maps data type to #

Notes:

Many options can be set via symbols (name = value), including:

Symbol	Description
<code>gds_export_dir</code>	Sets the cross-section direction to export. A value of 1 = X (YZ plane), 2 = Y (XZ plane), and 3 = Z (XY plane).
<code>gds_export_pos</code>	Sets the export cross-section position.
<code>gds_clipping</code>	Enables clipping the structure.
<code>gds_clip_xmin</code> <code>gds_clip_xmax</code> <code>gds_clip_ymin</code> <code>gds_clip_ymax</code> <code>gds_clip_zxin</code> <code>gds_clip_zmax</code>	Sets the clipping ranges. If unset or set to default, the whole structure will output.

DXF Utilities

RSoft `.ind` files can be converted to and from DXF masks via the `dx2ind` and `ind2dx2` utilities. Use the `-?` option to view the options supported by each utility.

mat2bp – Convert Matrix Data to RSoft Format

This program converts raw matrix data into the RSoft file format.

Syntax

mat2bp supports the following syntax and options:

```
usage: mat2bp input-file-name output-file-name
options:
  -?                help - prints this message
  -x<range>         set range in x (<range> = #,#)
  -y<range>         set range in y (<range> = #,#)
```

Usage

mat2bp converts a raw matrix of data contained in a data file into a file in the RSoft format which could then be used as an index profile, input field, etc. Since the RSoft file format contains information which relates the domain of the data, the user needs to provide this information in order to convert the matrix into the RSoft format. The `mat2bp` utility supports these options:

- | | |
|--------|--|
| -x#, # | This option sets the X domain for the data file. The range should be input as #, #, where the first number is the domain min, and the second number is the domain max. These numbers will be used to build the header for the new RSoft Data file that will be output. |
| -y#, # | This option sets the y domain for the data file. The range should be input as #, #, where the first number is the domain min, and the second number is the domain max. These numbers will be used to build the header for the new RSoft Data file that will be output. |

The utility will automatically add the appropriate header lines to the file.

e.g. to convert the file `data.dat` into a RSoft file `field.fld`, where the input file is defined over an X range of (-10,10) and a Y range of (-1,6):

```
mat2bp -x-10,10 -y-1,6 data.dat field.fld
```

mathmat – Do Arithmetic with Data Files

This program accepts multiple input files and combines the columns using mathematical expressions specified by the user. Any expression that would be valid in the CAD symbol table may be used. A circuit file may be specified to take advantage of existing symbols and new symbols may be defined in the command itself.

`mathmat` is often useful with BandSOLVE band structure output, since BandSOLVE results have the wavevector as the independent variable when often we wish to express results as a function of the frequency. Tutorial 6 in the BandSOLVE manual contains demonstrations of this technique.

Syntax

`mathmat` supports the following syntax and options:

usage: `mathmat <symbol_table_expr> [input_files]`

options:

<code>-help</code>	[show this message]
<code>-longhelp</code>	[show full help]
<code>-p <int></code>	[output precision (default=10)]
<code>-f</code>	[use fixed point format]
<code>-nohead</code>	[disable header]
<code>-trunc</code>	[truncate at shortest file]
<code>-bp <string></code>	[use symbol table from RSoft .ind file]
<code>-sym <string></code>	[define new symbols]
<code>-maxy</code>	[in all columns (except 0), set output_val to '.' if <code>abs(output_val) > maxy</code>]

Usage:

`mathmat` reads from standard input or a list of input files, performs operations on the columns defined by a list of symbol table expressions and writes the answer to standard output. The operations are performed on each row in the column separately. The columns in the first input file are named 'a0', 'a1', 'a2',... The columns in the second input file are named 'b0', 'b1', 'b2',... and so on. As a quick example, this command will take the average of columns 1 and 2 in a file and write it as a function of column 0:

```
mathmat 'a0, (a1+a2)/2' in.dat
```

It is also possible to create commands that perform operations on entire files that have the same shape. For example, this command will take the average of two files and write the results:

```
mathmat '(__a+__b)/2' in1.dat in2.dat
```

`mathmat` write output to standard output (usually the terminal or command prompt). To save data in a new file, use the redirection operator '`>`'. For example, the simple command above to take the average of two columns can be saved in a file `out.dat` with this command:

```
mathmat 'a0, (a1+a2)/2' in.dat > out.dat
```

Referencing a column which does not exist or using more letter prefixes than there are input files causes a fatal error.

These names are used in the symbol table expression which must be a comma separated list of normal math expressions using these variable names.

Options:

<code>-help</code> <code>-longhelp</code>	These options display help for this utility.
<code>-p <int></code>	This option sets the output precision used. The default is 10 digits.
<code>-f</code>	This option enables the use of a fixed point format.
<code>-nohead</code>	This option disables the use of <code>mathmat</code> headers.
<code>-trunc</code>	This option truncates the output data at the shortest file.
<code>-bp <string></code>	This option enables the use of symbols from an existing <code>.ind</code> file.
<code>-sym <string></code>	This option allows the definition of new symbols.

Examples:

eg. Take the average of columns 1 and 2, and write as a function of column 0:

```
mathmat 'a0, (a1+a2)/2' in.dat
```

eg. Take the average of columns 1 in two files, and write as a function of column 0 of first file:

```
mathmat 'a0, (a1+b1)/2' in1.dat in2.dat
```

eg. Calculate a complicated expression:

```
mathmat 'a0,sqrt(a1/pi)*(a2*3+b1*exp(b2))' in1.dat in2.dat
```

eg. Use symbols Period and delta defined in a file myhex.ind:

```
mathmat -bp myhex.ind 'a0*Period, delta*a1' in1.dat
```

eg. Define and use symbol scale:

```
mathmat 'a0*scale' -sym'scale=3.1' in1.dat
```

parutil – Generate nk data files from TCAD Sentaurus parameter files

The `parutil` utility performs operations using the TCAD Sentaurus parameter files and/or the complex refractive index (CRI) library from Sentaurus. It extracts wavelength dependent CRI information into a n/k data file which can be used in the RSoft tools.

Syntax

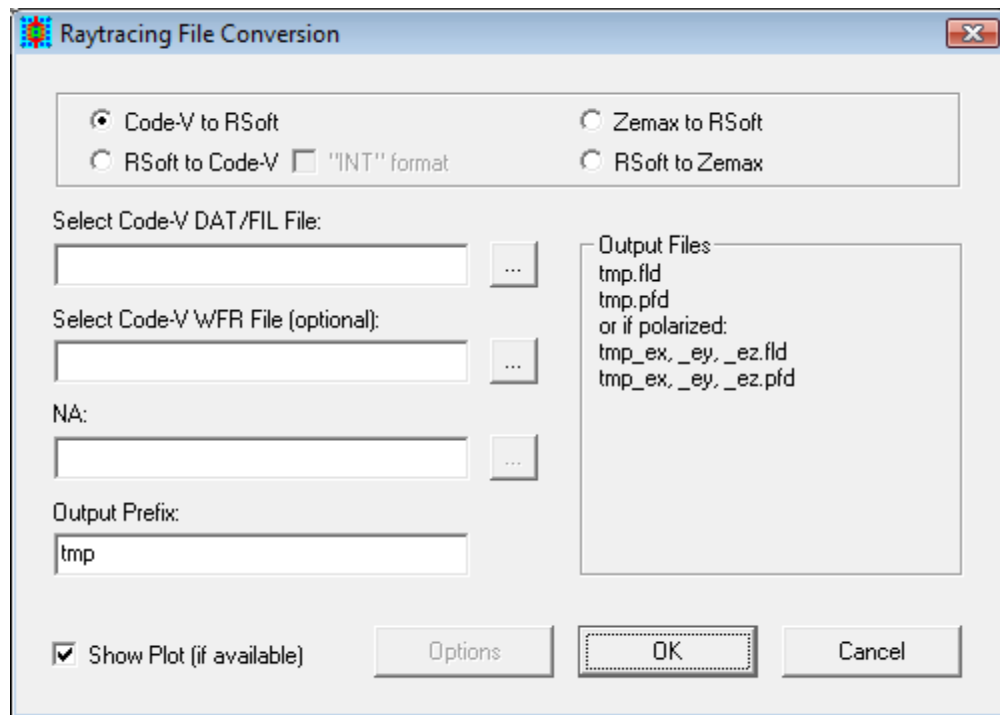
`parutil` supports the following syntax and options:

```
usage: parutil [options] <material> [<out-file>]
options:
  -h                help - prints this message
  -prefix=$         output prefix (<out-file>=<prefix>_nk.dat)
  -parfile=$        TCAD parameter file
  -swbnode#         TCAD node number for preprocessing parfile
  -checkmat         checks if material was found
```

`parutil` takes as input the material name, `<material>`, and creates a corresponding nk data file containing wavelength dependent CRI information. The CRI information is either obtained from the explicitly defined parameter file, `-parfile`, or from the material definition in the Sentaurus database. If the material definition is found in the explicitly specified `parfile` it takes precedence over the default definition in the Sentaurus database. If the `parfile` has preprocessing or `#include` directives a valid Sentaurus Workbench node number, associated with the RSoft tool, needs to be specified via the `-swbnode` option to preprocess the `parfile`. The name of the output file, `<out-file>`, to be generated can either be explicitly specified as a command line argument after `<material>`, or the output prefix to be used can be specified via the `-prefix` option. If neither option is specified, then the name of the output file would be `<matname>_nk.dat`. If the `-checkmat` option is selected it checks if the material is found either in the explicitly specified `parfile` or in Sentaurus Device CRI library and prints it to standard output; no n/k file gets created.

Ray-Tracing Conversion Utility

RSoft provides a utility to convert between popular Ray-Tracing file formats, including Synopsys' CODE V, and RSoft's field files. A description of the usage of this utility can be found in Tutorial 9 of the BeamPROP manual.



The Ray-Tracing Conversion Utility uses the `bp2codev`, `codev2bp`, and `bdconv` utilities to perform the conversion. These utilities are documented in [Appendix E](#). When using this utility, note that some options such as the wavelength, index, etc. can be set via the **Options** button. These options correspond to the same options that the command line utilities use.

rsanimate – Create Animations from Plot Files

This program is designed to create animations from a series of (up to 1000) WinPLOT plot files.

Animations can be created using the utility described in this section or using the Export Animation DataBROWSER feature described in Chapter 2 of the DataBROWSER manual.

Syntax:

`rsanimate` supports the following syntax and options:

```
usage: ranimate [options] <file-list>
options:
  -?                help - prints this message
  -o <file>        set output file name (must include .gif or .mpeg
                    extension)
  -p <opts>        set additional plot options (could use @ to include
                    a script)
```

Options:

- | | |
|----|--|
| -? | This option display help for this utility. |
| -o | Sets the output file name. GIF and MPEG files are supported; the names must include the .gif or .mpeg extension. |
| -p | Add additional WinPLOT plot options to each plot. |

Example:

To create an animation of the WinPLOT files `plot1.pcs`, `plot2.pcs`, and `plot3.pcs` and save it as `animation.gif`, use this command:

```
rsanimate -o animation.gif plot1.pcs plot2.pcs plot3.pcs
```

To create the same animation, but using the `/zw` WinPLOT command to automatically scale the color scale to the data, use the command:

```
rsanimate -o animation.gif -p /zw plot1.pcs plot2.pcs plot3.pcs
```

rscombine – Combine Frequency Results

rscombine is a utility which combines absorption monitor profiles obtained at multiple frequencies into a single absorbed photon density for use in an electronic simulator like Sentaurus Device.

It supports FullWAVE FDTD pulsed output, FullWAVE FDTD CW scan output, and DiffractMOD RCWA wavelength scan output, using the monitor information in the .ind file to get the names of the files. It can also apply weights through the -srcspecfile option and has an -apd option to produce absorbed photon density (see discussion for apd calculation in tdrutil in [Section E.S](#)). See full list of options below.

Syntax:

rscombine supports the following syntax and options:

```
usage: rscombine [options] <output-file-prefix>
general options:
-?                help - prints this message
-prefix=$         prefix used for data generation
-indfile=$        indfile used for data generation (default='prefix'.ind)
-srcspecfile=$    source spectrum filefrequency only) (default=whitelight)

extra options for apd usage for absorption monitors:
-apd              convert absorption to absorbed photon density before
combining
-int#             set plane wave intensity (W/cm^2, default=1)
-pow#             set guided wave power (W)
-outvar=$         name of output dataset (default=AbsorbedPhotonDensity)
-outunits=$       set units of output dataset (default=cm^-3*s^-1)

extra options for CW scan usage:
-spec#            specification 0-frequency, 1-wavelength (default=0)
-min#             minimum frequency/wavelength depending on spec
-max#             maximum frequency/wavelength depending on spec
-num#            # of frequency/wavelength points
```

Example:

To combine frequency simulation results calculated by running file file.ind with an **Output Prefix** of run4, calculating the Absorbed Photon Density, an input intensity of 4, and over a wavelength range of 0.3 to 2.2 μm , saving results with a prefix new_results, use this command:

```
rscombine -prefix=run4 -indfile=file.ind -apd -spec1 -int4 -min0.3
-max2.2 new_results
```

rspython – Python Environment

The `rspython` utility is a built-in Python interpreter capable of running Python scripts.

See [Appendix H](#) for details on supported Python versions and usage scenarios.

This utility can be run standalone and is also automatically used when processing Circuit References that contain `.py` files or running MOST User Simulators. See [Appendix H](#) for details on supported Python versions.

Syntax:

`rspython` supports the following syntax and options:

usage: `rspython <python-file>`

Note, running without a `<python-file>` will open an interactive Python environment.

shufflemat – Rearrange Data Files

This program is designed for reordering or extracting the columns in one or more data files. An online manual page can be obtained by typing “shufflemat - help” or “shufflemat - longhelp” at a command prompt.

Syntax:

shufflemat supports the following syntax and options:

```
usage: shufflemat [options] [output_files]
```

options:

-help	[show this message]
-longhelp	[show full help]
-t	[transpose input data]
-ec <string>	[extract named columns]
-pc <string>	[permute named columns]
-er <string>	[extract named rows]
-pr <string>	[permute named rows]
-elt <string>	[extract named element]
-p <int>	[output precision (default=10)]
-f	[use fixed point format]
-h	[keep column titles]
-numsym <string>	[treat this string in data files as a null value]
-nullval <float>	[set internal value for null symbols (default 0)]

shufflemat reads from standard input, rearranges or extracts the columns or rows and writes to specified output files, or to standard output if no output files are named. The columns and rows in the input file are denoted by integers starting from 0. As a quick example, this command will write columns 1, 2, and 3 of the file `in.dat` to standard output:

```
shufflemat -ec 0,2,3 < in.dat
```

shufflemat write output to standard output (usually the terminal or command prompt). To save data in a new file, use the redirection operator ‘>’. For example, the simple command above can be saved in a file `out.dat` with this command:

```
shufflemat -ec 0,2,3 < in.dat > out.dat
```

Options:

-help	This option displays the short help for this utility.
-longhelp	This option displays the long help for this utility.

<code>-t</code>	The <code>-t</code> (transpose) option takes the whole matrix and writes the transpose matrix to output. Any header lines in the input are ignored. Only one output file is allowed.
<code>-ec <string></code> <code>-er <string></code>	The ‘extract’ options take a string argument specifying which columns (<code>-ec</code>) or rows (<code>-er</code>) should be output. All other columns/rows are dropped. The argument consists of a comma separated list of column numbers. Ranges may be specified using a hyphen (-) and multiple lists may be separated by a semicolon (;) Each range is output to a different filename specified on the command line. There can be no white space in the range specifications. If semicolons are used, the extraction string must be protected from the shell by quotes (double-quotes for DOS/Windows).
<code>-pc <string></code> <code>-pr <string></code>	The ‘permute’ options write all input to output but performs permutations or cyclic rotations of specified columns (<code>-pc</code>) or rows (<code>-er</code>). Permutations are expressed as a space-separated list of integers inside parentheses. Several permutations can be expressed together, but may have no common column numbers. Permutations lists separated by semicolons will be directed to different output files. The permutation string must be always protected from the shell by quotes (double-quotes for DOS/Windows).
<code>-elt <string></code>	This option extracts a specific element from a file. The string argument should be the row/column of the element.
<code>-p <int></code>	This option sets the output precision used. The default is 10 digits.
<code>-f</code>	This option enables the use of a fixed point format.
<code>-h</code>	This option keeps the column titles.
<code>-numsym</code>	Sets the string used in the data file(s) for null values.

`-nullvall`

Sets the value used for null values in data files (default 0).

Examples:

eg. Place the transpose of the file in.dat in the file out.dat.

```
shufflemat -t < in.dat > out.dat
```

eg. Write columns 0, 2 and 3 of in.dat to standard output:

```
shufflemat -ec 0,2,3 < in.dat
```

eg. Write columns 1,2,3 and 5 of in.dat to standard output:

```
shufflemat -ec 1-3,5 < in.dat
```

eg. Write columns 1,2,3 to out1.dat and columns 4 and 6 to out2.dat:

```
shufflemat -ec "1-3;4,6" out1.dat out2.dat < in.dat
```

eg. Swap columns 2 and 3, writing to standard output:

```
shufflemat -pc "(2 3)" <ein.dat
```

eg. Rotate columns 2, 3 and 6 writing to standard output:

```
shufflemat -pc "(2 3 6)" <ein.dat
```

eg. Rotate columns 2, 3 and 6, and columns 4 and 5, writing to standard output:

```
shufflemat -pc "(2 3 6) (4 5)" <ein.dat
```

eg. Rotate columns 2, 3 and 6 in one output file and columns 4 and 5 in another file:

```
shufflemat -pc "(2 3 6); (4 5)" <ein.dat out1.dat out2.dat
```

eg. Extract element 2,3 from one file, writing to standard output:

```
shufflemat -elt2,3 <ein.dat
```

smatgen - Custom PDK Utility

See [Custom PDK Utility](#) section.

tailmon – Extract Final Value Utility

The `tailmon` utility extracts the final value from a text output file, useful when combining the result of multiple simulations. It is analogous to the Linux `tail` command.

Syntax:

`tailmon` supports the following syntax and options:

usage: `tailmon [options] <monfile`

options:

`-v<first-value>`

Options:

`-v#` Replace the value in the first column with #

Notes:

This utility uses typical redirection inputs/outputs, and takes input via the `<` operator, and will write output to the screen or a file via the `>` operator (write new file) or `>>` operator (append to file).

tdrutil – TDR File Utility

The TDR Utility (`tdrutil`) performs several useful operations using Sentaurus TDR files. The syntax for this utility is described in this section along with each of the 5 use scenarios and appropriate options.

Syntax:

`tdrutil` supports the following syntax and options:

```
usage: 1. Display Information:
        tdrutil [options] <infile.tdr>
      2. Extract/Convert Datasets to a Tensor Mesh:
        tdrutil [options] <infile.tdr>[:<dataset>] <outfile.tdr>
      3. Extract Datasets to a Compatible Mesh:
        tdrutil [options] <infile.tdr> <compatfile.tdr> <outfile.tdr>
      4. Generate Absorbed Photon Density from RSoft Monitor:
        tdrutil -apd [options] <infile.tdr> <outfile.tdr>
      5. Generate an RSoft CAD file with dynamic reference to a TDR file:
        tdrutil [options] <infile.tdr> <outfile.ind>
```

general options:

```
-?          help - prints this message
-info#      set information level (from 1 to 5, default=2)
```

options for usage 2:

```
[:<dataset>] set name of dataset to extract (default=first applicable)
-var=$       set name of dataset to extract (default=first applicable)
-getdnk      extract index perturbation (if present)
-sc#         scale dataset by argument (default=1)
-tgf#        set tensor grid factor (all dirs)
-tgs#        set tensor grid size (all dirs) (um)
-tg[x|y|z]#  set tensor grid size (indicated dir) (um)
```

options for usage 4:

```
-lam#        set wavelength (um)
-int#        set plane wave intensity (W/cm^2, default=1)
-pow#        set guided wave power (W)
-outvar=$    set name of output dataset (default=AbsorbedPhotonDensity)
-outunits=$  set units of output dataset (default=cm^-3s^-1)
```

Options:

The options for `tdrutil` are broken up into the 5 use scenarios. Each use is described here along with an example.

General Options:

- | | |
|---------------------|--|
| <code>-?</code> | This option display help for this utility. |
| <code>-info#</code> | Sets the information level used in utility log. Values are from 1 to 5, the highest level of detail. The default is 2. |

Usage 1 - Display TDR Information:

This command will display information about the TDR file, including details on the datasets in the file, statistics on the regions in the file, etc. The `-info#` option can be used to set the level of detail output. The syntax for this usage is:

```
tdrutil [options] <infile.tdr>
```

For example, the command:

```
tdrutil myTDRFile.tdr
```

displays information about the data in the TDR file `myTDRFile.tdr`.

Usage 2 - Extract/Convert Datasets to a Tensor Mesh:

This usage will extract a dataset from one TDR file and save it as a new TDR file. Several options are given to scale and/or resample the data. The syntax for this usage and associated options are:

```
tdrutil [options] <infile.tdr>[:<dataset>] <outfile.tdr>
```

- | | |
|---------------------------------|--|
| <code>[:<dataset>]</code> | Sets the name of the dataset to extract. If no name is specified, the first applicable set will be extracted. The <code>-var=*</code> option will extract all datasets. |
| <code>-var=\$</code> | |
| <code>-getdnk</code> | Extracts the n/k index perturbation from the carrier densities in the input TDR file if present. The model used for the extraction is documented in Section 8.I . Note that the options <code>-lambda</code> , <code>-fca</code> , and <code>-fca_file</code> can be used to set the wavelength, <code>fca_method</code> , and <code>fca_file</code> respectively. See Section 8.I for complete details. |
| <code>-sc#</code> | Scales the dataset by a value. |
| <code>-tgf#</code> | These options set the output tensor grid resolution: |
| <code>-tgs#</code> | |
| <code>-tg[x y z]</code> | |
| | |
- The `-tgf#` option sets the new tensor grid relative to the original grid by a factor for all directions.
 - The `-tgs#` option sets the new tensor grid in μm for all directions.
 - The `-tgx#`, `-tgy#`, and `-tgz#` options set the new tensor grid

in μm for the specified direction.

For example, the command:

```
tdrutil -tfs0.01 myTDRFile.tdr:absorption newTDRFile.tdr
```

will extract the ‘absorption’ dataset from the file `myTDRFile.tdr`, set the tensor grid to $0.01\ \mu\text{m}$ for all coordinates, and save the results in the new TDR file `newTDRFile.tdr`.

Usage 3 - Extract Datasets to a Compatible Mesh:

This usage will extract a dataset from one TDR file on the mesh contained in another TDR file and save the result as a new TDR file. The syntax for this usage is:

```
tdrutil [options] <infile.tdr> <compatfile.tdr> <outfile.tdr>
```

For example, the command:

```
tdrutil myTDRFile.tdr mesh.tdr newTDRFile.tdr
```

will extract the dataset from the file `myTDRFile.tdr`, put it on the mesh contained in the file `mesh.tdr`, and save it as the new TDR file `newTDRFile.tdr`.

Usage 4 - Generate Absorbed Photon Density from RSoft Monitor:

This option generates the Absorbed Photon Density (APD) from an RSoft monitor and saves it as a new TDR file. The APD is in units of $1/(\text{cm}^3 \cdot \text{s})$ defined as:

$$APD = \left(\frac{P_{in}}{hc/\lambda} \right) \times Abs \times 10^{12}$$

Where Abs is the absorption calculated by the simulator, and P_{in} is the input power in Watts. Note that the **Launch Power** should be set to 1 and **Launch Normalization** should be set to *Unit Power*.

The syntax for this usage and associated options are:

```
tdrutil -apd [options] <infile.tdr> <outfile.tdr>
```

<code>-lam#</code>	Sets the wavelength in μm .
<code>-int#</code>	Sets the plane wave intensity in W/cm^2 . The default is 1.
<code>-pow#</code>	Sets the guided wave power W .
<code>-outvar=\$</code>	Sets the name of the output dataset. The default is <code>AbsorbedPhotonDensity</code> .

For example, the command:

```
tdrutil -lam0.6 RSoftAbsMon.tdr newTDRFile.tdr
```

will compute the absorbed photon density from the absorption profile in the file `RSoftAbsMon.tdr` at a wavelength of $0.6\ \mu\text{m}$ and save it in the file `newTDRFile.tdr`.

Usage 5 - Generate an RSoft CAD file with Dynamic Reference to a TDR File:

Note that this process is automated when adding an RSoft step to an existing SWB project. Simply Edit the RSoft CAD file for the RSoft step and you will be prompted to select a boundary TDR file and the TDR Utility will create the RSoft CAD `.ind` file.

This usage performs the bootstrap process required to generate an RSoft `.ind` design file from a boundary TDR file. This process is only required once when adding an RSoft simulator to a Sentauros Workbench workflow. The geometry of the structure will be automatically and dynamically read from the TDR file. Note that it may be necessary to add additional CAD objects to the design for measurement/output, etc. See the TCAD Tutorials in the MultiPhysics Utility and TCAD Sentauros Interface manual for examples of this usage.

The syntax for this usage is:

```
tdrutil [options] <infile.tdr> <outfile.ind>
```

For example, this command:

```
tdrutil myTDRFile_bnd.tdr myRSoftDesign.ind
```

will create the file `myRSoftDesign.ind` that is dynamically linked to the boundary TDR file `myTDRFile_bnd.tdr`.

wgmode – Waveguide Mode Utility

This program calculates modal effective indices for slab waveguides, fibers, and diffused structures. It is the command line analog of the `slab_neff()` symbol table function described in [Section C.F.8](#).

This is an interactive program; you must answer the question at each step. The defaults for each question are shown in square brackets [], hit enter to use the default. Once all questions are answered, the program will output the computed modal properties. The program will automatically run again except the default values will be set to the most previous answers. This allows the properties of similar cases to be easily iterated through. To stop running the utility, type CTRL-C.

Example:

For example, the properties of the fundamental mode of a typical single mode fiber at $\lambda = 1.55$ μm can be found by running `wgmode` and answering the questions with 2, 0, 1, 1.55, 1.44, 1.45, 8.2 as shown here (the answers used for this example are highlighted below in **bold**):

```
Waveguide Structure [1=Slab,2=Fiber,3=Diffused] [ 1 ] : 2
Azimuthal Mode [m=0,...] [ 0 ] : 0
Radial Mode [n=1,...] [ 1 ] : 1
Wavelength [microns] [ 1.000000 ] : 1.55
Index of Clad (nclad) [ 1.000000 ] : 1.44
Index of Core (ncore) [ 1.010000 ] : 1.45
Diameter (d) [ 1.000000 ] : 8.2

The waveguide parameters for mode LP(0,1) are:
normalized frequency (V) : 2.8254066
normalized mode index (b) : 0.62111807
mode effective index (Neff) : 1.4462193
longitudinal propagation constant (Beta) : 5.8624929
transverse propagation constant (Kappa) : 0.42417888
transverse decay constant (Gamma) : 0.54310547 (1/G=1.841)
```

The output shown here shows, for example, that the computed mode effective index (Neff) is 1.4462193.

Appendix F

Color Scales

Color scale files provide a means to customize data presentation using RSoft products. All of the simulation products that use the RSoft CAD are capable of producing large amounts of data. Choosing which data to present and how to present it according to needs and taste is an important issue. Scale files are located in the directory `<rsoft_dir>\RLOTDIR`. Several default scale files are included, as well as several example files. You can also create and add your own custom files.

F.A. What are Scale Files?

A scale file is a color lookup table which is used to graphically display 2D data sets. It has three main components: a data range, a color lookup table, and a color shades value.

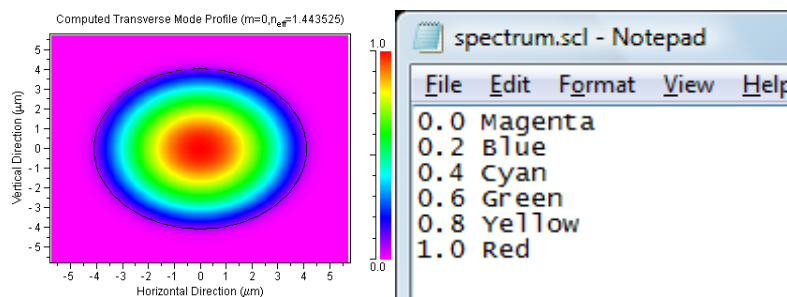


Figure 1: A simple fiber mode profile as seen in WinPLOT, along with the default scale file used by BeamPROP to display field profiles.

- *Data Range*

Each color scale is defined for a specific range. For example, most default color scales in BeamPROP have a range of [0,1], and FullWAVE a range of [-1,1]. The scale file used in Figure 1 is defined over [0,1]. The scale is set by the highest and lowest values in the first column; the rest of the numbers serve as documentation of the value range of each color.

A numerical value which lies outside the defined range of a scale file will be plotted with a default color. For values which are greater than the defined range, White is used, and for values which are less than the defined range, Black is used. The `/zw` WinPLOT command can be used to automatically scale a color scale to the data limits; see the WinPLOT manual for details.

- *Color Lookup Table*

A color scale is defined by a series of color steps. The color scale shown in Figure 1 is defined by 6 colors, and assigns the color Magenta to the range 0.0-0.2, Blue to the range 0.2-0.4, and Red to the value 1.0. The maximum number of color steps is 101.

- *Color Shades*

Upon close examination of the colors shown in Fig. 1, it may seem that they do not agree with the definition of the Color Lookup Table above, but, in fact, the program is blending the colors together. This effect is achieved through the use of **Color Shades**. This setting is not contained in the scale file, but can be set for each plot individually.

The **Color Shades** number sets the number of interpolated points between each defined color. For example, setting this option equal to 4 results in 4 color points interpolated between each defined color point. This option can be very useful for reducing display related CPU usage during simulations, especially for slower systems. Fig. 2 shows the effect of changing the color scale number.

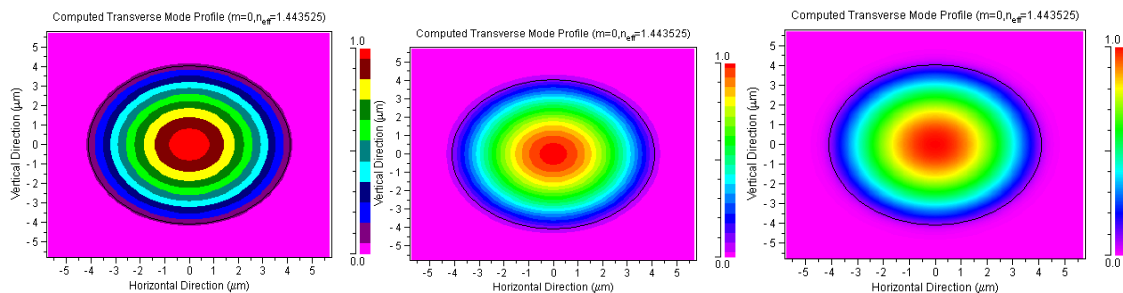


Figure 2: Effect of the color shade number. a) Color shades = 1, b) 4, and c) 32.

F.B. Using Color Scales

There are three ways to change the scale file:

F.B.1. Setting the scale file during a simulation

You can change the scale file used to render contour plots shown in a simulation window by opening the Simulation Parameters dialog, clicking the **Display...** button, clicking the **Scale File...** button, and then choosing the desired scale file. To set the number of color shades used, use the **Color Shades** field in the Display Properties box.

F.B.2. Changing the scale file after a simulation has completed

You can change the scale file used to display computed results in a completed simulation window by choosing View/Display/Options/Color Scale from the pull-down menu, and then selecting the desired scale file. You cannot change the number of color shades used after a simulation has completed.

F.B.3. Changing the scale file in a WinPLOT window.

To change the scale file used to display data in a WinPLOT window, first open a saved plot. This can be done via the **View Graphs** icon in the CAD interface or through your computers OS (Windows, Linux, etc). Once the plot is opened, select Options/3D Data Display/Options/Color Scale from the pull-down menu, and choose the desired .scl file. To make this change permanent, you should save the file after making the change.

You can change the number of color shades used by selecting Options/3D Data Display/Options/Color Shades.

Note: For advanced users, you can make these changes directly in the Edit Window. Use the `‘/cscl“FILE”’` command to set the color scale, and the `‘/cnum#’` command to set the color shades. If you want more information about WinPLOT commands, please consult the WinPLOT manual.

F.C. Scale File Format

Several color scale files are included in the RSoft distribution. The user can utilize these files for data presentation, or create customized scale files. Scale files are text based files which can be created/modified in any text editor, and should be saved in directory `<rsoft_dir>\RPLOTDIR`.

A scale file consists of several rows of number/color combinations. The number of rows determines the number of color steps in the scale. The scale file format is:

```
Min Value    Color
.
.
Max Value    Color
```

The first column contains the range information, and the second column contains the color lookup information. As stated before, the range information is only specified by the first and last

line, or min and max value. The other numbers only serve to document the color ranges. There are two ways to specify the color information in a scale file:

F.C.1. Using Predefined Color Keywords

You can use one of several predefined keywords to specify a color in a scale file. The color keywords currently recognized are shown in Fig. 3.

Color	Keyword	RGB Values
	White	255 255 255
	Yellow	255 255 000
	LtMagenta	255 000 255
	LtRed	255 000 000
	LtCyan	000 255 255
	LtGreen	000 255 000
	Blue	000 000 255
	DkGray	128 128 128
	LtGray	192 192 192
	Brown	128 064 000
	Magenta	128 000 128
	Red	128 000 000
	Cyan	000 128 128
	Green	000 128 000
	DkBlue	000 000 128
	Black	000 000 000

Figure 3: The predefined color keywords currently recognized with their corresponding RGB values.

An example of this type of color scale is the BeamPROP default scale shown in Fig. 1.

F.C.2. Using RGB Values

If a color you wish to use is not defined by a color keyword, you can directly input RGB values for the desired color. The values accepted range from 0 to 255. The first column is the R value, the second the G value, and the third the B value.

Example: The scale file `fireice.scl`, which is included in the distribution, is defined via RGB values.

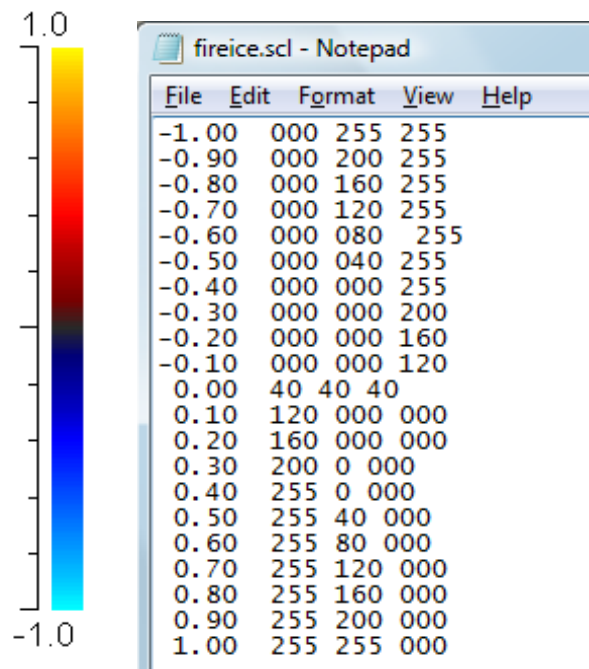


Figure 4: The color scale file fireice.scl which is defined through the use of RGB values.
This file can be found in the directory <rsoft_dir>/RPLOTDIR.

Appendix G

Polarization Conventions

While the concept of polarization gives rise to many of the most interesting effects in electromagnetism, the labeling of polarizations is a common cause of confusion since there is no universal agreement across different branches of photonics. This section describes the definitions used by RSoft software.

G.A. Background

First, some background information.

1D Structures:

In 1D, that is for structures in which the refractive index varies along only one dimension, Maxwell's equations are always separable into two sets of three equations. In one set of equations, the electric field is oriented along one of the invariant directions (i.e. the direction along which the refractive index does not vary); in the other the magnetic field lies along the invariant direction.

The simplest example of such a structure is the slab waveguide in which the field propagates along the z direction say, and the refractive index $n(x)$ is a function of only x . For this case, it is universally agreed that TE polarization refers to the electric field lying along the invariant direction y , and TM polarization refers to the magnetic field lying along y . Note that we could

equivalently refer to these states as *y-polarization* and *x-polarization* respectively, where we label the states by the direction in which the electric field points.

Such problems are separable in all RSoft tools.

2D Structures:

In 2D structures, i.e. structures in which the refractive index varies as a function of two dimensions, Maxwell's equations are again separable, *but only if the wave vector of the propagating solution lies in the plane of variation*. This means that for RSoft tools that model propagation along a waveguide/fiber (BeamPROP, DiffractMOD, ModePROP, and FemSIM) the equations are in fact not strictly separable. For the tools that permit propagation in the plane (FullWAVE and BandSOLVE) the equations are separable. The literature conventionally uses the labels TE to refer to states in which the electric field lies within the plane of variation, and TM to refer to states in which the electric field lies perpendicular to the plane of variation. As we see below, however, in the RSoft CAD these notations are reversed for consistency.

3D Structures:

In 3D structures, i.e. structures in which the refractive index varies in all three dimensions, Maxwell's equations are completely coupled and as a result, there is never a separation into TE and TM states.

G.B. The RSoft Convention

Optical scientists and engineers apply the labels 'TE' and 'TM' to several different types of electromagnetic modes. One class of modes is those that propagate along waveguides such as optical fibers or on-chip waveguides. Another class of modes are the eigenmodes of infinite structures such as the Bloch modes, which exist in photonic crystals. The usage of the terms TE and TM can be quite different between the various types of modes and a completely natural, uniform usage across all problems is both almost impossible to achieve and would be inconsistent with the most common usages in the literature.

Additionally, another possible source of confusion is the use of the terms 2D and 3D. In documenting the RSoft CAD environment and simulation tools, we count the propagation direction as a dimension, so a 2D structure is one whose cross-sectional profile varies as a function of one variable, usually x , while a 3D structure is one whose cross-sectional profile varies as a function of two variables, usually x and y . If we wish to indicate that a structure varies as a function of all three variables, we normally use the term 3D structure also. This convention occasionally gives rise to some confusing combinations of terminology. For example, a CAD representation of photonic crystal fiber along the z direction is regarded as a 3D structure. Propagating a field down the fiber in BeamPROP is performed in BeamPROP's 3D simulation mode, but the problem of finding the same fiber's modes in BandSOLVE would be solved in

BandSOLVE's 2D-XY mode. While not ideal, such variations in terminology are inevitable given the large number of tools that share the RSoft CAD. In most cases, there is in practice little confusion.

Now let us examine how polarization is labeled in RSoft tools.

2D CAD Designs:

The most basic structures that can be drawn in the RSoft CAD are waveguides that vary as a function of only x . The classic slab waveguide is the simplest example of this class. As there is no variation in the y -direction, the previous discussion indicates that these waveguides fall into the category of 2D CAD designs. For such structures, it is familiar from elementary waveguide optics that in a TE mode, the electric field points along y and in a TM mode, the magnetic field points along y . In this case the interpretation of 'TE' is the literal meaning: the electric field is transverse to the direction of propagation (the wave vector $\mathbf{k}$,) and implicitly the magnetic field is not transverse. Similarly, 'TM' literally means that the magnetic field is transverse to the direction of propagation and the electric field is not.

For 2D CAD designs, **this is the precise definition used in all RSoft tools** except BandSOLVE: 'TE' means that the electric field lies along Y ; 'TM' means that the magnetic field lies along Y . (The 2D rule for *BandSOLVE* is discussed below.) Observe that we could also label these states by the direction in which the electric field points. That is, TE polarization could equivalently be called y -polarization and TM polarization called x -polarization. This is important when we turn to 3D problems.

3D CAD Designs:

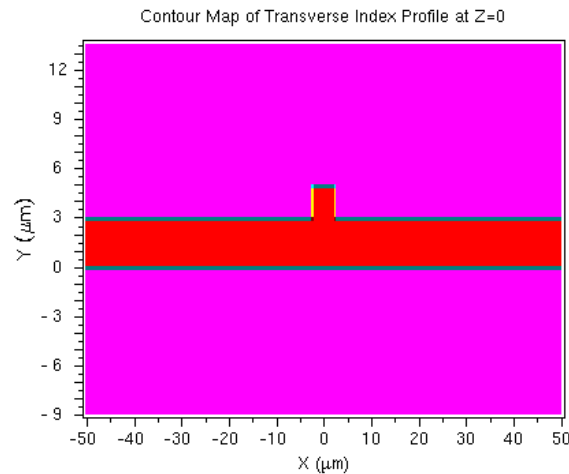
Now consider 3D CAD designs: those for which the waveguides vary in two transverse dimensions. For modal propagation along these waveguides, Maxwell's equations are not separable and the literal meaning of TE and TM modes cannot be applied—virtually all the modes are hybrid and neither field is transverse to the direction of propagation. This point is crucial so let us restate it more forcefully:

For all 3D dielectric waveguides, there are no modes that are truly TE or TM.

The only counter-examples to this statement are the TE_{0m} and TM_{0m} modes of azimuthally-invariant waveguides such as the step-index fiber or concentric Bragg ("Omniguide") fibers, and even then, most of the observed modes, including the fundamental mode, are hybrid LP-like states.

Nevertheless, in weakly guided structures (and even in quite strongly guided structures,) the modes can be close to transverse and the terms TE and TM are widely applied to modes of such structures. We therefore need to be precise about the way the labels are to be applied. The argument runs as follows:

Typically, the longitudinal component of one field is much larger than the longitudinal component of the other. Thus it can be useful to think of the modes as ‘quasi-TE’ and ‘quasi-TM’. Moreover, in many commonly encountered waveguides such as ribs, channels or diffused guides, the cross section varies more rapidly in one direction than the other, and as a result the strengths of the transverse components differ markedly and can be designated as major and minor components. So consider a rib waveguide with propagation along Z and where the layers of the guide are oriented along X:



This is the classic rib waveguide that is easily produced in the RSoft CAD. Then for a ‘quasi-TE’ mode, the longitudinal field E_z is small, and the major field E_x is much larger than the minor field E_y . For a ‘quasi-TM’ mode, the longitudinal field E_z is still small, but the major field is now E_y and the minor field is E_x . So in 3D simulations, the labels TE and TM should strictly be interpreted as quasi-TE and quasi-TM. This provides the explanation for one of the more confusing aspects of the RSoft CAD environment: for 3D structures, TE means the electric field is oriented predominantly along x, and TM means the field is oriented predominantly along y. While this appears to be in conflict with the 2D definitions, for which TE means the electric field points along Y, the rules are natural in the sense that in each case, TE refers to the polarization for which the electric field predominantly points along the transverse direction in which the index profile is most slowly varying. This is ‘natural’ only because most standard layered 3D waveguides drawn in the RSoft CAD tend to vary slowly along x. If for perverse reasons, you manage to construct a rib waveguide in which the layers are oriented along the y direction, then to find the mode that would conventionally be labeled quasi-TE, you will need to use the TM polarization setting.

3D CAD Designs Simulated via EIM:

When using the Effective Index Method, use the same conventions as for a 3D structure. The software will use the appropriate polarization when simulating the structure in 2D.

G.C. Simple rules to remember

The above discussion provides a detailed motivation for the behavior of the CAD with respect to polarization but is a lot to remember. For day-to-day work the following simple rules are easier:

2D Designs:

In 2D designs, TE means that the electric field lies strictly along Y; TM means that the magnetic field lies strictly along Y. In terms of the electric field, we can also label these as ‘y-polarization’ and ‘x-polarization’ respectively.

3D Designs:

In 3D designs there are (virtually) no true transverse modes, so we interpret the TE and TM buttons as meaning quasi-TE and quasi-TM for structures that have a natural orientation within the CAD. Their operational meaning is simply:

- For TE, the electric field is launched predominantly along x. This can be equivalently labeled as x-polarization.
- For TM, the electric field is launched predominantly along y. This can be equivalently labeled as y-polarization.

Remember also that in FullWAVE and full-vector BeamPROP, the field components are all coupled so that for certain polarization-rotating structures, the fields will not necessarily maintain these orientations during propagation.

G.D. Complications

There are some variations on these rules for certain tools:

BeamPROP:

For BeamPROP, please note that the handling of polarization is controlled by the **Vector Mode** setting. Please see the BeamPROP manual for more details.

3D FullWAVE:

In 3D FullWAVE, the definition of the launch polarization is complicated by the ability to rotate the propagation direction of the incoming field to any angle. In this case, the rules stated above are to be interpreted within the rotated launch frame.

BandSOLVE:

The polarization rules in BandSOLVE are quite different. The details are explained fully in the BandSOLVE manual but the pertinent facts are as follows:

- In the subset of 2D problems for which the Bloch vector lies in the plane of index variation, TE and TM are strictly interpreted in the sense of transverse fields. TE polarization means that the electric field is perpendicular to the Bloch vector (and to the plane of index variation). TM polarization means that the magnetic field is perpendicular to the Bloch vector. For standard 2D photonic crystal layouts in the XZ plane, this definition is consistent with the other tools in that for TE, the electric field lies along Y. However, the definition is the opposite of the normal usage in the photonic crystal literature.
- In all other cases, the only allowed polarization state is hybrid. However, a weaker form of separation into even and odd parity states is allowed for certain structures with symmetry planes. Please refer to the BandSOLVE manual for further details.

3D DiffractMOD:

In 3D DiffractMOD, the definition of the launch polarization is complicated by the ability to rotate the propagation direction of the incoming field to any angle as well as the definition of the primary direction. The rules stated above are to be interpreted within the rotated frame. In addition for 3D DiffractMOD computations, the polarization is defined by value of the **Polarization Angle** field. An angle of 0° corresponds to *p* polarization while 90° corresponds to *s* polarization. For normal incidence and a primary direction along +Z, this implies that with **Polarization Angle**= 0° the electric field lies along X, while for **Polarization Angle**= 90° the electric field lies along Y. Please refer to the DiffractMOD manual for further details.

Appendix H

Python Usage

The RSoft tools can utilize Python in various contexts, including:

- Via the `rspython` utility:
 - Manually running `rspython` (see [Appendix E](#))
 - Using a `.py` file within Circuit References (see [Section 6.F](#))
 - Using a Python-based MOST User Simulator (see MOST manual)
- Using a Python-based MOST User Metrics (see MOST manual)
- Using a Python-based MOST User Optimization Algorithms (see MOST manual)

This appendix documents supported Python versions, as well as several Python classes for layout scripting (`RSoftCircuit()` and supporting classes) as well as data manipulation (`RSoftUserfunction()` class). See [Chapter 10](#) for an overview of scripting, including several examples.

H.A. Python Version Support

The version of Python supported is 3.11 on both Windows and Linux.

Please note that as of the 2024.09 release, Python 2 support has been removed in both the API's described in this appendix, and in MOST. Refer to the MOST documentation for additional information. Users with existing Python 2 scripts or MOST metrics, etc. will need to update them to Python 3. In general, this requires changes in two areas. The first is directly related to Python 2/3 differences, the most common one being the `print` command, which are documented publicly on various websites. The second is related to the required switch from `numarray` to `numpy` and is described in the next section.

In addition to `rpython`, note that it is possible to use the API's described in subsequent sections with compatible distributions of Python from another source.

Please contact photonics_support@synopsys.com with questions.

Note: Numeric Array Support

The transition from using Python 2 to Python 3 in `rpython` is in part related to the language differences themselves as noted above, but is also impacted by the use of different numeric array packages. The numeric array package previously supported by `rpython` in Python 2.x was `numarray`, while Python version 3.x supports the newer `numpy`. In many cases, the user does not need to worry about this, as the `RSoftCircuit()` and `RSoftUserFunction()` classes are either agnostic to the choice or produce the corresponding arrays which are used in generally the same manner (e.g. when indexing). However, when the user is creating an array as an input to a member of these classes, `numpy` arrays must be used.

Please contact photonics_support@synopsys.com with questions.

H.B. Layout API: `RSoftCircuit()` class

The Python API can be used to script the layout of a `.ind` design file. It contains the main `RSoftCircuit()` class as well as the `RSoftComponent()` and `RSoftVertex()` classes. It is recommended that you use the API in conjunction with the circuit reference feature described in [Section 6.F](#). See [Chapter 10](#) for an overview of scripting, including several examples.

H.B.1. `RSoftCircuit()` class

Create a new `RSoftCircuit` object using this code:

```
from rstools import RSoftCircuit
c = RSoftCircuit()
```

The following class functions are supported:

```
read(filename)
```

Reads an existing `.ind` design file (string `filename`), for example:

```
c.read('myfile.ind')
```

```
write(filename)
```

Writes an `.ind` design file to the specified `filename` (string), for example:

```
c.write('myfile.ind')
```

```
set_symbol(name, value)
```

Set a symbol with the specified `name` and `value`. The `name` is a string, the `value` can be a number or a standard string expression that can use the built-in functions and/or other symbols. Examples:

```
c.set_symbol('Period', 0.5)
c.set_symbol('Radius', '0.35*Period')
```

```
delete_symbol(name)
```

Deletes a symbol with specified `name` (string), for example:

```
c.delete_symbol('Period')
```

```
load_settings(file)
```

Load settings from specified file (string). Equivalent to the **Load Settings** option in the Global Settings dialog (see [Section 3.B](#)). Example:

```
c.load_settings('SiSettings.ind')
```

```
set_exports(exports)
```

Set hierarchy exports; `exports` is a space-separated list of symbols to include. This feature enables the efficient passing of symbols from the parent to child files in cases where the symbol does not need to be an argument. See [Section 6.F.5](#) for details about hierarchy exports. Example:

```
c.set_exports('Period Angle')
```

```
set_default_tags(tags)
```

Set default tags; `tags` is a space-separated list of symbols to include. See [Section 6.K.5](#) for details about default tags. Example:

```
c.set_default_tags('grid_ppw_x=35 grid_size_y=0.001')
```

```
add_material(matname='', libname='', groupname='', datafile='', nr='',
ni='')
```

Adds a material to the design file from one of several sources, depending on the arguments.

Parameter	Description
-----------	-------------

matname	The material name
libname	The library name (optional)
groupname	The group name (optional)
datafile	Set the material properties; set either the name of an nk
nr	datafile or the real (nr) and optional imaginary (ni)
ni	values. If neither datafile nor nr/ni are present, the
	material is taken from the Material Library.

`add_monitor( position=(), dimensions=() )`

Adds a monitor to the design file.

Parameter	Description
position	A tuple that sets the starting (X,Y,Z) position of the segment. This is the starting vertex. The values can be numbers or standard expression strings.
dimensions	A tuple that sets the dimensions (width/height) of the starting vertex. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.

`add_portmonitor( position=(), dimensions=() )`

Adds a port monitor to the design file.

Parameter	Description
position	A tuple that sets the starting (X,Y,Z) position of the segment. This is the starting vertex. The values can be numbers or standard expression strings.
dimensions	A tuple that sets the dimensions (width/height) of the starting vertex. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.

`add_segment( position=(), offset=(), dimensions=(), dimensions_end=() )`

Adds a segment (see [Chapter 4](#)) to the design with the following properties:

Parameter	Description
position	A tuple that sets the starting (X,Y,Z) position of the

	segment. This is the starting vertex. The values can be numbers or standard expression strings.
offset	A tuple that sets the (X,Y,Z) offset of the ending vertex. This is the offset between the starting vertex and ending vertex. The values can be numbers or standard expression strings.
dimensions	A tuple that sets the dimensions (width/height) of the starting vertex. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.
dimensions_end	A tuple that sets the dimensions (width/height) of the ending vertex. If not set, the default value is equal to dimensions. The values can be numbers or standard expression strings.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```
seg1 = c.add_segment(position=(0,0,0), offset=(0,0, 'L'))
c.add_segment(position=(0,0,0), offset=(0,0, 'L'), dimensions=(0.5,0.2)

add_sbend( position=(), offset=(), dimensions=(), dimensions_end=()
           s_bend_type=0 )
```

Adds an s-bend segment (see [Section 6.B.3](#)) to the design with the following properties:

Parameter	Description
position	A tuple that sets the starting (X,Y,Z) position of the sbend. This is the starting vertex. The values can be numbers or standard expression strings.
offset	A tuple that sets the (X,Y,Z) offset of the ending vertex. This is the offset between the starting vertex and ending vertex. The values can be numbers or standard expression strings.
dimensions	A tuple that sets the dimensions (width/height) of the starting vertex. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.
dimensions_end	A tuple that sets the dimensions (width/height) of the ending vertex. If not set, the default value is equal to

dimensions. The values can be numbers or standard expression strings.

`sbend_type`

An integer that sets the sbend type. A value of 0 (default) is Circular-Arc, 1 is Cosine, and 2 is Raised-Sine.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```
sbend1 = c.add_sbend(position=(0,0,0),offset=(0,0,'L'),sbend_type=2)
c.add_sbend(position=(0,0,0),offset=(0,0,'L'),dimensions=(0.5,0.2)
```

```
add_arc( position=(), arcinfo=(), dimensions=(), dimensions_end=() )
```

Adds an arc segment (see [Section 6.B.3](#)) to the design with the following properties:

Parameter	Description
<code>position</code>	A tuple that sets the starting (X,Y,Z) position of the arc. This is the starting vertex. The values can be numbers or standard expression strings.
<code>arcinfo</code>	A tuple that sets the radius, initial angle, and final angle as (radius, angle-initial, angle-final). The values can be numbers or standard expression strings; angles should be in degrees.
<code>dimensions</code>	A tuple that sets the dimensions (width/height) of the starting vertex. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.
<code>dimensions_end</code>	A tuple that sets the dimensions (width/height) of the ending vertex. If not set, the default value is equal to dimensions. The values can be numbers or standard expression strings.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```
arc1 = c.add_arc(position=(0,0,0),arcinfo=('R',0,90))
c.add_arc(position=(0,0,0),arcinfo=('R',0,45),dimensions=(0.5,0.2)
```

```
add_lens( position=(), dimensions=(), shape=0 )
```

Adds a lens (see [Section 5.B](#)) to the design with the following properties:

Parameter	Description
<code>position</code>	A tuple that sets the (X,Y,Z) position of the lens. The values can be numbers or standard expression strings.
<code>dimensions</code>	A tuple that sets the dimensions (width/height) of the lens. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.
<code>shape</code>	An integer that sets the lens shape: a value of 0 (default) correspond to a cylinder, a value of 1 corresponds to a sphere.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```
spherel = c.add_lens(position=(0,0,0), dimensions=('D','D') shape=1)

add_circle( position=(), dimensions=(), shape=0 )
```

Adds a circle (see [Section 5.C](#)) to the design with the following properties:

Parameter	Description
<code>position</code>	A tuple that sets the (X,Y,Z) position of the circle. The values can be numbers or standard expression strings.
<code>dimensions</code>	A tuple that sets the dimensions (width/height) of the circle. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.
<code>shape</code>	An integer that sets the circle shape: a value of 0 (default) correspond to a cylinder, a value of 1 corresponds to a sphere.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```
circle1 = c.add_circle(position=(0,0,0), dimensions=('D','D')
                        shape=1)

add_polygon( position=(), x=None, y=None, file='', angle=0 )
```


Adds a polygon (see [Section 5.D](#)) to the design with the following properties:

Parameter	Description
position	A tuple that sets the (X,Y,Z) position of the polygon. The values can be numbers or standard expression strings.
X Y file	Defines the points of the polygon via x/y arrays or a data file. The x/y arrays should be 1D arrays of the same length.
angle	Sets the rotation angle for the polygon in degrees.

This function returns a `RSofComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```

polygon1 = c.add_polygon(position=(0,0,0), file='poygon.dat',
angle=45)

add_mark( position=(), dimensions=() )
```

Adds a mark (See [Section 5.E](#)) to the design with the following properties:

Parameter	Description
position	A tuple that sets the (X,Y,Z) position of the mark. The values can be numbers or standard expression strings.
dimensions	A tuple that sets the dimensions (width/height) of the mark. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.

This function returns a `RSofComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```

mark1 = c.add_mark(position=(0,0,0))

add_textblock( name, text='' )
```

Adds a text block to the design with the following properties:

Parameter	Description
name	The name of the text block.

<code>text</code>	The body of the text block.
-------------------	-----------------------------

This is an advanced option; contact technical support with questions.

```
add_vertex( position=(), dimensions=() )
```

Adds a vertex (See [Section 6.F.7](#)) to the design with the following properties:

Parameter	Description
<code>position</code>	A tuple that sets the (X,Y,Z) position of the vertex. The values can be numbers or standard expression strings.
<code>dimensions</code>	A tuple that sets the dimensions (width/height) of the vertex. If not set, a default value of ('width', 'height') is used. The values can be numbers or standard expression strings.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```
vertex1 = c.add_vertex(position=(0,0,0))
```

```
add_circuitref( position=(), file='', angle=0 )
```

Adds a circuit reference (See [Section 6.F](#)) to the design with the following properties:

Parameter	Description
<code>position</code>	A tuple that sets the (X,Y,Z) position of the circuit reference. The values can be numbers or standard expression strings.
<code>file</code>	Sets the file to be used for the circuit reference.
<code>angle</code>	Sets the rotation angle for the polygon in degrees.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

Examples:

```
ref1 = c.add_circuitref(position=(0,0,0))
```

```
add_polygon( position=(), x=None, y=None, file='', angle=0 )
```

Adds a circuit reference (See [Section 6.F](#)) to the design with the following properties:

Parameter	Description
<code>position</code>	A tuple that sets the (X,Y,Z) position of the circuit reference. The values can be numbers or standard expression strings.
<code>x</code>	Double 1D numeric array (same length as y)
<code>y</code>	Double 1D numeric array (same length as x)
<code>file</code>	Sets the file to be used for the circuit reference.
<code>angle</code>	Sets the rotation angle for the polygon in degrees.

This function returns a `RSoftComponent()` object which can be further configured, see [Section H.B.2](#).

```
attach( from_component, to_component, to_vertex, from_vertex,
attach_angles=0, attach_dimensions=0 )
```

This function enables users to connect components/vertices using the reference options described in [Section 4.B.1](#).

Parameter	Description
<code>from_component</code>	The <code>RSoftComponent()</code> object to attach to.
<code>to_component</code>	The <code>RSoftComponent()</code> object to attach.
<code>to_vertex</code>	Sets the vertex number to attach to.
<code>from_vertex</code>	Sets the vertex number to attach.
<code>attach_angles</code>	Set to 1 to dynamically pass the segment angle.
<code>attach_dimensions</code>	Set to 1 to dynamically pass the segment dimensions.

Example:

```
seg1 = c.add_segment()
seg2 = c.add_segment()
c.attach(seg2, seg1)
```

H.B.2. `RSoftComponent()` class

An `RSoftComponent` is created when adding components to an `RSoftCircuit()` object. The following class functions allow various component properties to be set:

```
attach_portmonitors()
```

Adds port monitors to unterminated vertices of an `RSoftComponent()` object.

For example:

```
cr = c.add_circuitreference(position=(0,0,0), file='a.gds:88/0')
cr.attach_portmonitors()
```

`attach_portvertices()`

Automatically detect vertex positions and add vertices.

For example:

```
cr = c.add_circuitreference(position=(0,0,0), file='a.gds:88/0')
cr.attach_portvertices()
cr.attach_portmonitors()
```

`color()`

Set the component color.

For example:

```
circle1 = c.add_circle(position=(0,0,0), shape=1)
circle1.color( '4' )
```

`dimensions( dimensions=() )`

Sets the dimensions of the component vertex, specifically a tuple that sets the dimensions (width/height). If the component has multiple vertices, this option sets the starting vertex dimensions. See `vertex()` function to set properties of other vertices. The values can be numbers or standard expression strings.

For example:

```
circle1 = c.add_circle(position=(0,0,0), shape=1)
circle1.dimensions( ('D','D') )
```

`draw_priority()`

Set the draw priority.

For example:

```
circle1 = c.add_circle(position=(0,0,0), shape=1)
circle1.draw_priority( 4 )
```

`mask_layer()`

Set the mask layer.

For example:

```
circle1 = c.add_circle(position=(0,0,0), shape=1)
circle1.mask_layer( 88 )
```

`material()`

Sets the material of the component via a string `matname`. Note that this option is designed to be used within a circuit reference. This option only assigns the material to the component, the `.ind` file generated must then be used as a circuit reference in a parent file where the material properties are defined in that parent file.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.material('SiO2')
```

`orientation()`

Sets the segment orientation of the component. Set to 0 for Z Axis, for Seg Axis, and 2 for Extended.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.orientation(0)
```

`polyinfo(x=None, y=None, file='', sides=0)`

Sets the polygon information for polygon segments. The `x` and `y` arguments are double 1D numeric arrays of the same length.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.structure('POLYGON')
seg1.polyinfo(sides=8)
```

`position( position=(), ref_type=-1 )`

Sets the position of the component vertex, specifically a tuple that sets the (X,Y,Z) coordinates. If the component has multiple vertices, this option sets the starting vertex dimensions. See `vertex()` function to set properties of other vertices. The optional `ref_type` option sets whether the position is absolute (0) or relative (-1). The values can be numbers or standard expression strings.

For example:

```
circle1 = c.add_circle(shape=1)
circle1.position( (1,1,1) )
```

`priority()`

Set the priority.

For example:

```
circle1 = c.add_circle(position=(0,0,0), shape=1)
circle1.priority( 4 )
```

```
rotation( angles=() )
```

Sets the rotation angles of the component in degrees, specially a tuple with the three rotation angles phi, theta, and psi that are used to rotate extended segments (see [Section 6.K.2](#)). The values can be numbers or standard expression strings.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.rotation( (0,90,0) )
```

```
set_name()
```

Sets the component name of the component. The argument is a string.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.set_name( "MySegment" )
```

```
set_property(name='', value='', sub='')
```

An advanced generic function to set any component property whose keyword in a .ind file is known. The optional `sub` argument sets a nested property. Note: you must use a valid property name; setting an invalid property name will result in a parse error when the design file is used.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.set_property('sidewall_angle', 80)
set1.set_property('combine_mode', 'COMBINE_ADD')

mon3 = c.add_monitor( position=(0,0,0) )
mon3.set_property('monitoroutputmask', 8)
```

```
set_symbol(name, value)
```

Set a symbol with the specified `name` and `value` in the local symbols of a circuit reference component. The `name` is a string, the `value` can be a number or a standard string expression that can use the built-in functions and/or other symbols. Examples:

```
c.set_symbol('Period', 0.5)
c.set_symbol('L', '2*width')
```

```
delete_symbol(name)
```

Deletes a symbol with specified `name` (string) from the local symbols of a circuit reference component, for example:

```
c.delete_symbol('Period')
```

```
set_tags()
```

Sets a component tag. The argument is a string of form "name=value".

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.set_tags( "tag=value" )
```

```
set_comment() )
```

Sets the component comment. The argument is a string.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.set_comment( "This is a comment!" )
```

```
structure()
```

Set the 3D Structure Type. Options include 'CHANNEL', 'FIBER', 'POLYGON', 'MULTILAYER', 'RIBBRIDGE', and 'DIFFUSED'.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
seg1.structure( 'FIBER' )
```

```
vertex( vindex=0 )
```

Returns an `RSoftVertex()` object for the requested vertex index `vindex`. See [Section H.B.3](#) for details on this type of object and how to set the dimensions and position of the vertex.

For example:

```
seg1 = c.add_segment( position=(0,0,0) )
vertex = seg1.vertex(1)
```

H.B.3. RSoftVertex() class

An `RSoftVertex` object can be extracted from an `RSoftComponent()` object, allowing additional control over these vertices. The following class functions allow various vertex properties to be set:

```
dimensions( dimensions=() )
```

Sets the dimensions of the vertex, specifically a tuple that sets the dimensions (width/height). The values can be numbers or standard expression strings.

For example:

```
seg = c.add_segment(position=(0,0,0))
vertex2 = seg.vertex(1)
vertex2.dimensions( ('D','D') )
```

```
position( position=(), ref_type=-1 )
```

Sets the position of the component vertex, specifically a tuple that sets the (X,Y,Z) coordinates. The optional `ref_type` option sets whether the position is absolute (0) or relative (-1). The values can be numbers or standard expression strings.

For example:

```
seg = c.add_segment()  
vertex2 = seg.vertex(1)  
vertex2.position( (0,0,0) )
```

H.B.4. RSoftMaterial() class

An `RSoftMaterial` object contains the definition of a material. The following class functions allow various material properties to be set:

```
set_property(name='', value='', sub='')
```

An advanced generic function to set any material property whose keyword in a `.ind` file is known. The optional `sub` argument sets a nested property. Note: you must use a valid property name; setting an invalid property name will result in a parse error when the design file is used.

For example:

```
mat = c.add_material( matname='MyMat')  
mat.set_property('anisotropic',1,'optical')  
mat.set_property('nr_xx',1.5,'optical')  
mat.set_property('nr_yy',1.6,'optical')  
mat.set_property('nr_zz',1.5,'optical')
```

H.B.5. Using Hierarchy Exports

In some cases, it is not necessary to pass all parameters to the Python script as arguments, hierarchy exports can also be used. Use the `set_exports()` function, see [Section 10.A.1](#) for an example.

H.B.6. Best Practices

It is recommended to follow these best practices when using the `RSoftCircuit()` class:

- See [Section 10.A](#) for several examples that illustrate possible design approaches
- When used within a circuit reference block, use with the `rsoft_layout()` function described in [Section 6.F.6](#). This simplifies the passing of arguments to the script. Any output will be written to a log file to assist in debugging.
- Use Hierarchy Exports, see [Section H.B.5](#).

- A material can be set via the see `RSoftComponent()` class, see details the `material()` function in [Section H.B.2](#).

H.B.7. Example Usage

Here are several simple example scripts to illustrate the usage of the `RSoftCircuit()` class:

See examples in [Section 10.A](#) for more examples

Example 1:

This example creates an 8x8 array of segments in the XY plane and saves as `array.ind`:

```
from rstools import RSoftCircuit

#Constants
N = 8
file = 'array.ind'

#Create new RSoftCircuit object
c = RSoftCircuit ()

#Set some symbols
c.set_symbol(Period, 0.5)
c.set_symbol(Radius, 'Period*0.3')
c.set_symbol(D, '2*Radius')
c.set_symbol(Length, 10)

for i in range(N):
    for j in range(N):
        x0 = ('{}*Period'.format(i))
        y0 = ('{}*Period'.format(j))
        z0 = 0
        c.add_segment( (x0,y0,z0), (0,0,'Length'), ('D','D'), ('D','D') )

#Save .ind file
c.write(file)
```

Example 2:

This example creates an NxN array of segments in the XY plane. Note that this example must be run via a circuit reference, if set, the Local Symbol `Arg1_num` will be used to set the array size (N).

```
from rstools import RSoftCircuit

def rsoft_main(ind_file='',
               N=int(10)):

    #Create new RSoftCircuit object
    c = RSoftCircuit ()
```

```

#Set some symbols
c.set_symbol(Period, 0.5)
c.set_symbol(Radius, 'Period*0.3')
c.set_symbol(D, '2*Radius')
c.set_symbol(Length, 10)

for i in range(N):
    for j in range(N):
        x0 = ('{}*Period'.format(i))
        y0 = ('{}*Period'.format(j))
        z0 = 0
        c.add_segment( (x0,y0,z0), (0,0,'Length'), ('D','D'), ('D','D') )

#Save .ind file
c.write(ind_file)

```

Example 3:

This example implements the same structure in Example 2, but sets the `Period`, `Radius`, and `Length` as Hierarchy Exports which can then easily be set in the Local Symbols table of the circuit reference.

```

from rstools import RSoftCircuit

def rsoft_main(ind_file='',
               N=int(10)):

    #Create new RSoftCircuit object
    c = RSoftCircuit ()

    #Set some symbols
    c.set_symbol(Period, 0.5)
    c.set_symbol(Radius, 'Period*0.3')
    c.set_symbol(D, '2*Radius')
    c.set_symbol(Length, 10)

    #Set Hierarchy Exports
    c.set_exports('Period', 'Radius', 'Length')

    for i in range(N):
        for j in range(N):
            x0 = ('{}*Period'.format(i))
            y0 = ('{}*Period'.format(j))
            z0 = 0
            c.add_segment( (x0,y0,z0), (0,0,'Length'), ('D','D'), ('D','D') )

    #Save .ind file
    c.write(ind_file)

```

Example 4:

See This example creates a tapered waveguide, including port monitors.

```

from rstools import RSoftCircuit

# inputs

```

```

name = 'wg_taper'

# structure properties
opts = {}
opts['Lin'] = 0.5
opts['Ltaper'] = 2
opts['W1'] = 0.5
opts['W2'] = 1.0
opts['H'] = 'height'

# simulation settings
opts['Dx'] = 0.02
opts['Dy'] = opts['Dx']
opts['Dz'] = opts['Dx']
opts['boundary_gap_x'] = 0.5
opts['boundary_gap_z'] = 0
opts['color_outline'] = -1
opts['domain_max'] = 'Lin*2+Ltaper'
opts['bpm_output_monitors'] = 1
opts['bpm_output_monitors_warned'] = 1
opts['free_space_wavelength'] = 1.55
opts['sim_tool'] = 'ST_FULLWAVE'
opts['grid_nonuniform'] = 1
opts['grid_size'] = 'Dx'
opts['grid_size_y'] = 'Dy'
opts['step_size'] = 'Dz'
opts['eim'] = 1
opts['launch_port'] = 1
opts['polarization'] = 0
opts['plot_aspectratio'] = 1

# create design file, load settings, add symbols
c = RSoftCircuit()
c.load_settings('SiSettings.ind')
for key in opts:
    c.set_symbol(key, opts[key])

# add/attach components
wg_in = c.add_segment(offset=(0,0,'Lin'), dimensions=('W1', 'H'))
wg_taper = c.add_segment(offset=(0,0,'Ltaper'), dimensions=('W1',
    'H'), dimensions_end=('W2', 'H'))
wg_out = c.add_segment(offset=(0,0,'Lin'), dimensions=('W2', 'H'))

c.attach(wg_taper, wg_in, 1,0,0)
c.attach(wg_out, wg_taper, 1,0,0)

# add/attach port positions
port1 = c.add_portmonitor(dimensions=('W1', 'height'))
port2 = c.add_portmonitor(dimensions=('W2', 'height'))

c.attach(port1, wg_in, 0,0,1)

```

```
c.attach(port2, wg_out, 1,0,1)

c.write('%s.ind'%name)
```

H.C. RSoft Data API: RSoftUserFunction()

The `RSoftUserFunction()` class can be used to pragmatically interact with RSoft data files. All standard RSoft file formats described in Appendix B are supported, including 1D, 2D, 3D, real, complex, uniform, and non-uniform data formats. Note that files in the TCAD Sentaurus TDR format are not currently supported. See [Chapter 10](#) for an overview of scripting, including examples.

H.C.1. Reference

Create a new `RSoftUserFunction` object using this code:

```
from rstools import RSoftUserFunction
uf = RSoftUserFunction()
```

The following class functions are supported:

`read(filename)`

Reads a data file (string `filename`) into the object, for example:

```
uf.read('data.dat')
```

`write(filename)`

Writes the object data to the specified `filename`, for example:

```
uf.write('data.dat')
```

`type()`

Returns the data type in the object, for example:

```
t = uf.type()
```

The types are in the data header of the file, they are 1 for amplitude, 2 for real, 3 for imaginary, 4 for real/imag, and 5 for amp/phase.

`isreal()`

Returns 1 if the data in the object is real (i.e. not complex), for example:

```
if (uf.isreal()):
    print('Data is real')
```

`iscomplex()`

Returns 1 if the data in the object is complex-valued, for example:

```
if (uf.iscomplex()):
    print('Data is complex')
```

```
get_arrays()
```

Returns a tuple of the arrays within the object, returns multiple arrays depending on rank of the data object. For example:

```
uf.read('data.dat')
(x,y,f) = uf.get_arrays()
```

where `data.dat` is a 2D data file and `x`, `y`, and `f` will contain the `x`, `y`, and data arrays from the file.

```
set_arrays(x,f)
set_arrays(x,y,f)
set_arrays(x,y,z,f)
```

Sets the arrays in the object, for example:

```
x = array([1,2,3,4.5])
f = array([1,4,9,16,25])
uf.set_arrays(x,f)
```

```
eval(x)
eval(x,y)
eval(x,y,z)
```

Returns the real value of the function at the given `x`, `y`, and/or `z` coordinate value. The data in the object will be linearly interpolated, similar to the `userdata()` functions described in [Section C.F.6](#). Note: this function returns 0 if the `x`, `y`, or `z` coordinates outside the range defined in the data object. Example:

```
value = uf.eval(1.5)
```

```
eval_complex(x)
eval_complex(x,y)
eval_complex(x,y,z)
```

Returns the complex value of the function at the given `x`, `y`, and/or `z` coordinate value. The data in the object will be linearly interpolated, similar to the `userdata()` functions described in [Section C.F.6](#). Note: this function returns 0 if the `x`, `y`, or `z` coordinates outside the range defined in the data object. Example:

```
value = uf.eval(1.5)  #value is a complex number, i.e. (1+7j)
```

```
get(i)
get(i,j)
get(i,j,k)
```

Returns the value of the data at the given `i`, `j`, and/or `k` position in the function array, for example:

```
value = uf.get(10)    #value is the 10th point in the function
```

```
rank()
```

Returns the rank (dimension) of the data in the object, for example:

```
if (uf.rank() == 2):  
    #do something if the data is 2D
```

```
nx()
```

```
ny()
```

```
nz()
```

Returns the number of X, Y, or Z points in the object, for example:

```
NX = uf.nx()
```

```
x(i)
```

```
y(i)
```

```
z(i)
```

Returns the X, Y, or Z coordinate for the ith position in the coordinate array, for example

```
x_coordinate = uf.x(4)
```

```
unwrap_phase(array)
```

Unwraps phase data in the array. See [Section H.C.2](#) for an example.

H.C.2. Example Usage

Here are several simple example scripts to illustrate the usage of the `RSoftUserFunction()` class:

See examples in [Section 10.C](#) for more examples

Example 1:

This example opens the data file `data.dat` and evaluates the function at `x=0.25`:

```
from rstools import *  
  
#Create new RSoftUserFunction object  
uf = RSoftUserFunction()  
  
#Open data file  
uf.read('data.dat')  
  
#Evaluate data at x = 0.25 (using interpolation as necessary)  
value = uf.eval(0.25)  
  
#Print value  
print(value)
```

Example 2:

This example opens the data file `data2.dat` squares the data values, and saves the file:

```
from rstools import *

#Create new RSoftUserFunction object
uf = RSoftUserFunction()

#Open data file
uf.read('data2.dat')

#Extract arrays and save in x (coordinate) and f (data value) arrays
(x,f) = uf.get_arrays()

#Square data values
f = f**2

#Store arrays
uf.set_arrays(x,f)

#Save file as data2_squared.dat
uf.write('data2_squared.dat')
```

Example 3:

This example illustrates how to unwrap phase in file `data.dat` and write the unwrapped data to the file `data_unwrapped.dat`.

```
from rstools import *

#Create new RSoftUserFunction object
uf = RSoftUserFunction()

#Open data file
uf.read('data.dat')

#Get arrays
(x,f) = uf.get_arrays()

#Unwrap phase in f array
unwrap_phase(f)

#Write arrays back to object
uf.set_arrays(x,f)

#Save unwrapped data to new file
uf.write('data_unwrapped.dat')
```

Appendix I

RSoft CAD Release Notes

This appendix summarizes the changes from previous versions of the RSoft CAD Environment. Notes for versions not listed here can be found in the BeamPROP manual.

Versions 2015.06 and Later

The release notes for version 2015.06 and later versions can be found here:

`<rsoft_dir>\readme\releasenotes`

Version 2014.09

- With this release, the RSoft Component Suite will no longer officially support 32-bit operating systems, Windows XP, Windows Vista, or Red Hat Enterprise Linux (RHEL) 4. If you have any questions, please contact rssoft_support@synopsys.com.
- A new Configure Licensing utility (Windows only) provides an easy way to reconfigure licensing options. This includes installing a new license key file and restarting SCL. This option can be opened from the Start Menu and requires administrative rights.
- Several improvements to SCL licensing implementation.
- Updated Linux installation routine.
- Added a new Tree Control to the RSoft CAD window. The tree lists all elements in a design, including components, circuit references, monitors, pathways, launch fields, user profiles and

tapers, layer tables, materials, and embedded circuits. All elements can now be selected and edited via the tree, in addition to previous methods.

- Added an expiration warning prior to expiration of a user's license key file.
- Expanded the circuit hierarchy feature to allow child design files to be embedded, or stored, within the parent file. A list of the embedded files in a parent design file can be viewed using the new Tree Control in the RSoft CAD.
- Added a Dynamic Array Wizard that simplifies the creation of dynamic arrays. A library of common unit cell elements is now included. The wizard will automatically embed the desired unit cell within the parent file.
- Added context-sensitive program usage tips designed to help new users learn to use the software and teach advanced users about new features.
- Added two new features for the automatic import of refractive index data from text files into the Material Editor. The Import NK Data option imports the data file into a nondispersive-type material using the userdata() functions. This approach is sufficient for accurately modeling dispersion in all simulation tools except FullWAVE pulse or impulse calculations. The Fit NK Data option attempts to fit the data to Lorentizan coefficients that can be used to define a dispersive-type material suitable for use with FullWAVE pulse or impulse calculations.
- Added pan/zoom control via keyboard and mouse.
- Added a new options toolbar to WinPLOT for easy access to commonly used plot options. This includes plot type, color scales, automatic data scaling, aspect ratio, coordinate system, axes scale type, complex data display, and symbol/line styles.
- Simulation tools (BandSOLVE and GratingMOD currently) that do not use the same simulation domain as the Compute Material Profile domain (the domain used when computing material profiles via the button in the left CAD toolbar) now display the simulation domain in the CAD as a dashed line to visually denote that the actual simulation domain is not displayed.
- Improved support for combined arc and width tapers for seg-axis and extended segments.
- Added several new preferences for the Tree Control and several new simulation tool options.

Version 2013.12

- All products in RSoft's Component Design Suite have been increased to version 2013.12 which is the Synopsys standard.
- Updating Licensing to use Synopsys Common Licensing (SCL).
- Updated Ray-Tracing interface to handle CODE V's complex and vector field formats.

- The RSoft field file header has been expanded to include named parameters in the header, including wavelength and refractive index.
- Added option to bduutil utility to convert an RSoft far-field into a Ray Data File.
- Added option to bduutil utility perform simple arithmetic operations on data files such as adding files.
- Improved far-field output of the bduutil utility. The `-fa` option now outputs the same data as the built-in far-field options of BeamPROP, FullWAVE, DiffractMOD, etc. Contact us with questions.
- New command line utilities to convert .ind files to DXF and GDS files.
- Enhanced Semiconductor options in the Material Editor.
- New rotatable 3D color-coded plots in WinPLOT.

Changes in Program Behavior

- Changed default file selection mode in DataBROWSER.
- Changed the default parameters and units for the Zemax portion of the Raytracing Converter. The new units are in microns.
- Changed case-sensitivity of data files on Linux. Previously, all data files referenced in an .ind design file were case-sensitive. Now, the program will first look for the file as specified in the design file, and if it is not found, then look an entirely lowercase name.

Changes form Version 8.2 to Version 9.0

The change log which details the history of changes from Version 8.2 to Version 9.0 is available in the file `README82.TXT`.

- All products in RSoft's Component Design Suite have been increased to version 9.0 with this release.
- Released LED Utility.
- Import/Export features in the Preferences dialog now effect the index generation.
- New PEC material in Material database.
- Components can now be selected by Component Name, if defined.
- New feature to offset a group of components by a fixed value.
- Maximum number of dispersion terms in the Material Editor increased from 6 to 24.
- Dynamically sized arrays of objects can be created with the hierarchy feature.
- Panning options with the mouse, scroll bars, and a keyboard shortcut.

- New features to simplify the drawing of common 3D objects.
- New 'polygon' structure type to easily assign a polygon cross-section to a component.
- Improved extended segments that allow for arbitrary position, tapering, and profiles.
- New Z Position Taper for extended segments.
- New 'twisting' taper which twists a segment's profile along the segment axis.
- Arc reference point can now be at the center of the arc.

Changes from Version 8.1 to Version 8.2

The change log which details the history of changes from Version 8.2 to Version 8.2 is available in the file `README81.TXT`.

- New group editing features.
- Improved undo functionality.
- New option to interpret user-defined profiles absolutely.
- Released the Solar Cell Utility and Multi-Physics Utility.
- Added new options for far field calculations of periodic fields.
- Added new symbol table functions that allow users to access the real and imaginary refractive index of a material at a specific wavelength (`nreal()` and `nimag()`).
- Many updates to specific simulation engines. See appropriate manual for details.

Changes from Version 8.0 to Version 8.1

The change log which details the history of changes from Version 8.0 to Version 8.1 is available in the file `README80.TXT`.

- Updated Material Editor that allows materials to be shared between design files.
- New stress modeling capabilities.
- New 1D mode solver TmmSIM.

Changes from Version 7.0 to Version 8.0

The change log which details the history of changes from Version 7.0 to Version 8.0 is available in the file `README70.TXT`.

- The CAD window has been updated with new graphics, a new 'view' toolbar, and new buttons.

- Several options which used to be in the Global Settings (Polarization, BPM Vector Mode, FDTD dispersion/non-linearity, and anisotropy) have been relocated to an appropriate simulation parameters window.
- Several items in the Additional Segment Properties window have been moved to the Component Properties window. These include the Display Color and Merge Priority.
- The BPM-based mode solvers included with the CAD are now controlled via the Mode Options window which can be opened via the Mode Calculation Parameters window (which can be opened via the button in the left toolbar). This includes the mode method and mode numbers to find.
- Multipane Mode: This allows the user to view different axis views individually or all at once. Additionally, a 3D view is available. These options are only available in 3D (in 2D, the XZ plane is displayed). Multipane view has two modes, 1P and 4P which can be chosen from the view toolbar. In 1P mode, the user can select one cut direction from the view toolbar to display the appropriate cross section. In 4P view, all three cross-sections can be seen at once, as well as a 3D view.
- Cut Mode: This allows the user to view a particular cut of a structure when in an axis view mode. For example, when viewing a Y cut (the XZ plane), the structure at a particular Y position can be seen. This is done via the +/- buttons on the view toolbar. The current cut position, as well as the cut step size are shown and can be changed by the user. To return to the normal view, click the 'All' button.
- Added a new drawing mode for out of plane segments. This allows the user to place segments that are perpendicular to the drawing window. For example, segments can be placed along the Z axis when viewing the XY plane. This is not enabled for all axis views.
- Added the Y coordinate to all component properties windows to parallel X and Z.
- Added an "End equals Start" feature to segments for index and geometry.
- The Compute Index Profile window now has control for cut position as well as a choice of a either real or imaginary index.
- The Compute Index Profile window now has a Output Options window.
- Added a new mode for defining the refractive index absolutely. This feature should be used with caution.
- Expanded the CAD preferences window with additional options.
- Launch Fields are now displayed in the CAD window.
- The simulation domain is now displayed in the CAD window.

Changes from Version 6.0 to Version 7.0

The change log which details the history of changes from Version 6.0 to Version 7.0 is available in the file `README60.TXT`.

- Added support to use a variable in file name fields to represent a data file. This allows for greater ease in using different data files for each step of a scan.
- Introduction of an entirely new index/grid/mesh generation algorithm which improves accuracy and allows more flexible layout.
- Added support for non-uniform grids. This can significantly improve accuracy while improving simulation speed.
- Added support for the Material Editor for all simulation tools.
- Added a status line for all simulation tools. The status line now shows the cursor coordinates, as well as other information such as the BeamPROP Z step or the FullWAVE time step.
- Enabled a cross-cut feature in Winplot.
- Added support for arbitrary polygons.
- Added ability to control the angular range of plots for the far field output options.
- Added three new fields to the Advanced Components Properties dialog, which can be accessed through the **More...** button. These allow users to name each component, add comments, and provide special tags for advanced use.
- Expanded the sidewall angle option to include multilayer structure types.

Changes from Version 5.1 to Version 6.0

The change log which details the history of changes from Version 5.1 to Version 6.0 is available in the file `README51.TXT`.

- Added a toolbar icon to bring up help for the currently active simulation program. The CAD/Simulation help icons are distinguished with the words CAD/SIM at the lower right of the icons, respectively.
- Implemented a new capability for z-dependent user-defined index profiles. To enable recognition, you must set the variable `change_profile = 1`. Do not set this variable unnecessarily as it can affect performance.
- Implemented a new capability for data files to be used in expressions to represent a function of up to three variables.
- Major updates to FullWAVE, including new features for dispersion, nonlinearity, and magnetic materials (including negative refractive index materials). More information in the FullWAVE manual.

- Improved consistency and functionality of index display options for showing XY, XZ, and YZ cross-sections. For FullWAVE, BandSOLVE, and DiffractMOD, using the `fddtd_index_profile = 1` option will give a more representative plot for XZ. This option will be made automatic and consistent with the other options in a future version.
- A new option for displaying the index profile of 3D geometries. The new option is called "3D Volume", and will generate a volumetric plot of the dielectric constant (i.e., N^2 , not N).
- A major new feature for hierarchical CAD layout. This feature lets you include references to circuits (`.ind` files) inside another circuit.
- A new, unified array layout generator.
- Corrected an error in the "disperse" utility which produced incorrect results for some combinations of options.
- Adds a feature, applicable to all products, which allows the imaginary part of the reference background index to be specified in the additional component properties dialog. This can be important when defining some structures to improve index averaging of the imaginary index at interfaces.
- During an anisotropic calculation, all index tensors can be saved via the `index_profile_all` option.
- General speed improvements.

Index

(
(Acceptors cm-3), 167

.
.ind file, 140, 141

1

1:1, 15
1P, 12, 15, 40

2

2.5D, 31
2D, 31

3

3D, 12, 31
3D Shape, 77
3D Structure Type, 25, 31, 39, 56,
57, 58, 69, 71, 74, 79, 83, 130
3D Structure Types, 128

4

4P, 12, 15, 40

A

Absolute index in User Profile, 101,
259

Accept Fit, 157
Accept Symbol, 14, 35
Add, 96
Add to Project, 147
All, 18
Allow Mixed Properties, 120
Allow Mixed Types, 120
Allow Redraw 2D, 48
Allow Redraw 3D, 48
Alloy X, 167
Alloy Y, 167
Angle, 55, 79
Angle Value, 55
anisotropic, 152, 154
Arc, 25, 37
Arc (XZ), 91, 92
Arc Data..., 71, 92
Arc Final Angle, 71, 92
Arc Initial Angle, 71, 92
Arc Radius, 92
Arc Reference, 71, 92
Arc Type, 92
Arrange Icons, 22
Array Layout, 22
Aspect Ratio, 25
Attach On Edit, 120
Attach Tolerance, 120
Auto, 41
AutoFit, 157
AutoFit Method, 157
Automatically Regrid after Zoom,
120
Azimuthal Mode #, 204

B

Back Radius, 74
Background Index, 12, 17, 31, 56,
58, 59, 60, 62, 64, 67, 83
Background Material, 31, 59, 60, 62,
64, 67
BCC, 115
Bend Origin, 98
Bounding Shape, 115

C

Cascade, 22
Center, 71
Center Color, 120
Center Thickness, 74
Centered Cells, 174
Change Order, 22
channel, 128
Chi2, 159
Chi3, 159
Chi3 Saturation Coef., 159
Choose Simulation Tool, 25
Circle, 25, 37

Circle Mode, 77
Circuit Reference, 25, 37, 105
Circuit Reference Box, 108, 120
Clear Log, 22
Clipping, 102
Close, 22
Close All, 22
Close All Browsers, 22
Close All Plots, 22
Cluster Settings, 22
color scale, 48, 120
Color Shades, 48, 308, 309
Combine Mode, 96
Comment, 123, 148
Component Color, 120
Component Delta-N, 12, 31
Component Diameter, 77
Component Fields, 130
Component Height, 31, 57, 58, 59,
60, 62, 64, 66, 67, 74, 77, 79,
83, 181
Component Material, 31
Component Name, 123
Component Tree, 120
Component Width, 18, 31, 50, 57,
58, 59, 60, 62, 64, 67, 74, 83,
181
Compute Modes, 22, 25, 202
Contents, 139
Continue Simulation, 282
Convert a GDS-II
DXF
or CIF file into an RSoft .ind
design file:, 102
Convert to Polygons, 22
Copy (Ctrl-C), 22
Copy Material, 146
Copy Selection, 25
Correlation Length, 99, 100
Cosine S-Bend, 92
Cover Index, 31, 60, 62, 64
Cover Material, 31, 60, 62, 64
Crystal Axes, 125, 154
Cubic, 115
Custom, 148
cut, 178
Cut (Ctrl-X), 22
Cut Direction, 102
Cut Position, 102
Cut Selection, 25
Cut View Controls, 25
Cylinder, 77

D

Data File, 88, 95
Data File..., 79
Data Mode, 79
Date Mode, 67

- Default, 83, 96, 148, 174
- Default (Fixed Z), 47
- Defect Size, 115
- Definition, 130
- Delete, 116
- Delete (Del), 22
- Delete Filter, 133
- Delete Group, 133
- Delete Material or Library, 146
- Delete Symbol, 35
- Diamond lattices, 115
- diffused, 83
- Dimensions, 136
- Direction Indicators, 120
- Dispersion/Nonlinearity, 155, 159
- Dispersive, 152, 155, 157
- display, 148
- Display Axes, 148
- Display Color, 54, 130, 148, 181
- Display Material Profile, 20, 22, 25, 45, 202, 207
- Display Mode, 47, 48
- Display Opts..., 148
- Display..., 48, 309
- dN/dT, 165
- Domain Max, 174, 204
- Domain Min, 20, 47, 174, 204
- Donors (cm-3), 167
- Draw Center Line, 120
- Draw Edge, 120
- Draw Interior, 120
- Draw Priority, 123, 130
- Drawing Preferences, 39
- Drawing Tool Options, 25
- Duplicate, 22
- Duplicate Selection, 25
- Dynamic Array, 25, 115
- Dynamic Color Shades, 120
- Dynamic Load, 67, 79
- Dynamically reference a GDS-II IND
 - or SPT file using a Circuit Reference component:, 102

E

- E/H Control Parameter, 123
- Edge Color, 120
- Edit Default Tags, 177
- Edit Default Tags..., 22
- Edit Filter, 133
- Edit Global Settings, 25
- Edit Hierarchy Exports..., 22
- Edit Launch Field, 25
- Edit Layers, 25, 65
- Edit Materials, 25
- Edit Pathway Monitors, 25
- Edit Pathways, 25, 116
- Edit Symbols, 14, 25, 35, 108

- Edit Table, 171
- Edit User Profiles, 25
- Edit User Tapers, 25
- Edit X Axis View, 25
- Edit Y Axis View, 25
- Edit Z Axis View, 25
- Effective Index Calculation, 31, 128
- Electrode/Heater Type, 123
- Electro-Optic, 164
- Embedded, 106
- Embedded Circuits, 106
- Embedded File, 106
- Enable Auto Clustering, 120
- Enable Free-Carrier Index Effects, 167
- Enable Incomplete Ionization, 167
- Enable Nonuniform, 174
- Enable Radial, 204
- Enable Scroll Bars, 120
- End, 125
- Eps Linear, 151
- Eps Nonlinear, 159
- Eps_xx, 164
- Eps_yy, 164
- Eps_zz, 164
- Epsilon (imag), 152
- Epsilon (real), 152
- Exit, 22
- exponential, 91
- Export Circuit, 22
- Export Mask Layers, 103
- Export Polygon Mode, 103
- Export:, 102
- Expression, 14, 35
- Extended, 69
- Extended Segments, 25, 37, 69, 71
- External File, 106, 181

F

- FCC, 115
- File Name, 102
- Final Angle, 92
- Fit NK Data..., 157
- Fit Tol, 157
- Flatten Circuit, 22
- Flip Horizontal, 22
- Flip Section Horizontally, 25
- Flip Section Vertically, 25
- Flip Vertical, 22
- Free, 92
- Free Space Wavelength, 31, 157
- Front Radius, 74
- Full, 22

G

- Gain Term, 155, 159
- gaussian, 83

- GDS file:, 106
- Global Settings, 22
- Global Symbols..., 108
- Go, 22
- GPU Settings, 22
- Grading Ratio, 174
- GratingMOD Grating Layout, 22
- Grid Edge, 177
- Grid Edge Factor, 174, 177
- Grid Edge Size, 174
- Grid Grading, 174, 177
- Grid Minimum Divisions, 177
- Grid Options..., 174
- Grid PPW, 177
- Grid Ratio, 177
- grid size, 174, 177
- Grid Size (Bulk), 174
- Group Libraries, 145
- Group Properties, 133

H

- Height, 81
- Height Taper, 91
- HeightCoded, 48
- Help for Active Sim Program, 25
- Help for CAD program, 25
- Hexagonal, 115
- Hexagonal (HCP), 115
- Hide Group, 133

I

- Import Circuit, 22
- Import NK Data..., 152
- Import Symbols, 22
- Import/Export Resolution, 69, 103
- Import:, 102
- In, 22
- Inactive, 83, 117
- Inactive Color, 120
- Index (imag part), 56, 66
- Index (imag), 152
- Index (Imag. part), 59, 60, 62, 67
- Index (real), 152
- Index Difference, 17, 56, 58, 59, 60, 62, 66, 67, 77, 83, 87
- Index Display Max, 48
- Index Display Min, 48
- Index Generation, 22
- Index Input Mode, 120
- Index Profile Type, 31, 74, 83, 87
- Index Taper, 91
- Initial Angle, 92
- Input Mode, 152, 154
- Insert, 22
- Interface Alignment, 174

K

Kappa_xx, 165
Kappa_yy, 165
Kappa_zz, 165
Keep these Settings Next Time, 143

L

Lambda Min Lambda Max, 157
Last, 22
Lattice Info..., 106, 115
Lattice Size, 115, 136, 137, 138
Lattice Type, 115, 141
Launch DataBROWSER, 22, 25
Launch MOST, 25
Launch Normalization, 303
Launch Power, 159, 303
Layer Table, 64, 66
LED, 22
lens, 25, 37
Lens Mode, 74
Lens Shape, 74
Line Thickness, 81
linear, 91
Linear Eps, 157
Litho Resolution, 99
Litho Roughness, 99, 100
Load Settings, 12, 31, 102
Local Symbols, 181
Local Symbols..., 108, 115
Locally Defined, 147, 151
Lorentzian, 157

M

Map File, 120
Mark Type, 81
Mask Layer, 103, 123, 130
Material, 58
Material Properties, 144, 147, 151, 181
Material Settings, 102
Material System, 167
Materials..., 144
Max, 87, 88, 94
Max # Lorentzians, 157
Max Level Undo, 120
Measurement Data, 148
Merge, 96
Merge Priority, 96, 130, 181
Min, 87, 88, 94
Min/Max, 89
Minimum Divisions, 174
Mode Options..., 202
Model Dimension, 12, 31, 83
Monitor, 25, 37
More..., 98, 122, 340
MOST Optimizer/Scanner, 22

MPI Display Whole Domain, 120
Mu Linear, 162
Mu Nonlinear, 163
multilayer, 31
Multi-Pane View, 120
Multi-Physics, 22
Multi-Threading Settings, 22

N

Name, 14, 35, 130
Neff (imag), 204
Neff (Real), 204
New, 116
New Circuit, 12, 25, 31
New Library, 145, 146
New Material, 144, 146
New Profile...or User #, 83
New Symbol, 14, 35
New Taper..., 91
New... (Ctrl-N), 22
None, 55, 91, 159, 174
Normalization, 159
Number of Sides, 67

O

Offset, 17, 55
Offset Value, 17, 55
OK, 9, 12, 18, 20, 202
Open Circuit, 25
Open Folder In Working Directory, 25
Open Log, 22
Open Terminal in Working Directory, 25
Open... (Ctrl-O), 22
Origin, 148
Out, 22
Outline Color, 48
Output All Index Tensor Elements, 49
Output Prefix, 49, 143, 148, 202, 204
Output..., 49
Overlay Grid, 178

P

P1, 166
P2, 166
P3, 166
Pan Mode, 25, 41
Paste (Ctrl-V), 22
Paste Clipboard Contents, 25
Paste Material, 146
Paste Offset, 120
Pathway Color, 120
pathway edit mode, 116
PDK Info file, 31, 120, 232

Perform Simulation, 9
Period, 115
Phi, 74, 125
Plot Aspect Ratio, 48
Plot Range/Resolution, 148
Points, 87, 88, 94
Poisson's Ratio (nu), 166
Pol (3D EIM), 128
polarization, 204
Polarization Angle, 317
Poly Info..., 67
polygon, 25, 37
Polygon File, 67
Polygon Mode, 79
Preferences, 22
Print (Ctrl-P), 22
Printer Setup, 22
Priority, 77
profile type, 57, 117
Profiles..., 87
Psi, 125

Q

Q-Finder, 22
quadratic, 91

R

R_x, 164
R_y, 164
R_z, 164
Radial Calculation, 31
Radius, 92
Raised Sine S-Bend, 92
Randomness, 142
Raster, 103
Raw, 103
Raytracing-Converters, 22
Rectangular, 115
Redo (Ctrl-Y), 22
Redo Last Change, 25
ReDraw, 22
Redraw Circuit, 25
Reference Phi, 69, 124
Reference Psi, 69, 124
Reference Theta, 69, 124
Reference Type, 17, 55
Reference X, 124
Reference Y, 124
Reference Z, 124
ReGrid, 22
Reject Symbol, 35
Relative To, 55
Rib/Ridge, 31
Rotate Selection, 25
Rotate..., 22
RSoft .ind file:, 106
Rubberband Color, 120

Run Script, 22

S

Save (Ctrl-S), 22
Save As..., 22
Save Circuit, 25
Save Library, 146
s-bend, 25, 37, 92
Scale Factor, 115
Scale File..., 309
Scale Files..., 48
Seg Orientation, 69
Seg-Axis, 69
Segment (In Plane), 15, 25, 37
Segment (Out of Plane), 25, 37
Select All, 22, 133
select mode, 25, 42
Select Tolerance, 120
Select/Edit Components..., 22
Semiconductor, 167
Send Forward/Send Backward, 25
Send to Front/Send to Back, 25
Set View, 22
Show 3D Pane, 120
Show 3D View, 25
Show All Draw Tools, 120
Show Color Scale, 48
Show Default Comp as Comp Color, 120
Show Group, 133
Show Launch Fields, 120
Show Simulation Domain, 120
sidewall angle, 59, 99, 123
Sidewall Angle Ref., 99, 123
Simple, 157
simulated bend, 98, 123
Simulated Bend Radius, 98, 123
Simulation Region, 25, 37
Simulation Tool, 12, 31
Single-Pane/Four-Pane, 25
Slab Height, 31, 62, 64, 66
Slab Index, 31, 62
Slab Material, 31, 62
Slice Grid, 47
Slice Output, 120
Slice Position Y, 47
Solar-Cell, 22
SolidModel, 48
Source Type, 87, 88, 94, 95
Special Effects, 142
Sphere, 77
Start, 125
Start with Tree Expanded, 120
Static Color Shades, 120
Static Load, 79
step index, 83
Straddled Cells, 174
Stress-Optic, 166

Structure Preference, 25, 58
Surface Color, 47, 48

T

Tables, 22
Tag, 123
Tags, 177
Tapered-Laser, 22
Tapers..., 93
Target Value, 130
TE, 204
Test, 87
Text Editor, 35, 50
Thermal Exp. (alpha), 166
Thermo-Optic, 165
Theta, 125
Tile, 22
TM, 204
TMM-Mode-Solver, 22
Toggle Component Tree, 15, 25, 30
Try Fit, 157

U

Undo, 15
Undo Last Change, 25
Undo(Ctrl-Z), 22
Unit Peak, 159
Unit Power, 159
Use Exact Geometry data in a
 Sentaurus TCAD TDR file in
 an RSoft .ind design file:, 102
Use Magnetic Materials, 162, 163
Use Material (Add to Project), 146
Use Material Colors, 120
Use Perpendicular Launch Width
 Convention, 120
Use Single Precision, 120
User #, 91

V

Vector Mode, 317
Via an Index Difference:, 56
Via Materials:, 56
View BSDF File, 22
View Graphs, 25, 309
View Grid, 178
View Phi, 48
View Single Plot, 22
View S-Matrix File, 22
View Theta, 48

W

Warn when automatically resizing
 grid, 120

Warnings, 22
WDM Router Layout..., 22
Width Taper, 91
Width/Length, 81

X

X, 12, 40, 55
X Alloy, 167
X Cut, 47
X Pos Taper, 91
X Scale, 79
X'Y' Rotation, 69
X'Y' Rotation End, 124
X'Y' Rotation Start, 124
X'Y' Rotation Taper, 124
xx(im), 154
xx(re), 154
XZ Rotation Angle, 106

Y

Y, 12, 40, 55
Y Alloy, 167
Y Cut, 47
Y Pos Taper, 91
Yes, 98
Young's Modulus (E), 166

Z

Z, 12, 40, 55
Z Cut, 47
Z Pos Taper, 91
Z Position, 204
Z Scale, 79
z-Axis, 69
Zoom Factor (>1.0), 120
Zoom Full, 25, 41
Zoom In, 25, 41
zoom mode, 25, 41
Zoom Out, 25, 41
Zoom/Full Affects All Panes, 120
Zoom/Full Border, 120
Zoom/Full Uses Simulation Domain, 120